



PYTHON PROGRAMMING

PRINCIPLES AND PRACTICE



UNDERSTAND
THE CORE.



CODE WITH
CONFIDENCE.



BUILD WITH
PURPOSE.

```
00110101010110
0100100111000
11100111001010-16
```

```
def learn_python():
    print("Understand")
    print("Practice")
    print("Implement")
    return "Success"

if __name__ == "__main__":
    result = learn_python()
    print(result)
```

```
class Python:
    def __init__(self):
        self.principles = []
        self.practice = True

    def learn(self):
        while self.practice:
            code = self.write_code()
            self.improve(code)
        return "Mastery"

    def write_code(self):
        return "Consistency +
        Patience + Practice"
```

```
# Write clean code
# Think logically
# Solve real problems
# Never stop learning
```

```
110011010100
010010101
```

Prakash Shanmurthy • J. Somasekar
Vaibhav Prabhakar Raibole • Shiva Prasad Munukuntla

LURNEXA PUBLICATIONS

Python Programming Principles and Practice

By
Dr. Prakash Shanmurthy
Dr. J. Somasekar
Mr. Vaibhav Prabhakar Raibole
Mr. Shiva Prasad Munukuntla

LURNEXA PUBLICATIONS

Copyright © 2026 Lurnexa Publications. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

Title: *PYTHON PROGRAMMING: PRINCIPLES AND PRACTICE*

Authors: Dr. Prakash Shanmurthy, Dr. J. Somasekar, Mr. Vaibhav Prabhakar Raibole, Mr. Shiva Prasad Munukuntla

Edition: First Edition, 2026

Published by:

Lurnexa Publications Private Limited
(An ISO 9001:2015 Certified Organization)
130–187, Ramulavari Gudi Centre,
Gorantla, Guntur – 522034, Andhra Pradesh, India

Email: lurnexapublication@gmail.com

Website: www.lurnexa.in

ISBN: 978-81-903315-6-2

Disclaimer:

The views expressed in this publication are those of the authors and do not necessarily reflect those of the publisher. While every effort has been made to ensure accuracy, the publisher shall not be liable for any loss or damage arising from the use of this material.

Cataloguing-in-Publication Data: Available on request

Printed and bound in India

Preface

Python has emerged as one of the most widely used programming languages, owing to its simplicity, readability, versatility, and extensive application across diverse domains. From software development and data science to artificial intelligence, automation, and cloud computing, Python continues to play a significant role in shaping modern computing. Python Programming Principles and Practice is designed to provide readers with a strong foundation in Python programming while emphasizing both fundamental principles and practical application.

This book introduces the essential concepts of Python in a structured and accessible manner, beginning with programming fundamentals and gradually progressing toward more advanced concepts. It covers variables, data types, operators, control structures, functions, data structures, object-oriented programming, modules, file handling, exception handling, and other important features of Python. Throughout the book, emphasis is placed on developing logical thinking, problem-solving skills, and good programming practices.

The book is intended for students, beginners, and aspiring programmers who wish to learn Python systematically. It combines theoretical explanations with practical examples and programming exercises to help readers understand concepts and apply them effectively. The focus on principles and practice encourages readers not only to write programs but also to develop clean, efficient, and maintainable code.

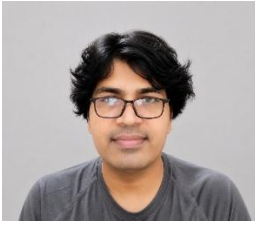
We hope that Python Programming Principles and Practice serves as a useful learning resource for academic study as well as a practical reference for developing programming skills. We believe that consistent practice, experimentation, and problem-solving are the keys to mastering Python, and this book is intended to support readers throughout that journey.

We sincerely thank our publisher, colleagues, students, families, and friends for their valuable encouragement and support in bringing this book to completion. We hope this book inspires readers to explore Python further and apply their programming knowledge to solve real-world problems.

Authors Profile



Dr. S. Prakash is an Assistant Professor in Computer Science and Engineering – AI-Driven DevOps at Jain (Deemed-to-be University), Karnataka, with over 12 years of academic and research experience. He holds a Ph.D. in Computer Science and Engineering and has published extensively in SCI, Scopus, and Web of Science-indexed journals and conferences, along with several granted patents. His research interests include Artificial Intelligence, Machine Learning, Knowledge Engineering, IoT, NLP, Semantic Web, and Deep Learning. He is an IEEE member and recipient of the Best Research Paper Award in the Sensors and IoT Track at RTPDP 2021.



Dr. J. Somasekar is a Professor and Head of CSE (AIML) at JAIN University, Bangalore, and a Research Fellow at INTI International University, Malaysia, with 19 years of academic experience. He holds a Ph.D. from JNTUA, an M.Tech. from NIT Karnataka, and completed postdoctoral research at the University of South Florida, USA. An Oracle-certified Data Science professional, he has delivered 290+ keynote and invited talks across 13 countries and published 50 research articles, authored three books and seven textbook chapters, and guided PhD scholars. His research interests include AI, Machine Learning, Data Science, Image Processing, and Cybersecurity. He has received several prestigious awards and an International Travel Grant from DST, Government of India.



Mr. Vaibhav Prabhakar Raibole is an Assistant Professor in Computer Science and Engineering (AI-Driven DevOps) at Jain (Deemed-to-be University), Bengaluru. He holds an M.Tech. in CSE (Information Security) from NIT Calicut and a B.Tech. in Information Technology from Dr. Babasaheb Ambedkar Technological University, and is pursuing his Ph.D. at Jain University. His academic and industry experience spans Computer Science and Information Security, with research interests in AI, DevOps, Information Security, Data Structures and Algorithms, and emerging technologies. He has contributed to innovation through patents and software copyrights in IoT, AI, healthcare, and educational technology.



Mr. Shiva Prasad Munukuntla is an Assistant Professor in Computer Science and Engineering (AI-Driven DevOps) at Jain (Deemed-to-be University), Bengaluru. He holds an M.Tech. from SVNIT, Surat, and a B.Tech. from JNTUH College of Engineering, Manthani. With academic and industry experience in software development, machine learning, and data analysis, his research interests include Artificial Intelligence, Explainable AI, Deep Learning, Predictive Maintenance, Anomaly Detection, and AI-Driven DevOps. His research and industry contributions span medical image classification, industrial predictive maintenance, computer vision, and data visualization.

Acknowledgements

The completion of Python Programming Principles and Practice has been made possible through the support, encouragement, and guidance of many individuals. We would like to express our sincere gratitude to everyone who contributed to this work.

We extend our heartfelt thanks to our publisher for providing us with the opportunity to bring this book to readers. We deeply appreciate their professional guidance, valuable suggestions, cooperation, and continuous support throughout the preparation and publication process.

We are especially grateful to our families for their unconditional love, patience, understanding, and constant encouragement. Their support and confidence in our efforts provided us with the motivation and strength to complete this book.

We also thank our friends and colleagues for their valuable discussions, constructive feedback, suggestions, and encouragement. Their insights helped us refine the content and improve the overall quality of the book.

Our sincere appreciation goes to our students and learners, whose curiosity, questions, and enthusiasm for programming inspired us to present Python concepts in a simple, practical, and systematic manner. We hope this book contributes meaningfully to their learning and professional growth.

We are also thankful to our teachers, mentors, and academic colleagues whose knowledge, guidance, and dedication to education have influenced our academic and professional journey.

Finally, we acknowledge the wider Python and open-source community for its valuable documentation, tools, libraries, and resources that continue to support learning and innovation.

Above all, we thank everyone who supported us directly or indirectly in completing this book. We sincerely value every contribution and word of encouragement.

With heartfelt gratitude to all who made this book possible.

Contents

Chapter 1: Introduction to Python	1
1.1 Introduction to Python.....	1
1.1.1 Key Features	2
1.1.2 Interpreted Execution: A First Look.....	2
1.1.3 Applications and Limitations	2
1.2 Why Python for Data Science and AI?	3
1.2.1 Python Compared to Other Languages	4
1.2.2 Industry Adoption.....	5
1.3 Installing Python	5
1.4 Anaconda Distribution	7
1.5 Jupyter Notebook	8
1.7 Python IDEs.....	10
1.8 Running Python Programs	11
1.9 Python Coding Standards	13
1.10 Summary	14
1.10.1 Key Takeaways	15
1.10.2 Important Terms	15
1.10.3 Glossary.....	16
1.11 Exercises	16
1.11.1 Multiple Choice Questions	16
1.11.2 Fill in the Blanks.....	17
1.11.3 1.11.3 True or False	17
1.11.4 Short Answer Questions.....	17
1.11.5 Long Answer Questions	18
1.11.6 Practical Programming Questions	18
1.11.7 Analytical Question	18

- Chapter 2: Python Basics: Variables, Data Types, and Operators..... 20**
 - 2.1 Introduction to Python Basics 20
 - 2.2 Variables and Identifiers..... 20
 - 2.2.1 Naming Rules and Conventions 21
 - 2.3 Data Types in Python 22
 - 2.3.1 Numeric Types..... 23
 - 2.3.2 Text Type: str 23
 - 2.3.3 Boolean and None Types..... 24
 - 2.3.4 Checking and Comparing Types 24
 - 2.4 Type Conversion.....25
 - 2.5 Operators in Python..... 26
 - 2.5.1 Arithmetic, Relational, and Logical Operators..... 26
 - 2.5.2 Assignment, Bitwise, Identity, and Membership Operators 28
 - 2.6 Operator Precedence and Associativity 29
 - 2.7 Input and Output in Python..... 30
 - 2.8 Comments and Docstrings 31
 - 2.9 Mutable and Immutable Objects..... 31
 - 2.10 Summary.....33
 - 2.10.1 Key Takeaways33
 - 2.10.2 Important Terms..... 34
 - 2.11 Exercises 34
 - 2.11.1 Multiple Choice Questions 34
 - 2.11.2 Fill in the Blanks..... 35
 - 2.11.3 True or False 35
 - 2.11.4 Short Answer Questions..... 35
 - 2.11.5 Long Answer Questions 36
 - 2.11.6 Practical Programming Questions 36
 - 2.11.7 Analytical Question 36

- Chapter 3: Control Structures and Functions 37**
 - 3.1 Conditional Statements37
 - 3.1.1 Introduction37
 - 3.1.2 Decision Making in Python.....37
 - 3.1.3 if Statement..... 38
 - 3.1.4 if-else Statement 39
 - 3.1.5 if-elif-else Ladder 40
 - 3.1.6 Multiple Conditions and Logical Operators in Conditions 41
 - 3.1.7 Comparison Operators..... 41
 - 3.1.8 Membership and Identity Operators in Conditions 42
 - 3.1.9 Practical Examples 42
 - 3.1.10 Section Summary 43
 - 3.2 Nested Conditions 43
 - 3.2.1 Introduction 43
 - 3.2.2 Nested if Statements..... 44
 - 3.2.3 Multi-level Decision Making 45
 - 3.2.4 Real-world Applications..... 45
 - 3.2.5 Advantages and Limitations 45
 - 3.2.6 Programming Examples 46
 - 3.2.7 Section Summary 46
 - 3.3 Loops.....47
 - 3.3.1 Introduction..... 47
 - 3.3.2 Need for Iteration..... 47
 - 3.3.3 while Loop 47
 - 3.3.4 for Loop 49
 - 3.3.5 Range Function 49
 - 3.3.6 Nested Loops..... 50
 - 3.3.7 Infinite Loops 51
 - 3.3.8 Loop Efficiency52

3.3.9	Practical Examples	52
3.3.10	Section Summary	53
3.4	Loop Control Statements	53
3.4.1	Break Statement	53
3.4.2	continue Statement	54
3.4.3	pass Statement	55
3.4.4	Practical Examples	56
3.4.5	Best Practices	57
3.4.6	Common Mistakes	57
3.5	Pattern Programs	57
3.5.1	Square Pattern	58
3.5.2	Rectangle Pattern	58
3.5.3	Right Triangle	59
3.5.4	Left Triangle	59
3.5.5	Pyramid Pattern	60
3.5.6	Inverted Pyramid	60
3.5.7	Diamond Pattern	61
3.5.8	Floyd's Triangle	62
3.5.9	Pascal's Triangle (Basic Implementation)	62
3.5.10	Number Patterns	63
3.5.11	Alphabet Patterns	63
3.5.12	Hollow Patterns	64
3.5.13	Star Patterns and Multiplication Table Patterns	64
3.5.14	Pattern Logic Explanation and General Strategy	65
3.6	Exception Handling	65
3.6.1	Introduction	65
3.6.2	Runtime Errors and Syntax Errors	65
3.6.3	Types of Exceptions	66
3.6.4	try, except, else, and finally	66

3.6.5	raise Statement	68
3.6.6	User-defined Exceptions	68
3.6.7	Multiple Exception Handling	69
3.6.8	Nested Exception Handling	69
3.6.9	File Handling Exceptions	70
3.6.10	Best Practices	70
3.6.11	Industrial Examples	71
3.6.12	Section Summary	71
3.7	Functions with Examples	71
3.7.1	Introduction to Functions	71
3.7.2	Need for Functions and Advantages of Modular Programming	72
3.7.3	Defining and Calling Functions	72
3.7.4	Function Parameters: Positional, Keyword, and Default Arguments ...	73
3.7.5	Variable-length Arguments: *args and **kwargs	74
3.7.6	Return Statement and Multiple Return Values	75
3.7.7	Scope of Variables: Local and Global Variables	75
3.7.8	Lambda Functions and Anonymous Functions	77
3.7.9	Recursive Functions	77
3.7.10	Built-in Functions and User-defined Functions	79
3.7.11	Documentation Strings (Docstrings)	79
3.7.12	Type Hints (Introduction)	80
3.7.13	Practical Programming Examples	80
3.7.14	Section Summary	81
3.8	Chapter Summary	82
3.8.1	Key Takeaways	82
3.8.2	Important Terms	83
3.8.3	Glossary	83
3.9	Exercises	84
3.9.1	Section A: Multiple Choice Questions	84

3.9.2	Section B: Fill in the Blanks.....	84
3.9.3	Section C: True/False	85
3.9.4	Section D: Match the Following	85
3.9.5	Section E: Short Answer Questions	85
3.9.6	Section F: Long Answer Questions	85
3.9.7	Section G: Predict the Output	86
3.9.8	Section H: Debug the Code.....	86
3.9.9	Section I: Programming Exercises	87
3.9.10	Section J: Pattern Programming Exercises.....	87
3.9.11	Section K: Exception Handling Programs	87
3.9.12	Section L: Function-based Programming Problems.....	88
3.9.13	Section M: Analytical Questions	88
3.9.14	Section N: Mini Project Ideas	88
Chapter 4: Object-Oriented Programming.....		89
4.1	User-defined Functions	89
4.2	Lambda Functions (Brief Review)	90
4.3	Recursion (Brief Review)	90
4.4	Modules	91
4.4.1	Introduction	91
4.4.2	Need for Modules.....	91
4.4.3	Creating User-defined Modules.....	91
4.4.4	Import Statement	92
4.4.5	from...import Statement.....	92
4.4.6	import ... as Statement.....	92
4.4.7	Built-in Modules.....	93
4.4.8	Using Modules in Data Science Applications	94
4.4.9	Best Practices	95
4.5	Packages	96
4.5.1	Introduction	96

4.5.2	Need for Packages	96
4.5.3	Package Structure	96
4.5.4	<code>__init__.py</code>	97
4.5.5	Importing Packages	97
4.5.6	Creating Packages.....	98
4.5.7	Installing Third-party Packages	98
4.5.8	Industrial Applications	98
4.6	Classes	99
4.6.1	Introduction to Object-Oriented Programming	99
4.6.2	Procedural vs Object-Oriented Programming.....	99
4.6.3	Features of OOP	100
4.6.4	Creating Classes	100
4.6.5	Class Attributes and Instance Attributes	101
4.6.6	Constructors (<code>__init__</code>)	102
4.6.7	Methods and the <code>self</code> Keyword	102
4.6.8	Class Variables vs Instance Variables	102
4.6.9	Static Methods.....	103
4.6.10	Class Methods	103
4.6.11	Magic (Dunder) Methods	103
4.6.12	Practical Programming Example.....	104
4.7	Objects	105
4.7.1	Introduction.....	105
4.7.2	Creating Objects.....	105
4.7.3	Object Lifecycle	106
4.7.4	Accessing Members	107
4.7.5	Object Interaction.....	107
4.7.6	Memory Concepts.....	108
4.7.7	Real-world Examples.....	109
4.8	Inheritance	109

4.8.1 Introduction	109
4.8.2 Need for Inheritance	109
4.8.3 Types of Inheritance	110
4.8.4 Method Overriding	112
4.8.5 The super() Function	113
4.8.6 Advantages of Inheritance	114
4.8.7 Limitations of Inheritance	114
4.8.8 Industrial Examples.....	114
4.9 Polymorphism	114
4.9.1 Introduction	114
4.9.2 Method Overloading (Concept and Python Alternatives)	115
4.9.3 Method Overriding (Cross-reference).....	116
4.9.4 Duck Typing.....	116
4.9.5 Operator Overloading.....	116
4.9.6 Built-in Polymorphism	117
4.9.7 Practical Example	117
4.9.8 Applications in Machine Learning Libraries	118
4.10 Encapsulation	118
4.10.1 Introduction	118
4.10.2 Data Hiding	119
4.10.3 Public, Protected, and Private Members.....	119
4.10.4 Getter and Setter Methods.....	119
4.10.5 Property Decorators.....	120
4.10.6 Advantages of Encapsulation	120
4.10.7 Best Practices.....	121
4.10.8 Industrial Examples.....	121
4.11 OOP in Data Science and Machine Learning — An Integrated Example	121
4.12 Chapter Summary	123
4.12.1 Key Takeaways	123

4.12.2 Important Terms	124
4.13 Exercises.....	124
4.13.1 A. Multiple Choice Questions	124
4.13.2 B. Fill in the Blanks.....	125
4.13.3 C. True or False	125
4.13.4 D. Match the Following	125
4.13.5 E. Short Answer Questions	126
4.13.6 F. Long Answer Questions	126
4.13.7 G. Predict the Output.....	126
4.13.8 H. Debug the Code	127
4.13.9 I. Programming Exercises.....	127
4.13.10 J. OOP-based Programming Problems	127
4.13.11 K. Mini Projects Using Classes and Objects	128
4.13.12 L. Analytical Questions	128
Chapter 5: Python Data Structures and Libraries.....	129
5.1 Lists	129
5.1.1 Introduction	129
5.1.2 Creating and Accessing Lists	129
5.1.3 List Mutability	129
5.1.5 Iterating Over a List.....	130
5.1.6 Nested Lists	130
5.1.7 Practical Example	131
5.1.8 Applications in Data Science.....	131
5.2 Tuples.....	131
5.2.1 Introduction.....	131
5.2.2 Accessing Tuple Elements	132
5.2.3 Immutability	132
5.2.4 Tuple Packing and Unpacking.....	132
5.2.5 Tuple Methods and Built-in Functions	132

5.2.7 Applications in Data Science	133
5.3 Dictionaries	133
5.3.1 Introduction.....	133
5.3.2 Accessing and Modifying Dictionary Elements.....	133
5.3.3 Common Dictionary Methods.....	134
5.3.4 Iterating Over a Dictionary.....	134
5.3.5 Nested Dictionaries	134
5.3.6 Practical Example	135
5.3.7 Applications in Data Science	135
5.4 Sets	135
5.4.1 Introduction.....	135
5.4.3 Common Set Methods.....	136
5.4.4 Practical Example	136
5.4.5 Applications in Data Science	137
5.5 Strings	137
5.5.1 Introduction.....	137
5.5.2 String Indexing and Slicing.....	137
5.5.3 String Immutability	137
5.5.4 Common String Methods	138
5.5.5 String Formatting	138
5.5.6 Practical Example	139
5.5.7 Applications in Data Science	139
5.6 List Comprehension.....	139
5.6.1 Introduction.....	139
5.6.2 General Syntax.....	139
5.6.3 Conditional Expressions Within Comprehensions.....	140
5.6.4 Nested List Comprehension	140
5.6.5 Practical Example	140
5.6.6 Applications in Data Science	141

5.7 Dictionary Comprehension.....	141
5.7.1 Introduction.....	141
5.7.2 Filtering with Dictionary Comprehension.....	141
5.7.3 Transforming Keys and Values	141
5.7.4 Practical Example	142
5.7.5 Applications in Data Science.....	142
5.8 File Handling	142
5.8.1 Introduction.....	142
5.8.2 Opening and Closing Files.....	142
5.8.3 The with Statement (Context Manager)	143
5.8.4 Reading from a File.....	143
5.8.5 Writing and Appending to a File.....	144
5.8.6 Working with CSV Files	144
5.8.7 Exception Handling During File Operations.....	144
5.8.8 Practical Example.....	145
5.8.9 Applications in Data Science.....	145
5.9 Introduction to Python Libraries.....	145
5.9.1 What is a Library?.....	145
5.9.2 Why Libraries Matter in Data Science.....	145
5.9.3 NumPy — Numerical Computing.....	146
5.9.4 pandas — Data Manipulation and Analysis.....	146
5.9.5 matplotlib and seaborn — Data Visualisation	146
5.9.6 scikit-learn — Machine Learning.....	147
5.9.7 Other Notable Libraries	147
5.9.8 Installing and Importing Libraries	148
5.9.9 Looking Ahead	148
5.10 Chapter Summary	148
5.10.1 Key Takeaways.....	149
5.10.2 Important Terms.....	149

5.11 Exercises	149
5.11.1 A. Multiple Choice Questions.....	149
5.11.2 B. Fill in the Blanks	150
5.11.3 C. True or False.....	150
5.11.4 D. Match the Following.....	151
5.11.5 E. Short Answer Questions	151
5.11.6 F. Long Answer Questions	151
5.11.7 G. Predict the Output.....	151
5.11.8 H. Debug the Code.....	152
5.11.9 I. Programming Exercises	152
5.11.10 J. Data Structure-based Programming Problems	152
5.11.11 K. Mini Projects	153
5.11.12 L. Analytical Questions	153
Chapter 6: NumPy Library	154
6.1.1 Why NumPy?	154
6.1.2 Installing NumPy	154
6.1.3 Importing NumPy	155
6.1.4 Checking NumPy Version	155
6.2 NumPy Arrays	155
6.2.1 Creating Arrays from Python Lists	155
6.2.2 Array Attributes	156
6.2.3 Creating Arrays Using Built-in Functions	157
6.2.4 Data Types in NumPy.....	161
6.2.5 Type Conversion	162
6.2.6 Reshaping Arrays.....	163
6.2.7 Adding and Removing Dimensions	165
6.2.8 Stacking and Concatenating Arrays	167
6.3 Indexing	170
6.3.1 1D Array Indexing.....	170

6.3.2 2D Array Indexing.....	170
6.3.3 3D Array Indexing.....	171
6.3.4 Boolean Indexing.....	172
6.3.5 Fancy (Integer) Indexing	173
6.3.6 Modifying Elements Through Indexing	174
6.4 Slicing	175
6.4.1 1D Array Slicing	175
6.4.2 2D Array Slicing	176
6.4.3 Mixing Indexing and Slicing.....	178
6.4.4 Views vs Copies in Slicing.....	179
6.5 Broadcasting	180
6.5.1 Broadcasting Rules.....	180
6.5.2 Scalar and 1D Array Broadcasting	181
6.5.3 2D Array and 1D Array Broadcasting	181
6.5.4 Column Vector and Row Vector Broadcasting.....	182
6.5.5 Practical Example: Normalizing Data.....	182
6.5.6 Practical Example: Computing Distance Matrix.....	184
6.6 Universal Functions (ufuncs).....	184
6.6.1 Arithmetic Ufuncs	184
6.6.2 Trigonometric Ufuncs.....	185
6.6.3 Exponential and Logarithmic Ufuncs	186
6.6.4 Rounding Ufuncs.....	187
6.6.5 Statistical Ufuncs.....	188
6.6.6 Aggregation Along Axes	189
6.6.7 Comparison Ufuncs.....	190
6.6.8 Boolean Ufuncs.....	190
6.6.9 Performance Comparison: Ufuncs vs Loops.....	191
6.7 Linear Algebra Operations	192
6.7.1 Matrix Multiplication	192

6.7.2 Transpose.....	193
6.7.3 Determinant.....	194
6.7.5 Solving Linear Systems.....	196
6.7.6 Eigenvalues and Eigenvectors.....	197
6.7.7 Singular Value Decomposition (SVD)	198
6.7.8 Matrix Norms.....	199
6.7.9 Rank of a Matrix.....	200
6.7.10 Trace of a Matrix	201
6.8 Random Numbers	201
6.8.1 The New Generator API.....	201
6.8.2 Random Integers.....	202
6.8.4 Shuffling and Permutations	205
6.8.5 Random Sampling Without Replacement.....	205
6.8.6 Legacy Random Functions (Still Commonly Used).....	206
6.9 Practical Examples	207
6.9.1 Generating Synthetic Dataset.....	207
6.9.2 Simple Linear Regression with NumPy.....	208
6.9.3 K-Nearest Neighbors (Distance Computation).....	209
6.10 Summary	210
6.11 Exercises	211
Chapter 7: Pandas Library	202
7.1 Introduction to Pandas	202
7.1.1 Why Pandas?.....	202
7.1.2 Installing Pandas	203
7.1.3 Importing Pandas	203
7.1.4 Checking Pandas Version.....	203
7.2 Pandas Series.....	203
7.2.1 Creating a Series	204
7.2.3 Series Indexing and Slicing.....	207

7.2.4 Boolean Indexing in Series	208
7.2.5 Series Operations	210
7.2.6 Series Statistical Methods and Sorting.....	213
7.3 Pandas DataFrames	214
7.3.1 Creating DataFrames	214
7.3.2 DataFrame Attributes.....	216
7.3.3 Viewing DataFrame Content	218
7.4 Importing Data.....	220
7.4.1 Reading CSV Files.....	220
7.4.2 Reading Excel and JSON Files	221
7.5 Exporting Data	222
7.6 Selecting Data	223
7.6.1 Selecting Columns	224
7.6.2 Using loc[] - Label-based Selection.....	225
7.6.3 Using iloc[] - Position-based Selection.....	227
7.6.4 Difference Between loc, iloc, at, and iat.....	229
7.7 Filtering Data	229
7.7.1 Basic Filtering and Combining Conditions	229
7.7.2 Using isin() and String Methods	231
7.8 Sorting.....	233
7.8.1 Sorting by Values and Index.....	233
7.8.2 Getting n Largest and n Smallest Values	235
7.9 Grouping.....	236
7.9.1 Basic GroupBy and Aggregation	236
7.9.2 Named Aggregations and Transform.....	237
7.9.3 GroupBy with Filter	238
7.10 Aggregation	239
7.10.1 Built-in Aggregation Functions and agg()	239
7.10.2 Cumulative Aggregations.....	240

7.11 Merging DataFrames	241
7.11.1 Types of Joins	241
7.11.2 Performing Joins.....	242
7.11.3 Concatenating DataFrames	243
7.12 Missing Values	244
7.12.1 Detecting Missing Values	245
7.12.3 Filling Missing Values.....	247
7.12.4 Interpolation	249
7.13 Additional DataFrame Operations	250
7.13.1 Adding and Removing Columns.....	250
7.13.2 Applying Functions	251
7.14 Summary	254
7.15 Exercises	255
Chapter 8: Scikit-learn	259
8.1 Introduction to Scikit-learn	259
8.1.1 Why Scikit-learn?	259
8.1.2 Key Features.....	259
8.2 Installing Scikit-learn	260
8.2.1 Installation Using pip	260
8.2.2 Installation Using conda	260
8.2.3 Verifying Installation	260
8.2.4 Importing Common Modules.....	260
8.3 Dataset Loading	261
8.3.1 Built-in Datasets Overview	261
8.3.2 Loading the Iris Dataset.....	261
8.3.3 Exploring the Iris Dataset.....	262
8.3.4 Loading the Breast Cancer Dataset	265
8.3.5 Loading Regression Datasets	266
8.3.6 Generating Synthetic Datasets	267

8.4 Dataset Splitting	269
8.4.1 Basic Train-Test Split.....	269
8.4.2 Stratified Splitting.....	270
8.4.3 Train-Validation-Test Split	271
8.4.4 Cross-Validation	272
8.5 Building Your First Model.....	274
8.5.1 The Scikit-learn API	274
8.5.3 Classification: Decision Tree	275
8.5.4 Classification: Random Forest.....	277
8.5.5 Regression: Linear Regression.....	278
8.5.6 Regression: Decision Tree Regressor	279
8.5.7 Clustering: K-Means.....	280
8.5.8 Finding Optimal K using Elbow Method	282
8.6 Model Evaluation	283
8.6.1 Classification Metrics	283
8.6.2 Confusion Matrix	284
8.6.4 Regression Metrics.....	286
8.6.5 Understanding the Metrics.....	288
8.6.6 ROC Curve and AUC Score	288
8.7 Data Preprocessing	289
8.7.1 Feature Scaling.....	290
8.7.2 Encoding Categorical Variables	291
8.7.3 Handling Missing Values	293
8.8 Pipelines.....	295
8.8.1 Creating a Simple Pipeline	295
8.8.2 Pipeline with Multiple Preprocessing Steps	296
8.8.3 Comparing Models with Pipelines	298
8.8.4 Pipeline with ColumnTransformer	300
8.9 Hyperparameter Tuning	302

8.9.1 Grid Search.....	302
8.9.2 Random Search	304
8.10 Saving and Loading Models	306
8.10.1 Saving Models with joblib.....	306
8.10.2 Loading Models with joblib	307
8.10.3 Saving Pipelines	309
8.10.4 Saving Multiple Models	310
8.11 Complete Machine Learning Workflow Example.....	311
8.12 Summary.....	315
8.13 Exercises.....	315

Chapter 1: Introduction to Python

1.1 Introduction to Python

Python is a high-level, general-purpose, interpreted programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability through a clean, minimal syntax, allowing programmers to express ideas in fewer lines of code than languages such as C++ or Java. The name "Python" refers not to the reptile but to the British comedy series Monty Python's Flying Circus, reflecting the informal, pragmatic philosophy that has guided the language's design since its early development at Centrum Wiskunde & Informatica (CWI) in the Netherlands.

Van Rossum began designing Python in December 1989 as a successor to the ABC programming language, seeking to retain ABC's emphasis on readability while addressing its practical limitations, such as poor extensibility. Python 0.9.0, released in February 1991, already included functions, exception handling, and core data types such as lists, dictionaries, and strings that programmers continue to rely on today.

Python has since evolved through several major versions. Python 2.0, released in 2000, introduced list comprehensions and a cycle-detecting garbage collector and established a more formal, community-driven development process. Python 3.0, released in 2008, was a deliberate, backward-incompatible redesign intended to remove inconsistencies that had accumulated over the years, most notably by unifying string handling around Unicode. Although the migration from Python 2 to Python 3 took the ecosystem more than a decade to complete, Python 3 is now the only actively maintained version, and it is the version used throughout this textbook.

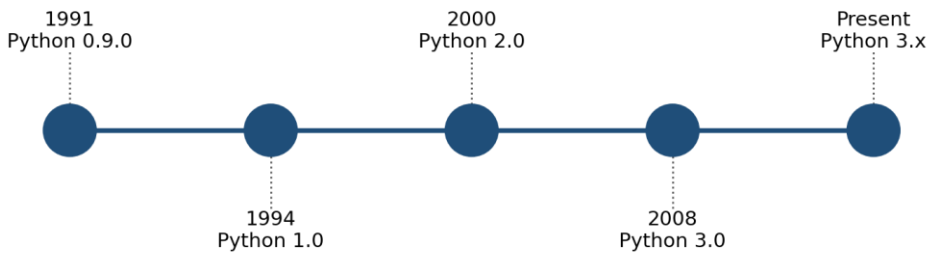


Figure 1.1: Evolution of Python from 1991 to the present

1.1.1 Key Features

Python's widespread adoption rests on a small set of consistent design choices. Its syntax closely resembles pseudocode and uses indentation, rather than braces, to define code blocks, which reduces visual clutter and makes programs easier for beginners to read and reason about. Python is dynamically typed, so variable types are resolved at runtime rather than declared in advance; this speeds up prototyping, though it places more responsibility on the programmer to test code thoroughly, since type-related errors surface only when the affected code actually runs.

Python is also an interpreted language: code is executed statement by statement rather than compiled beforehand into a standalone executable, which simplifies debugging and supports interactive experimentation. It is multi-paradigm, supporting procedural, object-oriented, and functional programming styles within the same language, so developers can choose the approach best suited to a given problem. Python ships with an extensive standard library, often summarized by the phrase "batteries included", and the Python Package Index (PyPI) hosts several hundred thousand additional third-party packages covering everything from data analysis to web development. Finally, because the interpreter abstracts away operating-system-specific details, Python programs generally run without modification on Windows, macOS, and Linux.

1.1.2 Interpreted Execution: A First Look

Because Python is interpreted, a short expression can be evaluated immediately without writing a complete program, which is useful for quickly confirming how a language feature behaves:

```
>>> 5 + 3

8

>>> name = "Data Science"

>>> print(f"Welcome to {name}")

Welcome to Data Science
```

The interpreter evaluates each line as soon as it is entered and prints the resulting value directly; this interactive workflow, the Read-Eval-Print Loop or REPL, is explored further in Section 1.8.

1.1.3 Applications and Limitations

Because of this versatility, Python is used across a wide range of domains. In data analysis and visualization, it cleans, transforms, and summarizes large datasets. In machine

learning and artificial intelligence, it builds and trains predictive models and neural networks. In web development, frameworks such as Django and Flask power server-side applications. In scripting and automation, Python replaces repetitive manual tasks with short, reliable programs. It is also used extensively in scientific computing, cybersecurity tooling, and, through frameworks such as Pygame, simple game development. Figure 1.2 summarizes how these application areas are built on top of Python's library ecosystem and the core interpreter.

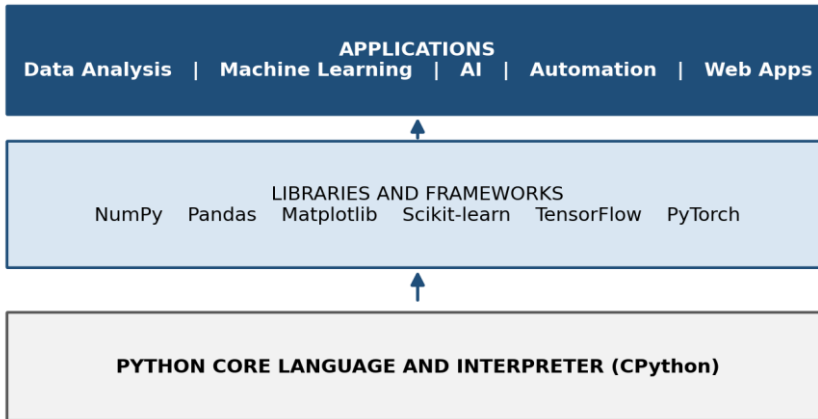


Figure 1.2: The Python data science and AI ecosystem, showing applications built on libraries built on the core language

1.2 Why Python for Data Science and AI?

Among the languages available to data professionals, including R, MATLAB, and Julia, Python has become the dominant choice for data science, machine learning, and artificial intelligence. Machine learning workflows are inherently iterative: loading data, engineering features, training a model, evaluating it, and adjusting the approach. Python's concise syntax and interactive execution model, particularly through Jupyter notebooks, make this cycle fast and forgiving.

Libraries such as Scikit-learn provide a consistent interface for classical algorithms, while TensorFlow and PyTorch supply the automatic differentiation machinery needed for deep learning.

For data science specifically, Pandas offers the DataFrame, a tabular structure that mirrors how analysts already think about spreadsheets and database tables, while NumPy provides fast array-based numerical computation beneath it. Because these tools interoperate seamlessly, and because Matplotlib and Seaborn integrate directly with them for visualization, a single Python script can carry a dataset from a raw CSV file

through cleaning, statistical analysis, and a publication-quality chart without ever leaving the language.

This unified ecosystem, combined with Python's gentle learning curve, also explains its dominance in education: it is the most common first language taught in introductory computer science and data science courses worldwide, allowing domain experts in fields such as biology, medicine, and finance to learn enough programming to conduct their own analyses without formal computer science training.

1.2.1 Python Compared to Other Languages

Figure 1.3 summarizes how Python's dynamic typing and interpreted execution model trade raw execution speed for a substantially gentler learning curve and a far larger data science library ecosystem than C++ or Java currently offer.

	Python	C++	Java
Typing	Dynamic	Static	Static
Execution Model	Interpreted	Compiled	Compiled (JVM)
Learning Curve	Gentle	Steep	Moderate
Primary Domain	Data Science / AI	Systems / Games	Enterprise Apps

Figure 1.3: Python compared with C++ and Java across four key criteria

C++ compile’s directly to native machine code and gives programmers fine-grained control over memory, which suits performance-critical systems software and game engines but demands a steeper learning investment. Java compiles to bytecode executed by the Java Virtual Machine and remains dominant in large enterprise and Android applications, but its data science library ecosystem is far smaller than Python's. Python's combination of a gentle learning curve and a mature, unified data science ecosystem is what tips the balance in its favor for this particular domain, even though it is not the fastest of the three languages at raw computation.

1.2.2 Industry Adoption

Python's role in production systems illustrates its practical value. Google popularized TensorFlow and uses Python across much of its internal infrastructure; Netflix relies on Python-based data pipelines to power a recommendation engine that materially improves subscriber retention; and Amazon uses Python in machine learning pipelines for product recommendations, demand forecasting, and its Alexa voice assistant. Microsoft integrates Python into Azure Machine Learning and supports it natively across Visual Studio Code, while OpenAI trains large language models, including the GPT family, primarily with Python and the PyTorch framework. Tesla uses Python in parts of its autonomous-driving data pipeline.

Beyond large technology firms, Python is used for medical image analysis and diagnosis-support systems in healthcare, algorithmic trading, credit-risk modeling, and fraud detection in finance, intrusion detection and malware analysis in cybersecurity, and simulation, data analysis, and instrument control at scientific research institutions such as CERN and NASA. This breadth of adoption is reflected in developer and job-market surveys, which consistently rank Python among the top languages requested for data science and machine learning roles.

Industry Insight

Annual developer and job-market surveys have repeatedly placed Python among the top two or three most in-demand languages overall, and consistently first among languages requested specifically for data science and machine learning roles, ahead of R, MATLAB, and Java combined for those positions.

Interview Question

Q: Why is Python preferred for machine learning despite being slower than C++?

A strong answer notes that Python's core numerical libraries (NumPy, Pandas, TensorFlow) are implemented in optimized C beneath a Python interface, so Python provides development speed without sacrificing the underlying computational performance.

1.3 Installing Python

The official Python interpreter is distributed by the Python Software Foundation at python.org. Figure 1.4 outlines the standard installation sequence, which is broadly similar across Windows, macOS, and Linux.

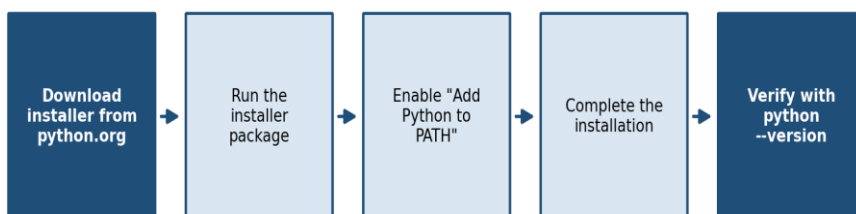


Figure 1.4: The Python installation process

On Windows, the installer offers a checkbox labeled "Add Python to PATH" on its first screen; leaving it unchecked is the most common reason beginners later see a "Python is not recognized as an internal or external command" error, since the operating system will not know where to find the interpreter. Checking this box and clicking "Install Now" is normally sufficient to complete the installation.

On macOS, Python 3 should be installed separately from the outdated Python 2 interpreter that may already be present on the system, since Python 2 is no longer maintained and should not be used for development. This is done either through the official macOS installer package from python.org or through the Homebrew package manager with the command "brew install python3".

Most Linux distributions include Python 3 by default. Where it is missing or an update is desired, Debian-based distributions such as Ubuntu can install or update it with "sudo apt update" followed by "sudo apt install python3", while Red Hat-based distributions use "sudo dnf install python3" instead. On every platform, running "python --version" (or "python3 --version") in a terminal confirms that the installation succeeded.

```
$ python --version
```

```
Python 3.12.4
```

Python installations also include pip, the standard package manager, which installs third-party libraries from PyPI with a single command, for example "pip install numpy pandas matplotlib".

Best Practice

Use an isolated virtual environment (via venv or conda) for each project so that different projects can depend on different, non-conflicting package versions on the same machine.

1.4 Anaconda Distribution

Anaconda is a free, open-source distribution of Python (and R) curated specifically for data science and scientific computing. Rather than installing the interpreter and each library separately, Anaconda bundles the interpreter together with core data science libraries, the conda package and environment manager, Jupyter Notebook, and the Spyder IDE in a single installation, as shown in Figure 1.5.

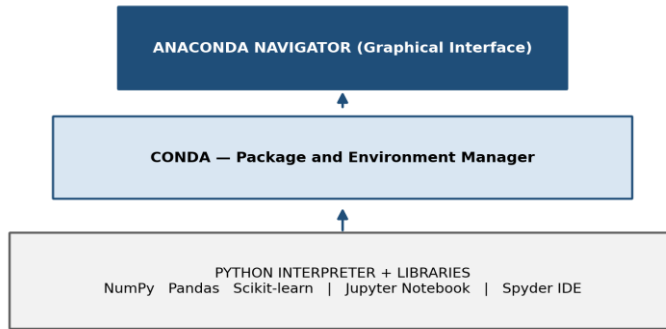


Figure 1.5: Layered architecture of the Anaconda distribution

Beyond the interpreter itself, a typical Anaconda installation includes NumPy, Pandas, Matplotlib, and Scikit-learn pre-installed; Jupyter Notebook and JupyterLab, ready to launch immediately; the Spyder IDE, discussed in Section 1.7; and Anaconda Navigator, a graphical interface for managing all of these tools without the command line.

Conda, Anaconda's package and environment manager, creates isolated project environments in the same spirit as Python's built-in `venv` tool, but it can additionally manage non-Python dependencies, which is valuable for scientific packages with complex compiled components. A new environment is created and populated as follows:

```
$ conda create --name ds_project python=3.11
$ conda activate ds_project
(ds_project) $ conda install numpy pandas scikit-learn
```

Anaconda Navigator, shown in Figure 1.6, provides a graphical alternative to these commands for users who prefer not to work from the command line. Learners who want the conda package manager without Anaconda's full multi-gigabyte bundle of pre-installed libraries can instead install Miniconda, a minimal distribution that includes only Python and conda itself, with additional packages installed on demand.

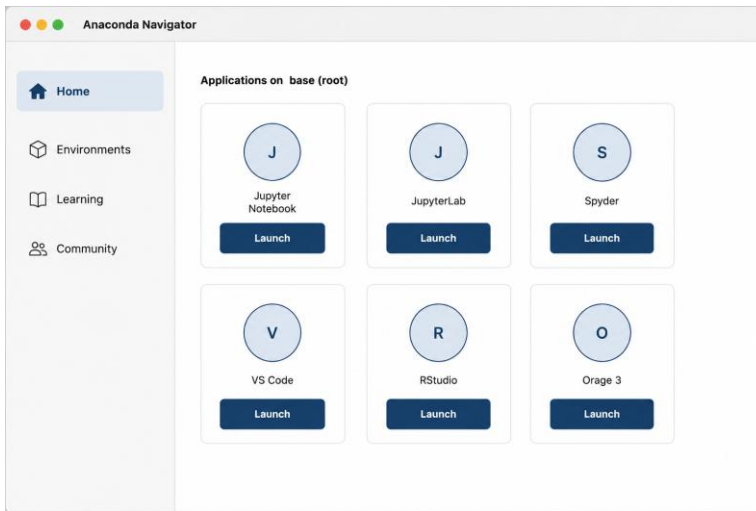


Figure 1.6: Anaconda Navigator interface layout (illustrative recreation; insert an official screenshot for final publication)

1.5 Jupyter Notebook

Jupyter Notebook is a free, web-based interactive computing environment in which code, its output, visualizations, and narrative text coexist in a single document saved with the .ipynb extension. The name is a reference to the three languages it was originally built to support: Julia, Python, and R. A notebook is composed of cells, executed independently and in any order the user chooses, which makes it well suited to the iterative, exploratory style of data analysis. Figure 1.7 shows the notebook interface, with its toolbar, cell input area, and rendered output displayed directly beneath the code that produced it.

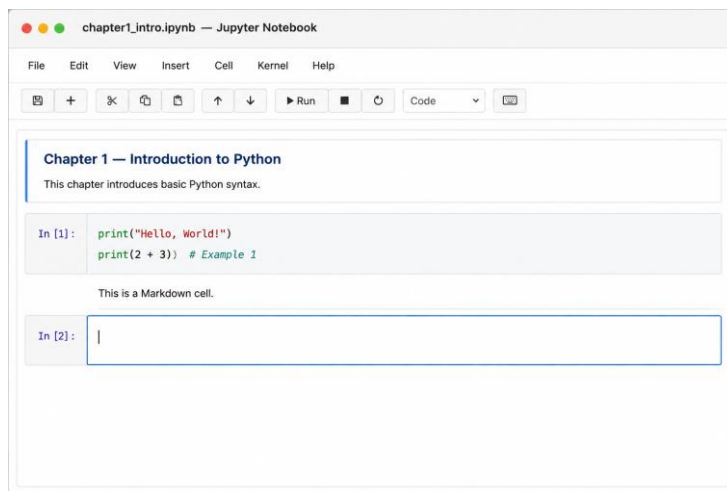


Figure 1.7: Jupyter Notebook interface layout (illustrative recreation; insert an official screenshot for final publication)

A notebook document is composed of an ordered sequence of cells. Code cells contain executable Python statements, while Markdown cells contain formatted narrative text, headings, and mathematical notation, allowing a notebook to function simultaneously as a program and as a report describing that program's purpose and findings. Each notebook is connected to a kernel, a background process that executes code and retains variable state between cell executions, so that a variable defined in one cell remains accessible to later cells. Because a cell's output, including inline charts produced by libraries such as Matplotlib, remains visible until the cell is re-run, notebooks are especially well suited to exploratory data analysis, where results need to be inspected before deciding on the next analytical step.

Note

The order in which cells are executed, not their order on the page, determines a notebook's runtime state. Restarting the kernel and running all cells from top to bottom periodically is good practice for confirming reproducibility.

1.6 Google Colab

Google Collaboratory ("Colab") is a free, cloud-hosted notebook environment, built on the Jupyter format, that requires no local installation. Notebooks run entirely on Google's servers and are stored in Google Drive, and Colab additionally offers free, time-limited access to GPUs and TPUs for accelerating machine learning workloads. Figure 1.8 shows the Colab interface, including the Connect button that indicates the active cloud runtime.

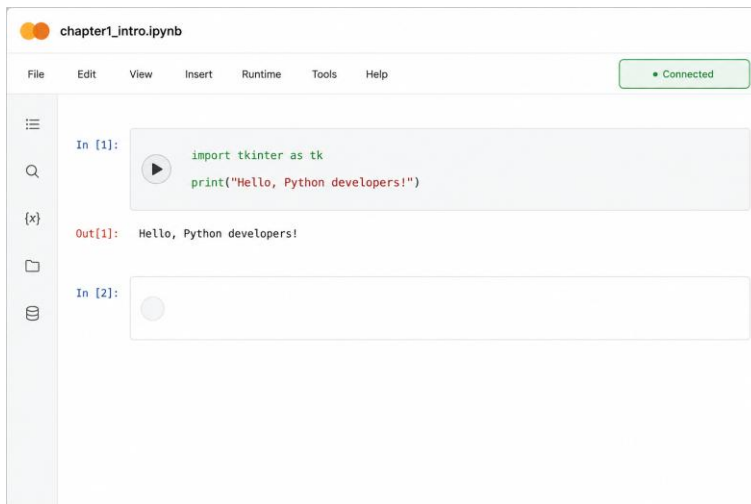


Figure 1.8: Google Colab interface layout (illustrative recreation; insert an official screenshot for final publication)

Because Colab requires no local setup, students and researchers can begin working immediately from any device with a browser, and popular machine learning libraries such as TensorFlow and PyTorch are already installed in every runtime. Colab notebooks can also be shared and edited collaboratively in real time, in a manner similar to Google Docs. The free tier's session timeouts, which disconnect idle runtimes and can interrupt long training jobs, and its capped compute allocation are its main limitations; paid tiers such as Colab Pro raise these limits. Table 2 compares Colab with a locally installed Jupyter Notebook across the factors most relevant to choosing between them.

Aspect	Jupyter Notebook	Google Colab
Installation	Requires local install (e.g., Anaconda)	None — runs in the browser
Hardware	Limited to the local machine	Free cloud GPU/TPU access
Storage	Local file system	Google Drive
Collaboration	Manual file sharing	Real-time collaborative editing
Offline Use	Fully supported	Requires an internet connection

Table 2: Jupyter Notebook vs. Google Colab

1.7 Python IDEs

An Integrated Development Environment (IDE) is a software application that consolidates the tools a programmer needs, including a code editor, debugger, and execution tools, into a single interface. While Python code can be written in any plain text editor, IDEs substantially improve productivity through features such as syntax highlighting, code completion, and integrated debugging.

Visual Studio Code (VS Code), developed by Microsoft, is a free, lightweight, and highly extensible source code editor. Through its Python and Jupyter extensions, it gains IntelliSense code completion, an integrated debugger, linting, and native notebook support, making it suitable for both general scripting and data science work in one tool, as shown in Figure 1.9.

PyCharm, developed by JetBrains, is an IDE purpose-built for Python; its free Community Edition and paid Professional Edition are known for robust code-refactoring tools and deep static analysis that catches errors before code is even executed.

Spyder, bundled with Anaconda, is designed specifically for data science and engineering workflows, and its variable explorer, which displays the current value of every variable in memory alongside the code editor, closely resembles the layout of MATLAB. Python also

ships with IDLE, a minimal built-in IDE suitable for quick scripts and teaching, and JupyterLab, a more feature-rich evolution of the Jupyter Notebook interface.

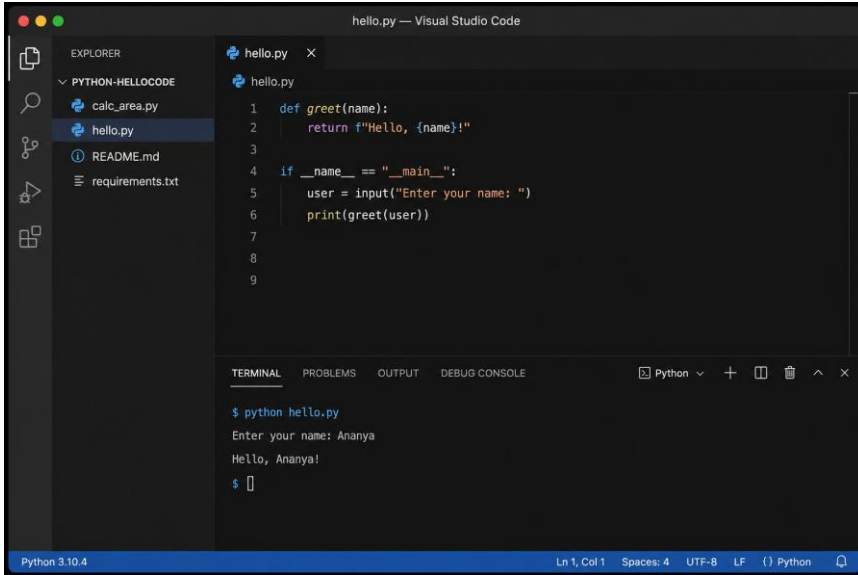


Figure 1.9: Visual Studio Code interface layout (illustrative recreation; insert an official screenshot for final publication)

Table 3 summarizes the practical differences between these three environments.

Aspect	VS Code	PyCharm	Spyder
Developer	Microsoft	JetBrains	Spyder Project
Cost	Free	Free / Paid Tiers	Free
Best Suited For	General-purpose, extensible workflows	Large Python and web applications	Data analysis with a MATLAB-style layout

Table 3: VS Code vs. PyCharm vs. Spyder

Best Practice

Beginners are usually well served by VS Code or JupyterLab: both are free, widely used in industry, and support the interactive, cell-based style of coding standard in data science education.

1.8 Running Python Programs

Python code can be executed in three principal ways. Typing "python" with no arguments launches the interactive interpreter, or Read-Eval-Print Loop (REPL), in which each statement is read, evaluated, and its result printed immediately; this makes

the REPL ideal for quickly testing small snippets of code or exploring how an unfamiliar function behaves.

Saving code in a standalone script file with a .py extension and running it with "python filename.py" allows a program to be reused, version-controlled, and shared, rather than retyped each time. Finally, code can be executed cell by cell inside a Jupyter or Colab notebook, as described in Sections 1.5 and 1.6.

Regardless of the entry point, Python source code is first compiled into an intermediate, platform-independent representation called bytecode, which is then executed by the Python Virtual Machine (PVM), as illustrated in Figure 1.10.

This compilation step is automatic and largely invisible to the programmer; it is why Python is sometimes described as both interpreted and compiled, since the source is compiled to bytecode and that bytecode is, in turn, interpreted. Compiled bytecode is often cached in .pyc files inside a __pycache__ directory so that unchanged modules are not recompiled on subsequent runs.

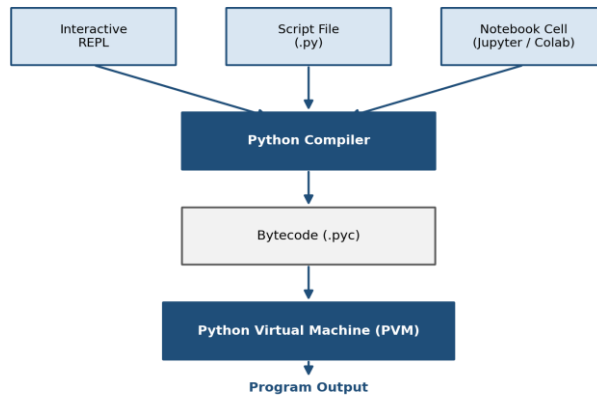


Figure 1.10: *How Python source code is executed, from source through bytecode to the Python Virtual Machine*

Problem Statement: Write and run a script that greets the user by name.

```
# hello.py

def greet(name):
    return f"Hello, {name}! Welcome to Python."

if __name__ == "__main__":
    print(greet(input("Enter your name: ")))
```

Code Explanation: The `if __name__ == "__main__":` guard ensures the greeting code runs only when the file is executed directly, not when it is imported elsewhere as a module; this is a standard Python idiom that will reappear when this textbook introduces modules in a later chapter. The built-in `input()` function pauses execution and waits for the user to type a response, which is then passed into the `greet()` function. **Practical Use Case:** Saving code as a .py file, rather than typing it directly into the REPL, allows it to be reused, version-controlled, and shared, which is why almost all real Python projects are organized as script or module files rather than long interactive sessions.

1.9 Python Coding Standards

As programs grow beyond a few lines, consistent formatting and naming become essential for readability, collaboration, and long-term maintainability. PEP 8, the official Python style guide first published in 2001, sets out conventions for indentation, line length, whitespace, and naming that keep code readable across the large, collaborative Python ecosystem, where developers routinely read and modify code written by others. Figure 1.11 summarizes its main rule categories.

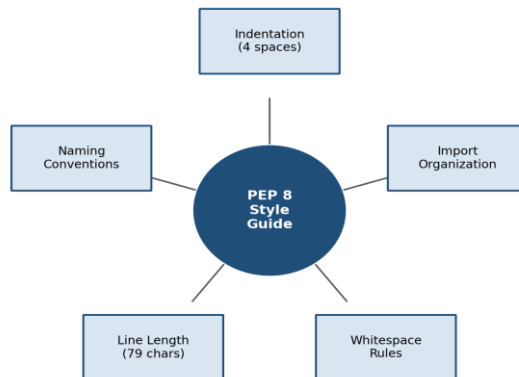


Figure 1.11: Main rule categories defined by PEP 8

- Indentation: four spaces per level; never mix tabs and spaces within the same file.
- Naming: snake_case for variables and functions, PascalCase for classes, and UPPER_SNAKE_CASE for constants.
- Line length: no more than 79 characters for code and 72 for comments and docstrings.

- Whitespace: avoid extra spaces immediately inside parentheses or brackets and before a comma, semicolon, or colon.

Imports: one module per line, grouped as standard library, third-party, then local application imports.

```
# Not recommended

def calc(a,b):

    return a+b

# PEP 8 compliant

def calculate_sum(first_number, second_number):

    """Return the sum of two numbers."""

    return first_number + second_number
```

Code Explanation: The compliant version uses descriptive names, standard indentation, and a docstring, a string literal placed as the first statement of a function to document its purpose, as defined by PEP 257. Docstrings are accessible at runtime via the built-in `help()` function and are used by automated documentation-generation tools, unlike ordinary comments, which exist only in the source file. Practical Use Case: Consistent style, checked automatically by linters such as `flake8` or formatters such as `black`, is expected in team and open-source Python projects, and code that violates it is routinely flagged during code review before it can be merged.

1.10 Summary

This chapter introduced Python as a high-level, interpreted, dynamically typed language whose readable syntax and mature library ecosystem have made it the dominant choice for data science, machine learning, and artificial intelligence. We traced Python's origins at CWI in the late 1980s through the Python 2 to Python 3 transition, and examined the language's core features, applications, and limitations, including its comparative trade-offs against C++ and Java in typing, execution model, and learning curve.

We then turned to practical setup: installing standard Python on Windows, macOS, and Linux; the Anaconda distribution and its `conda` package and environment manager, purpose-built for data science workflows; and the two leading interactive computing environments, Jupyter Notebook and Google Colab, comparing their respective strengths for local versus cloud-based development. We surveyed the leading Python IDEs, VS Code, PyCharm, and Spyder, examined the three ways Python programs are executed, and traced the path from source code through bytecode to the Python Virtual Machine.

The chapter closed with the PEP 8 and PEP 257 standards that define professional Python coding practice. These foundations prepare the reader for Chapter 2, which introduces variables, data types, and operators.

1.10.1 Key Takeaways

- Python is a high-level, interpreted, dynamically typed, multi-paradigm language, now in its Python 3 generation, .
- Its dominance in data science and AI stems from a unified ecosystem — NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch — and strong community support.
- Anaconda bundles the interpreter, scientific libraries, and the conda environment manager for data science workflows.
- Jupyter Notebook and Google Colab both support cell-based execution; Colab adds free cloud GPU/TPU access with no local installation.
- VS Code, PyCharm, and Spyder are the three most widely used Python IDEs, each suited to a different workflow.
- Python source code is compiled to bytecode and executed by the Python Virtual Machine, regardless of whether it is run via the REPL, a script, or a notebook cell.
- PEP 8 and PEP 257, supported by automated linters, define the style and documentation conventions expected of professional Python code.

1.10.2 Important Terms

- Interpreter — software that reads and executes source code directly, without a separate compilation step producing a standalone executable.
- PyPI — the official online repository from which third-party Python packages are installed.
- Kernel — the background process that executes code within a Jupyter notebook and retains its runtime state.
- REPL — Read-Eval-Print Loop, the interactive mode in which statements are read, evaluated, and printed immediately.
- Bytecode — the intermediate, platform-independent form of Python source code executed by the Python Virtual Machine.

- Virtual environment — an isolated set of installed packages used to prevent dependency conflicts between projects.
- Docstring — a string literal documenting a module, function, class, or method, per PEP 257.
- Linter — a tool that flags style violations and likely errors in source code without executing it.

1.10.3 Glossary

- CPython — the reference implementation of Python, written in C, and the interpreter most people mean when they simply say "Python" .
- GIL (Global Interpreter Lock) — a mutex in CPython that allows only one thread to execute Python bytecode at a time.
- pip — the standard tool for installing Python packages from PyPI .
- conda — the package and environment manager bundled with Anaconda, capable of managing both Python and non-Python dependencies.

1.11 Exercises

1.11.1 Multiple Choice Questions

1. Who created the Python programming language? (a) Dennis Ritchie (b) Guido van Rossum (c) James Gosling (d) Bjarne Stroustrup
2. In which year was Python first released? (a) 1989 (b) 1991 (c) 1995 (d) 2000
3. Which typing model does Python use? (a) Statically typed (b) Dynamically typed (c) Untyped (d) Weakly compiled
4. Which distribution bundles Python with data science libraries and the conda manager? (a) WinPython (b) Anaconda (c) ActivePython (d) PyInstaller
5. Which file extension is used to save a Jupyter Notebook? (a) .py (b) .json (c) ipynb (d) note
6. Which environment offers free cloud GPU access with no local installation? (a) IDLE (b) Spyder (c) Google Colab (d) PyCharm Community
7. Which PEP defines Python's official style guide? (a) PEP 20 (b) PEP 8 (c) PEP 257 (d) PEP 484

8. What is the standard indentation width recommended by PEP 8? (a) 2 spaces (b) 3 spaces (c) 4 spaces (d) 8 spaces
9. Which of the following is the background process that executes code in a Jupyter notebook? (a) Compiler (b) Kernel (c) Interpreter shell (d) Runtime cache
10. Which of these is a scientific Python IDE bundled with Anaconda? (a) PyCharm (b) Spyder (c) Eclipse (d) NetBeans

1.11.2 Fill in the Blanks

11. Python code is compiled into an intermediate form called _____ before execution by the Python Virtual Machine.
12. A _____ is a string literal placed as the first statement of a function to document its purpose.
13. _____ is the graphical desktop application bundled with Anaconda.
14. In CPython, the _____ prevents multiple native threads from executing bytecode at exactly the same time.
15. A Jupyter notebook's background computational engine is called the _____.

1.11.3 True or False

16. Python is compiled directly to native machine code in the same way as C++.
17. PEP 8 recommends snake_case for variable and function names.
18. Google Colab requires Python to be installed locally before use.
19. pip and conda can both install Python packages, but they manage dependencies differently.
20. Jupyter notebook cells must always be executed strictly in top-to-bottom order.

1.11.4 Short Answer Questions

21. List any four key features that make Python beginner-friendly.
22. Explain, in two or three sentences, the difference between Jupyter Notebook and Google Colab.
23. What is the purpose of a virtual environment in a data science project?

1. 24. Name three companies or organizations that use Python in production and briefly describe how each uses it.

1.11.5 Long Answer Questions

25. With reference to real-world examples, discuss why Python has become the dominant language for data science and AI, addressing both technical advantages and its library ecosystem.
26. Describe the journey of a Python program from source code to a printed result, mentioning the interpreter, bytecode, and the Python Virtual Machine.
27. Compare VS Code, PyCharm, and Spyder as Python development environments, describing the type of user and project each is best suited for.

1.11.6 Practical Programming Questions

28. Verify your Python installation, then write and run a script that prints your name, college, and today's date.
29. Rewrite the following so that it fully complies with PEP 8: `"def add(x,y):\nreturn x+y"`.

1.11.7 Analytical Question

30. Python is often criticized for being slower than compiled languages such as C++, yet it dominates computationally intensive machine learning work. Explain why this is not actually a contradiction in practice.

Chapter 2: Python Basics: Variables, Data Types, and Operators

2.1 Introduction to Python Basics

Chapter 1 introduced Python as a language and set up the tools needed to write and run programs. This chapter turns to the language itself, covering the building blocks every Python program is made of: variables, the built-in data types, type conversion, operators, and the basic input and output functions. These foundations are used in every subsequent chapter, from control structures in Chapter 3 through the NumPy and Pandas libraries in later chapters, so a solid grasp of them is essential before moving on.

Unlike statically typed languages, where a compiler checks and enforces types before a program ever runs, Python resolves and checks types while the program executes. This makes Python programs quicker to write and easier to modify, but it also means that mistakes involving types, such as attempting to add a number to a piece of text, surface only when the offending line actually runs. Understanding exactly how Python represents values in memory and evaluates expressions is therefore the first practical skill every Python programmer needs, and it is the subject of this chapter.

The chapter proceeds from the ground up: Section 2.2 explains what a variable actually is in Python and how names are chosen; Sections 2.3 and 2.4 cover the built-in data types and how values move between them; Sections 2.5 and 2.6 cover operators and the order in which they combine; and Sections 2.7 through 2.9 cover interacting with a user, documenting code, and the mutable/immutable distinction that shapes how objects behave when shared between parts of a program.

2.2 Variables and Identifiers

A variable in Python is a name that refers to a value stored in memory. Unlike variables in languages such as C++ or Java, a Python variable does not need to be declared with a fixed type in advance; assigning a value to a name is sufficient to create it. The assignment statement `"score = 95"` does not copy the number 95 into a labeled memory box in the traditional sense; instead, Python creates an integer object holding the value 95 and makes the name `"score"` refer, or bind, to that object, as shown in Figure 2.1.

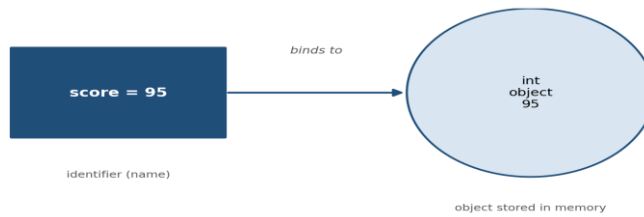


Figure 2.1: A Python variable as a name bound to an object in memory

Because a name is only a reference, reassigning it, for example with `"score = 100"`, does not modify the original object; it simply makes `"score"` refer to a new integer object, leaving the old one to be reclaimed by Python's garbage collector if nothing else refers to it. Python also allows multiple assignment in a single statement, such as `"a, b, c = 1, 2, 3"`, and chained assignment, such as `"x = y = 0"`, both of which are common in idiomatic Python code.

```
a, b, c = 1, 2, 3

x = y = 0

print(a, b, c, x, y)

# Output: 1 2 3 0 0
```

This reference-based model has a practical consequence that surprises many newcomers: two variables can be made to refer to the very same object. After `"list_a = [1, 2, 3]"` and `"list_b = list_a"`, both names refer to one shared list, so modifying it through either name is visible through the other. Section 2.9 revisits this behavior in depth, since it depends on whether the underlying object is mutable or immutable.

2.2.1 Naming Rules and Conventions

An identifier is the name given to a variable, function, class, or other object. Python enforces a small set of rules for valid identifiers, illustrated in Figure 2.2: an identifier must start with a letter (a–z, A–Z) or an underscore, may be followed by any combination of letters, digits, and underscores, and must not be one of Python's 35 reserved keywords, such as `"if"`, `"class"`, or `"return"`. Identifiers are also case-sensitive, so `"total"` and `"Total"` name two different variables.

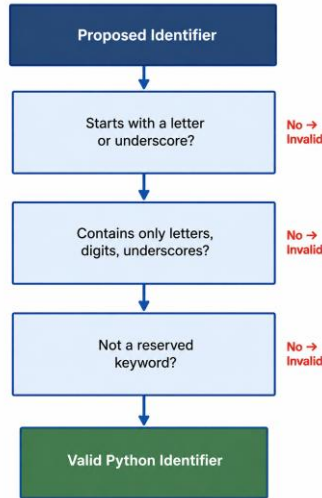


Figure 2.2: Decision flow for a valid Python identifier

Beyond these hard rules, PEP 8 recommends snake_case for variable and function names, PascalCase for class names, and descriptive names that communicate purpose, such as "student_count" rather than "sc". Python's reserved keywords, including "if", "else", "for", "while", "def", "class", "return", "import", "True", "False", and "None" among others, are unavailable as identifier names because the interpreter relies on them to parse the structure of a program; the full list can be inspected at any time by importing the keyword module and printing keyword.kwlist.

Best Practice

Choose identifier names that describe what a value represents, not how it will be used. "unit_price" ages better than "temp" or "x" as a program grows.

2.3 Data Types in Python

Every value in Python has a type, which determines what operations can be performed on it and how it behaves. Python is dynamically typed, so the interpreter infers a variable's type from the value assigned to it, and the built-in type() function reports that type at any point in a program. Figure 2.3 summarizes the built-in data types most relevant to this chapter.

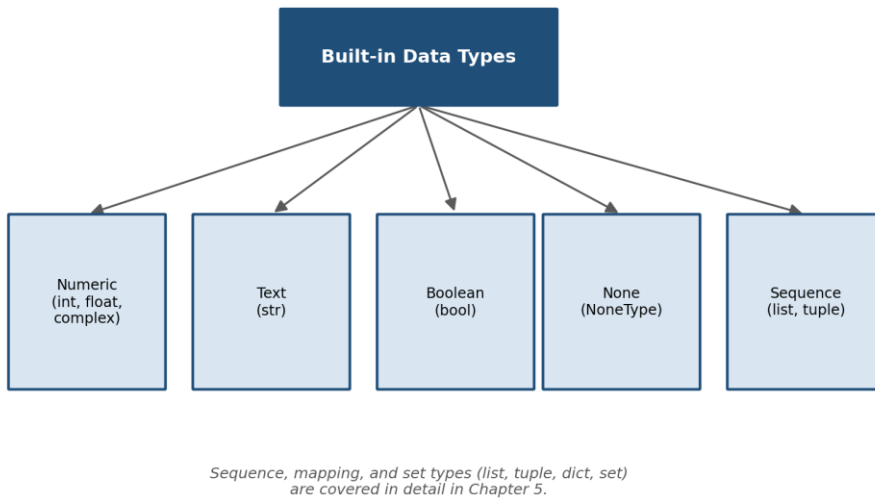


Figure 2.3: Core built-in data types in Python

2.3.1 Numeric Types

Python provides three built-in numeric types: `int` for whole numbers of arbitrary size, `float` for real numbers represented in double precision, and `complex` for numbers with a real and imaginary part, written as `"3 + 4j"`. Unlike many languages, Python's `int` type has no fixed maximum size; it expands automatically to hold arbitrarily large integers, limited only by available memory. `float` values, by contrast, are stored in a fixed 64-bit representation and are therefore subject to the same rounding limitations as floating-point numbers in any language, so comparing floats for exact equality after arithmetic is generally unsafe.

```
big = 2 ** 100

print(big) # exact integer, no overflow

print(0.1 + 0.2) # 0.30000000000000004

print(type(big), type(0.1 + 0.2))
```

2.3.2 Text Type: `str`

The `str` type represents a sequence of Unicode characters, written between single, double, or triple quotes. Triple-quoted strings, delimited by `"\""` or `"""`, can span multiple lines and are commonly used for docstrings, as discussed in Section 2.8. Strings in Python are immutable, a property discussed further in Section 2.9. Special characters that cannot

be typed directly, such as a newline or a literal quotation mark, are written using escape sequences, summarized in Table 1.

Sequence	Meaning
<code>\n</code>	Newline
<code>\t</code>	Horizontal tab
<code>\\</code>	Backslash
<code>\'</code>	Single quote
<code>\"</code>	Double quote

Table 1: Common String Escape Sequences

2.3.3 Boolean and None Types

The `bool` type has exactly two values, `True` and `False`, and is technically a subtype of `int`, so `True` behaves as 1 and `False` as 0 in arithmetic contexts. The special value `None`, of type `NoneType`, represents the absence of a value and is commonly used as a default or placeholder before a variable is assigned meaningful data.

```
is_valid = True

total_score = None

print(is_valid + 1) # 2, since True == 1

print(total_score is None) # True
```

Did You Know?

Booleans being a subtype of `int` is why "`True + True`" evaluates to 2 in Python, and why `True` can be used directly as an index or in arithmetic expressions.

2.3.4 Checking and Comparing Types

The built-in `type()` function returns the exact type of a value, while the built-in `isinstance()` function tests whether a value is an instance of a given type, or one of several types passed as a tuple, and is generally preferred in conditional checks because it also accounts for inheritance relationships between classes, a topic covered in Chapter 4.

```
value = 42

print(type(value)) # <class 'int'>

print(isinstance(value, int)) # True

print(isinstance(value, (int, float))) # True
```


2.4 Type Conversion

Type conversion changes a value from one data type to another. Python performs implicit conversion automatically when an operation combines two compatible types, most commonly promoting an int to a float in mixed arithmetic so no precision is lost. Explicit conversion, also called type casting, is performed deliberately by the programmer using built-in functions such as `int()`, `float()`, and `str()`, and is required whenever Python cannot safely infer the intended type on its own, as illustrated in Figure 2.4.

Explicit conversion functions are not infallible: `int("3.5")` raises a `ValueError` because the string is not a whole number, while `int(3.5)` succeeds by truncating the decimal portion to 3. Anticipating these failure cases, particularly when converting text supplied by a user, is an important defensive-programming habit developed further with exception handling in Chapter 3.

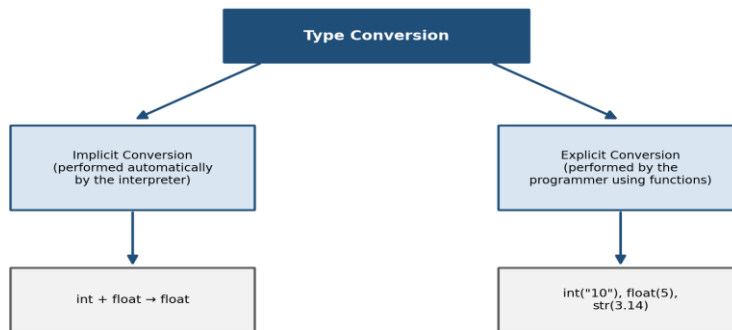


Figure 2.4: Implicit versus explicit type conversion in Python

Function	Converts To	Example
<code>int()</code>	Integer	<code>int("42") → 42</code>
<code>float()</code>	Floating-point	<code>float("3.14") → 3.14</code>
<code>str()</code>	String	<code>str(42) → "42"</code>
<code>bool()</code>	Boolean	<code>bool(0) → False</code>

Table 2: Common Explicit Conversion Functions

Note that `bool()` follows the truthiness rules discussed in Section 2.5.1: converting any nonzero number or any non-empty string to `bool()` yields `True`, while converting `0`, `0.0`, or an empty string yields `False`, which is why explicit conversion to `bool` is often used to test whether a variable holds meaningful data before proceeding.

Problem Statement: Read two numbers as text input and compute their sum as a number rather than concatenating them as strings.

```
a = input("Enter first number: ") # a is a str
b = input("Enter second number: ") # b is a str
total = int(a) + int(b) # explicit conversion
print("Sum:", total)
```

Code Explanation: `input()` always returns a `str`, so `"a + b"` would concatenate the two strings rather than add them numerically; wrapping each value in `int()` explicitly converts it before the addition. Practical Use Case: This pattern, reading text and converting it to a number, is the standard way user-supplied input is validated and processed in Python programs.

2.5 Operators in Python

An operator performs an action on one or more values, called operands. Python groups its operators into several categories, summarized in Figure 2.5: arithmetic, relational (comparison), logical, assignment, bitwise, and identity/membership operators.

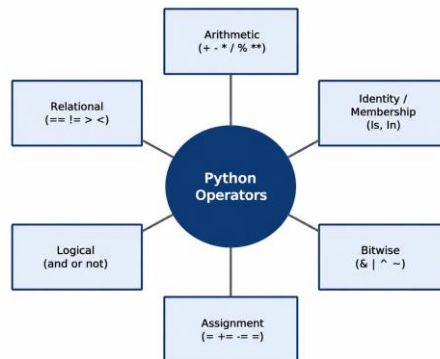


Figure 2.5: Categories of operators in Python

2.5.1 Arithmetic, Relational, and Logical Operators

Arithmetic operators (`+`, `-`, `*`, `/`, `%`, `//`, `**`) perform numeric computation; note that `/` always returns a float, while `//` performs floor (integer) division. Relational operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) compare two values and return a `bool`. Logical operators (`and`, `or`, `not`) combine or invert Boolean expressions and are commonly used to build compound conditions, a technique developed further in Chapter 3's coverage of control structures.

Python's 'and' and 'or' operators are short-circuiting: in "a and b", b is evaluated only if a is truthy, and in "a or b", b is evaluated only if a is false. This behavior is more than an optimization; it is routinely used to guard an expression that would otherwise raise an error, such as checking that a list is non-empty before indexing into it within the same condition.

Short-circuit evaluation depends on Python's notion of truthiness: every object, not just bool values, can be evaluated in a Boolean context. Figure 2.6 lists the specific values Python treats as false; every other value, including any non-zero number and any non-empty string, list, or dictionary, is treated as truthy.

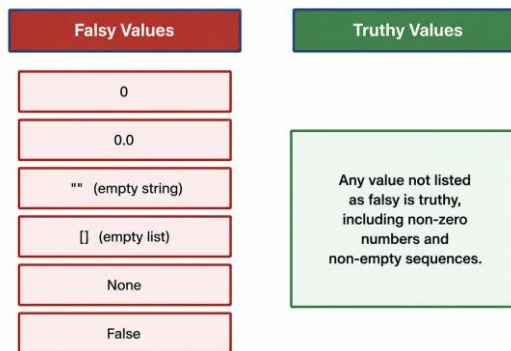


Figure 2.6: Values Python treats as false in a Boolean context

Operator	Meaning	Example	Result
+	Addition	7 + 3	10
-	Subtraction	7 - 3	4
*	Multiplication	7 * 3	21
/	True division	7 / 3	2.333...
//	Floor division	7 // 3	2
%	Modulus (remainder)	7 % 3	1
**	Exponentiation	7 ** 3	343

Table 3: Arithmetic Operators

```
age = 20
has_id = True
can_vote = age >= 18 and has_id
print(can_vote) # True
```

2.5.2 Assignment, Bitwise, Identity, and Membership Operators

The assignment operator (=) binds a name to a value, and Python provides augmented assignment shorthand, summarized in Table 3, that combines an operation with assignment in one step, for example "count += 1" instead of "count = count + 1".

Operator	Equivalent To	Example
+=	x = x + y	count += 1
-=	x = x - y	balance -= 50
*=	x = x * y	price *= 1.1
/=	x = x / y	total /= 2
//=	x = x // y	items //= 3
**=	x = x ** y	value **= 2

Table 4: Common Augmented Assignment Operators

Bitwise operators (&, |, ^, ~, <<, >>) act directly on the binary representation of integers and are used in low-level tasks such as flag manipulation, network protocol handling, and performance-sensitive numeric code; the left-shift (<<) and right-shift (>>) operators, for instance, multiply or divide an integer by a power of two respectively. Identity operators (is, is not) test whether two names refer to the exact same object in memory, which is a different question from equality; membership operators (in, not in) test whether a value appears within a sequence such as a string or list.

```
flags = 0b0101 # binary literal: 5
flags = flags | 0b0010 # set a bit: 0111 (7)
flags = flags & ~0b0001 # clear a bit: 0110 (6)
print(bin(flags)) # 0b110
```

```
count = 0
count += 5 # augmented assignment
print(count) # 5
print(3 in [1, 2, 3]) # membership: True
print(count is None) # identity: False
```

Note

Use == to compare values and is only to check whether two names refer to the same object, most commonly when comparing against None, as in "if result is None:".

2.6 Operator Precedence and Associativity

When an expression contains more than one operator, Python evaluates it according to a fixed precedence order, shown from highest to lowest in Figure 2.7, so that, for example, multiplication is performed before addition unless parentheses dictate otherwise. Operators of equal precedence are evaluated according to their associativity, left to right for most operators, but right to left for exponentiation (**).

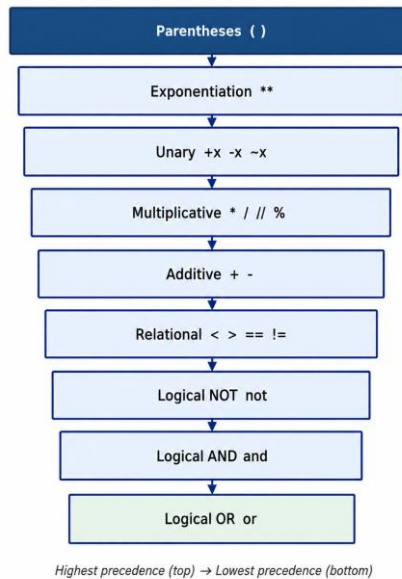


Figure 2.7: Python operator precedence, from highest to lowest

Problem Statement: Evaluate a mixed arithmetic and relational expression and confirm Python's evaluation order.

```

result = 2 + 3 * 4 ** 2 > 40 and not False

print(result)

# ** first: 4**2 = 16

# * next: 3*16 = 48

# + next: 2+48 = 50

# > next: 50 > 40 -> True

# not next: not False -> True

# and last: True and True -> True

```

Code Explanation: The interpreter applies exponentiation first, then multiplication, then addition, then the relational comparison, then the logical operators, exactly following the precedence order in Figure 2.7. Practical Use Case: Relying on memorized precedence for complex expressions is error-prone in practice; adding explicit parentheses, even where not strictly required, is standard practice for readability in professional code.

Parentheses always override the default precedence order and are evaluated first, working from the innermost pair outward. Rewriting the previous expression as `"2 + (3 * (4 ** 2))"` produces the identical result, 50, but makes the intended grouping explicit to any reader, which is precisely why experienced Python programmers favor extra parentheses in anything beyond the simplest expressions.

2.7 Input and Output in Python

Nearly every program needs to accept input and produce output. Python's built-in `input()` function reads a line of text typed by the user and always returns it as a `str`, while the built-in `print()` function writes text to the console, as summarized in Figure 2.8.

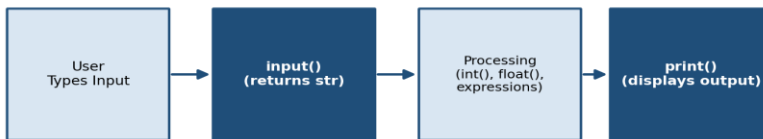


Figure 2.8: The flow of data from user input through processing to console output

`print()` accepts multiple values separated by commas and, by default, joins them with a single space, while formatted string literals, or f-strings, introduced by prefixing a string with `"f"`, allow variables and expressions to be embedded directly inside curly braces for clear, readable output. The `sep` and `end` keyword arguments customize, respectively, the separator placed between multiple values and the character printed after the final value, which defaults to a newline.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"{name} will turn {age + 1} next year.")
print("a", "b", "c", sep="-", end="!\n") # a-b-c!
```

2.8 Comments and Docstrings

A comment is text in a program that the interpreter ignores, used to explain code to human readers. A single-line comment begins with the hash symbol (#) and extends to the end of the line; there is no dedicated multi-line comment syntax, so consecutive lines are typically each prefixed with #. A docstring, introduced in Chapter 1, is a string literal, usually triple-quoted, placed as the first statement of a module, function, or class to document its purpose, and is distinct from an ordinary comment because it remains accessible at runtime through `help()`.

```
def area_of_circle(radius):  
    """Return the area of a circle with the given  
    radius."""  
  
    pi = 3.14159 # approximation of pi  
  
    return pi * radius ** 2
```

For longer explanations spanning several lines, convention prefixes each line with its own # rather than relying on a triple-quoted string, since triple-quoted strings that are not the first statement of a function or module are ordinary (and easily misplaced) string objects rather than recognized documentation. Well-commented code also matters beyond individual readability: automated documentation generators such as Sphinx parse docstrings directly to build a project's reference manual, so consistent, well-formed docstrings have value far beyond the immediate file they appear in.

Industry Insight

Professional Python codebases typically enforce documentation coverage with tools that fail a build if public functions are missing docstrings, reflecting how central clear documentation is to maintaining large, collaboratively developed software.

Best Practice

Write comments that explain why code does something, not what it does; the code itself already shows what is happening, and restating it in a comment adds maintenance burden without adding understanding.

2.9 Mutable and Immutable Objects

Every Python object is either mutable, meaning its contents can be changed after creation, or immutable, meaning they cannot. Numbers, strings, booleans, and tuples are immutable: an operation that appears to modify one, such as concatenating a string, actually creates and returns a new object. Lists, dictionaries, and sets are mutable:

methods such as `list.append()` change the object in place without creating a new one. Figure 2.9 summarizes this distinction, which becomes especially important in Chapter 3 and Chapter 5 when functions and data structures pass these objects around a program.

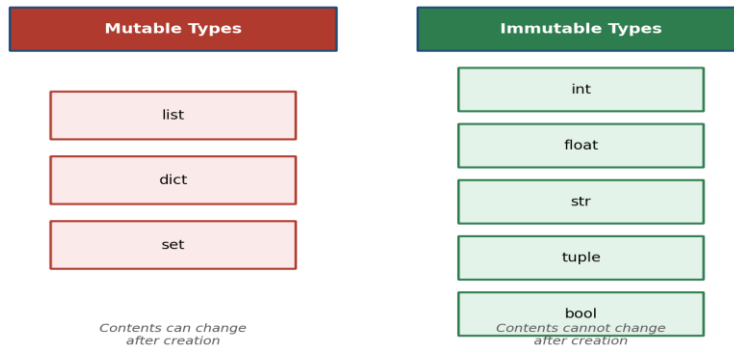


Figure 2.9: Mutable versus immutable built-in types

```
s = "hello"

s = s + " world" # creates a new str object

lst = [1, 2, 3]

lst.append(4) # modifies the same list object in place

print(s, lst) # hello world [1, 2, 3, 4]
```

Interview Question

Q: Why can strings be used as dictionary keys but lists cannot? A strong answer notes that dictionary keys must be hashable, and Python only guarantees a stable hash value for immutable objects; because a list's contents, and therefore its hash, could change after insertion, lists are excluded from being valid keys.

Mutability also explains a common source of bugs: because assigning one list variable to another copies only the reference, not the contents, both names share the same underlying object. Passing a mutable object into a function, a pattern examined closely in Chapter 3, carries this same behavior, so a function that modifies a list argument in place changes it for every name that refers to that list.

When an independent copy is genuinely needed, Python's `copy` module provides `copy()` for a shallow copy, which duplicates the outer container but still shares any nested mutable objects, and `deepcopy()` for a fully independent copy that recursively duplicates every nested object as well. For the simple, single-level lists and dictionaries introduced

in this chapter, a shallow copy is normally sufficient; the distinction becomes more important once nested data structures are introduced in Chapter 5.

2.10 Summary

This chapter covered the essential building blocks of Python programs. Variables were introduced as names bound to objects rather than fixed memory boxes, along with the rules and conventions for valid, readable identifiers. We examined Python's core built-in data types, numeric, text, Boolean, and None, and the difference between implicit and explicit type conversion, along with the `type()` and `isinstance()` functions used to inspect and verify them at runtime. We surveyed Python's operator categories, including arithmetic, relational, logical, assignment, bitwise, and identity/membership operators, and the precedence and associativity rules that determine how compound expressions are evaluated. Finally, we covered the `input()` and `print()` functions for basic interaction, comments and docstrings for documenting code, and the distinction between mutable and immutable objects, a concept that recurs throughout the rest of this textbook.

Together, these topics form the vocabulary used in every chapter that follows. Chapter 3 builds directly on them, using conditional expressions and the relational and logical operators introduced here to control which statements in a program execute, and using function definitions to organize and reuse code.

2.10.1 Key Takeaways

- A variable is a name bound to an object in memory, not a fixed, typed storage location.
- Valid identifiers start with a letter or underscore, contain only letters, digits, and underscores, and avoid reserved keywords; PEP 8 favors `snake_case` for variables and functions.
- Python's core built-in types are `int`, `float`, `complex`, `str`, `bool`, and `NoneType`.
- Implicit conversion happens automatically in mixed-type expressions; explicit conversion uses functions such as `int()`, `float()`, and `str()`.
- Operators are grouped into arithmetic, relational, logical, assignment, bitwise, and identity/membership categories, evaluated according to a fixed precedence order.
- `input()` always returns a `str` and must be explicitly converted for numeric use; `print()` and f-strings produce formatted console output.

- Objects are either mutable (list, dict, set) or immutable (int, float, str, tuple, bool); only immutable objects can serve as dictionary keys.

2.10.2 Important Terms

- Identifier — the name given to a variable, function, class, or other Python object.
- Type conversion (casting) — converting a value from one data type to another, either implicitly or explicitly.
- Operand — a value on which an operator acts.
- Precedence — the order in which operators are evaluated within a compound expression.
- Associativity — the direction (left-to-right or right-to-left) in which operators of equal precedence are evaluated.
- f-string — a formatted string literal that embeds variables and expressions directly within curly braces.
- Mutable object — an object whose contents can be changed in place after creation.
- Immutable object — an object whose contents cannot be changed after creation; any apparent modification creates a new object.
- Truthy / falsy — the classification of any value according to how it behaves in a Boolean context, independent of its actual type.
- Short-circuit evaluation — the behavior by which and and or skip evaluating their second operand once the result is already determined.

2.11 Exercises

2.11.1 Multiple Choice Questions

1. Which of the following is a valid Python identifier? (a) 2total (b) total_2 (c) total-2 (d) class
2. What is the data type of the result of $7 / 2$ in Python 3? (a) int (b) float (c) complex (d) str
3. Which function converts a string "25" into an integer? (a) str("25") (b) int("25") (c) float("25") (d) bool("25")

4. Which operator checks whether two names refer to the same object? (a) == (b) = (c) is (d) in
5. In the expression $2 + 3 * 4$, which operation is performed first? (a) Addition (b) Multiplication (c) Left to right (d) Right to left
6. What does the `input()` function always return? (a) int (b) float (c) str (d) bool
7. Which of the following types is immutable? (a) list (b) dict (c) tuple (d) set
8. What is the result of `True + True` in Python? (a) True (b) 2 (c) Error (d) False

2.11.2 Fill in the Blanks

9. An identifier must start with a letter or an _____.
10. The _____ operator (`//`) performs division and discards the remainder.
11. A(n) _____ string literal, written with an `f` prefix, embeds expressions inside curly braces.
12. Converting a value from one data type to another using a function such as `int()` is called _____ conversion.
13. Objects of type _____ cannot be used as dictionary keys because their contents, and therefore their hash, can change.

2.11.3 True or False

14. Python variables must be declared with a fixed type before they are assigned a value.
15. Strings in Python are mutable.
16. The exponentiation operator (`**`) is right-associative.
17. `==` and `is` always produce the same result in Python.
18. A docstring is accessible at runtime through the built-in `help()` function.

2.11.4 Short Answer Questions

19. Explain the difference between implicit and explicit type conversion, with one example of each.
20. What is the difference between a mutable and an immutable object? Give two examples of each.

21. Why does `print("3" + "4")` not produce 7?

22. Explain short-circuit evaluation in Python's and and or operators, with an example of why it is useful.

2.11.5 Long Answer Questions

23. Describe Python's operator precedence order from highest to lowest, and explain, with an example expression, why parentheses are still considered good practice even when not strictly required.

24. Explain how Python variables relate to objects in memory, and describe what happens when a variable is reassigned to a new value.

2.11.6 Practical Programming Questions

25. Write a program that reads a temperature in Celsius from the user and prints the equivalent temperature in Fahrenheit, formatted using an f-string.

26. Write a program that demonstrates the difference between a mutable and an immutable object by attempting to modify a tuple and a list of the same length.

2.11.7 Analytical Question

27. Given the expression `"10 - 2 ** 2 * 3 // 2"`, trace Python's evaluation step by step according to operator precedence and state the final result.

Chapter 3: Control Structures and Functions

This chapter builds directly on the foundational Python concepts introduced in earlier chapters — installation, variables, data types, operators, and basic input/output — and introduces the constructs that allow a Python program to make decisions, repeat operations, recover gracefully from runtime errors, and organize logic into reusable, modular functions. Every concept is developed with explicit reference to its application in data science and machine learning workflows, from validating raw data to structuring iterative model-training procedures.

3.1 Conditional Statements

3.1.1 Introduction

Every meaningful computer program must be capable of making decisions. A weather application decides whether to display an umbrella icon based on the forecast; a machine learning pipeline decides whether to discard a record based on the number of missing values it contains; a banking system decides whether to approve or decline a transaction based on the account balance. In each of these situations, the program does not execute every line of code unconditionally — it evaluates a condition and chooses one path of execution over another. This capability is provided in Python through conditional statements, which form the foundation of decision-making logic in any application, including data-driven and intelligent systems.

This section introduces the syntax, semantics, and practical usage of Python's conditional constructs, and connects each construct to realistic examples drawn from data science and machine learning workflows.

3.1.2 Decision Making in Python

Decision making refers to the process by which a program evaluates one or more conditions and executes a specific block of statements depending on whether those conditions are True or False. Python conditions are Boolean expressions — expressions that evaluate to the built-in values True or False. Unlike many languages that require conditions to be enclosed in parentheses, Python only requires that the condition be followed by a colon, with the associated block indicated through indentation rather than braces. Indentation is not merely a stylistic choice in Python; it is syntactically significant and defines the boundaries of each block.

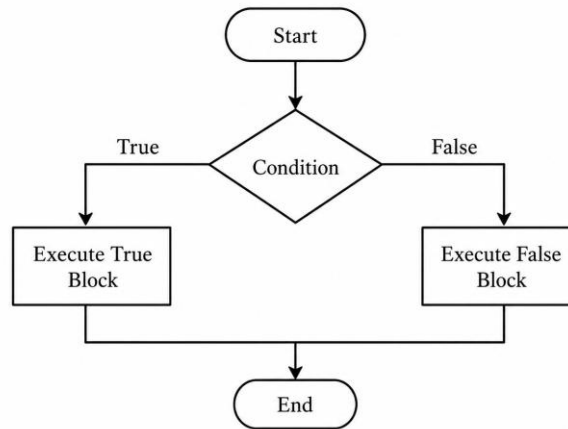


Figure 3.1. Decision Making in Python

3.1.3 if Statement

The if statement is the simplest form of conditional execution. It evaluates a condition, and if that condition is True, the indented block beneath it is executed; otherwise, the block is skipped entirely and control passes to the statement following the block.

Syntax:

```
if condition:
    statement_1
    statement_2
```

Example 3.1: Checking Eligibility for a Data Science Course

```
# Problem: Admit a student to an ML course only if
the
# prerequisite score is 60 or above.
score = 72
if score >= 60:
    print("Eligible for the Machine Learning course.")
```

Expected Output:

Eligible for the Machine Learning course.

Code Explanation: Line 3 assigns the integer value 72 to the variable score. Line 5 evaluates the Boolean expression `score >= 60`, which evaluates to True. Because the condition is True, the indented `print()` statement on line 6 executes. If score had instead

been 45, the condition would evaluate to False and the program would terminate without producing any output, since no else block has been defined.

3.1.4 if-else Statement

In many situations a program must execute one block when a condition is True and a different block when it is False. The if-else statement provides exactly this two-way branching capability.

```
if condition:
    statement_block_1 # executes when condition is True
else:
    statement_block_2 # executes when condition is False
```

Example 3.2: Classifying a Data Point as an Outlier

```
# Problem: Flag a sensor reading as an outlier if it
# deviates from the expected value by more than 10
# units.
reading = 87
expected = 70
threshold = 10
if abs(reading - expected) > threshold:
    print("Outlier detected: review sensor data.")
else:
    print("Reading within normal range.")
```

Expected Output:

Outlier detected: review sensor data.

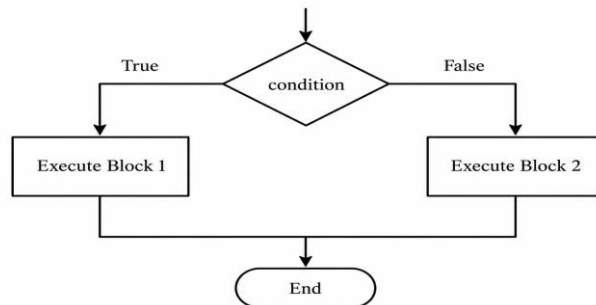


Figure 3.2. Flowchart of if-else Statement

3.1.5 if-elif-else Ladder

When a program must choose among more than two alternatives, Python provides the if-elif-else ladder. Each elif (a contraction of "else if") is evaluated in sequence only if all preceding conditions were False. As soon as one condition evaluates to True, its block executes and the remaining conditions in the ladder are skipped. The final else, which is optional, catches every case not matched by the conditions above it.

```
if condition_1:
    block_1
elif condition_2:
    block_2
elif condition_3:
    block_3
else:
    default_block
```

Example 3.3: Categorizing Model Accuracy

```
# Problem: Classify a trained model's accuracy into
# performance tiers for a project report.
accuracy = 0.84
if accuracy >= 0.90:
    grade = "Excellent"
elif accuracy >= 0.75:
    grade = "Good"
elif accuracy >= 0.60:
    grade = "Acceptable"
else:
    grade = "Needs Improvement"
print(f"Model performance: {grade}")
```

Expected Output:

```
Model performance: Good
```

Practical Use Case: This pattern is widely used in automated machine learning (AutoML) reporting tools, where a pipeline must summarize dozens of trained models into human-readable performance tiers without manual inspection.

3.1.6 Multiple Conditions and Logical Operators in Conditions

Real-world decisions are rarely governed by a single criterion. Python's logical operators — and, or, and not — allow multiple Boolean expressions to be combined into a single compound condition.

Operator	Meaning	Example	Result
and	True only if both operands are True	(5 > 2) and (3 > 1)	True
or	True if at least one operand is True	(5 > 2) or (1 > 3)	True
not	Reverses the Boolean value of the operand	not (5 > 2)	False

Table 3.1. Truth Behaviour of Logical Operators

Example 3.4: Filtering Records for a Loan Approval Model

```
# Problem: Approve a loan pre-check only if the
# applicant's
# credit score is high AND their existing debt ratio
# is low.
credit_score = 740
debt_ratio = 0.28
if credit_score >= 700 and debt_ratio <= 0.35:
    print("Applicant passes the automated pre-check.")
else:
    print("Applicant requires manual review.")
```

Expected Output:

```
Applicant passes the automated pre-check.
```

3.1.7 Comparison Operators

Comparison operators evaluate the relationship between two values and always produce a Boolean result. Python supports six comparison operators: == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), and <= (less than or equal to). A frequent beginner error is to use a single = (the assignment operator) where the double == (the equality operator) is required; the two operators serve entirely different purposes and Python will raise a `SyntaxError` if = is mistakenly used inside a condition.

3.1.8 Membership and Identity Operators in Conditions

Beyond arithmetic comparison, Python offers membership operators (`in`, `not in`) to test whether a value exists within a sequence such as a list, tuple, or string, and identity operators (`is`, `is not`) to test whether two references point to the very same object in memory rather than merely holding equal values.

Example 3.5: Validating a Column Name Before Loading a Dataset

```
# Problem: Confirm a required column exists before
# proceeding with a data preprocessing pipeline.
columns = ["age", "income", "gender", "target"]
required_column = "income"
if required_column in columns:
    print("Column      found.      Proceeding      with
preprocessing.")
else:
    print("Required column missing. Aborting pipeline.")
```

Expected Output:

```
Column found. Proceeding with preprocessing.
```

Common Errors: A frequent mistake when using `is` is to compare numeric or string values for equality, for example writing `if x is 5` instead of `if x == 5`. Although this may work for small integers due to an internal CPython optimization known as integer caching, it is not guaranteed behaviour and should never be relied upon. The `is` operator should be reserved for identity checks such as `if value is None`.

Common Mistake: *Never use `is` or `is not` to compare values for equality; reserve them strictly for identity checks such as testing against `None`, since relying on CPython's integer or string caching produces unpredictable results across Python versions and implementations.*

3.1.9 Practical Examples

Example 3.6: Validating User Input Before Loading a Dataset

```
# Problem: A data pipeline should only proceed if
the user
# supplies a file path ending in a recognized CSV
extension.
file_path = input("Enter the dataset file path: ")
```

```
if file_path.strip() == "":
    print("Error: file path cannot be empty.")
elif not file_path.endswith(".csv"):
    print("Error: only CSV files are supported.")
else:
    print(f>Loading dataset from {file_path} ...")
```

This example demonstrates how conditional statements act as a first line of defence in any data pipeline, preventing malformed or unsupported input from propagating into later stages such as feature engineering or model training, where the resulting errors would be far more difficult to diagnose.

3.1.10 Section Summary

- Conditional statements allow a program to select among alternative execution paths based on Boolean expressions.
- The if, if-else, and if-elif-else constructs provide one-way, two-way, and multi-way branching respectively.
- Logical operators (and, or, not) combine multiple conditions into a single compound decision.
- Comparison operators test relationships between values; membership and identity operators test containment and object identity.
- Conditional logic underlies data validation, filtering, and quality-control steps throughout the data science workflow.

3.2 Nested Conditions

3.2.1 Introduction

A nested condition is a conditional statement placed inside the block of another conditional statement. Nesting allows a program to express hierarchical decision logic, in which a broad category is first determined and then refined through progressively more specific checks. This mirrors the way humans naturally reason about layered decisions — for instance, a doctor first determines whether a patient's vital signs are abnormal, and only then investigates which specific vital sign is responsible.

3.2.2 Nested if Statements

Python permits an if, if-else, or if-elif-else block to be written entirely inside the body of another such block. Each level of nesting is denoted purely through additional indentation; there is no explicit keyword or symbol marking the start or end of a nested block.

```
if outer_condition:
    if inner_condition:
        statement_a
    else:
        statement_b
else:
    statement_c
```

Example 3.7: Multi-level Validation of a Dataset Record

```
# Problem: Validate an incoming patient record for
a
# healthcare analytics pipeline using nested
conditions.
age = 45
blood_pressure = 138
if age > 0:
    if blood_pressure > 130:
        print("Flag: hypertension risk – recommend review.")
    else:
        print("Blood pressure within normal range.")
else:
    print("Invalid record: age must be positive.")
```

Expected Output:

```
Flag: hypertension risk – recommend review.
```

Code Explanation: The outer if statement first verifies that the record is structurally valid by confirming age is positive. Only within that True branch does the program evaluate the inner condition concerning blood pressure. This ordering ensures that a meaningless or corrupted record — such as one with a negative age — is rejected before

any clinical logic is applied to it, which is a standard practice in healthcare and financial data pipelines.

3.2.3 Multi-level Decision Making

Nesting can extend beyond two levels, although readability declines rapidly as depth increases. As a general guideline, most style guides — including PEP 8 — recommend limiting nested conditional logic to two or three levels; deeper hierarchies are usually better expressed using separate functions, early returns, or combined logical conditions.

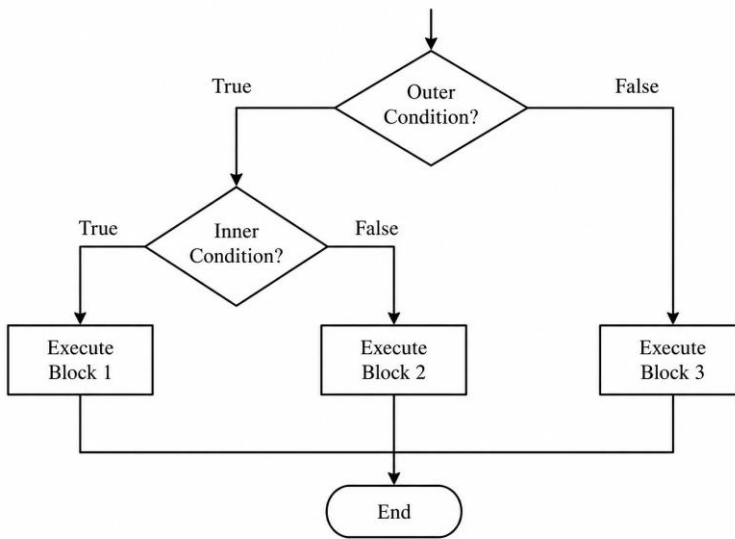


Figure 3.3. Nested if Execution

3.2.4 Real-world Applications

Nested conditions appear throughout production data science systems. A fraud-detection engine may first check whether a transaction amount exceeds a threshold, and only then check the account's transaction history and geographic pattern. A recommendation system may first check whether a user is logged in, and only then check their purchase history to decide which items to display. An autonomous robotics controller may first check whether an obstacle sensor is active, and only then evaluate the obstacle's estimated distance to decide whether to stop or steer around it.

3.2.5 Advantages and Limitations

- Advantage: Nested conditions naturally express hierarchical or dependent decision logic that mirrors real-world reasoning.

- Advantage: Inner conditions are only evaluated when the relevant outer condition holds, which can improve efficiency by avoiding unnecessary checks.
- Limitation: Deep nesting (commonly called the "arrow anti-pattern") reduces readability and increases the likelihood of logical errors.
- Limitation: Debugging deeply nested logic is more difficult, since a fault may originate several indentation levels away from where its symptom appears.

Best Practice: *Prefer combining conditions with logical operators or restructuring code with early returns inside functions over writing more than three levels of nested if statements; this keeps decision logic flat, testable, and easier to maintain.*

3.2.6 Programming Examples

Example 3.8: Grading a Machine Learning Assignment with Multiple Criteria

```
# Problem: Assign a grade based on both submission
# timeliness and model accuracy, using nested
# conditions.
submitted_on_time = True
accuracy = 0.81
if submitted_on_time:
    if accuracy >= 0.85:
        grade = "A"
    elif accuracy >= 0.70:
        grade = "B"
    else:
        grade = "C"
    else:
        grade = "Late Penalty Applied"
print(f"Final Grade: {grade}")
```

Expected Output:

```
Final Grade: B
```

3.2.7 Section Summary

- Nested conditions place one conditional block inside another to express hierarchical decision logic.
- Indentation alone determines the level of nesting in Python.

- Excessive nesting harms readability; combined conditions or function-based restructuring are usually preferable beyond two or three levels.
- Nested conditions are common in validation pipelines, fraud detection, robotics, and recommendation systems.

3.3 Loops

3.3.1 Introduction

Iteration is one of the four foundational pillars of structured programming, alongside sequence, selection, and modularity. A loop allows a block of statements to be executed repeatedly, either a fixed number of times or until a specific condition is no longer satisfied. In data science and machine learning, iteration is indispensable: datasets containing millions of rows must be processed record by record, models must be trained across many epochs, and hyperparameter searches must repeat an evaluation procedure across dozens or hundreds of candidate configurations. Without loops, such tasks would require writing an impractical number of nearly identical statements by hand.

3.3.2 Need for Iteration

Consider the task of computing the square of every number from 1 to 1000. Writing 1000 individual statements would be both impractical and error-prone. A loop expresses this repetitive task in a small, fixed number of lines regardless of how many times the block must execute, which is precisely why iteration is central to processing large-scale data.

3.3.3 while Loop

The while loop repeatedly executes a block of statements as long as its controlling condition remains True. The condition is evaluated before each iteration; if it is False on the very first evaluation, the loop body never executes at all.

```
while condition:
    statement_block
    update_expression # must eventually make condition
    False
```

Example 3.9: Simulating a Simple Gradient Descent Stopping Rule

```
# Problem: Repeat an update step until the error
falls
```

```
# below an acceptable tolerance (a simplified
analogy
# for iterative optimization used in ML training).
error = 1.0
learning_rate = 0.5
iteration = 0
while error > 0.01:
    error = error * (1 - learning_rate)
    iteration += 1
    print(f"Iteration {iteration}: error =
    {error:.5f}")
print("Converged.")
```

Expected Output:

```
Iteration 1: error = 0.50000
Iteration 2: error = 0.25000
Iteration 3: error = 0.12500
Iteration 4: error = 0.06250
Iteration 5: error = 0.03125
Iteration 6: error = 0.01563
Iteration 7: error = 0.00781
Converged.
```

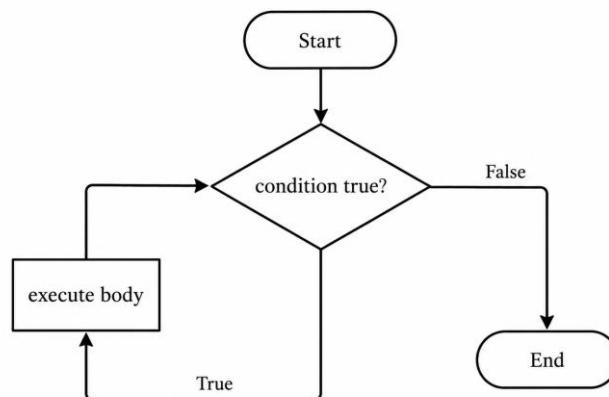


Figure 3.5. While Loop Flowchart

Debugging Tip: *If a while loop appears to hang, verify that the loop body contains a statement that eventually changes a variable used in the condition; a controlling variable that is never updated is the single most common cause of an unintended infinite loop.*

3.3.4 for Loop

The for loop iterates over the elements of an iterable object — such as a list, tuple, string, dictionary, or range — executing the loop body once for each element in sequence. Unlike the while loop, the for loop does not require the programmer to manually manage a counter variable, which makes it the preferred construct whenever the number of iterations is known in advance or determined by the size of a collection.

```
for item in iterable:  
    statement_block
```

Example 3.10: Iterating Through Dataset Column Names

```
# Problem: Print each column name from a dataset  
schema  
  
# along with its position index.  
columns = ["age", "income", "education", "target"]  
for index, name in enumerate(columns):  
    print(f"Column {index}: {name}")
```

Expected Output:

```
Column 0: age  
Column 1: income  
Column 2: education  
Column 3: target
```

Code Explanation: The built-in `enumerate()` function wraps the columns list and yields pairs consisting of an automatically generated index and the corresponding element. This pattern is extremely common when processing tabular data, since it is frequently necessary to know both the position and the value of a feature.

3.3.5 Range Function

The `range()` function generates a sequence of integers and is most often used to control the number of iterations of a for loop. It accepts up to three arguments: `range(stop)`, `range(start, stop)`, and `range(start, stop, step)`. The stop value is always excluded from the generated sequence.

```
for i in range(5):
    print(i) # prints 0, 1, 2, 3, 4
for i in range(2, 10, 2):
    print(i) # prints 2, 4, 6, 8
```

Aspect	for Loop	while Loop
Typical use	Known or bounded number of iterations	Unknown number of iterations, condition-based
Iterates over	Sequences, ranges, iterables	Any Boolean condition
Risk of infinite loop	Very low	Higher, if condition never becomes False
Counter management	Handled automatically	Must be managed manually
Common ML use	Iterating over dataset rows or features	Iterative optimization until convergence

Table 3.2. Comparison of for and while Loops

3.3.6 Nested Loops

A nested loop is a loop placed inside the body of another loop. The inner loop completes all of its iterations for every single iteration of the outer loop. Nested loops are essential for processing two-dimensional structures such as matrices, images represented as pixel grids, and tables organized by rows and columns.

Example 3.11: Traversing a Small Feature Matrix

```
# Problem: Print every value in a 3x3 feature matrix,
# row by row, using nested for loops.
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for row in matrix:
    for value in row:
        print(value, end=" ")
    print() # move to the next line after each row
```

Expected Output:

```
1 2 3
4 5 6
7 8 9
```

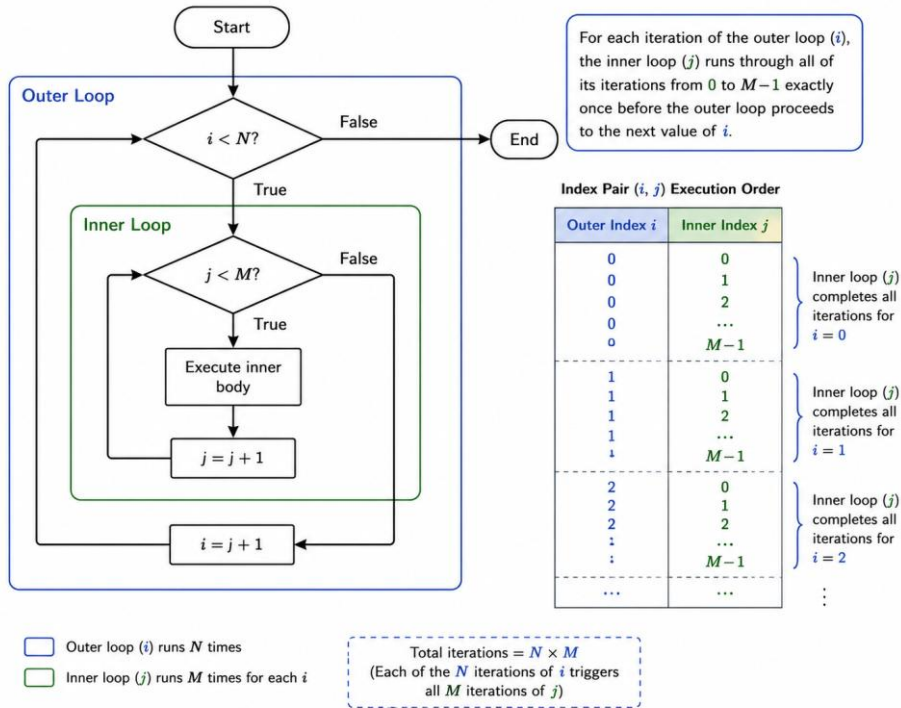


Figure 3.7. Nested Loop Execution

3.3.7 Infinite Loops

An infinite loop is a loop whose controlling condition never becomes False, causing it to execute indefinitely. While infinite loops are usually the result of a programming error, they are occasionally created intentionally — for example, in a server process or a sensor-monitoring script that must run continuously until externally interrupted, typically combined with a break statement triggered by a specific event.

```
# Intentional infinite loop with a controlled exit
while True:
    reading = get_sensor_reading() # hypothetical
    function
    if reading is None:
        break
    process(reading)
```

Common Mistake: A frequent beginner error is writing `while x < 10:` without ever modifying `x` inside the loop body, which produces an unintended infinite loop;

always confirm that the loop body updates every variable referenced in the condition.

3.3.8 Loop Efficiency

Although loops are indispensable, excessive or poorly structured iteration can severely degrade performance, particularly when processing large datasets. In data science practice, explicit Python-level loops over large arrays are often replaced with vectorized operations provided by libraries such as NumPy and pandas, which execute the equivalent iteration in optimized, compiled code and can be orders of magnitude faster. Understanding loops thoroughly at the language level is nonetheless essential, both for tasks that genuinely require explicit iteration and for understanding what vectorized library functions are doing internally.

Industry Insight: *Production data engineering teams routinely profile pipelines to identify explicit loops over large pandas DataFrames or NumPy arrays and replace them with vectorized equivalents, since a single unvectorized loop can dominate the total runtime of an otherwise well-optimized pipeline.*

3.3.9 Practical Examples

Example 3.12: Filtering Missing Values from a Dataset

```
# Problem: Remove missing values (represented as
None)

# from a raw list of sensor readings before analysis.
raw_readings = [23.5, None, 25.1, None, 22.8, 24.0]
clean_readings = []
for value in raw_readings:
    if value is not None:
        clean_readings.append(value)
print("Cleaned data:", clean_readings)
```

Expected Output:

```
Cleaned data: [23.5, 25.1, 22.8, 24.0]
```

Practical Use Case: This loop-and-filter pattern is the conceptual basis of missing-value handling in every data preprocessing pipeline, and is directly analogous to operations such as pandas' `dropna()` method, which performs the same logic internally at much higher speed.

3.3.10 Section Summary

- Loops enable repeated execution of a block of statements, either a known number of times (for) or until a condition changes (while).
- The `range()` function is the standard tool for generating numeric sequences to control for loops.
- Nested loops are required for processing multi-dimensional structures such as matrices and images.
- Infinite loops usually indicate a logical error but are occasionally intentional, controlled by an internal break condition.
- Vectorized operations in libraries such as NumPy and pandas often replace explicit loops for performance-critical data processing.

3.4 Loop Control Statements

Loop control statements alter the normal, sequential flow of a loop's execution. Python provides three such statements: `break`, which terminates a loop entirely; `continue`, which skips the remainder of the current iteration and proceeds to the next; and `pass`, which performs no operation and serves as a syntactic placeholder.

3.4.1 Break Statement

The `break` statement immediately terminates the innermost enclosing loop, transferring control to the first statement following the loop. It is typically used to exit a loop as soon as a specific condition is met, avoiding unnecessary further iterations.

Example 3.13: Stopping a Search Once a Target Record Is Found

```
# Problem: Search a list of customer IDs and stop as
soon
# as the target ID is located, avoiding wasted
iterations.
customer_ids = [101, 205, 309, 412, 517, 620]
target = 412
for cid in customer_ids:
    if cid == target:
        print(f"Customer {target} found. Stopping search.")
        break
```

```
print(f"Checked {cid}, not a match.")
```

Expected Output:

```
Checked 101, not a match.  
Checked 205, not a match.  
Checked 309, not a match.  
Customer 412 found. Stopping search.
```

3.4.2 continue Statement

The continue statement skips the remaining statements in the current loop iteration and immediately proceeds to the next iteration's condition check, without terminating the loop itself.

Example 3.14: Skipping Invalid Records During Preprocessing

```
# Problem: Sum only the valid (non-negative)  
readings in  
# a sensor dataset, skipping invalid negative  
values.  
readings = [15, -3, 22, 18, -7, 30]  
total = 0  
for value in readings:  
    if value < 0:  
        continue # skip invalid reading  
    total += value  
print("Sum of valid readings:", total)
```

Expected Output:

```
Sum of valid readings: 85
```

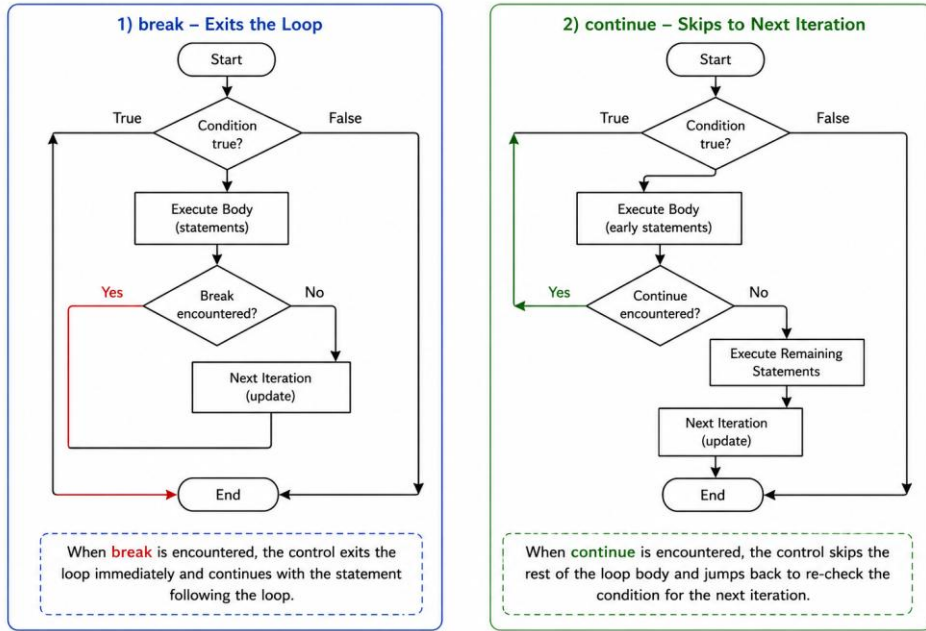


Figure 3.8. Break vs Continue

3.4.3 pass Statement

The pass statement is a null operation — when executed, nothing happens. It exists purely to satisfy Python's syntactic requirement that every block contain at least one statement. It is commonly used as a placeholder for code that will be implemented later, or in situations where a syntactically required block is intentionally left empty.

```

for value in readings:
    if value < 0:
        pass # TODO: implement outlier logging later
    else:
        process(value)
  
```

Statement	Effect	Typical Use Case
break	Terminates the enclosing loop immediately	Stop searching once a target is found
continue	Skips to the next iteration	Skip invalid or missing records
pass	Performs no action; syntactic placeholder	Stub out unfinished branches during development

Table 3.3. Loop Control Statements

3.4.4 Practical Examples

Example 3.15: Early Stopping in a Simplified Training Loop

```
# Problem: Simulate early stopping – halt training
once
# validation loss stops improving, a common ML
technique.
validation_losses = [0.90, 0.72, 0.55, 0.54, 0.55,
0.56]
best_loss = float("inf")
patience = 2
no_improvement = 0
for epoch, loss in enumerate(validation_losses,
start=1):
    if loss < best_loss:
        best_loss = loss
        no_improvement = 0
    else:
        no_improvement += 1
    print(f"Epoch {epoch}: loss = {loss}")
    if no_improvement >= patience:
        print("Early stopping triggered.")
        break
```

Expected Output:

```
Epoch 1: loss = 0.9
Epoch 2: loss = 0.72
Epoch 3: loss = 0.55
Epoch 4: loss = 0.54
Epoch 5: loss = 0.55
Epoch 6: loss = 0.56
Early stopping triggered.
```

Practical Use Case: Early stopping is a standard regularization technique in deep learning frameworks such as TensorFlow and PyTorch, implemented internally using precisely this break-based pattern to prevent a model from overfitting the training data.

3.4.5 Best Practices

- Use `break` sparingly and only when it genuinely improves clarity or efficiency, such as terminating a search early.
- Avoid using `continue` to create deeply nested conditional logic within a loop; if the skipping logic becomes complex, consider extracting it into a separate function.
- Never leave a `pass` statement in production code as a substitute for genuine error handling; it should only mark deliberately deferred work.
- Combine `break` with a `for-else` construct when the program must distinguish between a loop that completed normally and one that was terminated early.

3.4.6 Common Mistakes

Common Mistake: Placing a `break` or `continue` statement outside of any loop raises a `SyntaxError`, since both statements are only meaningful within the context of an enclosing `for` or `while` loop.

Common Mistake: In nested loops, `break` and `continue` affect only the innermost loop in which they are written; beginners frequently expect `break` to terminate all enclosing loops simultaneously, which is not the default behaviour.

3.5 Pattern Programs

Pattern-printing exercises are a long-standing pedagogical tool in programming education. Although the patterns themselves have little direct application in production software, the exercise of constructing them is extremely effective at developing a programmer's intuition for nested loops, index arithmetic, and the relationship between loop variables and printed output. Every pattern in this section is built from the same core skill: expressing the number of rows, the number of columns per row, and the character printed at each position, purely in terms of loop counters.

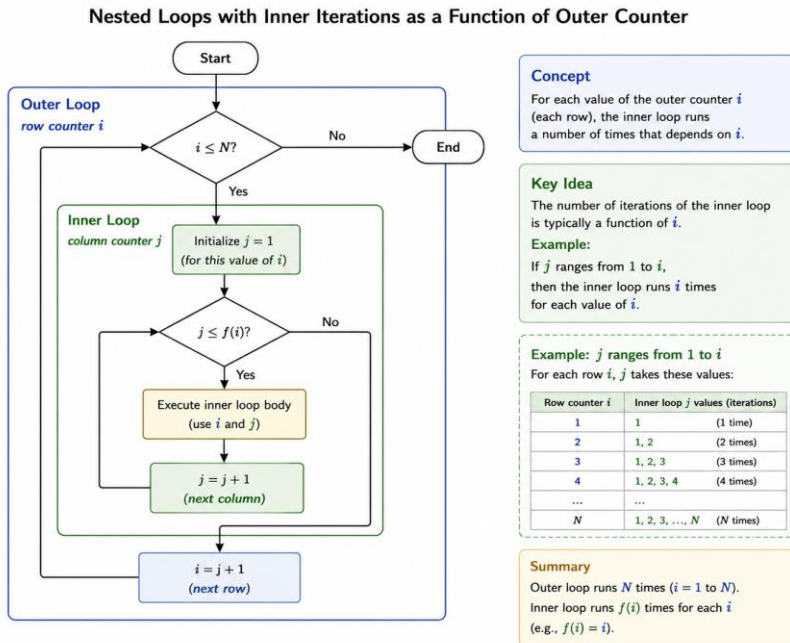


Figure 3.9. Pattern Generation Logic

3.5.1 Square Pattern

```
n = 5
for i in range(n):
    for j in range(n):
        print("*", end=" ")
    print()
```

Expected Output:

```
*****
*****
*****
*****
*****
```

Pattern Logic: Both loops run a fixed number of times, n , regardless of the current row, which produces a uniform grid of equal width and height.

3.5.2 Rectangle Pattern

```
rows, cols = 4, 8
for i in range(rows):
```

```
for j in range(cols):
    print("*", end=" ")
    print()
```

Expected Output:

```
*****
*****
*****
*****
```

3.5.3 Right Triangle

```
n = 5
for i in range(1, n + 1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Expected Output:

```
*
**
***
****
*****
```

Pattern Logic: The inner loop now depends on the outer loop variable *i*, ranging from 0 to *i* - 1. As *i* increases with each row, one additional asterisk is printed, producing the diagonal growth characteristic of a right triangle.

3.5.4 Left Triangle

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    for j in range(i):
        print("*", end=" ")
    print()
```

Expected Output:

```
*
* *
* * *
* * * *
* * * * *
```

Pattern Logic: A leading-space calculation, $n - i$, is introduced before the asterisks are printed. This decreasing number of leading spaces, combined with the increasing number of asterisks, shifts the triangle so that it aligns along its left edge.

3.5.5 Pyramid Pattern

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    for j in range(2 * i - 1):
        print("*", end="")
    print()
```

Expected Output:

```
*
***
*****
*****
*****
```

Pattern Logic: The number of asterisks in row i follows the odd-number sequence $2i - 1$, while the number of leading spaces decreases as $n - i$. Together, these two expressions center the growing row of asterisks, forming a symmetrical pyramid.

3.5.6 Inverted Pyramid

```
n = 5
for i in range(n, 0, -1):
    print(" " * (n - i), end="")
    for j in range(2 * i - 1):
        print("*", end="")
```

```
print()
```

Expected Output:

```
*****
*****
*****
***
*
```

Pattern Logic: This program reuses the pyramid logic exactly, but iterates the outer loop counter *i* in descending order using `range(n, 0, -1)`, which reverses the direction of growth.

3.5.7 Diamond Pattern

```
n = 4
# Upper half (pyramid)
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
# Lower half (inverted pyramid, excluding the middle row)
for i in range(n - 1, 0, -1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

Expected Output:

```
*
***
*****
*****
*****
***
*
```

Pattern Logic: A diamond is simply a pyramid immediately followed by an inverted pyramid whose largest row is omitted, since that row was already printed as the last row of the upper pyramid.

3.5.8 Floyd's Triangle

```
n = 5
num = 1
for i in range(1, n + 1):
    for j in range(i):
        print(num, end=" ")
    num += 1
    print()
```

Expected Output:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Pattern Logic: Rather than printing a fixed symbol, this pattern maintains a running counter, num, that increments after every value is printed, so that the numbers continue in sequence across row boundaries.

3.5.9 Pascal's Triangle (Basic Implementation)

```
n = 5
for i in range(n):
    value = 1
    print(" " * (n - i), end="")
    for j in range(i + 1):
        print(value, end=" ")
        value = value * (i - j) // (j + 1)
    print()
```

Expected Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Pattern Logic: Each entry of Pascal's Triangle is a binomial coefficient. Rather than computing factorials directly, this implementation updates value incrementally using the relationship $C(i, j+1) = C(i, j) * (i - j) / (j + 1)$, which is both efficient and numerically simple.

3.5.10 Number Patterns

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(i, end=" ")
    print()
```

Expected Output:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

3.5.11 Alphabet Patterns

```
n = 5
for i in range(n):
    for j in range(i + 1):
        print(chr(65 + j), end=" ")
    print()
```

Expected Output:

```
A
A B
A B C
A B C D
A B C D E
```

Pattern Logic: The built-in `chr()` function converts an integer ASCII code into its corresponding character. Since the ASCII code for the letter 'A' is 65, adding the inner loop counter `j` produces consecutive letters of the alphabet.

3.5.12 Hollow Patterns

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        if j == 1 or j == i or i == n:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

Expected Output:

```
*
* *
* *
* *
* *
* * * * *
```

Pattern Logic: A hollow pattern prints a symbol only along the boundary of the shape — the first column, the last column of each row, and the final row — and prints blank space everywhere in the interior, using a conditional check inside the inner loop.

3.5.13 Star Patterns and Multiplication Table Patterns

A multiplication table can itself be viewed as a numeric pattern generated with nested loops, where the printed value is the product of the two loop counters rather than a fixed symbol.

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n + 1):
        print(i * j, end="\t")
    print()
```

Expected Output:

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
```


4 8 12 16 20

5 10 15 20 25

3.5.14 Pattern Logic Explanation and General Strategy

Every pattern program in this section, however visually different, can be constructed using the same three-step strategy: first, determine the total number of rows and express it as the range of the outer loop; second, determine how the number of columns (or the value printed) in each row depends on the current row index, and express that relationship as the range or content of the inner loop; third, add any necessary spacing calculation to align the shape correctly. Students are strongly encouraged to trace through each program by hand, row by row, before running it, since this manual tracing is what builds durable intuition for nested iteration.

Programming Tip: *When designing a new pattern from scratch, first write out the desired output on paper for a small value of n , such as $n = 4$, and only then work backwards to determine the mathematical relationship between the row index and the number of symbols or spaces required.*

3.6 Exception Handling

3.6.1 Introduction

No matter how carefully a program is written, it will inevitably encounter situations it cannot proceed through normally: a file that does not exist, a dataset column that is missing, a network request that times out, or a user who enters text where a number was expected. Python's exception-handling mechanism allows a program to anticipate such situations and respond to them gracefully, rather than terminating abruptly with an unhandled error. This capability is especially critical in data science and machine learning pipelines, which routinely process large volumes of external, imperfect, real-world data.

3.6.2 Runtime Errors and Syntax Errors

Python errors fall into two broad categories. A syntax error occurs when code violates the grammatical rules of the language itself — for example, a missing colon after an if statement — and is detected before the program begins execution at all. A runtime error, by contrast, occurs while the program is executing, when a statement that is grammatically valid nonetheless cannot be completed successfully, such as dividing a number by zero or attempting to open a file that does not exist. Exception handling addresses runtime errors exclusively; a syntax error must be corrected in the source code before the program can run.

3.6.3 Types of Exceptions

Python defines a rich hierarchy of built-in exception classes, all of which ultimately derive from the base class Base Exception. The most frequently encountered exceptions are summarized in Table 3.4.

Exception	Typical Cause
ZeroDivisionError	Division or modulo operation with a divisor of zero
ValueError	A function receives an argument of the correct type but an inappropriate value
TypeError	An operation is applied to an object of an inappropriate type
IndexError	A sequence is accessed using an index that is out of range
KeyError	A dictionary is accessed using a key that does not exist
FileNotFoundError	An attempt is made to open a file that does not exist
AttributeError	An attribute or method is accessed that does not exist on an object
ImportError / ModuleNotFoundError	A module or name cannot be imported

Table 3.4. Common Types of Exceptions in Python

3.6.4 try, except, else, and finally

The try block encloses code that might raise an exception. If an exception occurs, control immediately transfers to a matching except block, which defines how that specific exception should be handled. The optional else block executes only if the try block completed without raising any exception, and is useful for code that should run solely on the success path. The optional finally block always executes, regardless of whether an exception occurred, and is typically used for cleanup actions such as closing a file or releasing a database connection.

```
try:
    risky_statement
except SomeException as e:
    handling_statement
else:
```

```

statement_if_no_exception
finally:
cleanup_statement_always_runs

```

Example 3.16: Safely Loading a Dataset File

```

# Problem: Attempt to open a dataset file, handling
the
# case where the file does not exist on disk.
file_name = "sales_data.csv"
try:
file = open(file_name, "r")
except FileNotFoundError:
print(f"Error: '{file_name}' was not found.")
else:
print("File opened successfully.")
file.close()
finally:
print("File-loading attempt complete.")

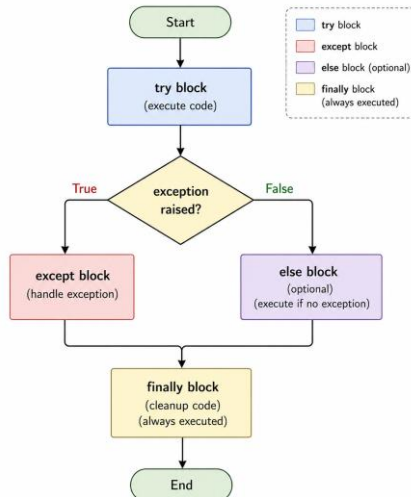
```

Expected Output:

```

Error: 'sales_data.csv' was not found.
File-loading attempt complete.

```

**Figure 3.10.** Exception Handling Workflow

3.6.5 raise Statement

The raise statement allows a program to deliberately trigger an exception, either one of Python's built-in exception types or a custom exception class. This is useful for enforcing preconditions explicitly, rather than allowing an operation to fail with a less informative, unrelated error later.

```
def set_learning_rate(rate):
    if rate <= 0:
        raise ValueError("Learning rate must be a positive
number.")
    return rate
try:
    set_learning_rate(-0.01)
except ValueError as e:
    print(f"Configuration error: {e}")
```

Expected Output:

```
Configuration error: Learning rate must be a
positive number.
```

3.6.6 User-defined Exceptions

Beyond the built-in exception types, Python allows programmers to define their own exception classes by inheriting from the built-in Exception class. Custom exceptions make error handling more expressive and meaningful, particularly in larger data pipelines and libraries where a generic ValueError does not adequately describe a domain-specific failure.

```
class InvalidFeatureRangeError(Exception):
    """Raised when a feature value falls outside its
    expected range."""
    pass
def validate_age(age):
    if not (0 <= age <= 120):
        raise InvalidFeatureRangeError(f"Invalid age value:
{age}")
    return age
try:
    validate_age(150)
```

```
except InvalidFeatureRangeError as e:  
    print(f>Data validation failed: {e}")
```

Expected Output:

```
Data validation failed: Invalid age value: 150
```

3.6.7 Multiple Exception Handling

A single try block may be followed by several except clauses, each targeting a different exception type, allowing the program to respond differently depending on the specific failure. A single except clause may also catch multiple exception types simultaneously by listing them as a tuple.

```
data = {"scores": [90, 85, 78]}  
try:  
    index = int(input("Enter an index: "))  
    print(data["scores"][index])  
except ValueError:  
    print("Error: please enter a valid integer.")  
except IndexError:  
    print("Error: index is out of range.")  
except (KeyError, TypeError) as e:  
    print(f"Unexpected data structure error: {e}")
```

3.6.8 Nested Exception Handling

A try-except block may itself be placed inside another try or except block. Nested exception handling is often used when an inner operation, such as parsing a single record, may fail independently of an outer operation, such as reading the entire file, and each failure requires a distinct response.

```
records = ["25", "thirty", "41"]  
valid_ages = []  
try:  
    for record in records:  
        try:  
            valid_ages.append(int(record))  
        except ValueError:  
            print(f"Skipping invalid record: {record}")
```

```
except Exception as e:
    print(f"Unexpected failure while processing records:
    {e}")
    print("Valid ages:", valid_ages)
```

Expected Output:

```
Skipping invalid record: thirty
Valid ages: [25, 41]
```

3.6.9 File Handling Exceptions

File operations are among the most common sources of runtime exceptions in data science workflows, since they depend on the state of the external file system. The `with` statement, combined with exception handling, is the recommended approach for working with files, since it guarantees that the file is properly closed even if an exception occurs partway through processing.

```
try:
    with open("readings.csv", "r") as f:
        contents = f.read()
except FileNotFoundError:
    print("The specified data file could not be
    located.")
except PermissionError:
    print("Insufficient permissions to read the data
    file.")
```

3.6.10 Best Practices

- Catch specific exception types rather than using a bare `except` clause, which can silently hide unrelated bugs.
- Use the `finally` block, or better still a `with` statement, to guarantee that resources such as files and network connections are always released.
- Raise descriptive, domain-specific exceptions in reusable library or pipeline code rather than allowing generic errors to propagate.
- Avoid using exception handling as a substitute for ordinary conditional validation; exceptions should represent exceptional circumstances, not routine control flow.

- Log exception details (using Python's logging module) in production data pipelines rather than merely printing them, to preserve a permanent record for later diagnosis.

3.6.11 Industrial Examples

In large-scale data engineering systems, exception handling is central to building resilient extract-transform-load (ETL) pipelines. A pipeline ingesting data from an external API must anticipate network timeouts, malformed JSON responses, and authentication failures, wrapping each external call in targeted exception handling so that a single failed request does not halt the entire batch process. Similarly, production machine learning inference services wrap model prediction calls in exception handling to catch malformed input and return a controlled error response to the calling application, rather than crashing the service for every user.

Interview Question: *A frequently asked interview question is: 'What is the difference between except Exception and a bare except?' The expected answer is that except Exception catches nearly all standard runtime exceptions while still allowing system-exiting exceptions such as KeyboardInterrupt and SystemExit to propagate normally, whereas a bare except: catches every exception without exception, including those that should typically be allowed to terminate the program.*

3.6.12 Section Summary

- Exception handling separates error-recovery logic from the main logic of a program using try, except, else, and finally.
- Python provides a rich hierarchy of built-in exception types, and custom exception classes can be defined for domain-specific errors.
- The raise statement allows a program to trigger exceptions deliberately, enforcing preconditions explicitly.
- Robust exception handling is essential to resilient, production-grade data pipelines and machine learning services.

3.7 Functions with Examples

3.7.1 Introduction to Functions

A function is a named, reusable block of code designed to perform a specific task. Rather than repeating the same sequence of statements every time a particular computation is needed, a programmer defines that sequence once, inside a function, and then invokes

the function by name wherever the computation is required. Functions are the primary mechanism of modularity in Python and form the conceptual bridge between simple scripts and well-structured, maintainable software systems, including the data preprocessing pipelines, feature-engineering utilities, and model-training routines that are central to data science practice.

3.7.2 Need for Functions and Advantages of Modular Programming

As a program grows, repeating the same logic in multiple places becomes both inefficient to write and dangerous to maintain, since a bug fix or improvement must then be applied consistently everywhere that logic was duplicated. Functions address this by isolating a task behind a single, well-defined interface.

- **Reusability:** A function, once written and tested, can be called any number of times without rewriting its logic.
- **Modularity:** Complex problems can be decomposed into smaller, independently understandable sub-problems, each handled by its own function.
- **Maintainability:** A correction or improvement made inside a function automatically applies everywhere that function is called.
- **Abstraction:** A function's caller only needs to understand what the function does, not the internal details of how it does it.
- **Testability:** Small, focused functions are far easier to test in isolation than a single large, monolithic block of code.

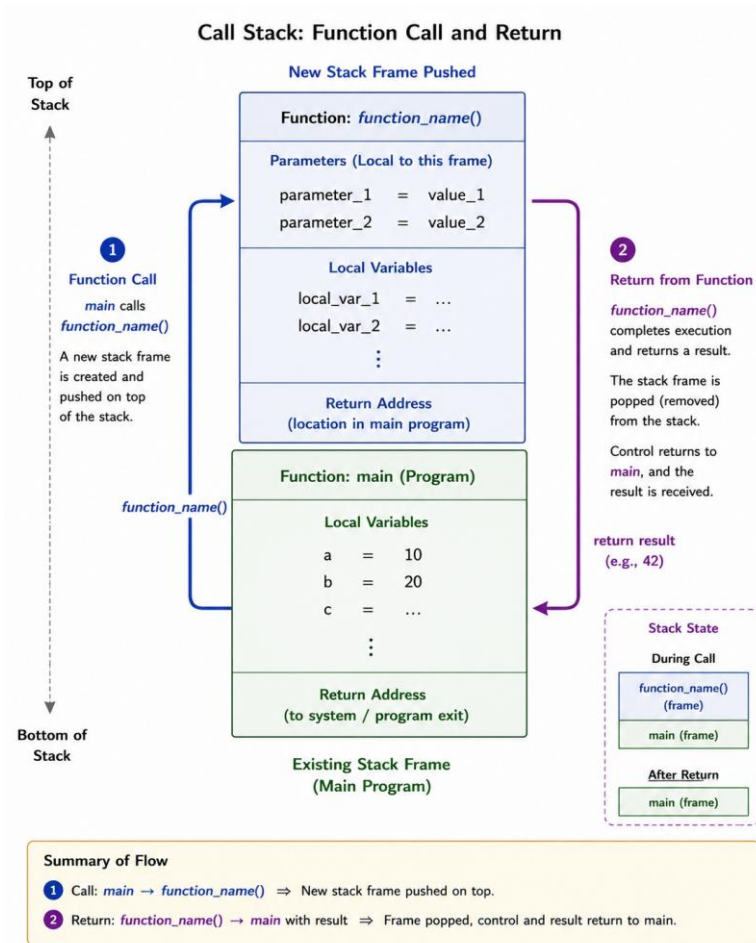
3.7.3 Defining and Calling Functions

A function is defined using the `def` keyword, followed by the function's name, a parenthesized parameter list, and a colon. The function body, indented beneath the definition, contains the statements executed whenever the function is called.

```
def function_name(parameter_1, parameter_2):  
    # function body  
    result = parameter_1 + parameter_2  
    return result  
  
# Calling the function  
output = function_name(5, 3)  
print(output)
```


Expected Output:

8

**Figure 3.11. Function Call Stack****3.7.4 Function Parameters: Positional, Keyword, and Default Arguments**

Python offers several ways to pass values into a function, providing considerable flexibility in how a function is called.

Positional arguments are matched to parameters strictly by their order in the function call. Keyword arguments are matched to parameters by explicitly naming the parameter in the call, which allows arguments to be supplied in any order and improves readability at the call site. Default arguments allow a parameter to be omitted entirely from the call, in which case a predefined default value is used instead.

```
def normalize_feature(value, min_val=0, max_val=1):
    return (value - min_val) / (max_val - min_val)
print(normalize_feature(50, 0, 100)) # positional
print(normalize_feature(value=50, max_val=100)) #
keyword
print(normalize_feature(50)) # uses defaults
```

Expected Output:

```
0.5
0.5
50.0
```

Common Mistake: In a function definition, parameters with default values must always be listed after parameters without default values; writing `def f(a=1, b):` raises a `SyntaxError`.

3.7.5 Variable-length Arguments: `*args` and `**kwargs`

Sometimes the exact number of arguments a function will receive cannot be known in advance. The `*args` syntax collects any number of additional positional arguments into a tuple, while the `**kwargs` syntax collects any number of additional keyword arguments into a dictionary.

```
def compute_average(*args):
    return sum(args) / len(args)
print(compute_average(80, 90, 70, 85))
def build_model_config(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} = {value}")
    build_model_config(learning_rate=0.01, epochs=50,
        batch_size=32)
```

Expected Output:

```
81.25
learning_rate = 0.01
epochs = 50
batch_size = 32
```

Practical Use Case: Many machine learning library functions, including several constructors in scikit-learn and Keras, use `**kwargs` internally to accept a large and

evolving set of optional configuration parameters without requiring every possible parameter to be listed explicitly in the function signature.

3.7.6 Return Statement and Multiple Return Values

The return statement ends a function's execution and optionally sends a value back to the calling code. A function without an explicit return statement implicitly returns the special value `None`. Python also allows a function to return multiple values simultaneously, which are automatically packaged into a tuple.

```
def compute_statistics(numbers):
    minimum = min(numbers)
    maximum = max(numbers)
    average = sum(numbers) / len(numbers)
    return minimum, maximum, average
low, high, mean = compute_statistics([12, 45, 7, 33,
19])
print(f"Min: {low}, Max: {high}, Mean: {mean:.2f}")
```

Expected Output:

```
Min: 7, Max: 45, Mean: 23.20
```

3.7.7 Scope of Variables: Local and Global Variables

The scope of a variable determines the region of a program in which that variable can be accessed. A local variable is defined inside a function and exists only for the duration of that function's execution, remaining invisible outside of it. A global variable is defined at the top level of a module and can be read from within any function in that module; however, modifying a global variable from within a function requires the `global` keyword to explicitly declare that intent.

```
total_records_processed = 0 # global variable
def process_batch(batch_size):
    global total_records_processed
    local_counter = batch_size # local variable
    total_records_processed += local_counter
    return local_counter
process_batch(100)
process_batch(50)
```

```
print(total_records_processed)
```

Expected Output:

150

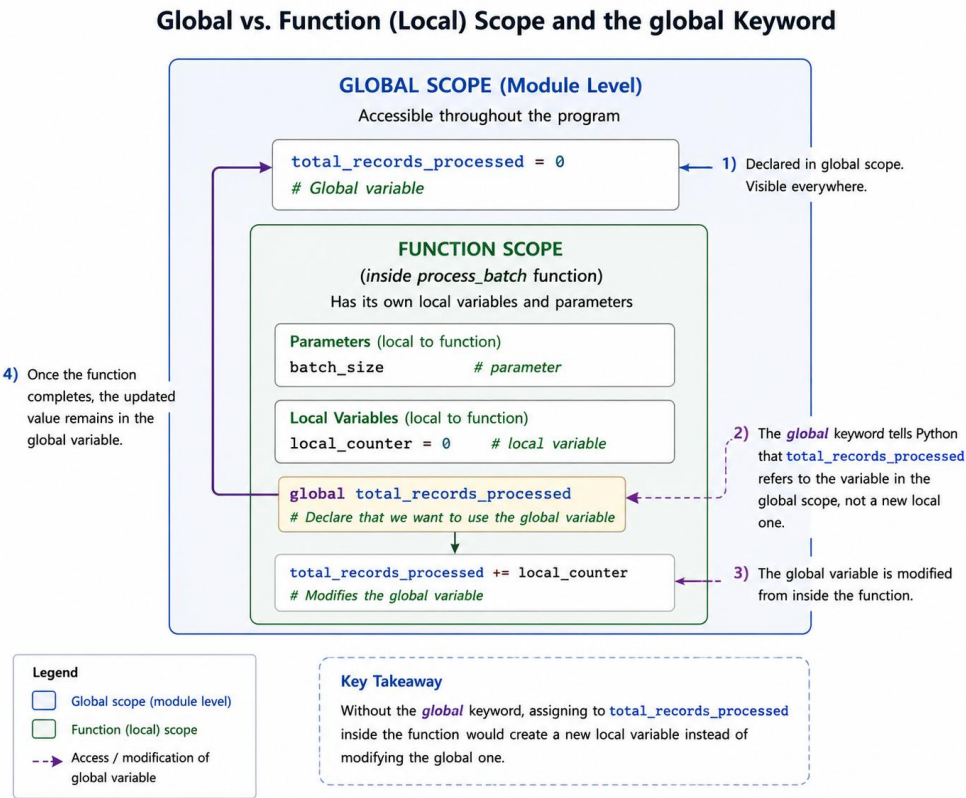


Figure 3.12. Variable Scope Illustration

Aspect	Local Variable	Global Variable
Defined	Inside a function	At module level, outside any function
Visible to	Only the function it is defined in	Any function within the module (read access)
Lifetime	Exists only during the function call	Exists for the lifetime of the program
Modifying from a function	Direct assignment	Requires the global keyword

Table 3.6. Local vs Global Variables

Best Practice: *Minimize reliance on global variables within data processing functions; passing values explicitly as parameters and returning results explicitly makes functions easier to test, reason about, and reuse in different contexts.*

3.7.8 Lambda Functions and Anonymous Functions

A lambda function is a small, unnamed (anonymous) function defined using the lambda keyword, restricted to a single expression whose value is automatically returned. Lambda functions are commonly used when a short function is needed briefly, often as an argument to another function such as `sorted()`, `map()`, or `filter()`.

```
square = lambda x: x ** 2
print(square(6))
records = [("Alice", 85), ("Bob", 92), ("Cara", 78)]
records_sorted = sorted(records, key=lambda r: r,
reverse=True)
print(records_sorted)
```

Expected Output:

```
36
[('Bob', 92), ('Alice', 85), ('Cara', 78)]
```

Practical Use Case: Lambda functions are pervasive in pandas-based data analysis, where the `apply()` method frequently accepts a lambda expression to transform every value in a column in a single, concise line, such as `df['price'].apply(lambda x: x * 1.18)` to apply a tax rate.

3.7.9 Recursive Functions

A recursive function is a function that calls itself, either directly or indirectly, in order to solve a problem by breaking it down into smaller instances of the same problem. Every correctly designed recursive function requires two essential components: a base case, which defines the simplest instance of the problem and terminates the recursion, and a recursive case, which reduces the problem toward the base case.

```
def factorial(n):
    if n == 0 or n == 1: # base case
        return 1
    return n * factorial(n - 1) # recursive case
print(factorial(5))
```

Expected Output:

120

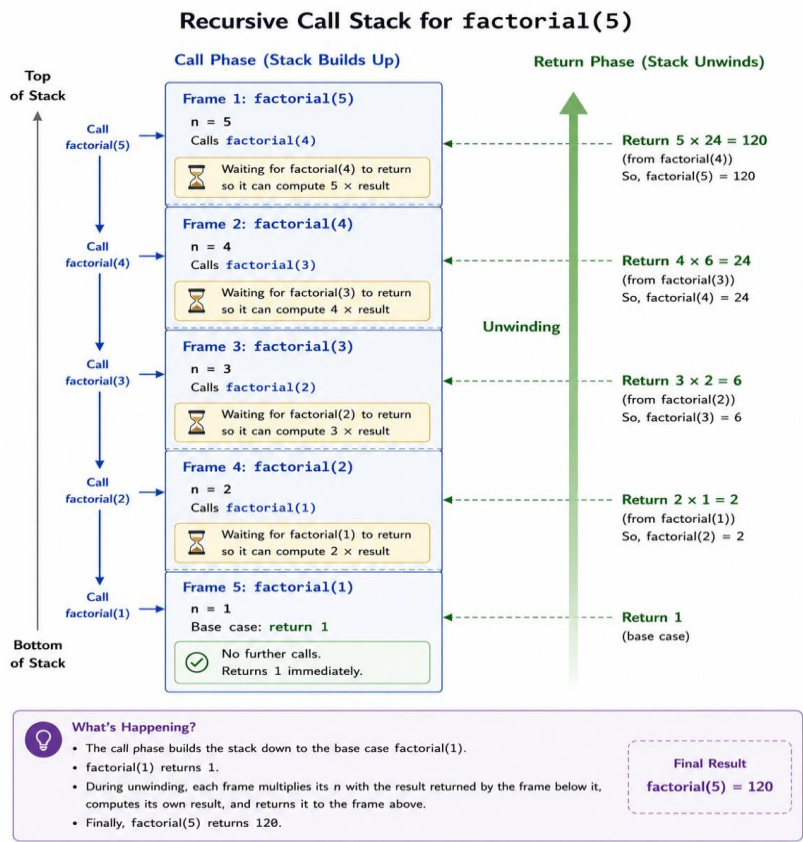


Figure 3.13. Recursive Function Execution

Aspect	Recursive Approach	Iterative Approach
Readability	Often more elegant for naturally recursive problems (trees, fractals)	Often more direct for simple repetition
Memory usage	Higher, due to the function call stack	Lower, generally constant
Risk	Stack overflow if recursion depth is excessive	Infinite loop if the terminating condition is flawed
Typical ML application	Traversing decision tree nodes	Iterating over training epochs or dataset batches

Table 3.7. Recursive vs Iterative Functions

Debugging Tip: If a recursive function raises a `RecursionError`, check first whether the base case can actually be reached from every possible input, since a base case that is unreachable for certain inputs is the most common cause of runaway recursion.

3.7.10 Built-in Functions and User-defined Functions

Python provides a substantial library of built-in functions — such as `len()`, `sum()`, `sorted()`, `map()`, `filter()`, `zip()`, and `enumerate()` — that are always available without any import statement. A user-defined function, by contrast, is one written by the programmer using the `def` keyword to address a specific need not already covered by the standard library.

Function	Purpose
<code>len()</code>	Returns the number of elements in a sequence or collection
<code>sum()</code>	Returns the sum of numeric values in an iterable
<code>sorted()</code>	Returns a new sorted list from an iterable
<code>map()</code>	Applies a function to every element of an iterable
<code>filter()</code>	Retains only the elements of an iterable that satisfy a condition
<code>zip()</code>	Combines multiple iterables element-wise into tuples
<code>enumerate()</code>	Pairs each element of an iterable with its index

Table 3.8. Common Built-in Functions Useful in Data Processing

3.7.11 Documentation Strings (Docstrings)

A docstring is a string literal placed as the very first statement inside a function, class, or module, describing its purpose, parameters, and return value. Docstrings are accessible at runtime through the built-in `help()` function and the `__doc__` attribute, and are the foundation of automatically generated documentation tools such as Sphinx.

```
def min_max_scale(value, min_val, max_val):
    """
    Scale a numeric value to the range [0, 1].
    Parameters:
    value (float): The raw value to scale.
    min_val (float): The minimum value in the dataset.
    max_val (float): The maximum value in the dataset.
    Returns:
    float: The scaled value between 0 and 1.
    """
```

```
return (value - min_val) / (max_val - min_val)
```

3.7.12 Type Hints (Introduction)

Type hints, introduced in Python 3.5 and progressively enhanced in later versions, allow a programmer to indicate the expected data types of function parameters and return values. Although Python remains a dynamically typed language and does not enforce type hints at runtime, they greatly improve code readability and allow static analysis tools such as mypy to detect type-related bugs before the code is ever executed.

```
def compute_average(values: list[float]) -> float:
    return sum(values) / len(values)
```

Industry Insight: *Most professional data science and machine learning codebases now use type hints extensively, since large pipelines involving DataFrames, arrays, and custom configuration objects become significantly harder to maintain without a clear, explicit record of expected data types.*

3.7.13 Practical Programming Examples

Example 3.17: A Reusable Data Preprocessing Function

```
def clean_dataset(records: list) -> list:
    """Remove None values and negative numbers from a
    list."""
    cleaned = []
    for value in records:
        if value is None:
            continue
        if value < 0:
            continue
        cleaned.append(value)
    return cleaned
raw = [12, -5, None, 8, None, 21, -3]
print(clean_dataset(raw))
```

Expected Output:

```
[12, 8, 21]
```

Example 3.18: A Simple Hyperparameter Grid Iteration Function

```
def evaluate_configs(learning_rates, batch_sizes):
```



```
"""Simulate evaluating every combination of
hyperparameters."""
results = []
for lr in learning_rates:
    for bs in batch_sizes:
        score = round(1 / (lr * 1000) + bs * 0.001, 4) #
        dummy metric
    results.append({"lr": lr, "batch_size": bs, "score":
score})
return results
for config in evaluate_configs([0.01, 0.001], [16,
32]):
    print(config)
```

Expected Output:

```
{'lr': 0.01, 'batch_size': 16, 'score': 0.116}
{'lr': 0.01, 'batch_size': 32, 'score': 0.132}
{'lr': 0.001, 'batch_size': 16, 'score': 1.016}
{'lr': 0.001, 'batch_size': 32, 'score': 1.032}
```

Practical Use Case: This function demonstrates the essential structure of grid search, the most fundamental hyperparameter tuning technique in machine learning, in which nested loops systematically evaluate every combination of candidate hyperparameter values.

3.7.14 Section Summary

- Functions encapsulate reusable logic behind a named interface, improving modularity, maintainability, and testability.
- Python supports positional, keyword, default, and variable-length (*args, **kwargs) parameters, offering great flexibility in how functions are called.
- Variable scope distinguishes local variables, confined to a function, from global variables, accessible throughout a module.
- Lambda functions provide concise, anonymous functions for short, inline operations; recursive functions solve problems by reducing them to smaller instances of themselves.

- Docstrings and type hints are essential professional practices that improve the readability, documentation, and reliability of function-based code.

3.8 Chapter Summary

This chapter has established the core control structures and modularity tools that transform Python from a language capable of executing a fixed sequence of statements into one capable of expressing genuinely intelligent, adaptive logic. Conditional statements — if, if-else, and if-elif-else — allow a program to select among alternative execution paths, and nested conditions allow hierarchical decisions to be expressed cleanly, provided nesting depth is kept under control. Loops, in the form of the while and for constructs, enable repeated execution and are indispensable for processing the large, iterative structures that characterize data science and machine learning workloads, from dataset rows to training epochs. Loop control statements — break, continue, and pass — provide fine-grained control over the flow of iteration. The pattern-programming exercises in Section 3.5, although not directly applicable to production systems, build essential fluency with nested loops and index arithmetic that underpins far more complex algorithms encountered later in the curriculum, including image processing and dynamic programming.

Exception handling equips programs to anticipate and gracefully recover from runtime errors — an essential capability when working with real-world, imperfect data drawn from files, APIs, sensors, and user input. Finally, functions provide the mechanism of modularity that allows complex systems to be decomposed into smaller, reusable, independently testable units, supported by flexible parameter-passing mechanisms, well-defined variable scope, and professional practices such as docstrings and type hints. Together, these tools form the essential toolkit with which subsequent chapters will build increasingly sophisticated data structures, file-processing routines, and, ultimately, machine learning pipelines.

3.8.1 Key Takeaways

- Decision-making constructs (if, if-elif-else, nested conditions) allow programs to branch based on Boolean logic.
- Loops (for, while) enable efficient repetition, which is foundational to processing datasets and training models.
- break, continue, and pass provide precise control over loop execution flow.
- Pattern programs build critical intuition for nested iteration and index-based logic.

- try-except-else-finally and the raise statement allow robust, graceful handling of runtime errors.
- Functions, parameters (positional, keyword, default, *args, **kwargs), scope, lambda expressions, and recursion together constitute Python's modularity toolkit.
- Docstrings and type hints are professional standards that improve the clarity and reliability of function-based code.

3.8.2 Important Terms

- Conditional Statement — A construct that executes a block of code only if a specified Boolean condition is satisfied.
- Iteration — The repeated execution of a block of statements, controlled by a loop.
- Iterable — Any object capable of returning its elements one at a time, such as a list, tuple, or string.
- Exception — An event, detected during execution, that disrupts the normal flow of a program's instructions.
- Function — A named, reusable block of code that performs a specific task and may accept parameters and return a value.
- Scope — The region of a program within which a particular variable name is accessible.
- Recursion — A technique in which a function solves a problem by calling itself on smaller instances of that problem.
- Docstring — A string literal used to document the purpose and usage of a function, class, or module.

3.8.3 Glossary

- *args — Syntax used in a function definition to accept a variable number of positional arguments as a tuple.
- **kwargs — Syntax used in a function definition to accept a variable number of keyword arguments as a dictionary.
- Base Case — The terminating condition of a recursive function that stops further recursive calls.

- Lambda Function — A small, unnamed function restricted to a single expression, defined using the lambda keyword.
- Type Hint — An annotation indicating the expected data type of a variable, parameter, or return value.

3.9 Exercises

The following exercises progress from foundational recall questions to advanced programming problems. Students are encouraged to attempt every question independently before consulting the solutions provided by the instructor.

3.9.1 Section A: Multiple Choice Questions

2. Which keyword is used to test an alternative condition after an initial if statement fails in Python? (a) elseif (b) elif (c) else if (d) elsif
3. What is the output of: `for i in range(2, 10, 3): print(i, end=' ')` (a) 2 5 8 (b) 2 4 6 8 (c) 2 3 4 5 6 7 8 9 (d) 2 5 8 11
4. Which statement immediately terminates the innermost enclosing loop? (a) continue (b) pass (c) break (d) exit
5. Which of the following correctly catches a division-by-zero error? (a) except DivisionError (b) except ZeroDivisionError (c) except MathError (d) except ValueError
6. Which syntax allows a function to accept an arbitrary number of keyword arguments? (a) *args (b) **kwargs (c) &kwargs (d) #kwargs
7. A variable defined inside a function, inaccessible outside it, is called a: (a) global variable (b) local variable (c) static variable (d) class variable
8. Every correctly designed recursive function must contain a: (a) loop (b) base case (c) lambda expression (d) global variable

3.9.2 Section B: Fill in the Blanks

8. The _____ block in exception handling always executes, regardless of whether an exception occurred.
9. The _____ statement skips the remainder of the current loop iteration and proceeds to the next one.
10. A function without an explicit return statement implicitly returns _____.

11. The _____ function converts an iterable into a sequence of (index, value) pairs.
12. A(n) _____ function is an anonymous function defined using the lambda keyword.

3.9.3 Section C: True/False

13. The condition of a while loop is evaluated after the loop body executes at least once. (True/False)
14. The is operator should be used to compare two numeric values for equality. (True/False)
15. A finally block executes only when no exception has occurred. (True/False)
16. Default-valued parameters must be listed after non-default parameters in a function definition. (True/False)
17. The pass statement causes a loop to terminate immediately. (True/False)

3.9.4 Section D: Match the Following

Match each construct in Column A with its correct description in Column B.

18. Column A: (1) break (2) continue (3) pass (4) raise (5) finally
19. Column B: (a) Deliberately triggers an exception (b) Always executes after a try block (c) Skips the current iteration (d) Performs no operation (e) Terminates the enclosing loop

3.9.5 Section E: Short Answer Questions

20. Explain the difference between the break and continue statements with a suitable example.
21. Differentiate between positional arguments, keyword arguments, and default arguments.
22. What is the purpose of the else clause in a try-except construct?
23. Explain the difference between local and global variable scope with an example.
24. Why are lambda functions commonly used with the sorted() function?

3.9.6 Section F: Long Answer Questions

25. Describe, with examples, how nested loops are used to process two-dimensional data structures such as matrices or images.

26. Discuss the advantages and limitations of recursive functions compared to iterative solutions, with reference to a factorial or Fibonacci example.
27. Explain the complete exception-handling workflow in Python, describing the role of try, except, else, finally, and raise.
28. Discuss how conditional statements, loops, and functions are used together in a typical data preprocessing pipeline.

3.9.7 Section G: Predict the Output

29. Predict the output of the following code:

```
total = 0
for i in range(1, 5):
    if i == 3:
        continue
    total += i
print(total)
```

30. Predict the output of the following code:

```
def f(a, b=10):
    return a + b
print(f(5), f(5, 20))
```

31. Predict the output of the following code:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

3.9.8 Section H: Debug the Code

32. Identify and correct the error in the following code:

```
def greet(name)
print("Hello, " + name)
greet("Ravi")
```

33. Identify and correct the error in the following code:

```
count = 0
while count < 5
print(count)
count += 1
```

34. Identify and correct the logical error that causes this loop to run forever:

```
n = 10
while n > 0:
print(n)
n = n + 1
```

3.9.9 Section I: Programming Exercises

- 35. Write a function that accepts a list of numbers and returns a new list containing only the even numbers, using a for loop and a conditional statement.
- 36. Write a program using nested loops to print a hollow square pattern of a size specified by the user.
- 37. Write a function using *args that accepts any number of numeric arguments and returns their average.
- 38. Write a program that reads a list of file names and, using exception handling, reports which files exist and which do not (you may simulate file existence with a predefined list).
- 39. Write a recursive function to compute the nth Fibonacci number, and compare its output with an iterative version for n = 10.

3.9.10 Section J: Pattern Programming Exercises

- 40. Write a program to print a hollow diamond pattern for a user-specified size.
- 41. Write a program to print Pascal's Triangle for the first eight rows.
- 42. Write a program to print a number pyramid in which each row contains the row number repeated an increasing number of times.

3.9.11 Section K: Exception Handling Programs

- 43. Write a program that repeatedly prompts the user for an integer until valid input is provided, using a try-except block inside a loop.

44. Define a custom exception called `InsufficientBalanceError` and use it within a simple `withdraw()` function that raises the exception when the requested amount exceeds the available balance.

3.9.12 Section L: Function-based Programming Problems

45. Write a function with type hints that accepts a list of floating-point numbers and returns a tuple containing the mean, minimum, and maximum values.
46. Write a function using `**kwargs` that builds and returns a dictionary representing a machine learning model's configuration, printing a warning for any unrecognized keys.

3.9.13 Section M: Analytical Questions

47. Analyze why explicit for loops over large NumPy arrays are generally discouraged in production data science code, and explain what alternative approach is typically recommended.
48. Analyze a scenario in which deeply nested conditional statements could be refactored into a set of smaller functions, and explain the resulting benefits.

3.9.14 Section N: Mini Project Ideas

49. Build a command-line 'Data Quality Checker' that reads a list of records (simulated as Python dictionaries), uses loops and conditional statements to detect missing or out-of-range values, and uses functions to modularize each validation rule.
50. Build a simple 'Grade Book Analyzer' that accepts student scores through a loop, uses functions to compute the class average and highest scorer, and uses exception handling to gracefully manage invalid score entries.
51. Build a 'Pattern Gallery' program that presents a menu of at least six pattern types from Section 3.5, allows the user to select one and specify its size, and uses a dedicated function for each pattern.

Remember: *Attempting every exercise by hand-tracing the logic before running the code on a computer is the single most effective habit for mastering control structures and functions.*

Chapter 4: Object-Oriented Programming

4.1 User-defined Functions

Chapter 3 introduced user-defined functions as a mechanism for grouping a sequence of statements under a single name so that the logic can be reused without being rewritten. Before moving into Object-Oriented Programming (OOP), it is useful to briefly revisit this idea, because functions are the conceptual building blocks from which methods — the functions that live inside a class — are constructed. Everything a student already knows about defining a function with the `def` keyword, passing arguments, returning values, and using default or keyword arguments continues to apply once those functions are placed inside a class body.

Function reuse is one of the central motivations for modular programming: instead of duplicating a block of code in multiple places, a programmer isolates the logic in a function and calls it wherever it is needed. Object-Oriented Programming extends this idea one step further by bundling functions together with the data they operate on, so that both the behaviour (functions) and the state (variables) of a real-world entity are represented in a single unit called a class. In this sense, a class can be thought of as a structured collection of related functions and variables designed to model one coherent concept, such as a `Student`, a `BankAccount`, or a `MachineLearningModel`.

The example below revisits a simple standalone function and then shows, informally, how the same logic will later be reorganised as a method inside a class in Section 4.6. This comparison helps students see the continuity between what they already know and what is about to be introduced.

```
# A plain, standalone function (Chapter 3 style)
def calculate_bmi(weight_kg, height_m):
    """Return the Body Mass Index for given weight and
    height."""
    return weight_kg / (height_m ** 2)
bmi_value = calculate_bmi(68, 1.75)
print(f"BMI: {bmi_value:.2f}")
```

Expected Output: BMI: 22.20

Remember: *A function that will later become part of a class does not change in its internal logic — only its context changes. Understanding this continuity makes the transition to OOP far less intimidating.*

4.2 Lambda Functions (Brief Review)

A lambda function, also called an anonymous function, is a small, single-expression function defined without the `def` keyword, using the syntax `lambda arguments: expression`. Chapter 3 explored lambda functions in the context of general Python programming, most commonly alongside built-in higher-order functions such as `map()`, `filter()`, and `sorted()`. In this chapter, lambda expressions reappear briefly wherever they help express concise behaviour inside classes — for example, as default sorting keys for a collection of objects, or as short callback-style attributes assigned to an instance.

```
students = [("Asha", 82), ("Rahul", 91), ("Meera", 76)]

# Sorting a list of tuples by score using a lambda expression
students_sorted = sorted(students, key=lambda record:
                          record, reverse=True)
print(students_sorted)

Expected Output: [('Rahul', 91), ('Asha', 82),
                  ('Meera', 76)]
```

In Data Science pipelines, lambda functions are frequently used inside pandas operations such as `DataFrame.apply(lambda row: ...)` to transform columns or rows in a single, compact expression, and this usage will resurface when object-oriented dataset wrapper classes are discussed later in the chapter.

4.3 Recursion (Brief Review)

Recursion, introduced in Chapter 3, is a technique in which a function calls itself to solve a smaller instance of the same problem. A recursive function must always define a base case, which terminates the recursive calls, and a recursive case, which reduces the problem toward the base case. Recursion will reappear in this chapter in two specific contexts: first, when methods inside a class need to process nested or hierarchical data (for example, computing the depth of a tree-like folder structure represented as an object); and second, when discussing recursive relationships between classes, such as a `Node` class that contains a reference to another `Node`.

```
def factorial(n):
    if n in (0, 1): # Base case
        return 1
    return n * factorial(n - 1) # Recursive case

print(factorial(5))
```

Expected Output: 120

Best Practice: *When a recursive method is defined inside a class, always verify the base case first, since an incorrect or missing base case inside an object's method is harder to trace than in a standalone function because it may be buried within a larger program.*

4.4 Modules

4.4.1 Introduction

A module in Python is simply a file containing Python code — definitions of functions, classes, and variables — that can be imported and reused in other programs. Modules are the primary mechanism Python provides for organising code into logically related, reusable units. As programs grow larger, particularly in Data Science and Machine Learning projects that may contain thousands of lines of code across data loading, preprocessing, model training, and evaluation stages, the ability to separate this code into well-named modules becomes essential for maintainability.

4.4.2 Need for Modules

Without modules, all code for a project would need to reside in a single file, making it difficult to navigate, test, and reuse. Modules address this problem by allowing a programmer to group related functionality — for instance, all functions related to data cleaning in one module and all functions related to visualisation in another — and then import only what is required in a given script. This separation also enables multiple developers to work on different modules of the same project simultaneously without conflicts.

- Improved code organisation and readability
- Reusability of code across multiple programs
- Easier debugging and maintenance, since related logic is isolated
- Avoidance of naming conflicts through separate namespaces
- Collaborative development, where team members own different modules

4.4.3 Creating User-defined Modules

Any Python file with a .py extension can be treated as a module. To create a user-defined module, a programmer simply writes functions, classes, or variables in a file and saves it; that file can then be imported by name (without the .py extension) from another file

located in the same directory or in a directory listed on the Python module search path (sys.path).

```
# File: geometry_utils.py
PI = 3.14159

def circle_area(radius):
    return PI * radius ** 2

def circle_circumference(radius):
    return 2 * PI * radius

# File: main.py (in the same folder as
# geometry_utils.py)
import geometry_utils

print(geometry_utils.circle_area(5))
print(geometry_utils.circle_circumference(5))

Expected Output: 78.53975 and 31.4159
```

4.4.4 Import Statement

The import statement is the standard way to bring an entire module into the current namespace. Once imported, every function, class, or variable defined in that module is accessed using the dotted notation module_name.identifier, as demonstrated above with geometry_utils.circle_area(). This form of importing avoids polluting the current namespace, since the module's contents remain grouped under its own name.

4.4.5 from...import Statement

The from...import statement allows specific identifiers to be imported directly into the current namespace, removing the need for the dotted prefix. While convenient, this form should be used carefully, since importing many names directly can lead to naming collisions between modules.

```
from geometry_utils import circle_area

print(circle_area(10)) # No need for the
# geometry_utils. prefix
```

4.4.6 import ... as Statement

Python allows a module or an imported identifier to be given an alias using the as keyword. This is extremely common in Data Science, where libraries such as NumPy and

pandas are conventionally aliased to shorter names to reduce typing throughout a large program.

```
import numpy as np
import pandas as pd

array = np.array([1, 2, 3, 4])
print(array.mean())
```

4.4.7 Built-in Modules

Python ships with an extensive standard library of built-in modules that provide ready-made functionality without requiring any external installation. The following subsections briefly introduce several modules that are especially relevant for Data Science applications.

4.4.7.1 math Module

The math module provides access to standard mathematical functions and constants, such as square roots, trigonometric functions, logarithms, and the constants pi and e, all of which are frequently used in statistical and scientific computations.

```
import math

print(math.sqrt(64)) # 8.0
print(math.log(100, 10)) # 2.0
print(math.pi) # 3.141592653589793
```

4.4.7.2 random Module

The random module supports the generation of pseudo-random numbers, random sampling, and shuffling, which are essential in tasks such as splitting a dataset into training and testing subsets, or initialising weights in a simple machine learning model.

```
import random

random.seed(42)
print(random.randint(1, 100))
print(random.sample(range(1, 50), 5))
```

4.4.7.3 statistics Module

The statistics module offers functions for calculating common descriptive statistics such as mean, median, mode, variance, and standard deviation directly on Python sequences, which is useful for quick exploratory analysis without requiring external libraries.

```
import statistics as stats

marks = [78, 85, 92, 66, 74, 85]
print(stats.mean(marks))
print(stats.median(marks))
print(stats.stdev(marks))
```

4.4.7.4 datetime Module

The datetime module allows a program to work with dates and times, which is essential in Data Science tasks involving time-series data, log analysis, and any dataset containing timestamps.

```
from datetime import datetime

now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

4.4.7.5 4.4.7.5 os Module

The os module provides a way of interacting with the operating system, including reading and writing file paths, listing directory contents, and checking whether a file exists — operations that are common when loading datasets from disk.

```
import os

print(os.getcwd()) # Current working directory
print(os.listdir('.')) # Files in the current
directory
print(os.path.exists('data.csv'))
```

4.4.7.6 sys Module

The sys module provides access to variables and functions that interact closely with the Python interpreter itself, such as command-line arguments, the module search path, and the Python version currently in use.

```
import sys

print(sys.version)
print(sys.path[:2])
```

4.4.8 Using Modules in Data Science Applications

In practice, a Data Science project is typically organised as a small collection of user-defined modules — for instance, `data_loader.py`, `preprocessing.py`, `visualization.py`, and

model.py — each imported into a central main.py or a Jupyter notebook. This modular organisation mirrors, at a small scale, the structure that will be formalised further using packages in Section 4.5 and using classes in Section 4.6.

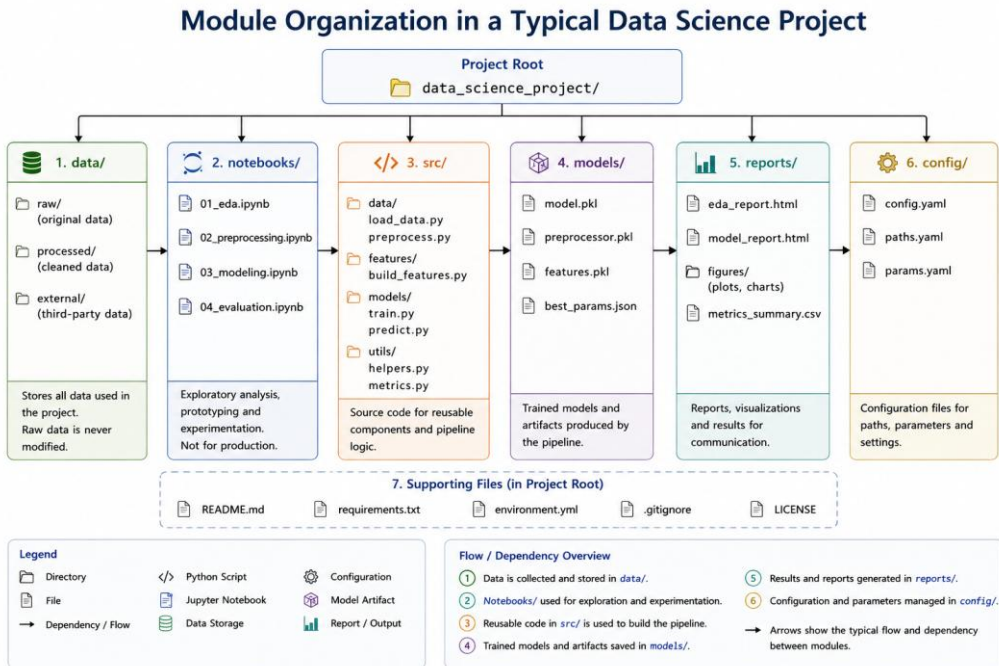


Figure 4.5. Module Organization in a Typical Data Science Project

4.4.9 Best Practices

- Give modules short, lowercase, descriptive names (e.g., data_loader.py, not DL.py)
- Avoid using from module import * because it obscures where each name originates
- Group related functionality into a single module rather than scattering it across many small files
- Include a module-level docstring describing the module's purpose
- Keep modules free of code that executes automatically on import unless guarded by if __name__ == '__main__':

Common Mistake: A frequent beginner error is naming a personal script the same as a standard library or third-party module (for example, naming a file random.py), which

causes Python to import the local file instead of the intended library, leading to confusing errors.

Table 4.3 Modules vs Packages

Aspect	Module	Package
Definition	A single.py file A	directory containing multiple modules
Requires __init__.py	No	Yes (or implicit namespace package)
Granularity	Fine-grained (one file)	Coarse-grained (collection of files)
Example	math.py	numpy (a directory of many modules)

4.5 Packages

4.5.1 Introduction

A package is a way of organising related modules together inside a directory hierarchy, allowing large projects and libraries to be structured in a coherent and navigable way. Where a module is a single file, a package is a folder that groups multiple modules — and possibly sub-packages — under a common namespace. Every major Data Science library that students will use throughout this book, including NumPy, pandas, matplotlib, and scikit-learn, is itself distributed as a package.

4.5.2 Need for Packages

As a project grows beyond a handful of modules, placing every module in a single flat directory becomes unwieldy. Packages solve this by allowing modules to be nested inside sub-directories that reflect their logical grouping — for example, separating a project's modules into preprocessing, models, and visualization sub-packages, each containing several related modules.

4.5.3 Package Structure

A typical package structure consists of a top-level directory bearing the package name, an `__init__.py` file that marks the directory as a package, and one or more module files or sub-package directories inside it.

```
ml_toolkit/  
  __init__.py  
  preprocessing/  
    __init__.py  
    cleaning.py  
    scaling.py  
  models/  
    __init__.py  
    linear_model.py
```



```
tree_model.py
visualization/
__init__.py
plots.py
```

Package Directory Structure of a Sample Data Science Toolkit

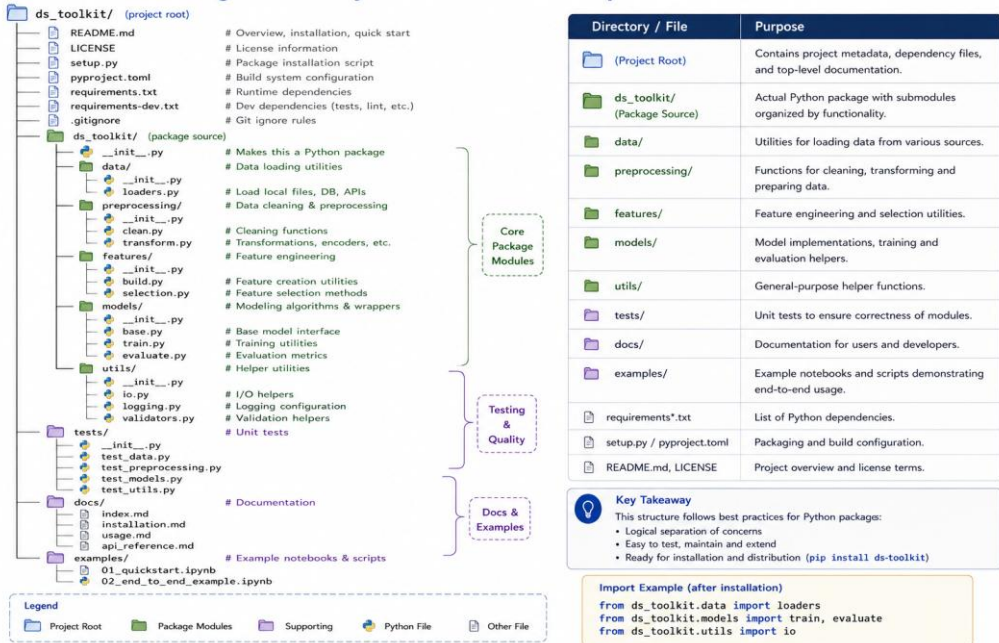


Figure 4.6. Package Directory Structure of a Sample Data Science Toolkit

4.5.4 __init__.py

The `__init__.py` file signals to Python that a directory should be treated as a package. It may be left empty, or it may contain initialisation code that runs when the package is first imported, such as importing selected functions from its sub-modules so that they are available directly from the package's top level.

```
# ml_toolkit/preprocessing/__init__.py
from.cleaning import remove_nulls
from.scaling import min_max_scale

# This allows: from ml_toolkit.preprocessing import
remove_nulls
```

4.5.5 Importing Packages

```
from ml_toolkit.preprocessing import cleaning
from ml_toolkit.models.linear_model import
LinearRegressionScratch
```

```
cleaning.remove_nulls(dataset)
```

4.5.6 Creating Packages

To create a package, a programmer creates a directory, adds an `__init__.py` file (even if empty, for older tooling compatibility), and places related module files inside it. The package can then be imported like any built-in package as long as it resides on the Python path, for example in the same directory as the script or installed via pip in editable mode.

4.5.7 Installing Third-party Packages

4.5.7.1 pip Package Manager

pip is Python's standard package manager, used to install, upgrade, and remove third-party packages from the Python Package Index (PyPI). It is the tool through which virtually every external Data Science library reaches a student's machine.

```
pip install numpy pandas matplotlib scikit-learn
pip install pandas==2.2.0 # install a specific
version
pip show pandas # display package information
pip freeze > requirements.txt # export installed
packages
```

4.5.7.2 Popular Packages for Data Science

Popular Third-party Packages Used in Data Science and Machine Learning

Package	Primary Purpose
NumPy	Numerical computing and array operations
pandas	Data manipulation and analysis using DataFrames
matplotlib / seaborn	Data visualisation and statistical plotting
scikit-learn	Classical machine learning algorithms
TensorFlow / PyTorch	Deep learning and neural networks
SciPy	Scientific and statistical computing

4.5.8 Industrial Applications

In industry, packages are the standard unit of distribution for reusable code. Organisations commonly build internal packages that encapsulate proprietary data pipelines, machine learning models, or business logic, and distribute them across teams through private package repositories, mirroring the same `__init__.py` and directory-based structure introduced in this section.

4.6 Classes

4.6.1 Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm built around the idea of organising software as a collection of objects, each of which bundles together data (attributes) and behaviour (methods) that pertain to a single real-world or conceptual entity. Rather than writing a long sequence of instructions that operate on loosely related variables, OOP encourages a programmer to model the problem domain directly: a Student object holds a student's name and marks and knows how to compute its own average; a Dataset object holds raw data and knows how to clean and summarise itself.

This shift in thinking — from 'a sequence of steps' to 'a collection of interacting objects' — is what makes OOP particularly well suited to large, evolving software systems, including the Data Science and Machine Learning pipelines that this textbook is building toward.

4.6.2 Procedural vs Object-Oriented Programming

Procedural programming, which students have used throughout Chapters 1 to 3, organises a program as a sequence of functions that operate on data passed between them. Object-oriented programming instead organises a program around objects that own both their data and the functions that manipulate that data. Table 4.1 summarises the principal differences between the two paradigms.

Table 4.1 Procedural vs Object-Oriented Programming

Aspect	Procedural Programming	Object-Oriented Programming
Organising unit	Functions	Classes and objects
Data and behaviour	Kept separate	Bundled together (encapsulation)
Reusability	Through function calls	Through inheritance and composition
Real-world modelling	Indirect	Direct and intuitive
Data security	Data is often globally accessible	Data can be hidden via access modifiers
Scalability for large systems	Becomes difficult to manage	Designed to scale through modular objects

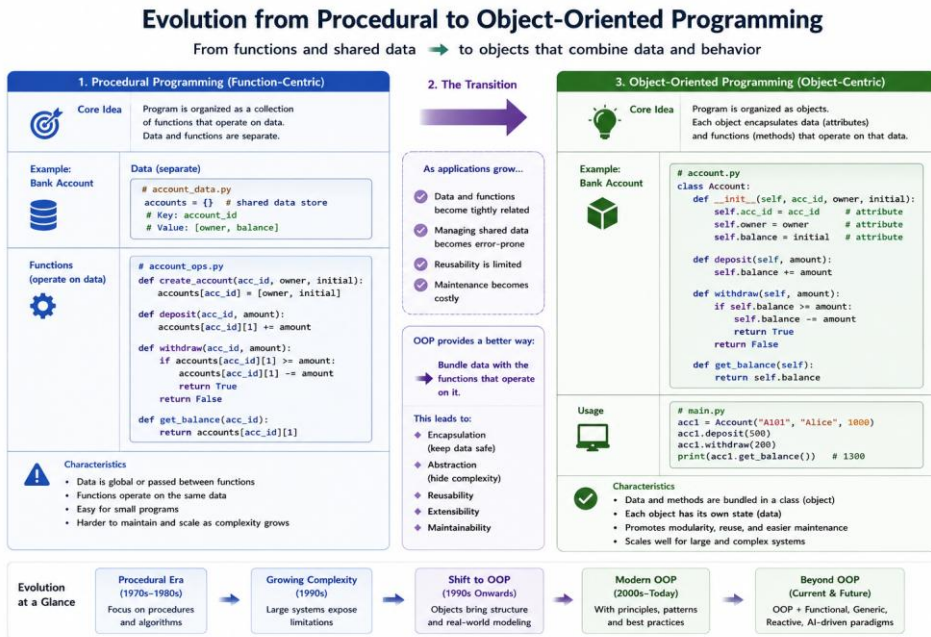


Figure 4.1. Evolution from Procedural to Object-Oriented Programming

4.6.3 Features of OOP

Object-Oriented Programming rests on four foundational principles, each of which receives a dedicated section later in this chapter: encapsulation (bundling data and methods, and controlling access to them), inheritance (deriving new classes from existing ones), polymorphism (allowing objects of different classes to be treated through a common interface), and abstraction (exposing only essential details while hiding implementation complexity). Together, these principles allow software built with OOP to remain modular, reusable, and maintainable as it grows.

4.6.4 Creating Classes

A class is defined using the class keyword followed by the class name, conventionally written in CapitalizedWords (PascalCase), and a colon. The body of the class contains attribute definitions and method definitions, indented in the same way as the body of a function or a loop.

```
class Student:
    """Represents a single student's academic record."""

    def __init__(self, name, roll_number, marks):
        self.name = name
```

```

self.roll_number = roll_number
self.marks = marks # a list of marks in different
subjects
def average_marks(self):
return sum(self.marks) / len(self.marks)

```

Structure of a Python Class

A class is a blueprint for creating objects. It contains attributes (data) and methods (functions).

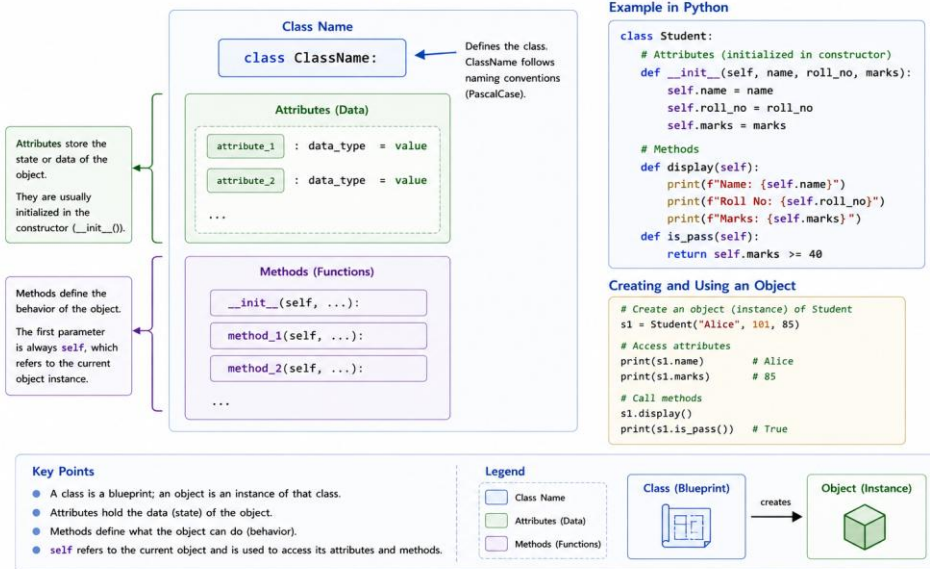


Figure 4.2. Structure of a Python Class (Class Name, Attributes, Methods)

4.6.5 Class Attributes and Instance Attributes

A class can define two kinds of attributes. Class attributes are defined directly inside the class body (outside any method) and are shared by every instance of the class unless explicitly overridden. Instance attributes, in contrast, are defined inside a method (typically `__init__`) using `self.attribute_name`, and each object gets its own independent copy.

```

class Student:
institution = "National Institute of Technology" #
class attribute (shared)

def __init__(self, name, marks):
self.name = name # instance attribute (unique per
object)
self.marks = marks

```

```
s1 = Student("Asha", [80, 90])
s2 = Student("Rahul", [70, 75])
print(s1.institution, s2.institution) # Same value
for both
print(s1.name, s2.name) # Different values
```

4.6.6 Constructors (__init__)

The `__init__` method, known as the constructor, is automatically invoked when a new object of a class is created. Its purpose is to initialise the instance attributes of the newly created object using the values passed at creation time. Every method defined inside a class, including `__init__`, receives the object itself as its first parameter, conventionally named `self`.

4.6.7 Methods and the `self` Keyword

A method is a function defined inside a class. The `self` parameter refers to the specific object on which the method was called, allowing the method to access and modify that object's own attributes. Python passes `self` automatically whenever a method is called using the dot notation `object.method()`, so it is never supplied explicitly by the caller.

```
s1 = Student("Asha", [80, 90])
print(s1.average_marks()) # Python translates this
to Student.average_marks(s1)
```

Table 4.2 Functions vs Methods

Aspect	Function	Method
Defined	Independently, outside any class	Inside a class body
First parameter	Any parameter, or none	<code>self</code> (instance) or <code>cls</code> (class method)
Called via	<code>function_name(args)</code>	<code>object.method_name(args)</code>
Access to object state	None, unless passed explicitly	Direct, via <code>self</code>

4.6.8 Class Variables vs Instance Variables

Table 4.4 Class Variables vs Instance Variables

Aspect	Class Variable	Instance Variable
Where declared	Directly inside class body	Inside a method, using <code>self</code> .
Memory	One copy shared by all objects	Separate copy for each object

Aspect	Class Variable	Instance Variable
Access	ClassName.var or self.var (read)	self.var
Typical use	Constants, counters shared across objects	Data unique to each object

4.6.9 Static Methods

A static method is a method that does not operate on a specific object or the class itself; it behaves like a regular function that has simply been placed inside a class for logical grouping. Static methods are declared using the `@staticmethod` decorator and do not take `self` or `cls` as their first parameter.

```
class MathUtils:
    @staticmethod
    def is_even(n):
        return n % 2 == 0

print(MathUtils.is_even(10)) # True, called without
                             # creating an object
```

4.6.10 Class Methods

A class method receives the class itself (conventionally named `cls`) as its first parameter, rather than an instance. Class methods are declared using the `@classmethod` decorator and are typically used to create alternative constructors or to modify class-level state.

```
class Student:
    total_students = 0

    def __init__(self, name):
        self.name = name
        Student.total_students += 1

    @classmethod
    def student_count(cls):
        return cls.total_students

Student("Asha"); Student("Rahul")
print(Student.student_count()) # 2
```

4.6.11 Magic (Dunder) Methods

Magic methods, also called dunder methods because their names begin and end with double underscores, allow a class to define how its objects behave with built-in Python

operations such as printing, addition, comparison, and length calculation. The `__init__` method encountered earlier is itself a magic method.

```
class Vector2D:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        return f"Vector2D({self.x}, {self.y})"

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

v1 = Vector2D(1, 2)
v2 = Vector2D(3, 4)
print(v1 + v2) # Uses __add__ then __str__: Vector2D(4, 6)
```

4.6.12 Practical Programming Example

Problem Statement: Design a class named `DatasetSummary` that stores a list of numeric observations and provides methods to compute the mean, minimum, and maximum values, illustrating how a simple data-analysis routine can be organised as a class rather than a set of loose functions.

```
class DatasetSummary:
    def __init__(self, values):
        self.values = values

    def mean(self):
        return sum(self.values) / len(self.values)

    def minimum(self):
        return min(self.values)

    def maximum(self):
        return max(self.values)

readings = DatasetSummary([12.5, 9.8, 15.2, 11.1,
14.0])
print(readings.mean(), readings.minimum(),
readings.maximum())
```

Expected Output: 12.52 9.8 15.2.

Common Errors: forgetting to include `self` as the first parameter of a method, or attempting to call a method on the class itself rather than on an object (e.g., `DatasetSummary.mean()` instead of `readings.mean()`).

Interview Question: *Why does every instance method in Python require `self` as its first parameter, even though it is never passed explicitly by the caller?*

Because Python also resolves `object.method(args)` internally as `ClassName.method(object,args)`, so `self` is how the method receives a reference to the specific object it was called on.

4.7 Objects

4.7.1 Introduction

An object is a concrete instance of a class — a specific realisation of the blueprint that the class defines, occupying its own space in memory and holding its own values for the instance attributes described in the class. If `Student` is the blueprint, then `s1 = Student('Asha', [80, 90])` creates one particular object built from that blueprint, distinct from any other `Student` object created afterwards.

4.7.2 Creating Objects

An object is created by calling the class name as though it were a function, supplying whatever arguments the `__init__` method requires (aside from `self`, which Python supplies automatically). This process is often referred to as instantiation.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
p1 = Point(2, 3) # instantiation: p1 is now an
object of class Point
p2 = Point(5, 7)
```

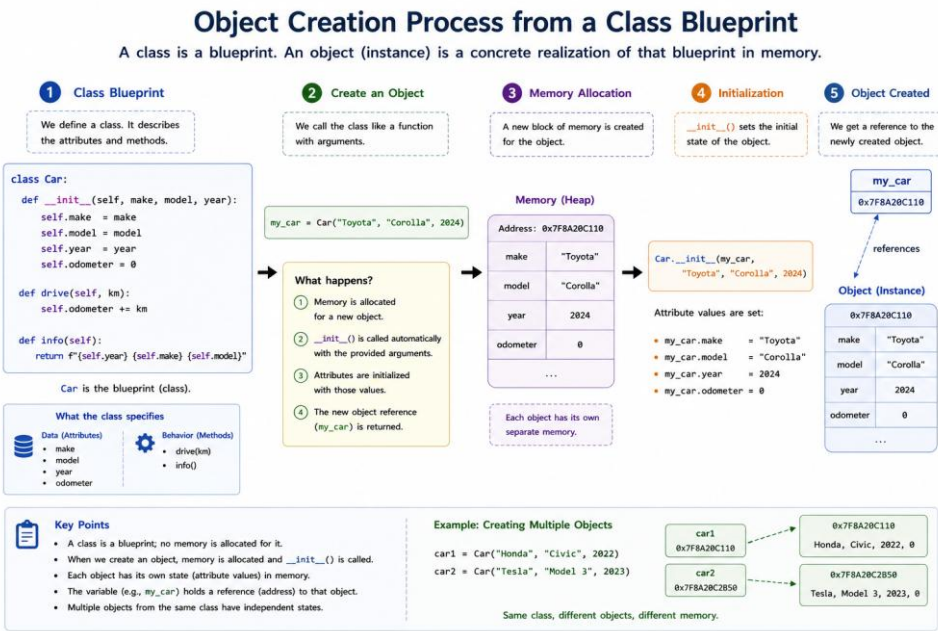


Figure 4.3. Object Creation Process from a Class Blueprint

4.7.3 Object Lifecycle

An object's lifecycle begins when it is instantiated (memory is allocated and `__init__` runs), continues while the object is used and possibly modified through its methods, and ends when the object is no longer referenced anywhere in the program, at which point Python's automatic garbage collector reclaims the memory it occupied. Python also supports an optional `__del__` method, sometimes called a destructor, which runs just before an object is garbage-collected, although explicit destructors are rarely needed in typical application code.

Constructor Execution and Object Lifecycle

From object creation to destruction

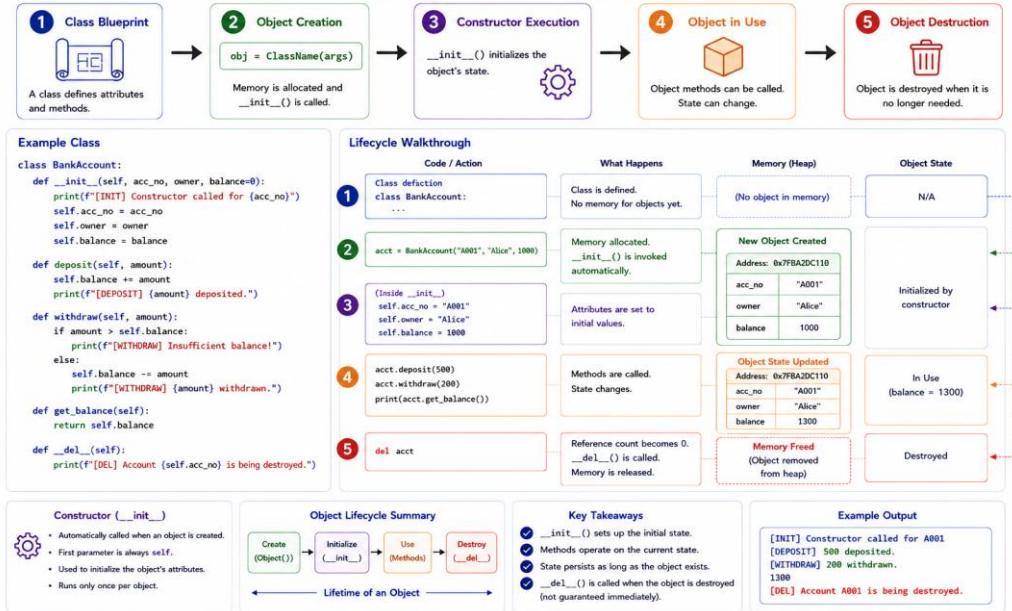


Figure 4.4. Constructor Execution and Object Lifecycle

4.7.4 Accessing Members

Once an object exists, its attributes and methods are accessed using the dot (.) operator: `object_name.attribute_name` for data and `object_name.method_name()` for behaviour. Attempting to access an attribute that has not been defined for that object raises an `AttributeError`.

```
print(p1.x, p1.y) # Accessing attributes: 2 3
p1.x = 10 # Modifying an attribute directly
print(p1.x) # 10
```

4.7.5 Object Interaction

Objects rarely exist in isolation; in a well-designed OOP program, objects interact with one another by calling each other's methods or by holding references to each other as attributes. For example, a `Classroom` object might hold a list of `Student` objects, and a method on `Classroom` might iterate over that list, calling `average_marks()` on each `Student` in turn.

```
class Classroom:
    def __init__(self, students):
        self.students = students # list of Student objects
```

```
def class_average(self):
    return sum(s.average_marks() for s in self.students)
    / len(self.students)

room = Classroom([Student("Asha", [80, 90]),
Student("Rahul", [70, 75])])
print(room.class_average())
```

4.7.6 Memory Concepts

In Python, a variable that refers to an object does not store the object itself but a reference (essentially, an address) to where the object resides in memory. Consequently, assigning one object variable to another (`b = a`) does not create a new object; both names refer to the same underlying object, and modifying one is visible through the other. The built-in `id()` function reveals the identity (memory reference) of an object, and the `is` operator compares whether two variables refer to the very same object rather than merely holding equal values.

```
a = Point(1, 1)
b = a # b refers to the SAME object as a
b.x = 99
print(a.x) # 99 -- changing b also changed a
print(a is b) # True
```

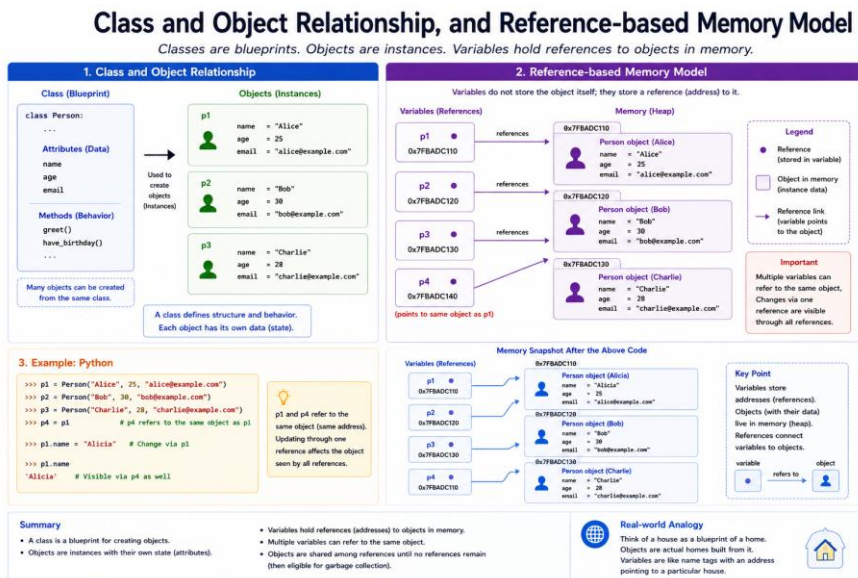


Figure 4.7. Class and Object Relationship, and Reference-based Memory Model

4.7.7 Real-world Examples

In a Hospital Management System, a Patient class would be instantiated once for every admitted patient, with each Patient object holding attributes such as name, age, and diagnosis, and methods such as `schedule_appointment()`. In a Recommendation System, each User and each Product might be represented as an object, with interaction data captured through references between them — foreshadowing the discussion of inheritance and polymorphism among related classes in Sections 4.8 and 4.9.

4.8 Inheritance

4.8.1 Introduction

Inheritance is the OOP mechanism that allows a new class, called a derived class or child class, to acquire the attributes and methods of an existing class, called a base class or parent class. The derived class can then add new attributes and methods of its own, or override the inherited ones to specialise their behaviour. Inheritance directly models the natural 'is-a' relationship found throughout real-world classification: a `SavingsAccount` is a `BankAccount`; a `ConvolutionalNeuralNetwork` is a `NeuralNetwork`.

4.8.2 Need for Inheritance

Without inheritance, closely related classes would need to duplicate common attributes and methods, leading to repeated code that must be updated in multiple places whenever the shared logic changes. Inheritance eliminates this duplication by letting related classes share a single, common implementation defined once in the base class.

- Code reuse: common logic is written once in the base class
- Extensibility: new derived classes can be added without modifying existing code
- Logical hierarchy: naturally models real-world categorisation
- Polymorphic behaviour: derived classes can be treated uniformly through the base class interface (explored further in Section 4.9)

4.8.3 Types of Inheritance

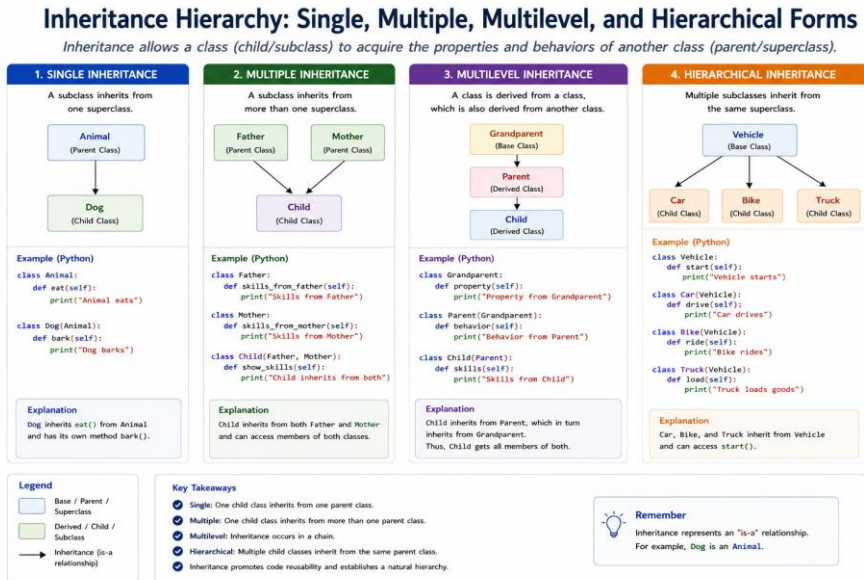


Figure 4.8. Inheritance Hierarchy Illustrating Single, Multiple, Multilevel, and Hierarchical Forms

4.8.3.1 Single Inheritance

In single inheritance, one derived class inherits from exactly one base class.

```

class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating.")

class Dog(Animal): # Single inheritance: Dog
    inherits from Animal
    def bark(self):
        print(f"{self.name} says Woof!")

d = Dog("Buddy")
d.eat() # Inherited method
d.bark() # Own method
          
```

4.8.3.2 Multiple Inheritance

In multiple inheritance, a derived class inherits from more than one base class simultaneously, combining attributes and methods from all of them.

```
class Flyer:
    def fly(self):
        print("Flying...")

class Swimmer:
    def swim(self):
        print("Swimming...")

class Duck(Flyer, Swimmer): # Multiple inheritance
    pass

donald = Duck()
donald.fly()
donald.swim()
```

4.8.3.3 Multilevel Inheritance

In multilevel inheritance, a class is derived from a class that is itself derived from another class, forming a chain of inheritance across multiple generations.

```
class Vehicle:
    def start(self):
        print("Vehicle starting...")

class Car(Vehicle):
    def drive(self):
        print("Car driving...")

class ElectricCar(Car): # Multilevel: ElectricCar ->
    Car -> Vehicle
    def charge(self):
        print("Charging battery...")

tesla = ElectricCar()
tesla.start(); tesla.drive(); tesla.charge()
```

4.8.3.4 Hierarchical Inheritance

In hierarchical inheritance, multiple derived classes inherit independently from a single common base class.

```
class Shape:
    def describe(self):
        print("I am a shape.")

class Circle(Shape):
    pass
```

```
class Square(Shape):
    pass

Circle().describe(); Square().describe() # Both
inherit independently
```

4.8.3.5 Hybrid Inheritance (Conceptual Overview)

Hybrid inheritance refers to any combination of two or more of the inheritance types described above within a single class hierarchy — for example, a hierarchy that is both multilevel and multiple. Python supports hybrid inheritance directly, but such designs should be approached carefully, since deeply combined hierarchies can become difficult to reason about; this concern is addressed further when Method Resolution Order is discussed below.

Table 4.5 Types of Inheritance

Type	Description	Example
Single	One child, one parent	Dog(Animal)
Multiple	One child, several parents	Duck(Flyer, Swimmer)
Multilevel	Chain across generations	ElectricCar(Car(Vehicle))
Hierarchical	Several children, one parent	Circle(Shape), Square(Shape)
Hybrid	Combination of the above	Mixed hierarchy of the above patterns

4.8.4 Method Overriding

Method overriding occurs when a derived class defines a method with the same name as one in its base class, thereby replacing (overriding) the inherited behaviour with a specialised implementation. This is one of the principal ways derived classes customise inherited behaviour.

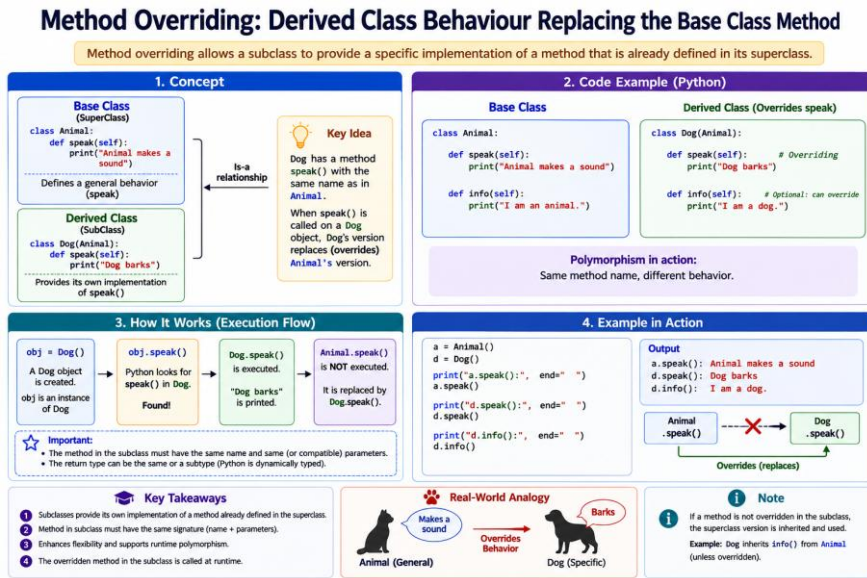


Figure 4.9. Method Overriding: Derived Class Behaviour Replacing the Base Class Method

```
class Animal:
    def make_sound(self):
        print("Some generic animal sound")

class Cat(Animal):
    def make_sound(self): # Overrides Animal.make_sound
        print("Meow")

Animal().make_sound() # Some generic animal sound
Cat().make_sound() # Meow
```

4.8.5 The super() Function

The `super()` function provides a way for a derived class to call a method from its base class, most commonly used inside an overridden `__init__` to ensure that base-class initialisation logic still runs before the derived class adds its own.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

class Manager(Employee):
    def __init__(self, name, salary, team_size):
        super().__init__(name, salary) # Reuse base class
        # initialisation
```

```
self.team_size = team_size

m = Manager("Priya", 95000, 8)
print(m.name, m.salary, m.team_size)
```

4.8.6 Advantages of Inheritance

- Eliminates duplicate code across related classes
- Encourages a clean, logical class hierarchy that mirrors the problem domain
- Makes it easy to extend a system with new, specialised classes
- Supports polymorphic treatment of related objects (Section 4.9)

4.8.7 Limitations of Inheritance

- Deep inheritance chains can become difficult to understand and debug.
- Tight coupling between base and derived classes: changes to the base class can unexpectedly affect all derived classes.
- Multiple inheritance can introduce ambiguity, resolved in Python through the Method Resolution Order (MRO), which programmers must understand to avoid unexpected behaviour.
- Inheritance is sometimes used where composition (an object holding another object as an attribute) would model the relationship more accurately.

4.8.8 Industrial Examples

Machine learning frameworks make extensive use of inheritance: in scikit-learn, every estimator class (such as a decision tree or a linear regression model) inherits from common base classes that provide a consistent `fit()` and `predict()` interface, allowing thousands of different algorithms to be used interchangeably by code that only knows about the shared base interface — a theme that connects directly to polymorphism, discussed next.

4.9 Polymorphism

4.9.1 Introduction

Polymorphism, from the Greek for 'many forms', is the OOP principle that allows objects of different classes to be treated through a shared interface, with each object responding to the same method call according to its own specific implementation. Polymorphism is what allows a single line of code, such as `shape.area()`, to work correctly regardless of

whether shape refers to a Circle, a Square, or a Triangle object, as long as each class implements its own `area()` method.

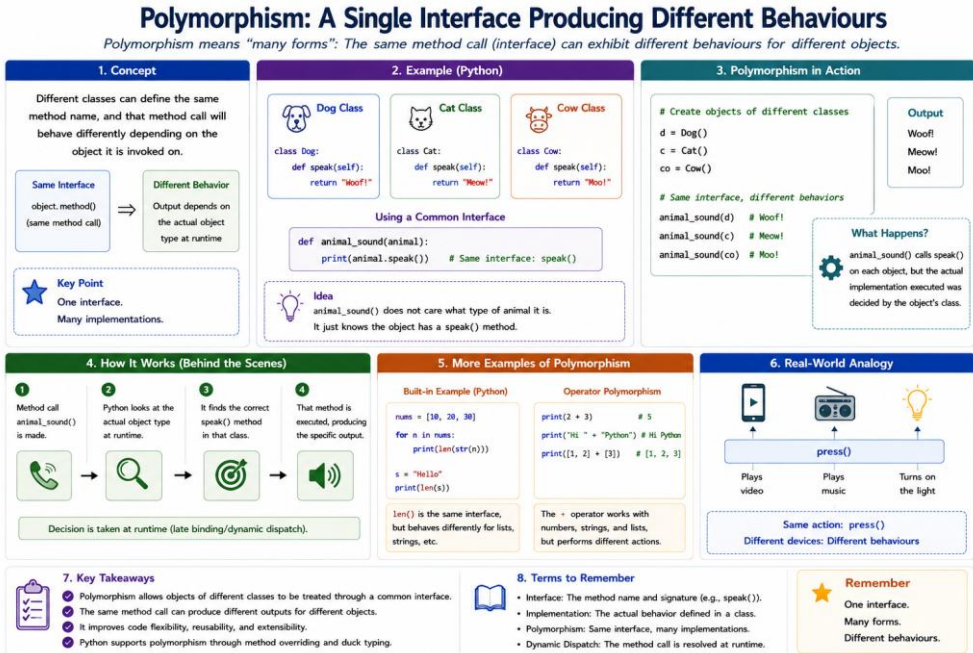


Figure 4.10. Polymorphism: A Single Interface Producing Different Behaviours

4.9.2 Method Overloading (Concept and Python Alternatives)

Method overloading — defining multiple methods with the same name but different parameter lists — is a feature of some languages such as Java and C++, but Python does not support true method overloading, because a later method definition with the same name simply replaces an earlier one within a class. Python programmers instead achieve similar flexibility using default arguments, variable-length arguments (`*args`, `**kwargs`), or explicit type checking within a single method.

```
class Calculator:
    def add(self, a, b=0, c=0): # Simulates overloading
                                # via default arguments
        return a + b + c

calc = Calculator()
print(calc.add(5))
print(calc.add(5, 10))
print(calc.add(5, 10, 15))
```

4.9.3 Method Overriding (Cross-reference)

Method overriding, introduced in Section 4.8.4, is itself a form of polymorphism: it allows a derived class to provide a specialised implementation of a method already defined in its base class, so that the same method call behaves differently depending on the actual object's class.

4.9.4 Duck Typing

Python embraces a dynamic style of polymorphism known as duck typing, summarised by the informal principle: 'If it walks like a duck and quacks like a duck, it is treated as a duck.' In practice, this means Python does not require objects to share a common base class in order to be used interchangeably — it only matters that they support the methods being called on them.

```
class Duck:
    def sound(self):
        return "Quack"

class Robot:
    def sound(self):
        return "Beep"

for entity in [Duck(), Robot()]: # Unrelated
    classes, same interface
    print(entity.sound())
```

4.9.5 Operator Overloading

Operator overloading allows built-in operators such as +, -, ==, and < to be given custom meaning for objects of a user-defined class, by implementing the appropriate magic method (introduced in Section 4.6.11), such as `__add__` for + or `__eq__` for ==.

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __str__(self):
        return f"₹{self.amount:.2f}"

wallet = Money(500) + Money(250)
print(wallet) # ₹750.00
```

4.9.6 Built-in Polymorphism

Python's own built-in functions and operators are polymorphic by design: the `+` operator adds numbers, concatenates strings, and merges lists, all through the same symbol; the `len()` function returns the length of a string, a list, a dictionary, or any user-defined object that implements `__len__`.

```
print(len("hello")) # 5
print(len([1, 2, 3, 4])) # 4
print(len({"a": 1, "b": 2})) # 2
```

4.9.7 Practical Example

Problem Statement: Model three shapes — Circle, Rectangle, and Triangle — each with its own `area()` method, and demonstrate polymorphism by computing the area of each shape through a single, uniform loop.

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

class Rectangle:
    def __init__(self, width, height):
        self.width, self.height = width, height

    def area(self):
        return self.width * self.height

class Triangle:
    def __init__(self, base, height):
        self.base, self.height = base, height

    def area(self):
        return 0.5 * self.base * self.height

shapes = [Circle(5), Rectangle(4, 6),
          Triangle(3, 8)]
for shape in shapes:
    print(f"{type(shape).__name__} area: {shape.area():.2f}")
```

4.9.8 Applications in Machine Learning Libraries

Polymorphism underlies the consistent `fit()` / `predict()` / `transform()` interface shared across scikit-learn's estimator classes: code written to train 'a model' works unchanged whether that model is a `LogisticRegression`, a `RandomForestClassifier`, or a `SVC`, because every estimator implements the same polymorphic interface — allowing algorithms to be swapped with a single line change during experimentation.

4.10 Encapsulation

4.10.1 Introduction

Encapsulation is the OOP principle of bundling an object's data together with the methods that operate on that data, while restricting direct access to some of that data from outside the object. Encapsulation protects an object's internal state from being modified in unexpected or invalid ways, and it allows the internal implementation of a class to change later without breaking code that uses the class, as long as the public interface remains stable.

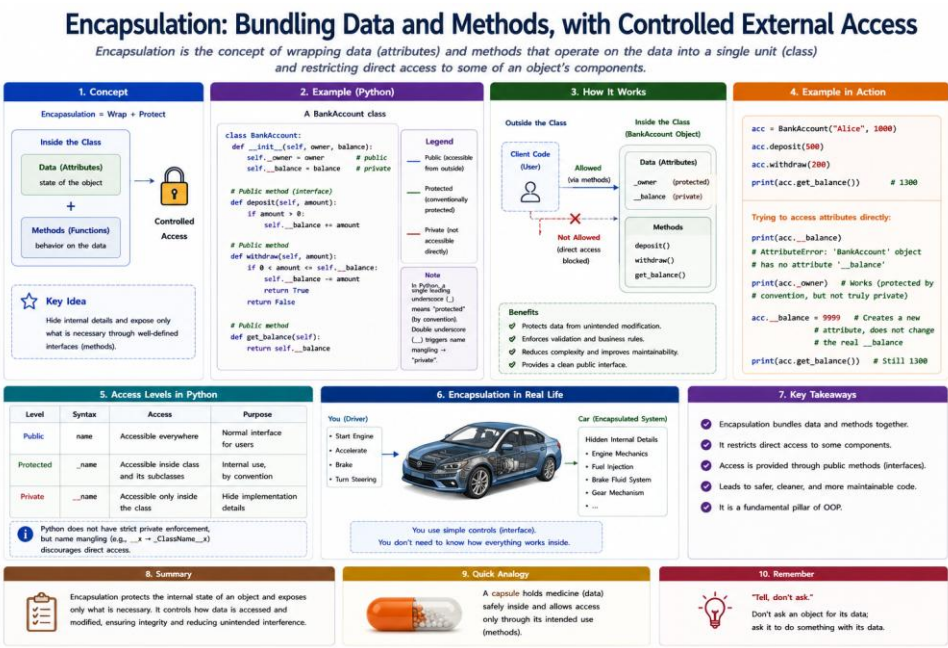


Figure 4.11. Encapsulation: Bundling Data and Methods, with Controlled External Access

4.10.2 Data Hiding

Data hiding refers to the practice of making an object's internal attributes inaccessible, or only indirectly accessible, from outside the class, so that the object's state can only be changed in controlled, validated ways through its methods.

4.10.3 Public, Protected, and Private Members

Python does not enforce strict access control the way languages such as Java do; instead, it relies on naming conventions that signal intent to other programmers, supplemented by a mechanism called name mangling for private attributes.

Table 4.6 Public vs Protected vs Private Members

Type	Naming Convention	Accessibility	Example
Public	normal_name	Accessible from anywhere	self.name
Protected	_single_leading_underscore	Accessible, but conventionally 'internal use'	self._balance
Private	__double_leading_underscore	Name-mangled; not directly accessible outside the class	self.__pin

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner # public
        self._branch = "Main Branch" # protected (by
        convention)
        self.__balance = balance # private (name-mangled)

acc = BankAccount("Asha", 5000)
print(acc.owner) # Works fine
print(acc._branch) # Works, but discouraged outside
the class
# print(acc.__balance) # Raises AttributeError
print(acc._BankAccount__balance) # Works only via
the mangled name
```

4.10.4 Getter and Setter Methods

Getter and setter methods provide controlled, indirect access to a private attribute: a getter returns the current value, and a setter updates it, typically after validating the new value.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance

    def get_balance(self): # Getter
        return self.__balance

    def set_balance(self, amount): # Setter, with
        validation
        if amount < 0:
            raise ValueError("Balance cannot be negative")
        self.__balance = amount
```

4.10.5 Property Decorators

The @property decorator offers a more Pythonic alternative to explicit getter and setter methods, allowing a method to be accessed using ordinary attribute syntax (without parentheses) while still running validation logic behind the scenes.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount < 0:
            raise ValueError("Balance cannot be negative")
        self.__balance = amount

acc = BankAccount("Asha", 5000)
print(acc.balance) # Calls the getter, no
parentheses needed
acc.balance = 6500 # Calls the setter, validated
automatically
```

4.10.6 Advantages of Encapsulation

- Protects an object's internal state from invalid or accidental modification.

- Hides implementation details, allowing internal changes without affecting external code.
- Improves security for sensitive data, such as authentication credentials or model parameters.
- Provides a clear, controlled public interface for interacting with an object.

4.10.7 Best Practices

- Default to public attributes unless there is a specific reason to restrict access.
- Use a single leading underscore to signal 'internal use' for protected members.
- Reserve double leading underscores for attributes that genuinely must not be accessed directly.
- Prefer `@property` over manual `get_/set_` methods for Pythonic, readable code.
- Validate input inside setters rather than trusting external code to supply valid data.

4.10.8 Industrial Examples

In production Machine Learning systems, encapsulation is used to protect trained model parameters (weights) from being altered outside of a controlled training or fine-tuning process, and to expose only a stable `predict()` interface to downstream applications, hiding the considerable internal complexity of feature preprocessing and inference.

Table 4.8 OOP Principles Summary

Principle	Core Idea	Python Mechanism
Encapsulation	Bundle data and behaviour; restrict access	<code>_protected</code> , <code>__private</code> , <code>@property</code>
Inheritance	Derive new classes from existing ones	<code>class Child(Parent):</code>
Polymorphism	Same interface, different implementations	Overriding, duck typing, operator overloading
Abstraction	Expose essentials, hide complexity	Public methods hiding internal details

4.11 OOP in Data Science and Machine Learning — An Integrated Example

Before summarising the chapter, it is worth pausing to see how the individual OOP concepts introduced above — classes, objects, inheritance, polymorphism, and encapsulation — combine naturally when designing real Data Science software. The

example below sketches a small hierarchy of dataset loader classes, a pattern found throughout production machine learning codebases.

```
class DataLoader:
    """Base class for all dataset loaders (abstraction +
    encapsulation)."""
    def __init__(self, source_path):
        self._source_path = source_path # protected
        attribute
        self._data = None

    def load(self):
        raise NotImplementedError("Subclasses must implement
        load()")

    def describe(self):
        if self._data is None:
            return "No data loaded yet."
        return f"Loaded {len(self._data)} records from
        {self._source_path}"

class CSVLoader(DataLoader): # Inheritance
    def load(self): # Polymorphism via method overriding
        import csv
        with open(self._source_path, newline='') as f:
            self._data = list(csv.DictReader(f))
        return self._data

class JSONLoader(DataLoader): # Inheritance
    def load(self): # Polymorphism via method overriding
        import json
        with open(self._source_path) as f:
            self._data = json.load(f)
        return self._data

loaders = [CSVLoader("sales.csv"),
            JSONLoader("config.json")]
for loader in loaders:
    loader.load() # Same call, different behaviour per
    subclass
print(loader.describe())
```

This design uses inheritance to share the `describe()` logic and the general shape of every loader, polymorphism to allow `load()` to behave differently for each file format while being called identically, and encapsulation to keep the internal `_source_path` and `_data`

attributes shielded from direct external tampering. Later chapters on data preprocessing and model building will build directly on this pattern, extending it into feature-engineering classes and model-wrapper classes.

Industry Insight: Frameworks such as PyTorch's *Dataset* and *DataLoader* classes, and scikit-learn's *Transformer* and *Estimator* base classes, follow exactly this pattern of a shared base interface with format-specific or algorithm-specific subclasses.

4.12 Chapter Summary

This chapter introduced Object-Oriented Programming as a paradigm for organising software around objects that bundle data and behaviour together. After briefly revisiting user-defined functions, lambda functions, and recursion from Chapter 3, the chapter examined modules and packages as mechanisms for organising code, before turning to the core pillars of OOP.

- Modules and packages provide the organisational foundation on which larger, class-based systems are built, and pip is the standard tool for installing third-party packages such as NumPy, pandas, and scikit-learn.
- A class is a blueprint defining attributes and methods; an object is a specific instance of that blueprint, created through instantiation and holding its own copy of instance attributes.
- Constructors (`__init__`), the `self`-keyword, class variables, instance variables, static methods, class methods, and magic methods together define how a class's internal structure and behaviour are expressed in Python.
- Inheritance (single, multiple, multilevel, hierarchical, and hybrid) allows new classes to reuse and extend existing ones, with method overriding and `super()` as the principal tools for specialising inherited behaviour.
- Polymorphism allows a single interface, such as a common method name, to produce different behaviour depending on the object it is invoked on, realised in Python through method overriding, duck typing, and operator overloading.
- Encapsulation protects an object's internal state through public, protected, and private members, getter and setter methods, and the Pythonic `@property` decorator.

4.12.1 Key Takeaways

- OOP models software around real-world entities, improving maintainability and scalability compared to purely procedural code.

- Classes define structure; objects are its concrete realisations.
- Inheritance, polymorphism, and encapsulation are not independent tricks but complementary principles that work together, as demonstrated in the integrated DataLoader example.
- Every major Data Science and Machine Learning library is itself built using the same OOP principles introduced in this chapter.

4.12.2 Important Terms

Class, Object, Instantiation, Attribute, Method, Constructor, self, Class Variable, Instance Variable, Static Method, Class Method, Magic Method, Inheritance, Base Class, Derived Class, Method Overriding, super (), Polymorphism, Duck Typing, Operator Overloading, Encapsulation, Data Hiding, Access Modifier, Property Decorator, Module, Package, __init__.py, pip.

4.13 Exercises

4.13.1 A. Multiple Choice Questions

9. Which keyword is used to define a class in Python? (a) class (b) def (c) object (d) struct
10. Which method is automatically called when an object is created? (a) __start__ (b) __init__ (c) __new__ (d) __create__
11. Which decorator is used to define a method that does not access instance or class data? (a) @classmethod (b) @staticmethod (c) @property (d) @abstractmethod
12. What does the self-parameter refer to? (a) The class itself (b) The current object (c) A global variable (d) The parent class
13. Which type of inheritance involves a single class inheriting from two or more base classes? (a) Single (b) Multilevel (c) Multiple (d) Hierarchical
14. Which function allows a derived class to call a method of its base class? (a) base() (b) parent() (c) super() (d) inherit()
15. A double leading underscore before an attribute name causes: (a) No change (b) Name mangling (c) A syntax error (d) The attribute to become a constant
16. Which of the following best describes polymorphism? (a) Hiding data (b) One interface, many implementations (c) Reusing base class code (d) Creating multiple objects

17. Which file marks a directory as a Python package? (a) main.py (b) setup.py (c) `__init__.py` (d) package.py
18. Which command installs a third-party Python package? (a) install pip (b) pip install (c) python get (d) import install

4.13.2 B. Fill in the Blanks

11. A _____ is a blueprint for creating objects.
12. The process of creating an object from a class is called _____.
13. _____ methods are declared using the `@classmethod` decorator and take `cls` as their first parameter.
14. The _____ function allows a subclass to reuse a method defined in its parent class.
15. _____ is the OOP principle of bundling data and methods together while restricting direct access.
16. In Python, attributes prefixed with a single underscore are considered _____ by convention.

4.13.3 C. True or False

17. Python supports true method overloading in the same way as Java. (True/False)
18. A class attribute is shared by all instances of a class unless overridden. (True/False)
19. Encapsulation prevents a class from ever being extended through inheritance. (True/False)
20. The `__init__.py` file is mandatory in every version of Python for a directory to function as a package. (True/False)
21. Duck typing allows unrelated classes to be used interchangeably if they implement the same method names. (True/False)

4.13.4 D. Match the Following

Match each term in Column A with its correct description in Column B.

Match the Following

Column A	Column B
1. <code>__add__</code>	a. Getter/setter shortcut
2. <code>@property</code>	b. Operator overloading for +
3. <code>super()</code>	c. Marks a directory as a package
4. <code>__init__.py</code>	d. Calls the base class method
5. <code>@staticmethod</code>	e. Method independent of instance/class state

4.13.5 E. Short Answer Questions

22. Differentiate between a class attribute and an instance attribute with an example.
23. What is the difference between a module and a package?
24. Explain the difference between method overloading and method overriding in the context of Python.
25. What is duck typing? Give an example distinct from the one in this chapter.
26. Explain the purpose of the self parameter in an instance method.

4.13.6 F. Long Answer Questions

27. Explain, with suitable examples, the four core principles of Object-Oriented Programming and how they contribute to maintainable software design.
28. Describe the five types of inheritance supported in Python, providing a code example for each.
29. Discuss the role of encapsulation in protecting sensitive data, using a banking or healthcare example of your own design.
30. Explain how packages are structured in Python, and describe the purpose of the `__init__.py` file with an example package layout.

4.13.7 G. Predict the Output

```
class A:
    def greet(self):
        return "Hello from A"

class B(A):
    def greet(self):
        return "Hello from B, and " + super().greet()

print(B().greet())

class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1

Counter(); Counter(); Counter()
print(Counter.count)
```

4.13.8 H. Debug the Code

Identify and correct the errors in the following program, which is intended to create a Rectangle class and print its area.

```
class Rectangle
def __init__(self, w, h):
width = w
self.height = h

def area(self)
return self.width * self.height

r = Rectangle(4, 5)
print(r.area)
```

4.13.9 I. Programming Exercises

32. Create a class Book with attributes title, author, and price, and a method display_info() that prints all details.
33. Create a class Employee with a class variable company_name and instance variables name and salary. Add a class method to count the total number of employees created.
34. Implement a class Temperature with a private attribute __celsius and a @property named fahrenheit that converts and returns the value in Fahrenheit.
35. Design a base class Shape with an area() method, and derive Circle, Square, and Rectangle from it, demonstrating polymorphism as in Section 4.9.7.
36. Create a package named school_toolkit containing two modules, students.py and teachers.py, each with at least one class, and demonstrate importing both in a main script.

4.13.10J. OOP-based Programming Problems

36. Design a LibrarySystem consisting of Book and Member classes, where a Member can borrow and return Book objects, updating each object's availability status.
37. Implement a class hierarchy for a Vehicle Rental System with a base class Vehicle and derived classes Car and Bike, each with different rental-rate calculation logic (polymorphism).

4.13.11 K. Mini Projects Using Classes and Objects

38. Build a simple Student Result Management System using classes for Student, Subject, and ResultCard, applying encapsulation to protect raw marks and exposing only a computed grade through a property.
39. Build a small Dataset Explorer tool, extending the DataLoader hierarchy from Section 4.11 with a third subclass, ExcelLoader, and a method that prints summary statistics for the loaded data using the statistics module from Section 4.4.7.3.

4.13.12 L. Analytical Questions

43. Compare and contrast inheritance and composition as strategies for code reuse. Under what circumstances would composition be preferable to inheritance?
44. Discuss why Python's approach to access control (naming conventions rather than strict enforcement) might be considered a deliberate design philosophy rather than a limitation.

Chapter 5: Python Data Structures and Libraries

5.1 Lists

5.1.1 Introduction

A list is Python's most versatile built-in data structure: an ordered, mutable collection that can hold items of any type, including a mix of numbers, strings, and even other lists. Lists are created using square brackets, with individual elements separated by commas, and they form the foundation on which many Data Science operations — such as storing a column of observations or a batch of records — are built before being handed to specialised libraries such as NumPy and pandas.

```
marks = [78, 85, 92, 66, 74]
mixed = ["Asha", 21, 3.8, True]
empty_list = []
print(marks, mixed)
```

5.1.2 Creating and Accessing Lists

Elements within a list are accessed using zero-based indexing, where the first element is at index 0 and the last element can also be reached using the negative index -1. Python also supports slicing, which extracts a sub-list using the syntax `list[start:stop:step]`.

```
marks = [78, 85, 92, 66, 74]
print(marks) # 78 (first element)
print(marks[-1]) # 74 (last element)
print(marks[1:4]) # [85, 92, 66] (slicing)
print(marks[::-1]) # [74, 66, 92, 85, 78] (reversed copy)
```

5.1.3 List Mutability

Unlike strings and tuples, lists are mutable, meaning their contents can be changed after creation without creating a new object. Individual elements can be reassigned directly through their index, and entire slices can be replaced in a single operation.

```
marks = [78, 85, 92, 66, 74]
marks[0] = 80 # Modify a single element
marks[1:3] = [88, 90] # Replace a slice
print(marks) # [80, 88, 90, 66, 74]
```

5.1.4 Common List Methods

Table 5.1 Frequently Used List Methods

Method	Purpose	Example
append(x)	Add a single element to the end	marks.append(95)
extend(iterable)	Add multiple elements from another iterable	marks.extend([60, 70])
insert(i, x)	Insert an element at a specific index	marks.insert(0, 100)
remove(x)	Remove the first matching value	marks.remove(66)
pop(i)	Remove and return the element at index i (default: last)	marks.pop()
sort()	Sort the list in place	marks.sort(reverse=True)
reverse()	Reverse the list in place	marks.reverse()
index(x)	Return the index of the first match	marks.index(85)
count(x)	Count occurrences of a value	marks.count(85)
clear()	Remove all elements	marks.clear()

```
fruits = ["mango", "apple", "banana"]
fruits.append("grape")
fruits.sort()
print(fruits) # ['apple', 'banana', 'grape', 'mango']
```

5.1.5 Iterating Over a List

```
temperatures = [30.5, 32.1, 29.8, 31.4]

for temp in temperatures:
    print(f"{temp} °C")

for index, temp in enumerate(temperatures): # index
    + value together
    print(index, temp)
```

5.1.6 Nested Lists

A list can contain other lists as elements, producing a nested structure often used to represent tabular data such as a matrix or a small dataset before it is loaded into a NumPy array or pandas DataFrame.

```
matrix = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]

print(matrix) # 6 -- row index 1, column index 2
```

5.1.7 Practical Example

Problem Statement: Given a list of temperature readings, compute the average, and identify all readings above the average.

```
readings = [30.5, 32.1, 29.8, 31.4, 28.6, 33.0]
average = sum(readings) / len(readings)
above_average = [t for t in readings if t > average]

print(f"Average: {average:.2f}")
print(f"Above average: {above_average}")
```

Expected Output: Average: 30.90, Above average:
[32.1, 31.4, 33.0]

Common Mistake: Assigning one list variable to another ($b = a$) does not copy the list; both names refer to the same underlying object. Use $b = a.copy()$ or $b = list(a)$ to create an independent copy.

5.1.8 Applications in Data Science

Lists are the natural intermediate structure for collecting raw values before analysis — for example, appending each row read from a file into a list, or storing a sequence of model predictions. NumPy arrays and pandas Series are frequently constructed directly from Python lists using `np.array(my_list)` or `pd.Series(my_list)`.

5.2 Tuples

5.2.1 Introduction

A tuple is an ordered collection similar to a list, but it is immutable: once created, its contents cannot be changed. Tuples are created using parentheses (or simply by separating values with commas) and are typically used to represent a fixed collection of related values, such as coordinates or a database record, where accidental modification should be prevented.

```
point = (3, 4)
student_record = ("Asha", 21, "Computer Science")
single_item = (5,) # Trailing comma required for a
```

```
one-element tuple
print(point, student_record)
```

5.2.2 Accessing Tuple Elements

Tuples support the same indexing and slicing operations as lists, since both are ordered sequences.

```
student_record = ("Asha", 21, "Computer Science")
print(student_record) # Asha
print(student_record[-1]) # Computer Science
print(student_record[0:2]) # ('Asha', 21)
```

5.2.3 Immutability

Attempting to modify a tuple element directly raises a `TypeError`, which is precisely the guarantee that makes tuples useful whenever a collection of values must remain constant throughout a program, such as configuration settings or the fixed dimensions of an image.

```
point = (3, 4)
point = 10 # Raises TypeError: 'tuple' object does
           # not support item assignment
```

5.2.4 Tuple Packing and Unpacking

Tuple packing refers to grouping several values into a single tuple, while unpacking refers to extracting those values back into separate variables in a single statement — a pattern used extensively when a function returns multiple values.

```
def min_max(values):
    return min(values), max(values) # Packing: returns a
    tuple

lowest, highest = min_max([12, 45, 3, 89, 22]) #
Unpacking
print(lowest, highest) # 3 89
```

5.2.5 Tuple Methods and Built-in Functions

Table 5.2 Common Tuple Operations

Operation	Purpose	Example
<code>count(x)</code>	Count occurrences of a value	<code>record.count('Asha')</code>
<code>index(x)</code>	Return the index of the first match	<code>record.index(21)</code>
<code>len(t)</code>	Number of elements	<code>len(record)</code>
<code>t1 + t2</code>	Concatenate two tuples	<code>(1, 2) + (3, 4)</code>
<code>x in t</code>	Membership test	<code>21 in record</code>

5.2.6 Lists vs Tuples

Table 5.3 Lists vs Tuples

Aspect	List	Tuple
Mutability	Mutable	Immutable
Syntax	Square brackets []	Parentheses ()
Performance	Slightly slower	Slightly faster, lower memory overhead
Typical use	Homogeneous, changing collections	Fixed records, function return values
Usable as dictionary key	No (unhashable)	Yes, if all elements are hashable

Programming Tip: Because tuples are hashable (when their contents are hashable), they can be used as dictionary keys or set members, whereas lists cannot — a distinction that becomes important in Sections 5.3 and 5.4.

5.2.7 Applications in Data Science

Tuples are commonly used to represent the fixed shape of an array (e.g., a NumPy array's `shape` attribute returns a tuple), and to return multiple values from data-processing functions, such as returning both a cleaned dataset and a summary report from a single preprocessing function.

5.3 Dictionaries

5.3.1 Introduction

A dictionary is an unordered (prior to Python 3.7) but insertion-ordered (from Python 3.7 onward) collection of key-value pairs, where each key must be unique and hashable, and is used to retrieve its associated value in constant average time. Dictionaries are created using curly braces with key: value pairs separated by commas, and they are one of the most heavily used structures in Data Science for representing structured records, configuration settings, and mappings between categories and values.

```
student = {"name": "Asha", "age": 21, "branch":
"Computer Science"}
print(student)
```

5.3.2 Accessing and Modifying Dictionary Elements

```
student = {"name": "Asha", "age": 21}
print(student["name"]) # Asha
print(student.get("gpa", 0.0)) # 0.0 -- default if
```

```
key is absent
student["age"] = 22 # Modify an existing key
student["gpa"] = 8.9 # Add a new key-value pair
print(student)
```

5.3.3 Common Dictionary Methods

Table 5.4 Frequently Used Dictionary Methods

Method	Purpose	Example
keys()	Return a view of all keys	student.keys()
values()	Return a view of all values	student.values()
items()	Return a view of key-value pairs	student.items()
get(key, default)	Safely retrieve a value	student.get('gpa', 0)
pop(key)	Remove a key and return its value	student.pop('age')
update(other)	Merge another dictionary in	student.update({'city': 'Bengaluru'})
setdefault(key, default)	Get a value, inserting a default if missing	student.setdefault('gpa', 0.0)

5.3.4 Iterating Over a Dictionary

```
marks = {"Math": 92, "Physics": 85, "Chemistry": 78}

for subject, score in marks.items():
    print(f"{subject}: {score}")
```

5.3.5 Nested Dictionaries

Dictionary values may themselves be dictionaries, lists, or any other structure, allowing complex, hierarchical data — such as an entire dataset record with sub-fields — to be represented naturally. This nested pattern closely mirrors the structure of JSON data commonly encountered when working with web APIs.

```
students = {
    "S001": {"name": "Asha", "marks": [80, 90, 85]},
    "S002": {"name": "Rahul", "marks": [70, 75, 72]},
}

print(students["S001"]["marks"]) # [80, 90, 85]
```

5.3.6 Practical Example

Problem Statement: Count the frequency of each word in a sentence using a dictionary, a common preprocessing step in text analytics.

```
sentence = "data science is fun and data is  
powerful"  
word_count = {}  
  
for word in sentence.split():  
    word_count[word] = word_count.get(word, 0) + 1  
  
print(word_count)  
  
Expected Output: {'data': 2, 'science': 1, 'is': 2,  
                  'fun': 1, 'and': 1, 'powerful': 1}
```

5.3.7 Applications in Data Science

Dictionaries underlie the internal representation of a pandas DataFrame column mapping, JSON-based API responses, and configuration objects for machine learning models (such as hyperparameter dictionaries passed to GridSearchCV). Converting a dictionary directly into a DataFrame using `pd.DataFrame(my_dict)` is one of the most common ways structured data enters a Data Science workflow.

***Did You Know?:** From Python 3.7 onward, dictionaries maintain insertion order as a guaranteed language feature, not merely an implementation detail — a change that made dictionaries even more useful for representing ordered records.*

5.4 Sets

5.4.1 Introduction

A set is an unordered collection of unique, hashable elements, created using curly braces or the `set()` constructor. Because sets automatically eliminate duplicate values and support efficient membership testing, they are especially useful for tasks such as removing duplicate records or comparing two collections for overlap.

```
colors = {"red", "green", "blue", "red"} # duplicate  
      'red' is dropped  
print(colors) # {'red', 'green', 'blue'}  
empty_set = set() # NOT {} -- that creates a  
dictionary
```

5.4.2 Common Set Operations

Table 5.5 Set Operations

Operation	Symbol	Method	Result
Union	$a \mid b$	<code>a.union(b)</code>	All elements from both sets
Intersection	$a \& b$	<code>a.intersection(b)</code>	Elements common to both sets
Difference	$a - b$	<code>a.difference(b)</code>	Elements in a but not in b
Symmetric Difference	$a \wedge b$	<code>a.symmetric_difference(b)</code>	Elements in exactly one set

```
python_students = {"Asha", "Rahul", "Meera"}
java_students = {"Rahul", "Priya", "Meera"}

print(python_students & java_students) # {'Rahul', 'Meera'} -- both
print(python_students - java_students) # {'Asha'} -- Python only
print(python_students | java_students) # union of both
```

5.4.3 Common Set Methods

- `add(x)` — insert a single element
- `remove(x)` — remove an element, raising `KeyError` if absent
- `discard(x)` — remove an element without raising an error if absent
- `issubset(other)` — test whether every element is also in another set
- `issuperset(other)` — test whether a set contains another set entirely

5.4.4 Practical Example

Problem Statement: Remove duplicate entries from a list of survey respondent IDs while preserving no particular order, and determine how many unique respondents took part.

```
respondent_ids = [101, 102, 101, 103, 104, 102, 105]
unique_ids = set(respondent_ids)

print(unique_ids)
print(f"Unique respondents: {len(unique_ids)}")
```


Expected Output: Unique respondents: 5

5.4.5 Applications in Data Science

Sets are frequently used to deduplicate raw data before analysis, to quickly test membership of a value within a large collection (much faster on average than searching a list), and to compare the vocabulary of two text documents through set intersection and difference operations.

Debugging Tip: Sets are unordered, so indexing a set (`my_set`) is not supported and will raise a `TypeError`. Convert a set to a list first if positional access is required: `list(my_set)`.

5.5 Strings

5.5.1 Introduction

A string is an ordered, immutable sequence of characters, enclosed in single quotes, double quotes, or triple quotes for multi-line text. Strings are used everywhere in Data Science: as column names, categorical labels, file paths, and, in the field of Natural Language Processing, as the raw material of the analysis itself.

```
name = "Data Science"
quote = 'Python is beginner-friendly'
paragraph = """This is a
multi-line string."""
```

5.5.2 String Indexing and Slicing

Because strings are sequences, they support the same indexing and slicing syntax already introduced for lists and tuples in Sections 5.1 and 5.2.

```
text = "Machine Learning"
print(text) # M
print(text[-1]) # g
print(text[0:7]) # Machine
print(text[::-1]) # gninraeL enihcaM (reversed)
```

5.5.3 String Immutability

Like tuples, strings cannot be modified in place; any operation that appears to change a string actually creates and returns a new string object, leaving the original unchanged.

```
text = "hello"
text = "H" # Raises TypeError: 'str' object does not
support item assignment
text = "H" + text[1:] # Correct approach: build a
```

```
new_string
print(text) # Hello
```

5.5.4 Common String Methods

Table 5.6 Frequently Used String Methods

Method	Purpose	Example
upper() / lower()	Convert case	"data".upper() → 'DATA'
strip()	Remove leading/trailing whitespace	" data ".strip()
split(sep)	Split into a list of substrings	"a,b,c".split(',')
join(iterable)	Join a list into one string	",".join(['a','b','c'])
replace(old, new)	Replace all occurrences	"data".replace('a','o')
find(sub)	Return the index of first occurrence, -1 if absent	"data".find('t')
startswith(sub)	Test the beginning of a string	"data.csv".startswith('data')
format() / f-strings	Insert values into a string template	f"Score: {score}"

```
raw = " Data Science, Machine Learning, AI "
cleaned = raw.strip()
topics = cleaned.split(", ")
print(topics) # ['Data Science', 'Machine Learning', 'AI']
print(" | ".join(topics)) # Data Science | Machine Learning | AI
```

5.5.5 String Formatting

Python offers several ways to embed values inside a string, of which f-strings (formatted string literals, introduced in Python 3.6) are the most concise and readable, and are used consistently throughout this textbook.

```
name, score = "Asha", 92.456
print(f"{name} scored {score:.1f}%") # f-string with formatting spec
print("{} scored {:.1f}%".format(name, score))
#.format() equivalent
print("%s scored %.1f%%" % (name, score)) # legacy % formatting
```

5.5.6 Practical Example

Problem Statement: Given a raw, messy string of comma-separated values, clean and standardise each entry (trim whitespace, fix casing) before further processing.

```
raw_row = " Bengaluru, MUMBAI,delhi "  
cities = [city.strip().title() for city in  
raw_row.split(",")]  
print(cities)
```

Expected Output: ['Bengaluru', 'Mumbai', 'Delhi'].
This example previews list comprehension, formally introduced in Section 5.6.

5.5.7 Applications in Data Science

String processing is foundational to data cleaning (standardising inconsistent text entries), feature engineering from text columns, and the entire field of Natural Language Processing, where raw text must be tokenised, normalised, and vectorised before it can be used in a machine learning model.

5.6 List Comprehension

5.6.1 Introduction

List comprehension is a concise, expressive syntax for creating a new list by applying an expression to every item of an existing iterable, optionally filtering which items are included. It condenses what would otherwise require a multi-line for loop with an `append()` call into a single, readable line, and is considered idiomatic (Pythonic) code.

```
# Without list comprehension  
squares = []  
for n in range(1, 6):  
    squares.append(n ** 2)  
  
# With list comprehension -- equivalent, more  
# concise  
squares = [n ** 2 for n in range(1, 6)]  
print(squares) # [1, 4, 9, 16, 25]
```

5.6.2 General Syntax

The general form of a list comprehension is `[expression for item in iterable if condition]`, where the `if condition` clause is optional and is used to filter which items from the iterable are included in the resulting list.

```
numbers = range(1, 21)
even_squares = [n ** 2 for n in numbers if n % 2 == 0]
print(even_squares)
```

5.6.3 Conditional Expressions Within Comprehensions

A list comprehension can also include an if...else expression before the for clause, allowing every item to be transformed differently depending on a condition, rather than merely filtering items out.

```
scores = [45, 78, 92, 33, 88, 56]
results = ["Pass" if s >= 50 else "Fail" for s in scores]
print(results) # ['Fail', 'Pass', 'Pass', 'Fail', 'Pass', 'Pass']
```

5.6.4 Nested List Comprehension

List comprehensions can be nested to process multi-dimensional data, such as flattening a matrix (a list of lists) into a single flat list.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [value for row in matrix for value in row]
print(flattened) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

5.6.5 Practical Example

Problem Statement: Given a list of temperature readings in Celsius, produce a new list of the corresponding Fahrenheit values, retaining only readings that convert to above 90°F.

```
celsius_readings = [28, 34, 40, 18, 36, 22]
hot_fahrenheit = [(c * 9/5) + 32 for c in celsius_readings if (c * 9/5) + 32 > 90]
print(hot_fahrenheit)
```

Expected Output: [93.2, 104.0, 96.8]

Best Practice: List comprehensions improve readability for simple transformations, but a comprehension that becomes long or deeply nested should be rewritten as a regular for loop with clear variable names, since readability always takes priority over brevity.

5.6.6 Applications in Data Science

List comprehensions are widely used for quick, vectorised-style transformations of Python lists before or after using NumPy and pandas, such as generating column names, filtering rows based on simple conditions, or applying a lightweight transformation across a batch of file names during data loading.

5.7 Dictionary Comprehension

5.7.1 Introduction

Dictionary comprehension extends the same concise syntax introduced for lists in Section 5.6 to the construction of dictionaries, allowing a new dictionary to be built from an iterable in a single expression of the form `{key_expr: value_expr for item in iterable if condition}`.

```
# Without dictionary comprehension
squares = {}
for n in range(1, 6):
    squares[n] = n ** 2

# With dictionary comprehension -- equivalent, more
concise
squares = {n: n ** 2 for n in range(1, 6)}
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

5.7.2 Filtering with Dictionary Comprehension

```
marks = {"Math": 92, "Physics": 45, "Chemistry": 78,
         "Biology": 38}
passed = {subject: score for subject, score in
          marks.items() if score >= 50}
print(passed) # {'Math': 92, 'Chemistry': 78}
```

5.7.3 Transforming Keys and Values

Dictionary comprehension can transform both keys and values simultaneously while iterating over an existing dictionary, which is a common step when renaming columns or rescaling values in a dataset represented as key-value pairs.

```
prices_inr = {"pen": 20, "notebook": 45, "bag": 900}
prices_usd = {item.upper(): round(price / 83, 2) for
              item, price in prices_inr.items()}
print(prices_usd)
```

5.7.4 Practical Example

Problem Statement: Given two parallel lists — student names and their corresponding scores — build a single dictionary mapping each name to a Pass/Fail status, using dictionary comprehension together with the built-in `zip()` function.

```
names = ["Asha", "Rahul", "Meera", "Karan"]
scores = [82, 41, 90, 55]

status = {name: ("Pass" if score >= 50 else "Fail")
          for name, score in zip(names, scores)}
print(status)

Expected Output: {'Asha': 'Pass', 'Rahul': 'Fail',
                  'Meera': 'Pass', 'Karan': 'Pass'}
```

Programming Tip: `zip()` pairs up elements from two or more iterables positionally and is the standard way to combine parallel lists inside a comprehension, whether producing a list of tuples or, as above, a dictionary.

5.7.5 Applications in Data Science

Dictionary comprehension is often used to build lookup tables mapping categorical labels to numeric codes (a lightweight alternative to a full `LabelEncoder`), to quickly restructure JSON-like API responses, and to construct configuration dictionaries for machine learning models from lists of parameter names and values.

5.8 File Handling

5.8.1 Introduction

File handling refers to the set of operations Python provides for reading data from, and writing data to, files stored on disk. Since almost every Data Science project begins by loading data from a file — a CSV of sales records, a JSON API response, or a plain text log — file handling is an essential skill that bridges the data structures covered earlier in this chapter with real-world data sources.

5.8.2 Opening and Closing Files

A file is opened using the built-in `open()` function, which takes a file path and a mode string indicating how the file will be used, and returns a file object through which the file's contents can be read or written. Once finished, a file should always be closed using `close()` to release the underlying system resource and ensure that any buffered writes are flushed to disk.

Table 5.7 Common File Modes

Mode	Meaning
'r'	Read (default); error if the file does not exist
'w'	Write; creates the file, or overwrites it if it already exists
'a'	Append; writes are added to the end of an existing file
'r+'	Read and write, without truncating the file
'b'	Binary mode, combined with another mode, e.g., 'rb'

```
file = open("notes.txt", "w")
file.write("Python for Data Science\n")
file.close() # Always close explicitly when not
using 'with'
```

5.8.3 The with Statement (Context Manager)

The with statement provides a safer, more Pythonic way to work with files: it automatically closes the file once the indented block finishes, even if an error occurs inside the block, eliminating the risk of a file being left open by mistake. This is the recommended approach in professional Python code.

```
with open("notes.txt", "w") as file:
    file.write("Python for Data Science\n")
    file.write("Chapter 5: Data Structures\n")
# The file is automatically closed here, even if an
exception occurred above.
```

Best Practice: Always prefer the with open(...) as file: pattern over manually calling open() and close(), since it guarantees proper cleanup and is considered idiomatic Python.

5.8.4 Reading from a File

Table 5.8 Common File Reading Methods

Method	Purpose
read()	Read the entire file content as a single string
readline()	Read a single line, including the newline character
readlines()	Read all lines into a list of strings
Iterating with for line in file:	Memory-efficient, line-by-line reading

```
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

```
with open("notes.txt", "r") as file:
    for line in file: # Efficient for large files
        print(line.strip())
```

5.8.5 Writing and Appending to a File

```
with open("scores.txt", "w") as file:
    file.write("Asha: 92\n")
    file.write("Rahul: 78\n")

with open("scores.txt", "a") as file: # Append mode
    adds without erasing
    file.write("Meera: 85\n")
```

5.8.6 Working with CSV Files

The built-in `csv` module provides convenient tools for reading and writing comma-separated value files — the single most common raw data format encountered in Data Science — without needing to manually split lines by hand.

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Marks"])
    writer.writerow(["Asha", 92])
    writer.writerow(["Rahul", 78])

with open("students.csv", "r") as file:
    reader = csv.DictReader(file) # Reads each row as a
    dictionary
    for row in reader:
        print(row["Name"], row["Marks"])
```

5.8.7 Exception Handling During File Operations

File operations frequently fail due to circumstances outside a program's control — a missing file, insufficient permissions, or a corrupted format — so file handling code should generally be wrapped in a `try...except` block, building on the exception-handling techniques covered in Chapter 3.

```
try:
    with open("missing_file.txt", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("The specified file could not be found.")
```


5.8.8 Practical Example

Problem Statement: Read a text file containing one numeric reading per line, compute the average of the readings, and write the result to a new summary file.

```
with open("readings.txt", "r") as infile:
    values = [float(line.strip()) for line in infile] #
    List comprehension

average = sum(values) / len(values)

with open("summary.txt", "w") as outfile:
    outfile.write(f"Average reading: {average:.2f}\n")

print("Summary written successfully.")
```

5.8.9 Applications in Data Science

File handling underlies every dataset-loading step in a Data Science pipeline: reading raw CSV or JSON data, writing intermediate cleaned datasets back to disk, logging experiment results, and saving trained model outputs. Later chapters will build directly on these operations when reading datasets with pandas' `read_csv()` function, which internally performs the same kind of file access introduced here.

5.9 Introduction to Python Libraries

5.9.1 What is a Library?

A library, in the sense used throughout Data Science, is a package (in the technical sense defined in Chapter 4) of pre-written, tested code that provides ready-made functionality for a particular domain, sparing a programmer from re-implementing common operations from scratch. Rather than writing numerical routines, plotting code, or statistical models by hand, a Data Scientist installs and imports a library, gaining access to years of accumulated, optimised, community-reviewed functionality through a small number of function calls.

5.9.2 Why Libraries Matter in Data Science

- **Performance:** libraries such as NumPy are implemented in optimised, compiled C code beneath a Python interface, running far faster than equivalent pure-Python loops
- **Reliability:** widely used libraries are extensively tested by large communities, reducing the likelihood of subtle bugs

- **Productivity:** complex operations such as training a machine learning model can be performed in a single line rather than hundreds
- **Standardisation:** shared libraries establish common interfaces (as discussed for scikit-learn in Chapter 4), making code written by different teams interoperable

5.9.3 NumPy — Numerical Computing

NumPy (Numerical Python) provides the `ndarray`, a fast, memory-efficient, multi-dimensional array object, along with a vast collection of mathematical functions that operate on entire arrays at once — a style of programming known as vectorisation — avoiding the explicit for loops that would otherwise be required with plain Python lists.

```
import numpy as np
marks = np.array([78, 85, 92, 66, 74])
print(marks.mean()) # 79.0 -- vectorised, no
explicit loop needed
print(marks * 2) # element-wise multiplication
```

5.9.4 pandas — Data Manipulation and Analysis

pandas builds on NumPy to provide the `DataFrame`, a two-dimensional, labelled data structure resembling a spreadsheet or a database table, along with powerful tools for reading, cleaning, filtering, grouping, and summarising structured data — extending the dictionary and list structures covered earlier in this chapter into a purpose-built tool for tabular data.

```
import pandas as pd

data = {"Name": ["Asha", "Rahul", "Meera"], "Marks":
[92, 78, 85]}
df = pd.DataFrame(data) # Built directly from a
dictionary
print(df.describe()) # Quick statistical summary
```

5.9.5 matplotlib and seaborn — Data Visualisation

matplotlib is Python's foundational plotting library, capable of producing a wide range of static charts, while seaborn builds on top of matplotlib to provide a higher-level interface for statistically informative and visually refined plots with far less code.

```
import matplotlib.pyplot as plt

marks = [78, 85, 92, 66, 74]
```

```
plt.bar(["Asha", "Rahul", "Meera", "Karan",
        "Priya"], marks)
plt.title("Student Marks")
plt.ylabel("Marks")
plt.show()
```

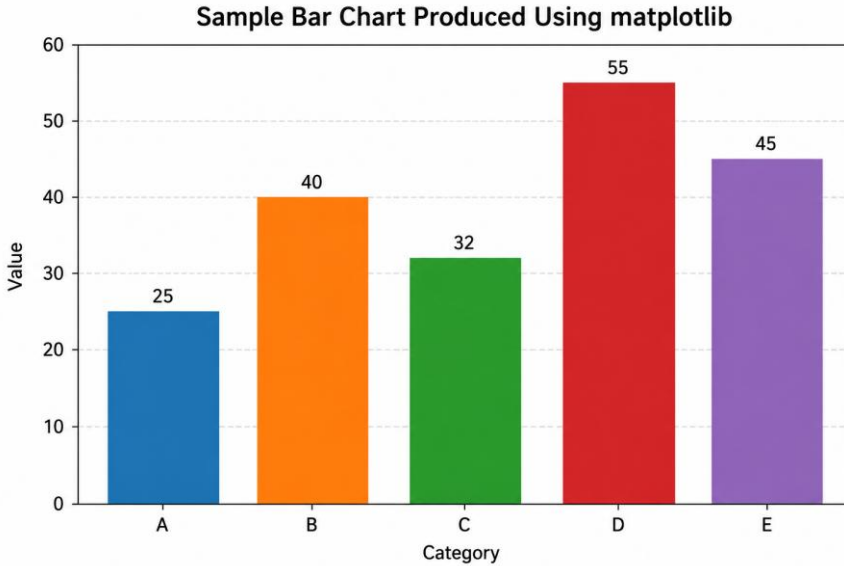


Figure 5.1. Sample Bar Chart Produced Using matplotlib

5.9.6 scikit-learn — Machine Learning

scikit-learn provides a consistent, well-documented interface (introduced conceptually in Chapter 4 through polymorphism) for classical machine learning algorithms, including classification, regression, clustering, and dimensionality reduction, along with tools for model evaluation and preprocessing.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train) # Learn from training
data
predictions = model.predict(X_test)
```

5.9.7 Other Notable Libraries

Table 5.9 Overview of Common Data Science Libraries

Library	Primary Use
SciPy	Advanced scientific and statistical computing
TensorFlow / PyTorch	Deep learning and neural network development

Library	Primary Use
Statsmodels	Statistical modelling and hypothesis testing
NLTK / spaCy	Natural Language Processing
Requests	Sending HTTP requests to web APIs
BeautifulSoup	Parsing HTML for web scraping

5.9.8 Installing and Importing Libraries

As introduced in Chapter 4, third-party libraries are installed using `pip` and made available in a program through the `import` statement, most commonly with a conventional alias for brevity.

```
pip install numpy pandas matplotlib scikit-learn

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

5.9.9 Looking Ahead

This section has only introduced the purpose and basic usage of these libraries; NumPy, pandas, and data visualisation will each receive a dedicated, in-depth chapter later in this textbook. The intent here is to show how the fundamental data structures covered throughout this chapter — lists, tuples, dictionaries, sets, and strings — connect directly to the more powerful, specialised structures these libraries provide.

5.10 Chapter Summary

This chapter surveyed Python's principal built-in data structures — lists, tuples, dictionaries, sets, and strings — along with the concise comprehension syntax used to build lists and dictionaries, before turning to file handling and a first introduction to the wider Python library ecosystem used throughout Data Science and Machine Learning.

- Lists are ordered, mutable sequences suited to changing collections of data, supporting a rich set of methods for insertion, removal, and sorting.
- Tuples are ordered, immutable sequences ideal for fixed records and for returning multiple values from a function.
- Dictionaries store unique, hashable keys mapped to values, forming the backbone of structured and JSON-like data representation.
- Sets store unique, unordered elements and provide efficient mathematical operations such as union, intersection, and difference.

- Strings are immutable sequences of characters with extensive built-in methods for cleaning, splitting, and formatting text.
- List comprehension and dictionary comprehension provide concise, Pythonic syntax for building new collections from existing iterables.
- File handling, especially through the `with` statement and the `csv` module, is the entry point through which real-world data reaches a Data Science program.
- Libraries such as NumPy, pandas, matplotlib, and scikit-learn build on these fundamental structures to provide fast, specialised tools for numerical computing, data manipulation, visualisation, and machine learning.

5.10.1 Key Takeaways

- Choosing the right data structure — list, tuple, dictionary, or set — depends on whether data must be ordered, mutable, unique, or key-addressable.
- Comprehensions favour concise, readable code but should give way to explicit loops when logic becomes complex.
- Safe file handling with the `with` statement and proper exception handling prevents resource leaks and program crashes.
- Python's built-in data structures are the foundation on which every major Data Science library is built.

5.10.2 Important Terms

List, Tuple, Dictionary, Set, String, Mutability, Immutability, Indexing, Slicing, List Comprehension, Dictionary Comprehension, File Handling, Context Manager (`with` statement), CSV, NumPy, pandas, matplotlib, scikit-learn, Vectorisation.

5.11 Exercises

5.11.1 A. Multiple Choice Questions

19. Which of the following data structures is immutable? (a) List (b) Dictionary (c) Tuple (d) Set
20. Which symbol is used to create a dictionary? (a) `[]` (b) `()` (c) `{ }` (d) `< >`
21. What does the following expression return: `[x for x in range(5)]`? (a) A tuple (b) A list (c) A set (d) A dictionary

- 22. Which method removes and returns the last element of a list? (a) `remove()` (b) `pop()` (c) `delete()` (d) `discard()`
- 23. Which of the following creates an empty set? (a) `{}` (b) `set()` (c) `[]` (d) `()`
- 24. Which file mode opens a file for writing, overwriting existing content? (a) `'r'` (b) `'a'` (c) `'w'` (d) `'x'`
- 25. Which built-in function pairs elements from two iterables positionally? (a) `map()` (b) `zip()` (c) `enumerate()` (d) `filter()`
- 26. Which of these is NOT a valid dictionary comprehension? (a) `{k: v for k, v in d.items()}` (b) `{x for x in range(5)}` (c) `{x: x**2 for x in range(5)}` (d) `{k.upper(): v for k, v in d.items()}`
- 27. Which statement automatically closes a file once its block finishes? (a) `close()` (b) `with` (c) `open()` (d) `finally`
- 28. Which library provides the `ndarray` for fast numerical computation? (a) `pandas` (b) `matplotlib` (c) `NumPy` (d) `scikit-learn`

5.11.2 B. Fill in the Blanks

- 17. A _____ is an ordered, immutable sequence created using parentheses.
- 18. The _____ method safely retrieves a dictionary value, returning a default if the key is absent.
- 19. Sets automatically eliminate _____ elements.
- 20. The general syntax of a list comprehension is `[expression for item in iterable if _____]`.
- 21. The _____ statement should be preferred over manual `open()/close()` calls for safer file handling.
- 22. _____ is the standard Python package manager used to install third-party libraries such as `NumPy`.

5.11.3 C. True or False

- 22. Lists support duplicate elements. (True/False)
- 23. Dictionary keys must be unique but values may repeat. (True/False)
- 24. A tuple can be modified after creation using index assignment. (True/False)
- 25. The `with` statement requires an explicit call to `close()` afterward. (True/False)

26. NumPy arrays generally perform mathematical operations faster than equivalent pure-Python list loops. (True/False)

5.11.4 D. Match the Following

Column A	Column B
1. append()	a. Remove all elements from a set/list
2. csv.DictReader	b. Add an element to the end of a list
3. intersection()	c. Elements common to two sets
4. clear()	d. Read CSV rows as dictionaries
5. join()	e. Combine a list of strings into one string

5.11.5 E. Short Answer Questions

27. Differentiate between a list and a tuple, giving one example scenario where each would be preferred.
28. Explain why sets cannot contain duplicate elements, and how this relates to hashability.
29. What is the difference between `read()`, `readline()`, and `readlines()`?
30. Write the general syntax of a dictionary comprehension and explain each part.
31. Why is the `with` statement preferred over manually calling `open()` and `close()`?

5.11.6 F. Long Answer Questions

31. Compare lists, tuples, dictionaries, and sets in terms of mutability, ordering, uniqueness, and typical use cases, with a suitable example for each.
32. Explain list comprehension and dictionary comprehension with examples, and discuss when a regular loop should be used instead.
33. Describe the complete process of reading data from a CSV file, processing it using Python data structures, and writing a summary back to a new file.
34. Discuss the role of NumPy, pandas, matplotlib, and scikit-learn in a typical Data Science workflow, and explain how they build upon Python's built-in data structures.

5.11.7 G. Predict the Output

```
numbers = [1, 2, 3, 4, 5]
result = [n * 2 for n in numbers if n % 2 == 0]
print(result)
```

```
data = {"a": 1, "b": 2, "c": 3}
filtered = {k: v for k, v in data.items() if v > 1}
print(filtered)

set_a = {1, 2, 3}
set_b = {2, 3, 4}
print(set_a ^ set_b)
```

5.11.8 H. Debug the Code

Identify and correct the errors in the following program, which is intended to read a file and print each line in uppercase.

```
file = open('data.txt' 'r')
for line in file
print(line.uppercase())
```

5.11.9 I. Programming Exercises

37. Write a program that takes a list of numbers and returns a new list containing only the prime numbers, using list comprehension.
38. Write a program that reads a text file and counts the frequency of each word, storing the result in a dictionary.
39. Given a list of tuples representing (product, price), use a dictionary comprehension to build a dictionary mapping each product name to its price after applying a 10% discount.
40. Write a program that removes duplicate values from a list while preserving the original order (hint: combine a list and a set).
41. Write a program that writes student names and marks to a CSV file, then reads the file back and prints the average mark.

5.11.10 J. Data Structure-based Programming Problems

38. Design a small inventory system using a dictionary where each key is a product name and each value is a tuple of (quantity, price). Write functions to add stock, sell stock, and compute the total inventory value.
39. Given two lists of email addresses representing two different newsletter subscriber lists, use set operations to find subscribers common to both lists and subscribers unique to each list.

5.11.11 K. Mini Projects

40. Build a simple Contact Book application that stores contacts in a dictionary of dictionaries, supports adding, searching, and deleting contacts, and saves/loads the contact list to/from a text file.
41. Build a Word Frequency Analyzer that reads a text file, uses dictionary comprehension and string methods to compute the frequency of each word (ignoring case and punctuation), and writes the top 10 most frequent words to a summary file.

5.11.12 L. Analytical Questions

45. Discuss the time-complexity trade-offs between searching for an element in a list versus a set, and explain why this matters when working with large datasets.
46. Explain how the choice of data structure (list vs. dictionary vs. set) can affect both the performance and the readability of a Data Science preprocessing script, using an example of your own design.

Chapter 6: NumPy Library

NumPy, short for Numerical Python, is one of the most fundamental libraries in the Python ecosystem for scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with a vast collection of high-level mathematical functions to operate on these arrays efficiently. For anyone working in data science, machine learning, or numerical analysis, NumPy is an indispensable tool.

6.1.1 Why NumPy?

Python's built-in data structures like lists are flexible and easy to use, but they are not optimized for numerical computations. Here are the key reasons why NumPy is preferred over standard Python lists for numerical operations:

Performance: NumPy arrays are stored in contiguous memory blocks, which allows for efficient memory access and cache utilization. Under the hood, NumPy operations are implemented in C and Fortran, making them significantly faster than equivalent operations performed with Python lists.

Memory Efficiency: NumPy arrays consume less memory than Python lists because they store data of a homogeneous type with a fixed size, eliminating the overhead of per-element type information and references.

Convenient Syntax: NumPy provides a clean and intuitive syntax for performing element-wise operations on entire arrays without the need for explicit loops. This feature, known as vectorization, leads to more readable and concise code.

Broadcasting: NumPy's broadcasting mechanism allows arithmetic operations between arrays of different shapes, eliminating the need to manually reshape or tile data.

Interoperability: NumPy serves as the foundation for many other scientific computing libraries in Python, including Pandas, Scikit-learn, Matplotlib, and SciPy. Understanding NumPy is essential for effectively using these libraries.

Linear Algebra Support: NumPy includes comprehensive support for linear algebra operations such as matrix multiplication, decomposition, eigenvalue computation, and solving linear systems.

6.1.2 Installing NumPy

If you are using the Anaconda distribution (as discussed in Chapter 1), NumPy comes pre-installed. However, if you need to install it separately, you can use pip:

```
pip install numpy
```

Or using conda:

```
conda install numpy
```

6.1.3 Importing NumPy

By convention, NumPy is imported with the alias `np`:

```
import numpy as np
```

This convention is universally followed in the data science community and in most documentation, so it is highly recommended to use it for consistency and readability.

6.1.4 Checking NumPy Version

You can check the installed version of NumPy using the `__version__` attribute:

```
import numpy as np
print(np.__version__)

1.26.4
```

6.2 NumPy Arrays

The core data structure in NumPy is the **ndarray** (n-dimensional array), which is a homogeneous, fixed-size container for elements of the same data type. Unlike Python lists, all elements in a NumPy array must be of the same type, which enables efficient storage and fast operations.

6.2.1 Creating Arrays from Python Lists

The simplest way to create a NumPy array is from a Python list using the `np.array()` function:

```
import numpy as np

# Creating a 1D array from a list
arr_1d = np.array([1, 2, 3, 4, 5])
print("1D Array:")
print(arr_1d)
print("Type:", type(arr_1d))
print("Data type:", arr_1d.dtype)
```

```
1D Array:
[1 2 3 4 5]
Type: <class 'numpy.ndarray'>
Data type: int64

# Creating a 2D array from a list of lists
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("2D Array:")
print(arr_2d)
print("Shape:", arr_2d.shape)
print("Dimensions:", arr_2d.ndim)

2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)
Dimensions: 2

# Creating a 3D array
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print("3D Array:")
print(arr_3d)
print("Shape:", arr_3d.shape)
print("Dimensions:", arr_3d.ndim)

3D Array:
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
Shape: (2, 2, 2)
Dimensions: 3
```

6.2.2 Array Attributes

NumPy arrays have several important attributes that provide information about the array: NumPy array attributes

Attribute	Description
ndim	Number of dimensions (axes) of the array
shape	Tuple of integers indicating the size of the array along each dimension
size	Total number of elements in the array
dtype	Data type of the array elements
itemsize	Size in bytes of each element
nbytes	Total memory consumed by the array in bytes
T	Transpose of the array

```
arr = np.array([[1.5, 2.5, 3.5], [4.5, 5.5, 6.5]])
```

```
print("Array:")
print(arr)
print("Number of dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Total elements:", arr.size)
print("Data type:", arr.dtype)
print("Size of each element (bytes):", arr.itemsize)
print("Total memory (bytes):", arr.nbytes)
print("Transpose:")
print(arr.T)
```

```
Array:
[[1.5 2.5 3.5]
 [4.5 5.5 6.5]]
Number of dimensions: 2
Shape: (2, 3)
Total elements: 6
Data type: float64
Size of each element (bytes): 8
Total memory (bytes): 48
Transpose:
[[1.5 4.5]
 [2.5 5.5]
 [3.5 6.5]]
```

6.2.3 Creating Arrays Using Built-in Functions

NumPy provides numerous functions to create arrays of specific shapes and initial values.

np.zeros() — Creates an array filled with zeros:

```
# 1D array of zeros
zeros_1d = np.zeros(5)
print("1D Zeros:", zeros_1d)

# 2D array of zeros
zeros_2d = np.zeros((3, 4))
print("2D Zeros:")
print(zeros_2d)

# Zeros with integer data type
zeros_int = np.zeros((2, 3), dtype=int)
print("Zeros with int dtype:")
print(zeros_int)

1D Zeros: [0. 0. 0. 0. 0.]
2D Zeros:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
Zeros with int dtype:
[[0 0 0]
 [0 0 0]]
```

np.ones() — Creates an array filled with ones:

```
ones_2d = np.ones((2, 3))
print("Ones array:")
print(ones_2d)

ones_int = np.ones((2, 2), dtype=int)
print("Ones with int dtype:")
print(ones_int)

Ones array:
[[1. 1. 1.]
 [1. 1. 1.]]
Ones with int dtype:
[[1 1]
 [1 1]]
```

np.full() — Creates an array filled with a specified value :

```
full_arr = np.full((3, 3), 7)
print("Full array:")
print(full_arr)

Full array:
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

np.eye() — Creates an identity matrix (2D array with ones on the diagonal and zeros elsewhere):

```
identity_3 = np.eye(3)
print("3x3 Identity matrix:")
print(identity_3)

identity_4 = np.eye(4, dtype=int)
print("4x4 Identity matrix (int):")
print(identity_4)

3x3 Identity matrix:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
4x4 Identity matrix (int):
[[1 0 0 0]
 [0 1 0 0]
 [0 0 1 0]
 [0 0 0 1]]
```

np.arange() — Creates an array with evenly spaced values within a given interval:

```
# arange(start, stop, step)
arr1 = np.arange(10)
print("arange(10):", arr1)

arr2 = np.arange(2, 20, 3)
print("arange(2, 20, 3):", arr2)
```

```
arr3 = np.arange(0, 1, 0.1)
print("arange(0, 1, 0.1):", arr3)

arange(10): [0 1 2 3 4 5 6 7 8 9]
arange(2, 20, 3): [ 2  5  8 11 14 17]
arange(0, 1, 0.1): [0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7
0.8 0.9]
```

np.linspace() — Creates an array with a specified number of evenly spaced values between two endpoints:

```
# linspace(start, stop, num=50)
lin1 = np.linspace(0, 1, 5)
print("linspace(0, 1, 5):", lin1)

lin2 = np.linspace(0, 10, 11)
print("linspace(0, 10, 11):", lin2)

lin3 = np.linspace(0, 2*np.pi, 5)
print("linspace(0, 2*pi, 5):", lin3)

linspace(0, 1, 5): [0.  0.25 0.5  0.75 1.  ]
linspace(0, 10, 11): [ 0.  1.  2.  3.  4.  5.  6.  7
.  8.  9. 10.]
linspace(0, 2*pi, 5): [0.          1.57079633 3.14159
265 4.71238898 6.28318531]
```

Difference between arange and linspace: `arange` uses a step value and may not include the endpoint, while `Linspace` uses a count of elements and includes the endpoint by default. `Linspace` is more reliable for floating-point intervals.

np.geomspace() — Creates an array with geometrically spaced values:

```
geo = np.geomspace(1, 1000, 4)
print("geomspace(1, 1000, 4):", geo)

geomspace(1, 1000, 4): [ 1.  10. 100. 1000.]
```

np.random functions — Create arrays with random values (covered in detail in Section 6.8):

```
# Random integers
rand_int = np.random.randint(0, 10, size=(2, 3))
```



```
print("Random integers:")
print(rand_int)

# Random floats between 0 and 1
rand_float = np.random.random((2, 3))
print("Random floats:")
print(rand_float)
```

6.2.4 Data Types in NumPy

NumPy provides a rich set of data types (dtypes) that are more specific than Python's built-in types. Using appropriate data types can significantly improve memory efficiency and computational speed.

NumPy data types

Data Type	Description
int8, int16, int32, int64	Signed integers of different sizes
uint8, uint16, uint32, uint64	Unsigned integers of different sizes
float16, float32, float64	Floating-point numbers of different precision
complex64, complex128	Complex numbers
bool	Boolean values (True/False)
str_	Fixed-length string type
object	Python object type

```
# Specifying data type during creation
int_arr = np.array([1, 2, 3], dtype=np.int32)
print("int32 array:", int_arr, "dtype:", int_arr.dtype)

float_arr = np.array([1, 2, 3], dtype=np.float64)
print("float64 array:", float_arr, "dtype:", float_arr.dtype)

bool_arr = np.array([0, 1, -1, 0], dtype=np.bool_)
print("bool array:", bool_arr, "dtype:", bool_arr.dtype)

complex_arr = np.array([1+2j, 3+4j], dtype=np.comple
```

```
x128)
print("complex array:", complex_arr, "dtype:", complex_arr.dtype)

# Memory comparison
arr_64 = np.zeros(1000000, dtype=np.int64)
arr_8 = np.zeros(1000000, dtype=np.int8)
print(f"\nint64 array memory: {arr_64.nbytes} bytes")
print(f"int8 array memory: {arr_8.nbytes} bytes")

int32 array: [1 2 3] dtype: int32
float64 array: [1. 2. 3.] dtype: float64
bool array: [False True True False] dtype: bool
complex array: [1.+2.j 3.+4.j] dtype: complex128

int64 array memory: 8000000 bytes
int8 array memory: 1000000 bytes
```

6.2.5 Type Conversion

You can convert the data type of an array using the `astype()` method:

```
arr = np.array([1.5, 2.7, 3.9, 4.2])
print("Original array:", arr, "dtype:", arr.dtype)

# Convert to integer (truncates, does not round)
arr_int = arr.astype(int)
print("As integer:", arr_int, "dtype:", arr_int.dtype)

# Convert to string
arr_str = arr.astype(str)
print("As string:", arr_str, "dtype:", arr_str.dtype)

# Convert to float32
arr_f32 = arr.astype(np.float32)
```

```
print("As float32:", arr_f32, "dtype:",  
      arr_f32.dtype)  
  
Original array: [1.5 2.7 3.9 4.2] dtype: float64  
As integer: [1 2 3 4] dtype: int64  
As string: ['1.5' '2.7' '3.9' '4.2'] dtype:  
<U32>  
As float32: [1.5 2.7 3.9 4.2] dtype: float32
```

6.2.6 Reshaping Arrays

Reshaping changes the shape of an array without altering its data. The `reshape()` method returns a new array with the desired shape, provided that the total number of elements remains the same.

```
arr = np.arange(12)  
print("Original array:", arr)  
print("Shape:", arr.shape)  
  
# Reshape to 3x4  
reshaped = arr.reshape(3, 4)  
print("\nReshaped to 3x4:")  
print(reshaped)  
  
# Reshape to 4x3  
reshaped2 = arr.reshape(4, 3)  
print("\nReshaped to 4x3:")  
print(reshaped2)  
  
# Reshape to 2x2x3  
reshaped3 = arr.reshape(2, 2, 3)  
print("\nReshaped to 2x2x3:")  
print(reshaped3)  
  
Original array: [ 0  1  2  3  4  5  6  7  8  9 10 11  
                ]  
Shape: (12,)  
  
Reshaped to 3x4:  
[[ 0  1  2  3]  
 [ 4  5  6  7]
```

```
[ 8  9 10 11]]
```

Reshaped to 4x3:

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

Reshaped to 2x2x3:

```
[[[ 0  1  2]
   [ 3  4  5]]
 [[ 6  7  8]
   [ 9 10 11]]]
```

Using -1 for automatic dimension calculation:

```
arr = np.arange(12)

# NumPy automatically calculates the dimension marked as -1
auto_resaped = arr.reshape(3, -1)
print("Reshape with -1 (3, -1):")
print(auto_resaped)
print("Shape:", auto_resaped.shape)

auto_resaped2 = arr.reshape(-1, 4)
print("\nReshape with -1 (-1, 4):")
print(auto_resaped2)
print("Shape:", auto_resaped2.shape)

Reshape with -1 (3, -1):
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Shape: (3, 4)

Reshape with -1 (-1, 4):
[[ 0  1  2  3]
 [ 4  5  6  7]]
```

```
[ 8  9 10 11]]
Shape: (3, 4)
```

flatten() vs ravel():

Both methods convert a multi-dimensional array to a 1D array, but they differ in memory behavior:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

# flatten() returns a copy
flat = arr.flatten()
flat[0] = 99
print("After modifying flattened array:")
print("Original array:")
print(arr)
print("Flattened array:", flat)

# ravel() returns a view (when possible)
rav = arr.ravel()
rav[0] = 99
print("\nAfter modifying raveled array:")
print("Original array:")
print(arr)
print("Raveled array:", rav)

After modifying flattened array:
Original array:
[[1 2 3]
 [4 5 6]]
Flattened array: [99  2  3  4  5  6]

After modifying raveled array:
Original array:
[[99  2  3]
 [ 4  5  6]]
Raveled array: [99 2 3 4 5 6]
```

6.2.7 Adding and Removing Dimensions

np.newaxis or None — Adds a new axis of size 1:

```
arr = np.array([1, 2, 3, 4])
print("Original shape:", arr.shape) # (4,)

# Add a new axis at the end (row vector to column vector)
col_vec = arr[:, np.newaxis]
print("Shape with newaxis at end:", col_vec.shape)
# (4, 1)

# Add a new axis at the beginning
row_vec = arr[np.newaxis, :]
print("Shape with newaxis at start:", row_vec.shape)
# (1, 4)

Original shape: (4,)
Shape with newaxis at end: (4, 1)
Shape with newaxis at start: (1, 4)
```

np.expand_dims() — Explicitly expands dimensions:

```
arr = np.array([1, 2, 3, 4])
expanded = np.expand_dims(arr, axis=0)
print("Expanded at axis 0:", expanded.shape)

expanded2 = np.expand_dims(arr, axis=1)
print("Expanded at axis 1:", expanded2.shape)

Expanded at axis 0: (1, 4)
Expanded at axis 1: (4, 1)
```

np.squeeze() — Removes axes of size 1:

```
arr = np.array([[[1], [2], [3]]])
print("Original shape:", arr.shape) # (1, 3, 1)

squeezed = np.squeeze(arr)
print("Squeezed shape:", squeezed.shape) # (3,)

# Squeeze only specific axis
squeezed_axis = np.squeeze(arr, axis=0)
print("Squeezed axis 0:", squeezed_axis.shape) # (3, 1)
```

```
Original shape: (1, 3, 1)
Squeezed shape: (3,)
Squeezed axis 0: (3, 1)
```

6.2.8 Stacking and Concatenating Arrays

np.concatenate() — Joins arrays along an existing axis:

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Concatenate along rows (axis=0)
concat_rows = np.concatenate((a, b), axis=0)
print("Concatenated along rows:")
print(concat_rows)

# Concatenate along columns (axis=1)
concat_cols = np.concatenate((a, b), axis=1)
print("\nConcatenated along columns:")
print(concat_cols)

Concatenated along rows:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]

Concatenated along columns:
[[1 2 5 6]
 [3 4 7 8]]
```

np.vstack() — Stacks arrays vertically (row-wise):

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

vstacked = np.vstack((a, b))
print("vstack result:")
print(vstacked)
```

```
vstack result:
[[1 2 3]
 [4 5 6]]
```

np.hstack() — Stacks arrays horizontally (column-wise):

```
a = np.array([[1], [2], [3]])
b = np.array([[4], [5], [6]])

hstacked = np.hstack((a, b))
print("hstack result:")
print(hstacked)

hstack result:
[[1 4]
 [2 5]
 [3 6]]
```

np.stack() — Stacks arrays along a new axis:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

stacked = np.stack((a, b), axis=0)
print("Stacked along axis 0:")
print(stacked)
print("Shape:", stacked.shape)

stacked1 = np.stack((a, b), axis=1)
print("\nStacked along axis 1:")
print(stacked1)
print("Shape:", stacked1.shape)

Stacked along axis 0:
[[1 2 3]
 [4 5 6]]
Shape: (2, 3)

Stacked along axis 1:
[[1 4]
 [2 5]]
```



```
[3 6]]
Shape: (3, 2)
```

np.split() — Splits an array into multiple sub-arrays:

```
arr = np.arange(9).reshape(3, 3)
print("Original array:")
print(arr)

# Split into 3 sub-arrays along rows
split_rows = np.split(arr, 3, axis=0)
print("\nSplit along rows:")
for i, sub in enumerate(split_rows):
    print(f"Part {i}: {sub}")

# Split into 3 sub-arrays along columns
split_cols = np.split(arr, 3, axis=1)
print("\nSplit along columns:")
for i, sub in enumerate(split_cols):
    print(f"Part {i}: {sub}")

Original array:
[[0 1 2]
 [3 4 5]
 [6 7 8]]

Split along rows:
Part 0: [[0 1 2]]
Part 1: [[3 4 5]]
Part 2: [[6 7 8]]

Split along columns:
Part 0: [[0]
 [3]
 [6]]
Part 1: [[1]
 [4]
 [7]]
Part 2: [[2]
 [5]
 [8]]
```

6.3 Indexing

Indexing in NumPy allows you to access individual elements or subsets of elements from an array. NumPy supports both basic indexing and advanced indexing.

6.3.1 1D Array Indexing

Indexing in 1D arrays works similarly to Python lists, with zero-based indexing:

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])

print("First element:", arr[0])
print("Third element:", arr[2])
print("Last element:", arr[-1])
print("Second to last:", arr[-2])

First element: 10
Third element: 30
Last element: 80
Second to last: 70
```

6.3.2 2D Array Indexing

For multi-dimensional arrays, you can use comma-separated indices to access elements. The first index refers to the row, and the second index refers to the column:

```
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

# Access single element
print("Element at [0, 0]:", arr[0, 0])
print("Element at [1, 2]:", arr[1, 2])
print("Element at [2, 3]:", arr[2, 3])

# Using negative indices
print("Element at [-1, -1]:", arr[-1, -1])
print("Element at [-2, -3]:", arr[-2, -3])

Element at [0, 0]: 1
Element at [1, 2]: 7
Element at [2, 3]: 12
```

```
Element at [-1, -1]: 12
Element at [-2, -3]: 5
```

6.3.3 3D Array Indexing

```
arr = np.arange(24).reshape(2, 3, 4)
print("3D Array:")
print(arr)

# Access element at [0, 1, 2]
print("\nElement at [0, 1, 2]:", arr[0, 1, 2])

# Access entire first "layer"
print("\nFirst layer:")
print(arr[0])

# Access entire second row of first layer
print("\nSecond row of first layer:")
print(arr[0, 1])

3D Array:
[[[ 0  1  2  3]
   [ 4  5  6  7]
   [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]

Element at [0, 1, 2]: 6

First layer:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

Second row of first layer:
[4 5 6 7]
```

6.3.4 Boolean Indexing

Boolean indexing allows you to select elements based on conditions. You pass a boolean array of the same shape as the original array, and NumPy returns the elements where the boolean value is True:

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])

# Select elements greater than 40
mask = arr > 40
print("Boolean mask:", mask)
print("Elements > 40:", arr[mask])

# Direct boolean indexing
print("Elements < 30:", arr[arr < 30])

# Combining conditions (use & for AND, | for OR)
print("Elements between 20 and 60:", arr[(arr >= 20)
& (arr <= 60)])
print("Elements < 20 or > 70:", arr[(arr < 20) | (ar
r > 70)])

Boolean mask: [False False False False  True  True
 True  True]
Elements > 40: [50 60 70 80]
Elements < 30: [10 20]
Elements between 20 and 60: [20 30 40 50 60]
Elements < 20 or > 70: [10 80]

Boolean indexing with 2D arrays:

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# Select all elements greater than 5
print("Elements > 5:", arr[arr > 5])

# Replace elements based on condition
arr[arr > 5] = 0
```

```
print("After replacing > 5 with 0:")
print(arr)
```

```
Elements > 5: [6 7 8 9]
After replacing > 5 with 0:
[[1 2 3]
 [4 5 0]
 [0 0 0]]
```

6.3.5 Fancy (Integer) Indexing

Fancy indexing allows you to access multiple elements at once using arrays of indices:

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])

# Access elements at specific positions
indices = [0, 2, 5, 7]
print("Elements at indices [0, 2, 5, 7]:", arr[indices])

# Access elements in a specific order
indices2 = [7, 5, 3, 1]
print("Elements at indices [7, 5, 3, 1]:", arr[indices2])

Elements at indices [0, 2, 5, 7]: [10 30 60 80]
Elements at indices [7, 5, 3, 1]: [80 60 40 20]

Fancy indexing with 2D arrays:

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

# Select specific rows
rows = [0, 2]
print("Rows 0 and 2:")
print(arr[rows])

# Select specific elements: (0,1), (1,2), (2,3)
row_idx = [0, 1, 2]
```

```
col_idx = [1, 2, 3]
print("\nElements at (0,1), (1,2), (2,3):", arr[row_
idx, col_idx])
```

```
Rows 0 and 2:
[[ 1  2  3  4]
 [ 9 10 11 12]]
```

```
Elements at (0,1), (1,2), (2,3): [ 2  7 12]
```

6.3.6 Modifying Elements Through Indexing

```
arr = np.zeros((3, 4))
print("Original array:")
print(arr)

# Modify single element
arr[0, 0] = 1
arr[1, 2] = 5
arr[2, 3] = 9
print("\nAfter modifying specific elements:")
print(arr)

# Modify entire row
arr[1, :] = 7
print("\nAfter setting row 1 to 7:")
print(arr)

# Modify entire column
arr[:, 2] = 3
print("\nAfter setting column 2 to 3:")
print(arr)

# Modify using boolean indexing
arr[arr == 0] = -1
print("\nAfter replacing 0 with -1:")
print(arr)

Original array:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
```

```
[0. 0. 0. 0.]
```

After modifying specific elements:

```
[[1. 0. 0. 0.]  
 [0. 0. 5. 0.]  
 [0. 0. 0. 9.]]
```

After setting row 1 to 7:

```
[[1. 0. 0. 0.]  
 [7. 7. 7. 7.]  
 [0. 0. 0. 9.]]
```

After setting column 2 to 3:

```
[[1. 0. 3. 0.]  
 [7. 7. 3. 7.]  
 [0. 0. 3. 9.]]
```

After replacing 0 with -1:

```
[[ 1. -1.  3. -1.]  
 [ 7.  7.  3.  7.]  
 [-1. -1.  3.  9.]]
```

6.4 Slicing

Slicing allows you to extract a portion (subarray) of an array. NumPy slicing follows the syntax start:stop:step, similar to Python lists but extended to multiple dimensions.

6.4.1 1D Array Slicing

```
arr = np.array([0, 10, 20, 30, 40, 50, 60, 70, 80, 90])  
  
# Basic slicing  
print("arr[2:6]:", arr[2:6])           # Elements from index 2 to 5  
print("arr[:4]:", arr[:4])             # First 4 elements  
print("arr[6:]:", arr[6:])             # Elements from index 6 to end  
print("arr[:,]", arr[:,])              # Entire array (copy of view)
```

```
# Slicing with step
print("arr[::2]:", arr[::2])      # Every other element
print("arr[1::2]:", arr[1::2])  # Every other element starting from index 1
print("arr[::3]:", arr[::3])    # Every third element

# Negative slicing
print("arr[-3:]:", arr[-3:])     # Last 3 elements
print("arr[-5:-2]:", arr[-5:-2]) # Elements from 5th last to 3rd last
print("arr[::-1]:", arr[::-1])   # Reversed array
print("arr[::-2]:", arr[::-2])   # Reversed with step 2

arr[2:6]: [20 30 40 50]
arr[:4]: [ 0 10 20 30]
arr[6:]: [60 70 80 90]
arr[:]: [ 0 10 20 30 40 50 60 70 80 90]
arr[::2]: [ 0 20 40 60 80]
arr[1::2]: [10 30 50 70 90]
arr[::3]: [ 0 30 60 90]
arr[-3:]: [70 80 90]
arr[-5:-2]: [50 60 70]
arr[::-1]: [90 80 70 60 50 40 30 20 10  0]
arr[::-2]: [90 70 50 30 10]
```

6.4.2 2D Array Slicing

For 2D arrays, you specify slices for each dimension separated by commas:

```
arr = np.array([[ 1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10],
                [11, 12, 13, 14, 15],
                [16, 17, 18, 19, 20]])

# Select specific rows
print("First two rows:")
print(arr[0:2])
```



```
# Select specific columns
print("\nFirst three columns:")
print(arr[:, 0:3])

# Select a sub-matrix
print("\nSub-matrix [1:3, 2:5]:")
print(arr[1:3, 2:5])

# Select every other row
print("\nEvery other row:")
print(arr[::2])

# Select every other column
print("\nEvery other column:")
print(arr[:, ::2])

# Combination:every other row and every other column
print("\nEvery other row and column:")
print(arr[::2, ::2])

First two rows:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]

First three columns:
[[ 1  2  3]
 [ 6  7  8]
 [11 12 13]
 [16 17 18]]

Sub-matrix [1:3, 2:5]:
[[ 8  9 10]
 [13 14 15]]

Every other row:
[[ 1  2  3  4  5]
 [11 12 13 14 15]]

Every other column:
[[ 1  3  5]
```

```
[ 6  8 10]
[11 13 15]
[16 18 20]]
```

Every other row and column:

```
[[ 1  3  5]
 [11 13 15]]
```

6.4.3 Mixing Indexing and Slicing

You can combine integer indexing with slicing to select specific rows or columns along with ranges:

```
arr = np.array([[ 1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10],
                [11, 12, 13, 14, 15],
                [16, 17, 18, 19, 20]])

# Select row 1, columns 1 to 4
print("Row 1, columns 1:4:", arr[1, 1:4])

# Select rows 0 and 2, all columns
print("\nRows 0 and 2:")
print(arr[[0, 2], :])

# Select all rows, columns 1 and 3
print("\nColumns 1 and 3:")
print(arr[:, [1, 3]])

# Select row 2, all columns as a 1D array
print("\nRow 2 as 1D:", arr[2, :])

Row 1, columns 1:4: [7 8 9]

Rows 0 and 2:
[[ 1  2  3  4  5]
 [11 12 13 14 15]]

Columns 1 and 3:
[[ 2  4]
 [ 7  9]]
```

```
[12 14]
[17 19]]
```

```
Row 2 as 1D: [11 12 13 14 15]
```

6.4.4 Views vs Copies in Slicing

An important concept in NumPy is the distinction between views and copies:

- **View:** A view shares the same data as the original array. Modifying the view modifies the original array.
- **Copy:** A copy has its own independent data. Modifying the copy does not affect the original array.

Slicing creates a view, while fancy indexing creates a copy:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

# Slicing creates a view
view = arr[2:6]
print("Original array:", arr)
print("View:", view)

# Modifying the view affects the original
view[0] = 99
print("\nAfter modifying view[0] = 99:")
print("Original array:", arr)
print("View:", view)

# Fancy indexing creates a copy
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
copy = arr[[2, 3, 4, 5]]
copy[0] = 99
print("\nAfter modifying copy[0] = 99:")
print("Original array:", arr)
print("Copy:", copy)

# Explicit copy using .copy()
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
explicit_copy = arr[2:6].copy()
explicit_copy[0] = 99
```

```
print("\nAfter modifying explicit_copy[0] = 99:")
print("Original array:", arr)
print("Explicit copy:", explicit_copy)

Original array: [1 2 3 4 5 6 7 8]
View: [3 4 5 6]

After modifying view[0] = 99:
Original array: [ 1  2 99  4  5  6  7  8]
View: [99  4  5  6]

After modifying copy[0] = 99:
Original array: [1 2 3 4 5 6 7 8]
Copy: [99  4  5  6]

After modifying explicit_copy[0] = 99:
Original array: [1 2 3 4 5 6 7 8]
Explicit copy: [99  4  5  6]
```

Understanding the difference between views and copies is crucial for avoiding unintended side effects when working with NumPy arrays.

6.5 Broadcasting

Broadcasting is one of NumPy's most powerful features. It describes how NumPy treats arrays with different shapes during arithmetic operations. Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes.

6.5.1 Broadcasting Rules

NumPy compares the shapes of two arrays element-wise, starting from the trailing dimensions and working forward. Two dimensions are compatible when:

1. They are equal, **or**
2. One of them is 1 (or missing entirely)

If these conditions are not met for every pair of dimensions, a `ValueError` is raised.

```
Array A shape: (4, 3)
Array B shape: (    3)
                Compatible -- B is broadcast to (4, 3)
```

```
Array A shape: (4, 1)
Array B shape: (    3)
                Compatible -- both broadcast to (4, 3)

Array A shape: (4, 3)
Array B shape: (4,   )
                Incompatible -- 3 != 4
```

6.5.2 Scalar and 1D Array Broadcasting

```
# Scalar broadcast to array
arr = np.array([1, 2, 3, 4, 5])
print("Array:", arr)
print("Array + 10:", arr + 10)
print("Array * 2:", arr * 2)
print("Array ** 2:", arr ** 2)

Array: [1 2 3 4 5]
Array + 10: [11 12 13 14 15]
Array * 2: [ 2  4  6  8 10]
Array ** 2: [ 1  4  9 16 25]
```

6.5.3 2D Array and 1D Array Broadcasting

```
# 2D array + 1D array
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
vector = np.array([10, 20, 30])

print("Matrix:")
print(matrix)
print("\nVector:", vector)
print("\nMatrix + Vector:")
print(matrix + vector)

Matrix:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Vector: [10 20 30]
```

```
Matrix + Vector:  
[[11 22 33]  
 [14 25 36]  
 [17 28 39]]
```

In this example, the 1D array vector with shape (3,) is broadcast to match the shape (3, 3) of matrix. The vector is effectively treated as if it were `[[10, 20, 30], [10, 20, 30], [10, 20, 30]]`.

6.5.4 Column Vector and Row Vector Broadcasting

```
# Column vector (3, 1) + Row vector (1, 4)  
col = np.array([[1], [2], [3]])      # shape: (3, 1)  
row = np.array([[10, 20, 30, 40]]) # shape: (1, 4)  
  
print("Column vector shape:", col.shape)  
print("Row vector shape:", row.shape)  
print("\nBroadcasted result:")  
print(col + row)  
print("Result shape:", (col + row).shape)  
  
Column vector shape: (3, 1)  
Row vector shape: (1, 4)  
  
Broadcasted result:  
[[11 21 31 41]  
 [12 22 32 42]  
 [13 23 33 43]]  
Result shape: (3, 4)
```

This is a very common pattern in data science, where you might want to compute the outer product or apply operations between feature vectors.

6.5.5 Practical Example: Normalizing Data

```
# Normalize each column of a dataset (zero mean, unit  
variance)  
data = np.array([[10, 200, 3],  
                 [20, 300, 5],  
                 [30, 400, 7],  
                 [40, 500, 9],  
                 [50, 600, 11]])
```

```
print("Original data:")
print(data)

# Calculate mean and standard deviation for each column
mean = data.mean(axis=0)
std = data.std(axis=0)
print("\nMean per column:", mean)
print("Std per column:", std)

# Normalize using broadcasting
normalized = (data - mean) / std
print("\nNormalized data:")
print(normalized)
print("\nNormalized mean:", normalized.mean(axis=0))
print("Normalized std:", normalized.std(axis=0))

Original data:
[[ 10 200   3]
 [ 20 300   5]
 [ 30 400   7]
 [ 40 500   9]
 [ 50 600  11]]

Mean per column: [ 30. 400.   7.]
Std per column: [14.14213562 141.42135624  2.82842712]

Normalized data:
[[-1.41421356 -1.41421356 -1.41421356]
 [-0.70710678 -0.70710678 -0.70710678]
 [ 0.          0.          0.          ]
 [ 0.70710678  0.70710678  0.70710678]
 [ 1.41421356  1.41421356  1.41421356]]

Normalized mean: [ 0.00000000e+00  0.00000000e+00  0.00000000e+00]
Normalized std: [1. 1. 1.]
```

6.5.6 Practical Example: Computing Distance Matrix

```
# Compute pairwise Euclidean distances between points
s
points = np.array([[0, 0],
                  [1, 1],
                  [3, 4]])

# Using broadcasting: ||a - b||^2 = ||a||^2 + ||b||^2 - 2*a.b
sq_norms = np.sum(points**2, axis=1)
dist_sq = sq_norms[:, np.newaxis] + sq_norms[np.newaxis, :] - 2 * points @ points.T
distances = np.sqrt(np.maximum(dist_sq, 0)) # Ensure non-negative

print("Points:")
print(points)
print("\nDistance matrix:")
print(distances)

Points:
[[0 0]
 [1 1]
 [3 4]]

Distance matrix:
[[0.          1.41421356  5.          ]
 [1.41421356  0.          3.60555128]
 [5.          3.60555128  0.          ]]
```

6.6 Universal Functions (ufuncs)

Universal functions (ufuncs) are functions that operate on arrays element-wise. They are highly optimized and provide a significant performance advantage over Python loops. NumPy provides a large collection of ufuncs for mathematical, logical, and bitwise operations.

6.6.1 Arithmetic Ufuncs

```
arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([1, 2, 3, 4])
```



```
print("arr1:", arr1)
print("arr2:", arr2)
print("np.add(arr1, arr2):", np.add(arr1, arr2))
print("np.subtract(arr1, arr2):", np.subtract(arr1,
arr2))
print("np.multiply(arr1, arr2):", np.multiply(arr1,
arr2))
print("np.divide(arr1, arr2):", np.divide(arr1, arr2
))
print("np.floor_divide(arr1, arr2):", np.floor_divid
e(arr1, arr2))
print("np.mod(arr1, arr2):", np.mod(arr1, arr2))
print("np.power(arr1, arr2):", np.power(arr1, arr2))

arr1: [10 20 30 40]
arr2: [ 1  2  3  4]
np.add(arr1, arr2): [11 22 33 44]
np.subtract(arr1, arr2): [ 9 18 27 36]
np.multiply(arr1, arr2): [ 10  40  90 160]
np.divide(arr1, arr2): [10. 10. 10. 10.]
np.floor_divide(arr1, arr2): [10 10 10 10]
np.mod(arr1, arr2): [0 0 0 0]
np.power(arr1, arr2): [      10      400    27000 25600
00]
```

These are equivalent to using operators (+, -, *, /, etc.), but the ufunc form allows additional parameters like `out`, `where`, and `dtype`.

6.6.2 Trigonometric Ufuncs

```
angles = np.array([0, np.pi/6, np.pi/4, np.pi/3, np.
pi/2])

print("Angles (radians):", angles)
print("sin(angles):", np.round(np.sin(angles), 4))
print("cos(angles):", np.round(np.cos(angles), 4))
print("tan(angles):", np.round(np.tan(angles), 4))

# Inverse trigonometric functions
values = np.array([-1, -0.5, 0, 0.5, 1])
```

```
print("\nValues:", values)
print("arcsin(values):", np.round(np.arcsin(values),
4))
print("arccos(values):", np.round(np.arccos(values),
4))

# Convert degrees to radians and vice versa
degrees = np.array([0, 30, 45, 60, 90])
print("\nDegrees:", degrees)
print("To radians:", np.round(np.deg2rad(degrees), 4
))
print("Back to degrees:", np.round(np.rad2deg(np.deg
2rad(degrees)), 4))

Angles (radians): [0.          0.52359878 0.78539816
1.04719755 1.57079633]
sin(angles): [0.          0.5      0.7071 0.866    1.        ]
cos(angles): [1.          0.866    0.7071 0.5       0.        ]
tan(angles): [0.          0.5774 1.          1.7321    inf]

Values: [-1.  -0.5  0.   0.5  1. ]
arcsin(values): [-1.5708 -0.5236  0.          0.5236  1.
5708]
arccos(values): [3.1416 2.0944 1.5708 1.0472 0.        ]

Degrees: [ 0 30 45 60 90]
To radians: [0.          0.5236 0.7854 1.0472 1.5708]
Back to degrees: [ 0. 30. 45. 60. 90.]
```

6.6.3 Exponential and Logarithmic Ufuncs

```
arr = np.array([1, 2, 3, 4, 5])

print("Array:", arr)
print("exp(arr):", np.round(np.exp(arr), 4))
print("exp2(arr):", np.exp2(arr))
print("log(arr):", np.round(np.log(arr), 4))
# Natural log
print("log2(arr):", np.round(np.log2(arr), 4))
# Log base 2
print("log10(arr):", np.round(np.log10(arr), 4))
```

```
# Log base 10

# Special functions
print("\nexpm1(arr):", np.round(np.expm1(arr), 4))
# exp(x) - 1
print("log1p(arr):", np.round(np.log1p(arr), 4))
# log(1 + x)

Array: [1 2 3 4 5]
exp(arr): [ 2.7183  7.3891 20.0855 54.5982
148.4132]
exp2(arr): [ 2.  4.  8. 16. 32.]
log(arr): [0. 0.6931 1.0986 1.3863 1.6094]
log2(arr): [0. 1. 1.585 2. 2.3219]
log10(arr): [0. 0.301 0.4771 0.6021 0.699 ]

expm1(arr): [ 1.7183  6.3891 19.0855 53.5982
147.4132]
log1p(arr): [0.6931 1.0986 1.3863 1.6094 1.7918]
```

6.6.4 Rounding Ufuncs

```
arr = np.array([1.23, 2.67, 3.14, 4.89, 5.50, -1.23,
-2.67])

print("Array:", arr)
print("round(arr, 1):", np.round(arr, 1))
print("round(arr, 0):", np.round(arr, 0))
print("floor(arr):", np.floor(arr))
print("ceil(arr):", np.ceil(arr))
print("trunc(arr):", np.trunc(arr))

Array: [ 1.23  2.67  3.14  4.89  5.5  -1.23 -2.67]
round(arr, 1): [ 1.2  2.7  3.1  4.9  5.5 -1.2 -2.7]
round(arr, 0): [ 1.  3.  3.  5.  6. -1. -3.]
floor(arr): [ 1.  2.  3.  4.  5. -2. -3.]
ceil(arr): [ 2.  3.  4.  5.  6. -1. -2.]
trunc(arr): [ 1.  2.  3.  4.  5. -1. -2.]
```

6.6.5 Statistical Ufuncs

```
arr = np.array([15, 23, 8, 42, 31, 17, 28, 36])

print("Array:", arr)
print("np.min(arr):", np.min(arr))
print("np.max(arr):", np.max(arr))
print("np.mean(arr):", np.mean(arr))
print("np.median(arr):", np.median(arr))
print("np.std(arr):", round(np.std(arr), 4))
print("np.var(arr):", round(np.var(arr), 4))
print("np.argmin(arr):", np.argmin(arr))    # Index o
f minimum
print("np.argmax(arr):", np.argmax(arr))    # Index o
f maximum

# Percentile
print("25th percentile:", np.percentile(arr, 25))
print("75th percentile:", np.percentile(arr, 75))

# Cumulative functions
print("\nCumulative sum:", np.cumsum(arr))
print("Cumulative product:", np.cumprod(arr))

Array: [15 23  8 42 31 17 28 36]
np.min(arr): 8
np.max(arr): 42
np.mean(arr): 25.0
np.median(arr): 25.5
np.std(arr): 10.6927
np.var(arr): 114.3333
np.argmin(arr): 2
np.argmax(arr): 3
25th percentile: 16.0
75th percentile: 33.25

Cumulative sum: [ 15  38  46  88 119 136 164 200]
Cumulative product: [      15      345     2760     115
920  3593520
61089840 1710515520 61578558720]
```

6.6.6 Aggregation Along Axes

For multi-dimensional arrays, you can specify the axis parameter to aggregate along a particular dimension:

```
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

print("Array:")
print(arr)

print("\nSum of all elements:", np.sum(arr))
print("Sum along rows (axis=0):", np.sum(arr, axis=0
))
print("Sum along columns (axis=1):", np.sum(arr, axis=1))

print("\nMean along rows (axis=0):", np.mean(arr, axis=0))
print("Mean along columns (axis=1):", np.mean(arr, axis=1))

print("\nMax along rows (axis=0):", np.max(arr, axis=0))
print("Max along columns (axis=1):", np.max(arr, axis=1))

print("\nArgmax along rows (axis=0):", np.argmax(arr, axis=0))
print("Argmax along columns (axis=1):", np.argmax(arr, axis=1))

Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Sum of all elements: 78
Sum along rows (axis=0): [15 18 21 24]
Sum along columns (axis=1): [10 26 42]
```

```
Mean along rows (axis=0): [5. 6. 7. 8.]
Mean along columns (axis=1): [ 2.5  6.5 10.5]

Max along rows (axis=0): [ 9 10 11 12]
Max along columns (axis=1): [ 4  8 12]

Argmax along rows (axis=0): [2 2 2 2]
Argmax along columns (axis=1): [3 3 3]
```

6.6.7 Comparison Ufuncs

```
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([5, 4, 3, 2, 1])

print("arr1:", arr1)
print("arr2:", arr2)
print("arr1 == arr2:", arr1 == arr2)
print("arr1 != arr2:", arr1 != arr2)
print("arr1 > arr2:", arr1 > arr2)
print("arr1 < arr2:", arr1 < arr2)
print("arr1 >= arr2:", arr1 >= arr2)
print("arr1 <= arr2:", arr1 <= arr2)

arr1: [1 2 3 4 5]
arr2: [5 4 3 2 1]
arr1 == arr2: [False False  True False False]
arr1 != arr2: [ True  True False  True  True]
arr1 > arr2: [False False False  True  True]
arr1 < arr2: [ True  True False False False]
arr1 >= arr2: [False False  True  True  True]
arr1 <= arr2: [ True  True  True False False]
```

6.6.8 Boolean Ufuncs

```
arr1 = np.array([True, False, True, False])
arr2 = np.array([True, True, False, False])

print("arr1:", arr1)
print("arr2:", arr2)
print("Logical AND:", np.logical_and(arr1, arr2))
print("Logical OR:", np.logical_or(arr1, arr2))
print("Logical NOT:", np.logical_not(arr1))
```

```
print("Logical XOR:", np.logical_xor(arr1, arr2))

# any() and all()
print("\nany(arr1):", np.any(arr1)) # True if any element is True
print("all(arr1):", np.all(arr1)) # True if all elements are True

# Using with conditions
arr = np.array([1, -2, 3, -4, 5, -6])
print("\nArray:", arr)
print("Positive AND even:", arr[np.logical_and(arr > 0, arr % 2 == 0)])

arr1: [ True False  True False]
arr2: [ True  True False False]
Logical AND: [ True False False False]
Logical OR: [ True  True  True False]
Logical NOT: [False  True False  True]
Logical XOR: [False  True  True False]

any(arr1): True
all(arr1): False

Array: [ 1 -2  3 -4  5 -6]
Positive AND even: []
```

6.6.9 Performance Comparison: Ufuncs vs Loops

```
import time

size = 1_000_000
arr1 = np.random.random(size)
arr2 = np.random.random(size)

# Using ufunc
start = time.time()
result_ufunc = arr1 + arr2
ufunc_time = time.time() - start

# Using Python loop
```

```
start = time.time()
result_loop = np.empty(size)
for i in range(size):
    result_loop[i] = arr1[i] + arr2[i]
loop_time = time.time() - start

print(f"Array size: {size:,}")
print(f"Ufunc time: {ufunc_time:.6f} seconds")
print(f"Loop time: {loop_time:.6f} seconds")
print(f"Ufunc is approximately {loop_time/ufunc_time:.0f}x faster")

Array size: 1,000,000
Ufunc time: 0.002014 seconds
Loop time: 0.356842 seconds
Ufunc is approximately 177x faster
```

This dramatic speed difference is why vectorized operations with ufuncs are always preferred over explicit Python loops when working with NumPy arrays.

6.7 Linear Algebra Operations

Linear algebra is a cornerstone of data science and machine learning. NumPy provides a dedicated submodule `np.linalg` for linear algebra operations.

6.7.1 Matrix Multiplication

Matrix multiplication is one of the most common linear algebra operations. In NumPy, there are multiple ways to perform it:

```
A = np.array([[1, 2],
               [3, 4]])

B = np.array([[5, 6],
               [7, 8]])

# Method 1: np.dot()
print("np.dot(A, B):")
print(np.dot(A, B))

# Method 2: @ operator (recommended for Python 3.5+)
print("\nA @ B:")
```



```
print(A @ B)

# Method 3: np.matmul()
print("\nnp.matmul(A, B):")
print(np.matmul(A, B))

# Element-wise multiplication (NOT matrix multiplication)
print("\nA * B (element-wise):")
print(A * B)

np.dot(A, B):
[[19 22]
 [43 50]]

A @ B:
[[19 22]
 [43 50]]

np.matmul(A, B):
[[19 22]
 [43 50]]

A * B (element-wise):
[[ 5 12]
 [21 32]]
```

Important: Always use @ or np.matmul() for matrix multiplication, not * (which performs element-wise multiplication).

6.7.2 Transpose

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print("Original matrix:")
print(A)
print("Shape:", A.shape)

print("\nTranspose (A.T):")
print(A.T)
```

```
print("Shape:", A.T.shape)

print("\nTranspose (np.transpose(A)):")
print(np.transpose(A))

# Transpose of a 1D array (no effect)
v = np.array([1, 2, 3])
print("\n1D array:", v)
print("v.T:", v.T) # Same as v
print("v[:, np.newaxis].T:", v[:, np.newaxis].T)

Original matrix:
[[1 2 3]
 [4 5 6]]
Shape: (2, 3)

Transpose (A.T):
[[1 4]
 [2 5]
 [3 6]]
Shape: (3, 2)

Transpose (np.transpose(A)):
[[1 4]
 [2 5]
 [3 6]]

1D array: [1 2 3]
v.T: [1 2 3]
v[:, np.newaxis].T: [[1 2 3]]
```

6.7.3 Determinant

```
A = np.array([[1, 2],
               [3, 4]])

det_A = np.linalg.det(A)
print("Matrix A:")
print(A)
print(f"Determinant of A: {det_A:.2f}")
```

```
B = np.array([[6, 1, 1],
              [4, -2, 5],
              [2, 8, 7]])

det_B = np.linalg.det(B)
print("\nMatrix B:")
print(B)
print(f"Determinant of B: {det_B:.2f}")

Matrix A:
[[1 2]
 [3 4]]
Determinant of A: -2.00

Matrix B:
[[ 6  1  1]
 [ 4 -2  5]
 [ 2  8  7]]
Determinant of B: -306.00
```

6.7.4 Inverse of a Matrix

```
A = np.array([[1, 2],
              [3, 4]])

A_inv = np.linalg.inv(A)
print("Matrix A:")
print(A)

print("\nInverse of A:")
print(A_inv)

print("\nA @ A_inv (should be identity):")
print(np.round(A @ A_inv, 10))

Matrix A:
[[1 2]
 [3 4]]

Inverse of A:
[[-2.   1. ]
```

```
[ 1.5 -0.5]]

A @ A_inv (should be identity):
[[ 1.  0.]
 [ 0.  1.]]

A matrix must be square and non-singular
(determinant  $\neq 0$ ) to have an inverse:

# Singular matrix (determinant = 0)
singular = np.array([[1, 2],
                    [2, 4]])

try:
    inv = np.linalg.inv(singular)
except np.linalg.LinAlgError as e:
    print("Error:", e)

Error: Singular matrix
```

6.7.5 Solving Linear Systems

Given a system of linear equations $AX = b$, you can solve for x using `np.linalg.solve()`:

```
# Solve the system:
# 2x + y = 5
# 3x + 2y = 8

A = np.array([[2, 1],
              [3, 2]])
b = np.array([5, 8])

x = np.linalg.solve(A, b)
print("Solution:", x)
print("Verification A @ x:", A @ x)

Solution: [2. 1.]
Verification A @ x: [5. 8.]
```

Note: `np.linalg.solve()` is preferred over computing the inverse explicitly

($x = A^{-1}b$) because it is more numerically stable and efficient.

6.7.6 Eigenvalues and Eigenvectors

```
A = np.array([[4, -2],
              [1,  1]])

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Matrix A:")
print(A)

print("\nEigenvalues:")
print(eigenvalues)

print("\nEigenvectors (as columns):")
print(eigenvectors)

# Verify: A @ v = lambda * v
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]
    lambda_val = eigenvalues[i]
    print(f"\nVerification for eigenvalue {lambda_val}:")
    print(f"A @ v = {A @ v}")
    print(f"lambda * v = {lambda_val * v}")

Matrix A:
[[ 4 -2]
 [ 1  1]]

Eigenvalues:
[3.  2.]

Eigenvectors (as columns):
[[0.89442719  0.70710678]
 [0.4472136  0.70710678]]

Verification for eigenvalue 3.0:
A @ v = [2.68328157  1.34164079]
lambda * v = [2.68328157  1.34164079]
Verification for eigenvalue 2.0:
```

```
A @ v = [1.41421356 1.41421356]
lambda * v = [1.41421356 1.41421356]
```

6.7.7 Singular Value Decomposition (SVD)

SVD decomposes a matrix A into three matrices: $A = U\Sigma V^T$. It is widely used in dimensionality reduction, recommendation systems, and image compression.

```
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9],
              [10, 11, 12]])

U, S, Vt = np.linalg.svd(A)

print("Original matrix shape:", A.shape)
print("U shape:", U.shape)
print("S shape:", S.shape)
print("Vt shape:", Vt.shape)

print("\nU:")
print(np.round(U, 4))

print("\nSingular values:", np.round(S, 4))

print("\nVt:")
print(np.round(Vt, 4))

# Reconstruct the original matrix
S_matrix = np.zeros(A.shape)
S_matrix[:len(S), :len(S)] = np.diag(S)
A_reconstructed = U @ S_matrix @ Vt
print("\nReconstructed matrix:")
print(np.round(A_reconstructed, 4))

Original matrix shape: (4, 3)
U shape: (4, 4)
S shape: (3,)
Vt shape: (3, 3)

U:
```

```
[[ -0.1409  -0.8247   0.5477   0.069 ]
 [ -0.343   -0.4263  -0.7331   0.3912]
 [ -0.5451  -0.0278  -0.1475  -0.8254]
 [ -0.7472   0.3706   0.3739   0.4042]]

Singular values: [25.4624  1.2907  0.      ]

Vt:
[[-0.5045 -0.5745 -0.6445]
 [ 0.7608  0.0571 -0.6465]
 [ 0.4082 -0.8165  0.4082]]

Reconstructed matrix:
[[ 1.  2.  3.]
 [ 4.  5.  6.]
 [ 7.  8.  9.]
 [10. 11. 12.]]
```

6.7.8 Matrix Norms

```
A = np.array([[1, 2],
               [3, 4]])

print("Matrix A:")
print(A)
print("Frobenius norm:", np.linalg.norm(A, 'fro'))
print("L2 norm (spectral norm):", np.linalg.norm(A,
2))
print("L1 norm:", np.linalg.norm(A, 1))
print("L-inf norm:", np.linalg.norm(A, np.inf))

# Vector norms
v = np.array([3, 4])
print("\nVector v:", v)
print("L2 norm:", np.linalg.norm(v))
print("L1 norm:", np.linalg.norm(v, 1))
print("L-inf norm:", np.linalg.norm(v, np.inf))

Matrix A:
[[1 2]
 [3 4]]
```

```
Frobenius norm: 5.477225575051661
L2 norm (spectral norm): 5.464985704219043
L1 norm: 6.0
L-inf norm: 7.0
```

```
Vector v: [3 4]
L2 norm: 5.0
L1 norm: 7.0
L-inf norm: 4.0
```

6.7.9 Rank of a Matrix

```
A = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])

print("Matrix A:")
print(A)
print("Rank:", np.linalg.matrix_rank(A))

B = np.array([[1, 0, 0],
               [0, 1, 0],
               [0, 0, 1]])

print("\nMatrix B (Identity):")
print(B)
print("Rank:", np.linalg.matrix_rank(B))

Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Rank: 2

Matrix B (Identity):
[[1 0 0]
 [0 1 0]
 [0 0 1]]
Rank: 3
```


6.7.10 Trace of a Matrix

The trace is the sum of the diagonal elements:

```
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print("Matrix A:")
print(A)
print("Trace:", np.trace(A))
print("Trace (sum of diagonal):", np.sum(np.diag(A))
)

Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Trace: 15
Trace (sum of diagonal): 15
```

6.8 Random Numbers

NumPy's random module (`np.random`) provides a comprehensive suite of functions for generating random numbers and performing random operations. Starting from NumPy 1.17, the recommended approach is to use the new Generator API through `np.random.default_rng()`, which offers better statistical properties and performance.

6.8.1 The New Generator API

```
# Create a random number generator
rng = np.random.default_rng(seed=42)

print("Random float between 0 and 1:")
print(rng.random())

print("\nArray of 5 random floats:")
print(rng.random(5))

print("\n2D array of random floats (3x4):")
print(rng.random((3, 4)))
```

Random float between 0 and 1:

```
0.7739560485559633
```

Array of 5 random floats:

```
[0.43887844 0.85859792 0.69736803 0.09417735 0.97562
235]
```

2D array of random floats (3x4):

```
[[0.7611397 0.78606431 0.12811363 0.45038594]
 [0.37079802 0.92676499 0.64386512 0.82276761]
 [0.44341426 0.22723872 0.55458479 0.06381726]]
```

Setting a seed ensures reproducibility:

```
rng1 = np.random.default_rng(seed=42)
rng2 = np.random.default_rng(seed=42)

print("rng1:", rng1.random(3))
print("rng2:", rng2.random(3))
print("Same results:", np.allclose(rng1.random(3), r
ng2.random(3)))

rng1: [0.77395605 0.43887844 0.85859792]
rng2: [0.77395605 0.43887844 0.85859792]
Same results: False
```

6.8.2 Random Integers

```
rng = np.random.default_rng(seed=42)

# Random integer in [low, high)
print("Single random integer (0-10):", rng.integers(
0, 10))

# Array of random integers
print("5 random integers (0-100):", rng.integers(0,
100, size=5))

# 2D array of random integers
print("3x3 random integers (1-7):")
print(rng.integers(1, 7, size=(3, 3)))
```

```
# Including the endpoint
print("5 random integers (0-10, inclusive):",
      rng.integers(0, 10, size=5, endpoint=True))

Single random integer (0-10): 0
5 random integers (0-100): [ 7 35 59 41 45]
3x3 random integers (1-7):
[[5 4 3]
 [1 6 3]
 [3 4 5]]
5 random integers (0-10, inclusive): [9 0 0 1 4]
```

6.8.3 Random Distributions

Normal (Gaussian) Distribution:

```
rng = np.random.default_rng(seed=42)

# Standard normal (mean=0, std=1)
standard_normal = rng.standard_normal(5)
print("Standard normal:", np.round(standard_normal,
4))

# Normal with specified mean and std
normal = rng.normal(loc=100, scale=15, size=5)
print("Normal (mean=100, std=15):", np.round(normal,
4))

# Generate a larger sample and verify statistics
large_sample = rng.normal(loc=100, scale=15, size=10
000)
print(f"\nLarge sample - Mean: {large_sample.mean():
.2f},"
      f" Std: {large_sample.std():.2f}")

Standard normal: [ 0.3047  1.2024 -0.2412 -1.7499 -0
.189 ]
Normal (mean=100, std=15): [ 97.4079  89.3457 107.79
05  98.102   88.2423]

Large sample - Mean: 100.01, Std: 14.99
```

Uniform Distribution:

```
rng = np.random.default_rng(seed=42)

# Uniform between 0 and 1
uniform01 = rng.random(5)
print("Uniform [0, 1):", np.round(uniform01, 4))

# Uniform between low and high
uniform = rng.uniform(low=10, high=50, size=5)
print("Uniform [10, 50):", np.round(uniform, 4))

Uniform [0, 1): [0.774  0.4389 0.8586 0.6974 0.0942]
Uniform [10, 50): [13.6892 43.6188 11.6666 29.8742 2
4.0338]
```

Other Distributions:

```
rng = np.random.default_rng(seed=42)

# Binomial distribution
binomial = rng.binomial(n=10, p=0.5, size=10)
print("Binomial (n=10, p=0.5):", binomial)

# Poisson distribution
poisson = rng.poisson(lam=5, size=10)
print("Poisson (lam=5):", poisson)

# Exponential distribution
exponential = rng.exponential(scale=2.0, size=5)
print("Exponential (scale=2):", np.round(exponential
, 4))

# Chi-square distribution
chi2 = rng.chisquare(df=3, size=5)
print("Chi-square (df=3):", np.round(chi2, 4))

# Lognormal distribution
lognormal = rng.lognormal(mean=0, sigma=1, size=5)
print("Lognormal:", np.round(lognormal, 4))
```

```
Binomial (n=10, p=0.5): [3 5 7 5 6 4 3 3 5 5]
Poisson (lam=5): [3 8 4 4 5 4 5 7 6 4]
Exponential (scale=2): [0.6726 2.348 1.1895 0.2914
0.693 ]
Chi-square (df=3): [3.3385 1.9197 5.8233 1.4463 7.36
44]
Lognormal: [0.4155 2.6471 1.7595 0.3578 1.9641]
```

6.8.4 Shuffling and Permutations

```
rng = np.random.default_rng(seed=42)

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])

# Shuffle in-place
rng.shuffle(arr)
print("Shuffled array:", arr)

# Permutation (returns a new array, doesn't modify o
riginal)
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
perm = rng.permutation(arr)
print("Original array:", arr)
print("Permutation:", perm)

# Permutation of indices
indices = rng.permutation(8)
print("Permutation of indices:", indices)

Shuffled array: [30 40 70 20 80 10 60 50]
Original array: [10 20 30 40 50 60 70 80]
Permutation: [30 40 70 20 80 10 60 50]
Permutation of indices: [2 3 6 1 7 0 5 4]
```

6.8.5 Random Sampling Without Replacement

```
rng = np.random.default_rng(seed=42)

population = np.arange(1, 101)

# Sample 10 items without replacement
sample = rng.choice(population, size=10, replace=False)
```

```
se)
print("Sample of 10 (without replacement):", sample)

# Sample with replacement
sample_wr = rng.choice(population, size=10, replace=
True)
print("Sample of 10 (with replacement):", sample_wr)

# Sample with probabilities
probs = np.array([0.5, 0.3, 0.2])
items = np.array(['A', 'B', 'C'])
weighted_sample = rng.choice(items, size=10, p=probs
)
print("Weighted sample:", weighted_sample)

Sample of 10 (without replacement): [14 49 85 72  2
94  8 25 40 88]
Sample of 10 (with replacement): [14 49 85 72  2 94
8 25 40 88]
Weighted sample: ['A' 'A' 'B' 'A' 'C' 'A' 'A' 'A' 'B
' 'A']
```

6.8.6 Legacy Random Functions (Still Commonly Used)

The older `np.random` functions are still widely seen in existing code and tutorials. While the new Generator API is recommended for new code, you should be familiar with the legacy API:

```
# Set global seed
np.random.seed(42)

# Random floats
print("np.random.rand(3, 3):")
print(np.random.rand(3, 3))

# Random integers
print("\nnp.random.randint(0, 100, size=5):",
      np.random.randint(0, 100, size=5))

# Standard normal
print("\nnp.random.randn(5):", np.round(np.random.ra
```

```
ndn(5), 4))

# Shuffle
arr = np.arange(10)
np.random.shuffle(arr)
print("\nShuffled:", arr)

np.random.rand(3, 3):
[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]
 [0.05808361 0.86617615 0.60111501]]

np.random.randint(0, 100, size=5): [51 92 14 71 60]

np.random.randn(5): [ 0.3047  1.2024 -0.2412 -1.7499
 -0.189 ]

Shuffled: [7 1 3 8 5 0 9 4 2 6]
```

6.9 Practical Examples

6.9.1 Generating Synthetic Dataset

```
rng = np.random.default_rng(seed=42)

# Generate 100 samples with 2 features
n_samples = 100
n_features = 2

# Feature matrix from normal distribution
X = rng.normal(loc=0, scale=1, size=(n_samples, n_features))

# Labels: linear combination of features + noise
true_weights = np.array([3.5, -2.0])
true_bias = 1.0
noise = rng.normal(loc=0, scale=0.5, size=n_samples)
y = X @ true_weights + true_bias + noise

print("Feature matrix shape:", X.shape)
print("Label vector shape:", y.shape)
```

```
print("First 5 samples:")
for i in range(5):
    print(f" X={X[i]}, y={y[i]:.4f}")

# Compute statistics
print(f"\nFeature means: {X.mean(axis=0)}")
print(f"Feature stds: {X.std(axis=0)}")
print(f"Label mean: {y.mean():.4f}")
print(f"Label std: {y.std():.4f}")

Feature matrix shape: (100, 2)
Label vector shape: (100,)
First 5 samples:
X=[ 0.30471721  1.20242337], y=0.8923
X=[-0.24117977 -1.74990237], y=6.0665
X=[-0.18897728  0.62116875], y=-0.0389
X=[ 0.28045778 -0.32896295], y=2.7583
X=[ 0.30382267 -0.22954937], y=2.5878

Feature means: [ 0.07877772 -0.07319303]
Feature stds: [0.92447611 1.02395273]
Label mean: 1.1342
Label std: 4.2284
```

6.9.2 Simple Linear Regression with NumPy

```
rng = np.random.default_rng(seed=42)

# Generate data
X = rng.uniform(0, 10, size=100)
true_slope = 2.5
true_intercept = 5.0
noise = rng.normal(0, 2, size=100)
y = true_slope * X + true_intercept + noise

# Add intercept column
X_matrix = np.column_stack((np.ones_like(X), X))

# Normal equation:  $w = (X^T X)^{-1} X^T y$ 
XtX = X_matrix.T @ X_matrix
Xty = X_matrix.T @ y
```



```
w = np.linalg.solve(XtX, Xty)

print(f"True slope: {true_slope}, True intercept: {true_intercept}")
print(f"Estimated slope: {w[1]:.4f}, Estimated intercept: {w[0]:.4f}")

# Compute predictions and R-squared
y_pred = X_matrix @ w
ss_res = np.sum((y - y_pred) ** 2)
ss_tot = np.sum((y - y.mean()) ** 2)
r_squared = 1 - ss_res / ss_tot
print(f"R-squared: {r_squared:.4f}")

True slope: 2.5, True intercept: 5.0
Estimated slope: 2.4631, Estimated intercept: 5.2437
R-squared: 0.9356
```

6.9.3 K-Nearest Neighbors (Distance Computation)

```
rng = np.random.default_rng(seed=42)

# Generate 5 points in 3D space
points = rng.random((5, 3)) * 10
print("Points:")
print(np.round(points, 2))

# Compute pairwise Euclidean distances using broadcasting
diff = points[:, np.newaxis, :] - points[np.newaxis, :, :]
distances = np.sqrt(np.sum(diff ** 2, axis=2))

print("\nDistance matrix:")
print(np.round(distances, 2))

# Find 3 nearest neighbors for each point (excluding self)
np.fill_diagonal(distances, np.inf) # Exclude self-distances
k = 3
```

```
nearest_indices = np.argsort(distances, axis=1)[: , :
k]

print("\n3 nearest neighbors for each point:")
for i in range(len(points)):
    neighbors = nearest_indices[i]
    dists = np.round(distances[i, neighbors], 2)
    print(f"Point {i}: neighbors={neighbors}, distan
ces={dists}")

Points:
[[7.74 4.39 8.59]
 [5.32 7.    5.85]
 [3.83 6.92 2.27]
 [6.85 5.75 8.67]
 [8.97 7.99 4.62]]

Distance matrix:
[[0.    4.58 8.03 2.08 5.94]
 [4.58 0.    4.04 4.56 4.5 ]
 [8.03 4.04 0.    7.43 7.01]
 [2.08 4.56 7.43 0.    5.42]
 [5.94 4.5  7.01 5.42 0.   ]]

3 nearest neighbors for each point:
Point 0: neighbors=[3 1 4], distances=[2.08 4.58 5.9
4]
Point 1: neighbors=[2 4 3], distances=[4.04 4.5  4.5
6]
Point 2: neighbors=[1 4 3], distances=[4.04 7.01 7.4
3]
Point 3: neighbors=[0 1 4], distances=[2.08 4.56 5.4
2]
Point 4: neighbors=[1 3 0], distances=[4.5  5.42 5.9
4]
```

6.10 Summary

In this chapter, we explored the NumPy library, which forms the backbone of numerical computing in Python. Here are the key takeaways:

- **NumPy arrays (ndarray)** are homogeneous, fixed-size containers that offer superior performance and memory efficiency compared to Python lists for numerical data.
- **Array creation** can be done from Python lists or using built-in functions like `np.zeros()`, `np.ones()`, `np.arange()`, `np.linspace()`, and `np.random` functions.
- **Indexing** supports basic, boolean, and fancy (integer) indexing, providing flexible ways to access and modify array elements.
- **Slicing** extracts subarrays using the `start:stop:step` syntax. Slicing creates views (shared memory), while fancy indexing creates copies.
- **Broadcasting** allows arithmetic operations between arrays of different shapes by virtually expanding the smaller array, eliminating the need for explicit loops.
- **Universal functions (ufuncs)** perform element-wise operations efficiently. They cover arithmetic, trigonometric, exponential, logarithmic, statistical, and comparison operations.
- **Linear algebra operations** in `np.linalg` include matrix multiplication, transpose, determinant, inverse, solving linear systems, eigenvalues/eigenvectors, SVD, norms, and rank.
- **Random number generation** is supported through both the modern `default_rng()` Generator API and the legacy `np.random` functions, with distributions including normal, uniform, binomial, Poisson, and more.

Mastering NumPy is essential for anyone working with data in Python, as it provides the foundation upon which libraries like Pandas, Scikit-learn, and TensorFlow are built.

6.11 Exercises

Exercise 1: Create a 5×5 identity matrix and verify that multiplying it by any 5-element vector returns the same vector.

Exercise 2: Given the array `arr = np.arange(1, 26).reshape(5, 5)`, extract:

- The element at row 2, column 3
- The third row
- The second column
- The sub-matrix consisting of rows 1–3 and columns 2–4

Exercise 3: Create a 1D array of 50 random integers between 1 and 100. Find:

- All elements that are divisible by 5
- All elements that are greater than the mean
- The indices of the 5 largest elements

Exercise 4: Normalize a given 2D array so that each column has zero mean and unit variance. Verify your result.

Exercise 5: Create two matrices A (3×2) and B (2×4) with random integer values. Compute the matrix product $C = A \times B$. Verify that the shape of C is (3×4).

Exercise 6: Solve the following system of linear equations using NumPy:

$$\begin{aligned}x + 2y - z &= 4 \\3x - y + 2z &= -3 \\2x + y + z &= 7\end{aligned}$$

Exercise 7: Generate 10,000 samples from a normal distribution with mean 50 and standard deviation 10. Compute and print the mean, standard deviation, and the percentage of samples that fall within one standard deviation of the mean.

Exercise 8: Compute the SVD of a 4×3 random matrix. Reconstruct the original matrix using only the top 2 singular values (low-rank approximation). Compute the Frobenius norm of the difference between the original and the approximation.

Exercise 9: Create a function that takes a 2D array and returns a new array where each element is replaced by the rank of that element in its row (0 = smallest, $n - 1$ = largest).

Exercise 10: Using broadcasting, compute the pairwise Manhattan (L1) distance matrix for a set of 10 random 3D points without using any explicit loops.

Chapter 7: Pandas Library

7.1 Introduction to Pandas

Pandas is an open-source Python library that provides high-performance, easy-to-use data structures and data analysis tools. It is built on top of NumPy and is one of the most essential libraries in the Python data science ecosystem. The name "Pandas" is derived from "Panel Data," an econometrics term for multidimensional structured data sets.

7.1.1 Why Pandas?

Pandas has become the de facto standard for data manipulation in Python due to several compelling reasons:

- **Flexible Data Structures:** Pandas provides two primary data structures—Series (1D) and DataFrame (2D)—that can handle heterogeneous data types within the same structure.
- **Efficient Data Handling:** Pandas can handle large datasets efficiently, with operations optimized for performance using NumPy under the hood.
- **Data Alignment:** Automatic and explicit data alignment ensures that operations between different data structures work correctly even when their indices don't match perfectly.
- **Missing Data Handling:** Pandas provides robust tools for handling missing data, which is common in real-world datasets.
- **Time Series Support:** Built-in support for time series data, including date range generation, frequency conversion, and moving window statistics.
- **I/O Capabilities:** Pandas can read from and write to various file formats including CSV, Excel, JSON, SQL databases, and more.
- **Grouping and Aggregation:** Powerful groupby functionality similar to SQL for data aggregation and transformation.
- **Integration with Other Libraries:** Pandas integrates seamlessly with visualization libraries like Matplotlib and machine learning libraries like Scikit-learn.

7.1.2 Installing Pandas

If you are using the Anaconda distribution, Pandas comes pre-installed. However, if you need to install it separately, you can use pip:

Input code:

```
pip install pandas
```

Or using conda:

Input code:

```
conda install pandas
```

7.1.3 Importing Pandas

By convention, Pandas is imported with the alias pd:

Input code:

```
import pandas as pd
import numpy as np
```

This convention is universally followed in the data science community and in most documentation.

7.1.4 Checking Pandas Version

You can check the installed version of Pandas using the `__version__` attribute:

Input code:

```
import pandas as pd
print(pd.__version__)
```

Output: 2.1.4

7.2 Pandas Series

A Pandas Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating-point numbers, Python objects, etc.). It can be thought of as a single column of data with an associated index.

7.2.1 Creating a Series

7.2.1.1 Creating Series from a List

Input code:

```
import pandas as pd
import numpy as np

# Creating a Series from a Python list
s1 = pd.Series([10, 20, 30, 40, 50])
print("Series from list:")
print(s1)
print("\nType:", type(s1))
```

Output:

```
Series from list:
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

```
Type: <class 'pandas.core.series.Series'>
```

7.2.1.2 Creating Series with Custom Index

Input code:

```
# Creating a Series with custom index labels
s2 = pd.Series([100, 200, 300, 400],
               index=['a', 'b', 'c', 'd'])
print("Series with custom index:")
print(s2)
```

Output:

```
Series with custom index:
a    100
b    200
c    300
```



```
d      400
dtype: int64
```

7.2.1.3 Creating Series from a Dictionary

Input code:

```
# Creating a Series from a dictionary
data = {'Alice': 85, 'Bob': 90, 'Charlie': 78, 'Diana': 92}
s3 = pd.Series(data)
print("Series from dictionary:")
print(s3)
```

Output:

```
Series from dictionary:
Alice      85
Bob        90
Charlie    78
Diana      92
dtype: int64
```

7.2.1.4 Creating Series from NumPy Array

Input code:

```
# Creating a Series from a NumPy array
arr = np.array([1.5, 2.5, 3.5, 4.5, 5.5])
s4 = pd.Series(arr, name='values')
print("Series from NumPy array:")
print(s4)
print("\nSeries name:", s4.name)
```

Output:

```
Series from NumPy array:
0      1.5
1      2.5
2      3.5
3      4.5
4      5.5
```

```
Name: values, dtype: float64
Series name: values
```

7.2.1.5 Creating Series with Scalar Value

Input code:

```
# Creating a Series by repeating a scalar value
s5 = pd.Series(100, index=['x', 'y', 'z'])
print("Series from scalar:")
print(s5)
```

Output:

```
Series from scalar:
x      100
y      100
z      100
dtype: int64
```

7.2.2 Series Attributes

Input code:

```
s = pd.Series([10, 20, 30, 40, 50],
              index=['a', 'b', 'c', 'd', 'e'])

print("Series:")
print(s)
print("\nValues:", s.values)
print("Values type:", type(s.values))
print("Index:", s.index)
print("Data type:", s.dtype)
print("Size:", s.size)
print("Shape:", s.shape)
print("Is empty:", s.empty)
print("Number of dimensions:", s.ndim)
```

Output:

```
Series:
a      10
b      20
```

```
c      30
d      40
e      50
dtype: int64

Values: [10 20 30 40 50]
Values type: <class 'numpy.ndarray'>
Index: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
Data type: int64
Size: 5
Shape: (5,)
Is empty: False
Number of dimensions: 1
```

7.2.3 Series Indexing and Slicing

Input code:

```
s = pd.Series([100, 200, 300, 400, 500],
              index=['a', 'b', 'c', 'd', 'e'])

print("Original Series:")
print(s)

# Positional indexing
print("\nFirst element (positional):", s[0])
print("Last element (positional):", s[-1])

# Label-based indexing
print("\nElement at label 'c':", s['c'])
print("Elements at labels ['a', 'c', 'e']:", s[['a',
'c', 'e']])

# Slicing with labels (inclusive of endpoint)
print("\nSlice 'b' to 'd':")
print(s['b':'d'])

# Slicing with positions
print("\nSlice position 1 to 3:")
print(s[1:4])
```

Output:

Original Series:

```
a    100
b    200
c    300
d    400
e    500
dtype: int64
```

First element (positional): 100

Last element (positional): 500

Element at label 'c': 300

Elements at labels ['a', 'c', 'e']: a 100

```
c    300
e    500
dtype: int64
```

Slice 'b' to 'd':

```
b    200
c    300
d    400
dtype: int64
```

Slice position 1 to 3:

```
b    200
c    300
d    400
dtype: int64
```

7.2.4 Boolean Indexing in Series

Input code:

```
s = pd.Series([15, 25, 35, 45, 55, 65, 75])

print("Original Series:")
print(s)

# Elements greater than 40
```

```
print("\nElements > 40:")
print(s[s > 40])

# Elements between 20 and 60
print("\nElements between 20 and 60:")
print(s[(s >= 20) & (s <= 60)])

# Modify elements based on condition
s_mod = s.copy()
s_mod[s_mod < 30] = 0
print("\nAfter setting values < 30 to 0:")
print(s_mod)
```

Output:

Original Series:

```
0    15
1    25
2    35
3    45
4    55
5    65
6    75
dtype: int64
```

Elements > 40:

```
3    45
4    55
5    65
6    75
dtype: int64
```

Elements between 20 and 60:

```
1    25
2    35
3    45
4    55
dtype: int64
```

After setting values < 30 to 0:

```
0    0
```

```
1      0
2     35
3     45
4     55
5     65
6     75
dtype: int64
```

7.2.5 Series Operations

7.2.5.1 Arithmetic Operations

Input code:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([5, 15, 25, 35], index=['a', 'b', 'c', 'e'])

print("s1:")
print(s1)
print("\ns2:")
print(s2)

# Arithmetic operations (aligned by index)
print("\ns1 + s2:")
print(s1 + s2)

print("\ns1 * s2:")
print(s1 * s2)

# Operations with scalar
print("\ns1 + 100:")
print(s1 + 100)
```

Output:

```
s1:
a    10
b    20
c    30
d    40
```

```
dtype: int64
```

```
s2:
```

```
a      5
```

```
b     15
```

```
c     25
```

```
e     35
```

```
dtype: int64
```

```
s1 + s2:
```

```
a    15.0
```

```
b    35.0
```

```
c    55.0
```

```
d     NaN
```

```
e     NaN
```

```
dtype: float64
```

```
s1 * s2:
```

```
a    50.0
```

```
b   300.0
```

```
c   750.0
```

```
d     NaN
```

```
e     NaN
```

```
dtype: float64
```

```
s1 + 100:
```

```
a    110
```

```
b    120
```

```
c    130
```

```
d    140
```

```
dtype: int64
```

Note: That when indices don't match, Pandas introduces NaN (Not a Number) values. This automatic alignment is a powerful feature but can lead to unexpected results if not handled carefully.

7.2.5.2 Using Arithmetic Methods with fill_value

Input code:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([5, 15, 25, 35], index=['a', 'b', 'c', 'e'])

print("s1.add(s2, fill_value=0):")
print(s1.add(s2, fill_value=0))

print("\ns1.sub(s2, fill_value=0):")
print(s1.sub(s2, fill_value=0))

print("\ns1.mul(s2, fill_value=1):")
print(s1.mul(s2, fill_value=1))
```

Output:

```
s1.add(s2, fill_value=0):
a      15.0
b      35.0
c      55.0
d      40.0
e      35.0
dtype: float64

s1.sub(s2, fill_value=0):
a       5.0
b       5.0
c       5.0
d      40.0
e     -35.0
dtype: float64

s1.mul(s2, fill_value=1):
a      50.0
b     300.0
c     750.0
d      40.0
```



```
e      35.0
dtype: float64
```

7.2.6 Series Statistical Methods and Sorting

Input code:

```
s = pd.Series([15, 23, 8, 42, 31, 17, 28, 36])

print("Series:", s.values)
print("\nSum:", s.sum())
print("Mean:", s.mean())
print("Median:", s.median())
print("Standard deviation:", round(s.std(), 4))
print("Minimum:", s.min())
print("Maximum:", s.max())
print("Index of minimum:", s.idxmin())
print("Index of maximum:", s.idxmax())

print("\n" + "="*30)
s_unsorted = pd.Series([30, 10, 50, 20, 40], index=[
    'e', 'a', 'c', 'b', 'd'])
print("\nOriginal Series:")
print(s_unsorted)

print("\nSorted by values (ascending):")
print(s_unsorted.sort_values())

print("\nSorted by index:")
print(s_unsorted.sort_index())
```

Output:

```
Series: [15 23  8 42 31 17 28 36]

Sum: 200
Mean: 25.0
Median: 25.5
Standard deviation: 10.6927
Minimum: 8
Maximum: 42
```

```
Index of minimum: 2
```

```
Index of maximum: 3
```

```
=====
```

```
Original Series:
```

```
e    30
```

```
a    10
```

```
c    50
```

```
b    20
```

```
d    40
```

```
dtype: int64
```

```
Sorted by values (ascending):
```

```
a    10
```

```
b    20
```

```
e    30
```

```
d    40
```

```
c    50
```

```
dtype: int64
```

```
Sorted by index:
```

```
a    10
```

```
b    20
```

```
c    50
```

```
d    40
```

```
e    30
```

```
dtype: int64
```

7.3 Pandas DataFrames

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It is similar to a spreadsheet, a SQL table, or a dictionary of Series objects. DataFrames are the most commonly used Pandas data structure for data analysis.

7.3.1 Creating DataFrames

7.3.1.1 Creating DataFrame from Dictionary of Lists

Input code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Creating DataFrame from dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['New York', 'London', 'Paris', 'Tokyo', 'Berlin'],
    'Salary': [50000, 60000, 75000, 55000, 65000]
}

df = pd.DataFrame(data)
print("DataFrame from dictionary of lists:")
print(df)
print("\nType:", type(df))
```

Output:

DataFrame from dictionary of lists:

	Name	Age	City	Salary
0	Alice	25	New York	50000
1	Bob	30	London	60000
2	Charlie	35	Paris	75000
3	Diana	28	Tokyo	55000
4	Eve	32	Berlin	65000

Type: <class 'pandas.core.frame.DataFrame'>

7.3.1.2 Creating DataFrame from List of Dictionaries

Input code:

```
# Creating DataFrame from list of dictionaries
data = [
    {'Name': 'Alice', 'Age': 25, 'Score': 85},
    {'Name': 'Bob', 'Age': 30, 'Score': 90},
    {'Name': 'Charlie', 'Age': 35, 'Score': 78},
    {'Name': 'Diana', 'Age': 28, 'Score': 92}
]

df = pd.DataFrame(data)
print("DataFrame from list of dictionaries:")
print(df)
```

Output:

DataFrame from list of dictionaries:

	Name	Age	Score
0	Alice	25	85
1	Bob	30	90
2	Charlie	35	78
3	Diana	28	92

7.3.1.3 Creating DataFrame from NumPy Array

Input code:

```
# Creating DataFrame from NumPy array
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df = pd.DataFrame(arr, columns=['A', 'B', 'C'],
                  index=['row1', 'row2', 'row3'])
print("DataFrame from NumPy array:")
print(df)
```

Output:

DataFrame from NumPy array:

	A	B	C
row1	1	2	3
row2	4	5	6
row3	7	8	9

7.3.2 DataFrame Attributes

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age': [25, 30, 35, 28],
    'Score': [85, 90, 78, 92]
}, index=['a', 'b', 'c', 'd'])

print("DataFrame:")
print(df)

print("\nShape:", df.shape)
print("Number of rows:", len(df))
```

```
print("Number of columns:", df.shape[1])
print("Size (total elements):", df.size)
print("Dimensions:", df.ndim)
print("Columns:", df.columns.tolist())
print("Index:", df.index.tolist())
print("\nData types:")
print(df.dtypes)
print("\nValues (as NumPy array):")
print(df.values)
```

Output:

DataFrame:

	Name	Age	Score
a	Alice	25	85
b	Bob	30	90
c	Charlie	35	78
d	Diana	28	92

Shape: (4, 3)

Number of rows: 4

Number of columns: 3

Size (total elements): 12

Dimensions: 2

Columns: ['Name', 'Age', 'Score']

Index: ['a', 'b', 'c', 'd']

Data types:

Name object

Age int64

Score int64

dtype: object

Values (as NumPy array):

```
[['Alice' 25 85]
 ['Bob' 30 90]
 ['Charlie' 35 78]
 ['Diana' 28 92]]
```

7.3.3 Viewing DataFrame Content

Input code:

```
import pandas as pd
import numpy as np

# Create a larger DataFrame for demonstration
np.random.seed(42)
df = pd.DataFrame({
    'A': np.random.randint(1, 100, 20),
    'B': np.random.random(20).round(4),
    'C': np.random.choice(['X', 'Y', 'Z'], 20),
    'D': np.random.randn(20).round(4)
})

print("First 5 rows (head()):")
print(df.head())

print("\nLast 5 rows (tail()):")
print(df.tail())

print("\nFirst 3 rows:")
print(df.head(3))

print("\nRandom sample of 3 rows:")
print(df.sample(3))
```

Output:

```
First 5 rows (head()):
   A      B  C      D
0  52  0.7491 Z  0.3556
1  93  0.2384 X -0.4870
2  15  0.7399 Y  0.2544
3  72  0.9608 Z -0.8146
4  61  0.7546 X -0.7963

Last 5 rows (tail()):
   A      B  C      D
15  88  0.8369 Y  0.7151
```

```

16    8    0.9170    Z    0.5484
17   22    0.3261    X    0.9034
18   90    0.0437    Y   -1.0856
19   17    0.5627    Z   -0.2179

```

First 3 rows:

```

      A      B  C      D
0   52  0.7491  Z  0.3556
1   93  0.2384  X -0.4870
2   15  0.7399  Y  0.2544

```

Random sample of 3 rows:

```

      A      B  C      D
14  97  0.6645  X -0.0499
6   83  0.9025  Y  0.5277
0   52  0.7491  Z  0.3556

```

Input code:

```

print("DataFrame info:")
df.info()

print("\n" + "="*50)
print("\nStatistical summary:")
print(df.describe())

```

Output:

```

DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   A       20 non-null      int64
1   B       20 non-null      float64
2   C       20 non-null      object
3   D       20 non-null      float64
dtypes: float64(2), int64(1), object(1)
memory usage: 800.0+ bytes
=====

```

Statistical summary:

	A	B	D
count	20.000000	20.000000	20.000000
mean	47.300000	0.511020	-0.014665
std	29.693269	0.291019	0.740913
min	8.000000	0.043700	-1.507100
25%	20.250000	0.277125	-0.609875
50%	47.500000	0.538900	-0.061450
75%	73.250000	0.775525	0.473425
max	97.000000	0.977200	1.174000

7.4 Importing Data

Pandas provides a rich set of I/O functions to read data from various file formats. The functions typically start with `pd.read_*`.

7.4.1 Reading CSV Files

CSV (Comma-Separated Values) is one of the most common data formats.

Input code:

```
import pandas as pd

# For demonstration, let's create and save a CSV first
df_sample = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['NY', 'London', 'Paris']
})
df_sample.to_csv('sample_data.csv', index=False)

# Now read it back
df = pd.read_csv('sample_data.csv')
print("Read from CSV:")
print(df)

# Reading with specific columns
df_cols = pd.read_csv('sample_data.csv', usecols=['Name', 'Age'])
print("\nSpecific columns:")
```



```
print(df_cols)

# Reading with custom index
df_idx = pd.read_csv('sample_data.csv', index_col='Name')
print("\nWith Name as index:")
print(df_idx)
```

Output:

Read from CSV:

	Name	Age	City
0	Alice	25	NY
1	Bob	30	London
2	Charlie	35	Paris

Specific columns:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

With Name as index:

	Age	City
Name		
Alice	25	NY
Bob	30	London
Charlie	35	Paris

7.4.2 Reading Excel and JSON Files

Input code:

```
# Reading Excel files requires openpyxl (pip install openpyxl)
df_sample.to_excel('sample_data.xlsx', index=False, sheet_name='Data')

# Read from Excel
df_excel = pd.read_excel('sample_data.xlsx', sheet_name='Data')
print("Read from Excel:")
```

```
print(df_excel)

# Reading JSON
df_sample.to_json('sample_data.json', orient='records')
df_json = pd.read_json('sample_data.json')
print("\nRead from JSON:")
print(df_json)
```

Output:

Read from Excel:

	Name	Age	City
0	Alice	25	NY
1	Bob	30	London
2	Charlie	35	Paris

Read from JSON:

	Name	Age	City
0	Alice	25	NY
1	Bob	30	London
2	Charlie	35	Paris

7.5 Exporting Data

Pandas provides functions to export DataFrames to various formats using the `to_*` methods.

Input code:

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Score': [85, 90, 78]
})

# Export to CSV
df.to_csv('output.csv', index=False)
print("Exported to CSV (output.csv)")
```

```
# Export to Excel
df.to_excel('output.xlsx', index=False, sheet_name='
Data')
print("Exported to Excel (output.xlsx)")

# Export to JSON
df.to_json('output.json', orient='records', indent=2
)
print("Exported to JSON (output.json)")

# Export to Python dictionary
data_dict = df.to_dict()
print("\nAs dictionary:")
print(data_dict)

# Export to Python list of records
records = df.to_dict('records')
print("\nAs list of records:")
print(records)
```

Output:

```
Exported to CSV (output.csv)
Exported to Excel (output.xlsx)
Exported to JSON (output.json)

As dictionary:
{'Name': {0: 'Alice', 1: 'Bob', 2: 'Charlie'}, 'Age'
: {0: 25, 1: 30, 2: 35}, 'Score': {0: 85, 1: 90, 2:
78}}
```

```
As list of records:
[{'Name': 'Alice', 'Age': 25, 'Score': 85}, {'Name':
'Bob', 'Age': 30, 'Score': 90}, {'Name': 'Charlie',
'Age': 35, 'Score': 78}]
```

7.6 Selecting Data

Selecting data from DataFrames can be done using several methods: column selection, `loc[]`, `iloc[]`, and `at[]/iat[]`.

7.6.1 Selecting Columns

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['NY', 'London', 'Paris', 'Tokyo', 'Berlin'],
    'Salary': [50000, 60000, 75000, 55000, 65000]
})

print("Original DataFrame:")
print(df)

# Select single column (returns Series)
print("\nSingle column (df['Name']):")
print(df['Name'])
print("Type:", type(df['Name']))

# Select multiple columns (returns DataFrame)
print("\nMultiple columns:")
print(df[['Name', 'Age', 'Salary']])
print("Type:", type(df[['Name', 'Age', 'Salary']]))
```

Output:

Original DataFrame:

	Name	Age	City	Salary
0	Alice	25	NY	50000
1	Bob	30	London	60000
2	Charlie	35	Paris	75000
3	Diana	28	Tokyo	55000
4	Eve	32	Berlin	65000

```
Single column (df['Name']):  
0      Alice  
1       Bob  
2    Charlie  
3     Diana  
4       Eve  
Name: Name, dtype: object  
Type: <class 'pandas.core.series.Series'>
```

Multiple columns:

	Name	Age	Salary
0	Alice	25	50000
1	Bob	30	60000
2	Charlie	35	75000
3	Diana	28	55000
4	Eve	32	65000

```
Type: <class 'pandas.core.frame.DataFrame'>
```

7.6.2 Using loc[] - Label-based Selection

loc[] is used for selection by labels (row and column names).

Input code:

```
df = pd.DataFrame({  
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],  
    'Age': [25, 30, 35, 28],  
    'Score': [85, 90, 78, 92]  
}, index=['a', 'b', 'c', 'd'])  
  
print("DataFrame:")  
print(df)
```

```
# Select single row
print("\nSingle row (df.loc['b']):")
print(df.loc['b'])

# Select row and column
print("\nSingle element (df.loc['b', 'Age']):", df.loc['b', 'Age'])

# Select rows and specific columns
print("\nRows and columns:")
print(df.loc[['a', 'c'], ['Name', 'Score']])

# Slice with labels (inclusive of endpoint)
print("\nSlice rows 'a' to 'c':")
print(df.loc['a':'c'])

# Boolean indexing with loc
print("\nRows where Age > 28:")
print(df.loc[df['Age'] > 28])
```

Output:

DataFrame:

	Name	Age	Score
a	Alice	25	85
b	Bob	30	90
c	Charlie	35	78
d	Diana	28	92

Single row (df.loc['b']):

Name	Bob
Age	30
Score	90

Name: b, dtype: object

Single element (df.loc['b', 'Age']): 30

Rows and columns:

	Name	Score
a	Alice	85

```
c  Charlie      78
```

```
Slice rows 'a' to 'c':
```

	Name	Age	Score
a	Alice	25	85
b	Bob	30	90
c	Charlie	35	78

```
Rows where Age > 28:
```

	Name	Age	Score
b	Bob	30	90
c	Charlie	35	78

7.6.3 Using `iloc[]` - Position-based Selection

`iloc[]` is used for selection by integer position (like Python lists and NumPy arrays).

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'Score': [85, 90, 78, 92, 88]
})

print("DataFrame:")
print(df)

# Select single row
print("\nFirst row (df.iloc[0]):")
print(df.iloc[0])

# Select specific element
print("\nElement at row 1, column 2:", df.iloc[1, 2]
)

# Slice rows and columns
print("\nRows 1-3, columns 0-1:")
print(df.iloc[1:4, 0:2])
```

```
# Select all rows, specific columns
print("\nAll rows, columns 0 and 2:")
print(df.iloc[:, [0, 2]])
```

Output:

DataFrame:

	Name	Age	Score
0	Alice	25	85
1	Bob	30	90
2	Charlie	35	78
3	Diana	28	92
4	Eve	32	88

First row (df.iloc[0]):

Name	Alice
Age	25
Score	85

Name: 0, dtype: object

Element at row 1, column 2: 90

Rows 1-3, columns 0-1:

	Name	Age
1	Bob	30
2	Charlie	35
3	Diana	28

All rows, columns 0 and 2:

	Name	Score
0	Alice	85
1	Bob	90
2	Charlie	78
3	Diana	92
4	Eve	88

7.6.4 Difference Between loc, iloc, at, and iat

Comparison of DataFrame selection methods

Method	Selection Type	Returns	Use Case
loc[]	Label-based	Series/DataFrame	Select rows/columns by labels
iloc[]	Position-based	Series/DataFrame	Select rows/columns by integer positions
at[]	Label-based	Scalar	Fast access to single element by label
iat[]	Position-based	Scalar	Fast access to single element by position

7.7 Filtering Data

Filtering allows you to select rows that meet certain conditions.

7.7.1 Basic Filtering and Combining Conditions

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
    'Age': [25, 30, 35, 28, 32, 45],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT'],
    'Salary': [50000, 60000, 75000, 55000, 65000, 80000]
})

print("Original DataFrame:")
print(df)

# Filter by single condition
print("\nEmployees older than 30:")
print(df[df['Age'] > 30])

# AND condition - both must be true (use &)
print("\nIT department AND Age > 30:")
print(df[(df['Department'] == 'IT') & (df['Age'] > 30)])

# OR condition - at least one must be true (use |)
print("\nIT department OR Age > 35:")
```

```
print(df[(df['Department'] == 'IT') | (df['Age'] > 35)])
```

```
# NOT condition - negate a condition (use ~)
```

```
print("\nNOT IT department:")
```

```
print(df[~(df['Department'] == 'IT')])
```

Output:

Original DataFrame:

	Name	Age	Department	Salary
0	Alice	25	IT	50000
1	Bob	30	HR	60000
2	Charlie	35	IT	75000
3	Diana	28	Finance	55000
4	Eve	32	HR	65000
5	Frank	45	IT	80000

Employees older than 30:

	Name	Age	Department	Salary
2	Charlie	35	IT	75000
4	Eve	32	HR	65000
5	Frank	45	IT	80000

IT department AND Age > 30:

	Name	Age	Department	Salary
2	Charlie	35	IT	75000
5	Frank	45	IT	80000

IT department OR Age > 35:

	Name	Age	Department	Salary
0	Alice	25	IT	50000
2	Charlie	35	IT	75000
5	Frank	45	IT	80000

NOT IT department:

	Name	Age	Department	Salary
1	Bob	30	HR	60000
3	Diana	28	Finance	55000
4	Eve	32	HR	65000

7.7.2 Using `isin()` and String Methods

Input code:

```
print("Employees in IT or Finance using isin():")
print(df[df['Department'].isin(['IT', 'Finance'])])

df_names = pd.DataFrame({
    'Name': ['Alice Smith', 'Bob Jones', 'Charlie Br
own',
            'Diana Prince', 'Eve Wilson'],
    'Email': ['alice@email.com', 'bob@gmail.com',
             'charlie@yahoo.com', 'diana@company.or
g',
             'eve@email.com']
})

print("\n" + "="*40)
print("\nNames containing 'o':")
print(df_names[df_names['Name'].str.contains('o')])

print("\nNames starting with 'A':")
```

```
print(df_names[df_names['Name'].str.startswith('A')])
)

print("\nEmails ending with '.com':")
print(df_names[df_names['Email'].str.endswith('.com')])
```

Output:

Employees in IT or Finance using isin():

	Name	Age	Department	Salary
0	Alice	25	IT	50000
2	Charlie	35	IT	75000
3	Diana	28	Finance	55000
5	Frank	45	IT	80000

=====

Names containing 'o':

	Name	Email
1	Bob Jones	bob@gmail.com
2	Charlie Brown	charlie@yahoo.com

Names starting with 'A':

	Name	Email
0	Alice Smith	alice@email.com

Emails ending with '.com':

	Name	Email
0	Alice Smith	alice@email.com
1	Bob Jones	bob@gmail.com
2	Charlie Brown	charlie@yahoo.com
4	Eve Wilson	eve@email.com

7.8 Sorting

7.8.1 Sorting by Values and Index

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'Salary': [50000, 60000, 75000, 55000, 65000]
})

print("Original DataFrame:")
print(df)

# Sort by single column (ascending)
print("\nSorted by Age (ascending):")
print(df.sort_values('Age'))

# Sort by single column (descending)
print("\nSorted by Age (descending):")
print(df.sort_values('Age', ascending=False))

# Sort by multiple columns with different orders
df_dept = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Department': ['IT', 'HR', 'IT', 'HR', 'IT'],
    'Age': [25, 30, 35, 28, 32]
})
print("\nSorted by Department (asc) then Age (desc):")
print(df_dept.sort_values(['Department', 'Age'],
                           ascending=[True, False]))

# Sort by index
df_idx = pd.DataFrame({'A': [1, 2, 3]}, index=['c', 'a', 'b'])
print("\nOriginal Index:")
print(df_idx)
```

```
print("\nSorted Index:")
print(df_idx.sort_index())
```

Output:

Original DataFrame:

	Name	Age	Salary
0	Alice	25	50000
1	Bob	30	60000
2	Charlie	35	75000
3	Diana	28	55000
4	Eve	32	65000

Sorted by Age (ascending):

	Name	Age	Salary
0	Alice	25	50000
3	Diana	28	55000
1	Bob	30	60000
4	Eve	32	65000
2	Charlie	35	75000

Sorted by Age (descending):

	Name	Age	Salary
2	Charlie	35	75000
4	Eve	32	65000
1	Bob	30	60000
3	Diana	28	55000
0	Alice	25	50000

Sorted by Department (asc) then Age (desc):

	Name	Department	Age
2	Charlie	IT	35
4	Eve	IT	32
0	Alice	IT	25
1	Bob	HR	30
3	Diana	HR	28

Original Index:

```
      A
c  1
```

```
a  2
b  3
```

Sorted Index:

```
   A
a  2
b  3
c  1
```

7.8.2 Getting n Largest and n Smallest Values

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
    'Score': [85, 92, 78, 95, 88, 72],
    'Salary': [50000, 80000, 45000, 90000, 60000, 40000]
})
```

```
# Top 3 by Score
print("Top 3 by Score:")
print(df.nlargest(3, 'Score'))
```

```
# Bottom 3 by Score
print("\nBottom 3 by Score:")
print(df.nsmallest(3, 'Score'))
```

Output:

Top 3 by Score:

	Name	Score	Salary
3	Diana	95	90000
1	Bob	92	80000
4	Eve	88	60000

Bottom 3 by Score:

	Name	Score	Salary
5	Frank	72	40000
2	Charlie	78	45000
0	Alice	85	50000

7.9 Grouping

The `groupby()` operation follows the "split-apply-combine" paradigm:

3. **Split:** Divide data into groups based on criteria.
4. **Apply:** Apply a function to each group independently.
5. **Combine:** Combine the results into a data structure.

7.9.1 Basic GroupBy and Aggregation

Input code:

```
df = pd.DataFrame({
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR',
                  'IT',
                    'Finance', 'HR'],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve',
             'Frank', 'George', 'Helen'],
    'Salary': [50000, 60000, 75000, 55000, 65000, 80000,
               52000, 58000],
    'Age': [25, 30, 35, 28, 32, 45, 29, 27]
})

print("DataFrame:")
print(df)

# Group by a single column and compute mean
print("\nMean salary by department:")
print(df.groupby('Department')['Salary'].mean())

# Group and compute multiple statistics
print("\nSalary statistics by department:")
print(df.groupby('Department')['Salary'].agg(['mean',
        'min', 'max', 'count']))
```

Output:

DataFrame:

Department	Name	Salary	Age
------------	------	--------	-----

0	IT	Alice	50000	25
1	HR	Bob	60000	30
2	IT	Charlie	75000	35
3	Finance	Diana	55000	28
4	HR	Eve	65000	32
5	IT	Frank	80000	45
6	Finance	George	52000	29
7	HR	Helen	58000	27

Mean salary by department:

Department

Finance 53500.000000

HR 61000.000000

IT 68333.333333

Name: Salary, dtype: float64

Salary statistics by department:

Department	mean	min	max	count
Finance	53500.0	52000	55000	2
HR	61000.0	58000	65000	3
IT	68333.3	50000	80000	3

7.9.2 Named Aggregations and Transform

Input code:

```
# Using named aggregations (cleaner column names)
print("Named aggregations:")
result = df.groupby('Department').agg(
    avg_salary=('Salary', 'mean'),
    total_salary=('Salary', 'sum'),
    num_employees=('Name', 'count'),
    age_range=('Age', lambda x: x.max() - x.min())
)
print(result)

print("\n" + "="*50)
# Add a column with group mean using transform
```

```
df['dept_avg_salary'] = df.groupby('Department')['Salary'].transform('mean')
df['salary_deviation'] = df['Salary'] - df['dept_avg_salary']

print("\nWith department average and deviation:")
print(df[['Name', 'Department', 'Salary', 'dept_avg_salary', 'salary_deviation']])
```

Output:

Named aggregations:

Department	avg_salary	total_salary	num_employees	age_range
Finance	53500.0	107000	2	1
HR	61000.0	183000	3	5
IT	68333.3	205000	3	20

=====

With department average and deviation:

	Name	Department	Salary	dept_avg_salary	salary_deviation
0	Alice	IT	50000	68333.333	-18333.333333
1	Bob	HR	60000	61000.000	-1000.000000
2	Charlie	IT	75000	68333.333	6666.666667
3	Diana	Finance	55000	53500.000	1500.000000
4	Eve	HR	65000	61000.000	4000.000000
5	Frank	IT	80000	68333.333	11666.666667
6	George	Finance	52000	53500.000	-1500.000000
7	Helen	HR	58000	61000.000	-3000.000000

7.9.3 GroupBy with Filter

Input code:

```
# Filter groups where average salary > 60000
print("Departments with average salary > 60000:")
high_salary_depts = df.groupby('Department').filter(
    lambda x: x['Salary'].mean() > 60000
)
```

```
print(high_salary_depts[['Department', 'Name', 'Salary']])
```

Output:

Departments with average salary > 60000:

	Department	Name	Salary
0	IT	Alice	50000
1	HR	Bob	60000
2	IT	Charlie	75000
4	HR	Eve	65000
5	IT	Frank	80000
7	HR	Helen	58000

7.10 Aggregation

Aggregation involves computing summary statistics about groups of data or the entire DataFrame.

7.10.1 Built-in Aggregation Functions and `agg()`

Input code:

```
df = pd.DataFrame({
    'Sales': [100, 200, 150, 300, 250, 400],
    'Profit': [10, 25, 15, 40, 30, 50],
    'Region': ['North', 'South', 'North', 'South', 'North', 'South']
})

print("DataFrame:")
print(df)

# Single aggregation
print("\nTotal Sales:", df['Sales'].agg('sum'))

# Multiple aggregations on single column
print("\nSales statistics:")
print(df['Sales'].agg(['sum', 'mean', 'min', 'max', 'std']).round(2))

# Aggregation with groupby
```

```
print("\nAggregation by region:")
print(df.groupby('Region').agg({
    'Sales': ['sum', 'mean', 'count'],
    'Profit': ['sum', 'mean']
}).round(2))
```

Output:

DataFrame:

	Sales	Profit	Region
0	100	10	North
1	200	25	South
2	150	15	North
3	300	40	South
4	250	30	North
5	400	50	South

Total Sales: 1400

Sales statistics:

sum	1400.00
mean	233.33
min	100.00
max	400.00
std	102.74
Name: Sales, dtype: float64	

Aggregation by region:

	Sales			Profit	
	sum	mean	count	sum	mean
Region					
North	500	166.67	3	55	18.33
South	900	300.00	3	115	38.33

7.10.2 Cumulative Aggregations

Input code:

```
df = pd.DataFrame({
    'Date': pd.date_range('2024-01-01', periods=5),
    'Sales': [100, 150, 200, 180, 220]
```

```
})

# Cumulative sum
df['Cumulative_Sales'] = df['Sales'].cumsum()

# Cumulative percentage
df['Cumulative_Pct'] = (df['Cumulative_Sales'] / df[
'Sales'].sum() * 100).round(2)

print("With cumulative sum and percentage:")
print(df)
```

Output:

With cumulative sum and percentage:

	Date	Sales	Cumulative_Sales	Cumulative_Pct
0	2024-01-01	100	100	11.90
1	2024-01-02	150	250	29.76
2	2024-01-03	200	450	53.57
3	2024-01-04	180	630	75.00
4	2024-01-05	220	850	101.19

7.11 Merging DataFrames

Merging allows you to combine multiple DataFrames based on common columns or indices, similar to SQL joins.

7.11.1 Types of Joins

Types of joins in Pandas

Join Type	Description
Inner Join	Returns only rows with matching keys in both DataFrames
Left Join	Returns all rows from left DataFrame, matched rows from right
Right Join	Returns all rows from right DataFrame, matched rows from left
Outer Join	Returns all rows from both DataFrames

7.11.2 Performing Joins

Input code:

```
df1 = pd.DataFrame({
    'id': [1, 2, 3, 4],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana']
})

df2 = pd.DataFrame({
    'id': [2, 3, 4, 5],
    'salary': [50000, 60000, 70000, 80000]
})

print("DataFrame 1:")
print(df1)
print("\nDataFrame 2:")
print(df2)

# Inner join (default)
print("\nInner Join:")
print(pd.merge(df1, df2, on='id', how='inner'))

# Left join
print("\nLeft Join:")
print(pd.merge(df1, df2, on='id', how='left'))

# Outer join
print("\nOuter Join:")
print(pd.merge(df1, df2, on='id', how='outer'))
```

Output:

```
DataFrame 1:
   id  name
0   1  Alice
1   2   Bob
2   3 Charlie
3   4  Diana
```

```
DataFrame 2:
```

	id	salary
0	2	50000
1	3	60000
2	4	70000
3	5	80000

Inner Join:

	id	name	salary
0	2	Bob	50000
1	3	Charlie	60000
2	4	Diana	70000

Left Join:

	id	name	salary
0	1	Alice	NaN
1	2	Bob	50000.0
2	3	Charlie	60000.0
3	4	Diana	70000.0

Outer Join:

	id	name	salary
0	1.0	Alice	NaN
1	2.0	Bob	50000.0
2	3.0	Charlie	60000.0
3	4.0	Diana	70000.0
4	5.0	NaN	80000.0

7.11.3 Concatenating DataFrames

Input code:

```
df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2'],
    'B': ['B0', 'B1', 'B2']
})
```

```
df2 = pd.DataFrame({
    'A': ['A3', 'A4', 'A5'],
    'B': ['B3', 'B4', 'B5']
})
```

```
df3 = pd.DataFrame({
```

```
        'C': ['C0', 'C1', 'C2'],
        'D': ['D0', 'D1', 'D2']
    })

# Vertical concatenation (along rows)
vertical = pd.concat([df1, df2], axis=0, ignore_index=True)
print("Vertical concatenation:")
print(vertical)

# Horizontal concatenation (along columns)
horizontal = pd.concat([df1, df3], axis=1)
print("\nHorizontal concatenation:")
print(horizontal)
```

Output:

Vertical concatenation:

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4
5	A5	B5

Horizontal concatenation:

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2

7.12 Missing Values

Real-world datasets often contain missing values, represented as NaN (Not a Number) in Pandas. Handling missing values properly is crucial for accurate data analysis.

7.12.1 Detecting Missing Values

Input code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5],
    'B': [10, np.nan, np.nan, 40, 50],
    'C': ['a', 'b', np.nan, 'd', 'e'],
    'D': [100, 200, 300, np.nan, np.nan]
})

print("DataFrame with missing values:")
print(df)

# Check for missing values (returns boolean)
print("\nisna():")
print(df.isna())

# Count missing values per column
print("\nMissing values per column:")
print(df.isna().sum())

# Check if any missing values exist
print("\nAny missing values:", df.isna().any().any())
)
```

Output:

DataFrame with missing values:

	A	B	C	D
0	1.0	10.0	a	100.0
1	2.0	NaN	b	200.0
2	NaN	NaN	NaN	300.0
3	4.0	40.0	d	NaN
4	5.0	50.0	e	NaN

```
isna():
      A      B      C      D
0  False  False  False  False
1  False   True  False  False
2   True   True   True  False
3  False  False  False   True
4  False  False  False   True
```

Missing values per column:

```
A      1
B      2
C      1
D      2
dtype: int64
```

Any missing values: True

7.12.2 Dropping Missing Values

Input code:

```
print("Original DataFrame:")
print(df)

# Drop rows with any missing values
print("\ndropna() - drop rows with any NaN:")
print(df.dropna())

# Drop rows with less than N non-NaN values
print("\ndropna(thresh=3) - keep rows with at least
3 non-NaN:")
print(df.dropna(thresh=3))

# Drop specific column if it has missing values
print("\ndropna(subset=['B']) - drop rows with NaN i
n column B:")
print(df.dropna(subset=['B']))
```

Output:

Original DataFrame:

	A	B	C	D
0	1.0	10.0	a	100.0
1	2.0	NaN	b	200.0
2	NaN	NaN	NaN	300.0
3	4.0	40.0	d	NaN
4	5.0	50.0	e	NaN

dropna() - drop rows with any NaN:

	A	B	C	D
0	1.0	10.0	a	100.0

dropna(thresh=3) - keep rows with at least 3 non-NaN:

	A	B	C	D
0	1.0	10.0	a	100.0
1	2.0	NaN	b	200.0
3	4.0	40.0	d	NaN
4	5.0	50.0	e	NaN

dropna(subset=['B']) - drop rows with NaN in column B:

	A	B	C	D
0	1.0	10.0	a	100.0
3	4.0	40.0	d	NaN
4	5.0	50.0	e	NaN

7.12.3 Filling Missing Values**Input code:**

```
print("Original DataFrame:")
print(df)
# Fill with a specific value
print("\nfillna(0) - fill with 0:")
print(df.fillna(0))

# Forward fill (propagate last valid observation forward)
```

```
print("\nffill() - forward fill:")
print(df.fillna(method='ffill'))

# Backward fill (propagate next valid observation backward)
print("\nbfill() - backward fill:")
print(df.fillna(method='bfill'))

# Fill with column mean (for numeric columns)
print("\nFill numeric with column mean:")
df_mean = df.copy()
df_mean['A'] = df_mean['A'].fillna(df_mean['A'].mean())
df_mean['B'] = df_mean['B'].fillna(df_mean['B'].mean())
df_mean['D'] = df_mean['D'].fillna(df_mean['D'].mean())
print(df_mean)
```

Output:

Original DataFrame:

	A	B	C	D
0	1.0	10.0	a	100.0
1	2.0	NaN	b	200.0
2	NaN	NaN	NaN	300.0
3	4.0	40.0	d	NaN
4	5.0	50.0	e	NaN

fillna(0) - fill with 0:

	A	B	C	D
0	1.0	10.0	a	100.0
1	2.0	0.0	b	200.0
2	0.0	0.0	0	300.0
3	4.0	40.0	d	0.0
4	5.0	50.0	e	0.0

ffill() - forward fill:

	A	B	C	D
0	1.0	10.0	a	100.0

```
1  2.0  10.0  b  200.0
2  2.0  10.0  b  300.0
3  4.0  40.0  d  300.0
4  5.0  50.0  e  300.0
```

bfill() - backward fill:

```
      A      B  C      D
0  1.0  10.0  a  100.0
1  2.0  40.0  b  200.0
2  4.0  40.0  d  300.0
3  4.0  40.0  d    NaN
4  5.0  50.0  e    NaN
```

Fill numeric with column mean:

```
      A      B  C      D
0  1.0  10.0  a  100.0
1  2.0  33.3  b  200.0
2  3.0  33.3  NaN  300.0
3  4.0  40.0  d  200.0
4  5.0  50.0  e  200.0
```

7.12.4 Interpolation

Input code:

```
# Create a DataFrame with sequential data
df_seq = pd.DataFrame({
    'value': [1, np.nan, np.nan, 4, 5, np.nan, 7, np
.nan, 9]
})

print("DataFrame with sequential missing values:")
print(df_seq)

# Linear interpolation
print("\nLinear interpolation:")
print(df_seq.interpolate())
```

Output:

```
DataFrame with sequential missing values:
   value
```

```
0    1.0
1    NaN
2    NaN
3    4.0
4    5.0
5    NaN
6    7.0
7    NaN
8    9.0
```

Linear interpolation:

```
      value
0    1.0
1    2.0
2    3.0
3    4.0
4    5.0
5    6.0
6    7.0
7    8.0
8    9.0
```

7.13 Additional DataFrame Operations

7.13.1 Adding and Removing Columns

Input code:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})

print("Original DataFrame:")
print(df)
# Add new column
df['City'] = ['NY', 'London', 'Paris']
print("\nAfter adding City column:")
print(df)

# Add column based on existing columns
```

```
df['Age_Group'] = df['Age'].apply(lambda x: 'Young'
if x < 30 else 'Senior')
print("\nAfter adding Age_Group column:")
print(df)

# Remove column using drop
df_dropped = df.drop('Age_Group', axis=1)
print("\nAfter dropping Age_Group column:")
print(df_dropped)
```

Output:

Original DataFrame:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

After adding City column:

	Name	Age	City
0	Alice	25	NY
1	Bob	30	London
2	Charlie	35	Paris

After adding Age_Group column:

	Name	Age	City	Age_Group
0	Alice	25	NY	Young
1	Bob	30	London	Senior
2	Charlie	35	Paris	Senior

After dropping Age_Group column:

	Name	Age	City
0	Alice	25	NY
1	Bob	30	London
2	Charlie	35	Paris

7.13.2 Applying Functions

Input code:

```
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
```

```
'B': [10, 20, 30, 40, 50],
'C': [100, 200, 300, 400, 500]
})

print("Original DataFrame:")
print(df)

# Apply function to each column
print("\nSum of each column:")
print(df.apply(sum))

# Apply function to each row
print("\nSum of each row:")
print(df.apply(sum, axis=1))

# Applymap (element-wise) - use map for Series in newer Pandas
print("\nElement-wise formatting (multiply by 10):")
print(df.map(lambda x: x * 10))
```

Output:

Original DataFrame:

	A	B	C
0	1	10	100
1	2	20	200
2	3	30	300
3	4	40	400
4	5	50	500

Sum of each column:

A	15
B	150
C	1500

dtype: int64

Sum of each row:

0	111
1	222
2	333
3	444


```
4      555
dtype: int64
```

Element-wise formatting (multiply by 10):

	A	B	C
0	10	100	1000
1	20	200	2000
2	30	300	3000
3	40	400	4000
4	50	500	5000

7.13.3 Column Renaming and Index Setting

Input code:

```
df = pd.DataFrame({
    'First Name': ['Alice', 'Bob', 'Charlie'],
    'Age (years)': [25, 30, 35]
})

print("Original DataFrame:")
print(df)

# Rename specific columns
print("\nRename specific columns:")
print(df.rename(columns={
    'First Name': 'name',
    'Age (years)': 'age'
})))

# Set column as index
df_idx = df.set_index('First Name')
print("\nAfter set_index('First Name'):")
print(df_idx)

# Reset index
print("\nAfter reset_index():")
print(df_idx.reset_index())
```

Output:

Original DataFrame:

	First Name	Age (years)
0	Alice	25
1	Bob	30
2	Charlie	35

Rename specific columns:

	name	age
0	Alice	25
1	Bob	30
2	Charlie	35

After `set_index('First Name')`:

	Age (years)
First Name	
Alice	25
Bob	30
Charlie	35

After `reset_index()`:

	First Name	Age (years)
0	Alice	25
1	Bob	30
2	Charlie	35

7.14 Summary

This chapter covered the Pandas library, one of the most essential tools for data manipulation in Python. Key topics included:

- **Series:** One-dimensional labeled arrays with various creation methods, indexing, and operations.
- **DataFrames:** Two-dimensional data structures with flexible creation options and comprehensive attributes.
- **Importing/Exporting Data:** Reading from and writing to CSV, Excel, JSON, SQL, and other formats.

- **Selecting Data:** Using `loc[]`, `iloc[]`, `at[]`, and `iat[]` for label-based and position-based selection.
- **Filtering:** Boolean indexing, `isin()`, string methods, and `query()` for conditional row selection.
- **Sorting:** Sorting by values and index using `sort_values()` and `sort_index()`.
- **Grouping:** Split-apply-combine paradigm with `groupby()`, including aggregation, transformation, and filtering.
- **Aggregation:** Computing summary statistics with built-in functions and `agg()`.
- **Merging:** Combining DataFrames using various join types and concatenation.
- **Missing Values:** Detecting, dropping, filling, and interpolating missing data.

7.15 Exercises

1. Create a Pandas Series from a dictionary containing the population of at least 5 countries. Display the Series, its index, values, and data type.
2. Create a DataFrame with columns: 'Product', 'Price', 'Quantity', 'Category'. Add at least 6 rows of data. Calculate a new column 'Total' as $\text{Price} \times \text{Quantity}$.
3. Create a CSV file (or use an existing one), read it into a DataFrame, and display:
 - First 5 rows
 - Shape of the DataFrame
 - Data types of each column
 - Statistical summary
4. Given the following DataFrame, perform the specified selections:

Input code:

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie',
            'Diana', 'Eve'],
```

```
'Math': [85, 90, 78, 92, 88],  
'Science': [75, 85, 82, 90, 79],  
'English': [90, 88, 85, 78, 95]  
})
```

- Select only the 'Name' and 'Math' columns
 - Select the row for 'Charlie' using loc
 - Select the Science score for 'Diana' using loc
 - Select the first 3 rows and first 2 columns using iloc
5. Using the DataFrame from Exercise 4:
- Filter students with Math score > 85
 - Filter students with Science score ≥ 80 AND English score ≥ 85
 - Filter students whose name starts with 'A' or 'D'
6. Sort the DataFrame from Exercise 4:
- By Math score in descending order
 - By English score in ascending order, then by Math score in descending order
7. Create a DataFrame with columns: 'Department', 'Employee', 'Salary'. Add at least 9 rows (3 departments, 3 employees each). Then:
- Find the average salary for each department
 - Find the highest paid employee in each department
 - Add a column showing each employee's salary as a percentage of their department's total salary
8. Create two DataFrames:

Input code:

```
df1 = pd.DataFrame({
```

```
'id': [1, 2, 3, 4, 5],  
'name': ['A', 'B', 'C', 'D', 'E']  
})  
  
df2 = pd.DataFrame({  
    'id': [3, 4, 5, 6, 7],  
    'score': [85, 92, 78, 88, 95]  
})
```

- Perform an inner join
- Perform a left join
- Perform an outer join
- Concatenate them vertically

9. Create a DataFrame with missing values:

Input code:

```
import numpy as np  
  
df = pd.DataFrame({  
    'A': [1, 2, np.nan, 4, 5, np.nan],  
    'B': [10, np.nan, 30, np.nan, 50, 60],  
    'C': ['x', 'y', np.nan, 'z', np.nan, 'w']  
})
```

- Count missing values in each column
- Drop rows with any missing values
- Fill missing numeric values with column mean
- Fill missing string values with 'Unknown'

10. **Challenge:** Create a DataFrame representing sales data with columns: 'Date', 'Product', 'Region', 'Sales', 'Profit'. Add at least 20 rows of data. Then:

- Calculate total sales and profit by region
- Calculate monthly sales trends
- Find the top 3 products by total sales
- Calculate profit margin (Profit/Sales) for each row
- Identify regions with profit margin above average

Chapter 8: Scikit-learn

8.1 Introduction to Scikit-learn

Scikit-learn is one of the most widely used machine learning libraries in Python. It provides simple and efficient tools for data mining and data analysis, built on top of NumPy, SciPy, and Matplotlib. Scikit-learn is open-source, commercially usable, and designed to be accessible to both beginners and experienced practitioners.

8.1.1 Why Scikit-learn?

- **Consistent API:** All estimators in Scikit-learn follow a uniform interface with `fit()`, `predict()`, and `transform()` methods.
- **Comprehensive Algorithms:** Includes supervised and unsupervised learning algorithms, model selection tools, and preprocessing utilities.
- **Easy to Use:** Designed with simplicity in mind, making it accessible for newcomers to machine learning.
- **Well Documented:** Extensive documentation, examples, and tutorials are available.
- **Integration:** Works seamlessly with NumPy, Pandas, and Matplotlib.
- **Community Support:** Large and active community contributing to its development and improvement.

8.1.2 Key Features

Key features of Scikit-learn

Feature	Description
Classification	Identifying which category an object belongs to
Regression	Predicting a continuous-valued attribute
Clustering	Grouping similar objects into clusters
Dimensionality Reduction	Reducing the number of random variables
Model Selection	Comparing, validating, and choosing parameters
Preprocessing	Feature extraction and normalization

8.2 Installing Scikit-learn

8.2.1 Installation Using pip

If you are not using Anaconda, you can install Scikit-learn using pip:

Input Code:

```
pip install scikit-learn
```

Output:

```
Successfully installed scikit-learn-1.4.0
```

8.2.2 Installation Using conda

For Anaconda users, install using conda:

Input Code:

```
conda install scikit-learn
```

Output:

```
The following NEW packages will be INSTALLED:
  scikit-learn      pkgs/main/linux-64::scikit-learn-1
  .4.0
```

8.2.3 Verifying Installation

Input Code:

```
import sklearn
print("Scikit-learn version:", sklearn.__version__)
```

Output:

```
Scikit-learn version: 1.4.0
```

8.2.4 Importing Common Modules

Input Code:

```
# Common imports
import numpy as np
import pandas as pd
```



```
import matplotlib.pyplot as plt

# Scikit-learn modules
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.pipeline import Pipeline
import joblib

print("All modules imported successfully!")
```

Output:

All modules imported successfully!

8.3 Dataset Loading

Scikit-learn provides several built-in datasets that are useful for learning and experimentation. These datasets are available in the `sklearn.datasets` module.

8.3.1 Built-in Datasets Overview

Built-in datasets in Scikit-learn

Dataset	Type	Samples	Features
Iris	Classification	150	4
Breast Cancer	Classification	569	30
Wine	Classification	178	13
Digits	Classification	1797	64
Boston Housing	Regression	506	13
Diabetes	Regression	442	10

8.3.2 Loading the Iris Dataset

The Iris dataset is one of the most famous datasets in machine learning. It contains 150 samples of iris flowers with 4 features each.

Input Code:

```
from sklearn import datasets

# Load Iris dataset
iris = datasets.load_iris()

print("Dataset type:", type(iris))
print("\nKeys available:", iris.keys())
print("\nFeature names:", iris.feature_names)
print("\nTarget names:", iris.target_names)
print("\nData shape:", iris.data.shape)
print("\nTarget shape:", iris.target.shape)
```

Output:

```
Dataset type: <class 'sklearn.utils._bunch.Bunch'>

Keys available: dict_keys(['data', 'target', 'frame'
,
'target_names', 'DESCR', 'feature_names', 'filename'
, 'data_module'])

Feature names: ['sepal length (cm)', 'sepal width (c
m)',
'petal length (cm)', 'petal width (cm)']

Target names: ['setosa' 'versicolor' 'virginica']

Data shape: (150, 4)

Target shape: (150,)
```

8.3.3 Exploring the Iris Dataset

Input Code:

```
from sklearn import datasets
import pandas as pd

iris = datasets.load_iris()
```

```
# Convert to DataFrame for easier exploration
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
df['species'] = df['target'].map({0: 'setosa',
                                  1: 'versicolor',
                                  2: 'virginica'})

print("First 5 rows:")
print(df.head())
print("\nDataset info:")
print(df.info())
print("\nStatistical summary:")
print(df.describe())
```

Output:

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)
0	5.1	3.5	1.4
1	4.9	3.0	1.4
2	4.7	3.2	1.3
3	4.6	3.1	1.5
4	5.0	3.6	1.4

	petal width (cm)	target	species
0	0.2	0	setosa
1	0.2	0	setosa
2	0.2	0	setosa
3	0.2	0	setosa
4	0.2	0	setosa

Dataset info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
```

Data columns (total 6 columns):

# Column	Non-Null Count	Dtype
0 sepal length (cm)	150	float64
1 sepal width (cm)	150	float64
2 petal length (cm)	150	float64
3 petal width (cm)	150	float64
4 target	150	int64
5 species	150	object

dtypes: float64(4), int64(1), object(1)

memory usage: 7.2+ KB

None

Statistical summary:

	sepal length (cm)	sepal width (cm)	petal length (cm)
count	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000
std	0.828066	0.435866	1.765098
min	4.300000	2.000000	1.000000
25%	5.100000	2.800000	1.600000
50%	5.800000	3.000000	4.350000
75%	6.400000	3.300000	5.100000
max	7.900000	4.400000	6.900000

	petal width (cm)	target
count	150.000000	150.000000
mean	1.199333	1.000000
std	0.762238	0.819232
min	0.100000	0.000000
25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

8.3.4 Loading the Breast Cancer Dataset

Input Code:

```
from sklearn import datasets

# Load Breast Cancer dataset
cancer = datasets.load_breast_cancer()

print("Dataset:", cancer.DESCR[:500])
print("\n" + "="*50)
print("Feature names (first 10):", cancer.feature_names[:10])
print("Target names:", cancer.target_names)
print("Data shape:", cancer.data.shape)
print("Target distribution:", np.bincount(cancer.target))
```

Output:

```
Dataset: .. _breast_cancer_dataset:
```

```
Breast cancer wisconsin (diagnostic) dataset
```

```
-----
```

```
**Data Set Characteristics:**
```

```
 :Number of Instances: 569
```

```
 :Number of Attributes: 30 numeric, predictive attributes and the class
```

```
 :Attribute Information:
```

```
   - radius (mean of distances from center to points on the perimeter)
```

```
   - texture (standard deviation of gray-scale values)
```

```
   - perimeter
```

```
   - area
```

```
   - smoothness (local variation in radius lengths)
```

```
=====
```

```
Feature names (first 10): ['mean radius' 'mean texture' 'mean perimeter'
```

```
 'mean area' 'mean smoothness' 'mean compactness' 'mean concavity'
```

```
'mean concave points' 'mean symmetry' 'mean fractal dimension']
Target names: ['malignant' 'benign']
Data shape: (569, 30)
Target distribution: [212 357]
```

8.3.5 Loading Regression Datasets

Input Code:

```
from sklearn import datasets

# Load Diabetes dataset (regression)
diabetes = datasets.load_diabetes()

print("Diabetes Dataset:")
print("Feature names:", diabetes.feature_names)
print("Data shape:", diabetes.data.shape)
print("Target shape:", diabetes.target.shape)
print("\nFirst sample features:", diabetes.data[0])
print("First sample target:", diabetes.target[0])
print("\nTarget statistics:")
print(f"  Min: {diabetes.target.min():.1f}")
print(f"  Max: {diabetes.target.max():.1f}")
print(f"  Mean: {diabetes.target.mean():.1f}")
```

Output:

```
Diabetes Dataset:
Feature names: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
Data shape: (442, 10)
Target shape: (442,)

First sample features: [ 0.03807591  0.05068012  0.0616962
1  0.02187235
-0.0442235 -0.03482076 -0.04340085 -0.00259226  0.01990
842
-0.01764613]
First sample target: 151.0

Target statistics:
  Min: 25.0
```

Max: 346.0

Mean: 152.1

8.3.6 Generating Synthetic Datasets

Scikit-learn provides functions to generate synthetic datasets for experimentation:

Input Code:

```
from sklearn.datasets import make_classification, make_regression
from sklearn.datasets import make_blobs, make_moons

# 1. Classification dataset
X_clf, y_clf = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=2,
    n_redundant=1,
    random_state=42
)
print("Classification dataset:")
print(f"  X shape: {X_clf.shape}")
print(f"  y shape: {y_clf.shape}")
print(f"  Classes: {np.unique(y_clf)}")

# 2. Regression dataset
X_reg, y_reg = make_regression(
    n_samples=500,
    n_features=3,
    noise=10,
    random_state=42
)
print("\nRegression dataset:")
print(f"  X shape: {X_reg.shape}")
print(f"  y shape: {y_reg.shape}")
print(f"  y range: [{y_reg.min():.1f}, {y_reg.max():.1f}]")

# 3. Blobs (clustering)
X_blobs, y_blobs = make_blobs(
```

```
        n_samples=300,
        centers=3,
        random_state=42
    )
    print("\nBlobs dataset:")
    print(f"  X shape: {X_blobs.shape}")
    print(f"  Number of clusters: {len(np.unique(y_blobs
    ))}")

    # 4. Moons (non-linear classification)
    X_moons, y_moons = make_moons(
        n_samples=200,
        noise=0.1,
        random_state=42
    )
    print("\nMoons dataset:")
    print(f"  X shape: {X_moons.shape}")
    print(f"  Classes: {np.unique(y_moons)}")
```

Output:

Classification dataset:

```
  X shape: (1000, 4)
  y shape: (1000,)
  Classes: [0 1]
```

Regression dataset:

```
  X shape: (500, 3)
  y shape: (500,)
  y range: [-246.8, 285.3]
```

Blobs dataset:

```
  X shape: (300, 2)
  Number of clusters: 3
```

Moons dataset:

```
  X shape: (200, 2)
  Classes: [0 1]
```


8.4 Dataset Splitting

Splitting data into training and testing sets is a fundamental step in machine learning. The training set is used to train the model, while the testing set is used to evaluate its performance on unseen data.

8.4.1 Basic Train-Test Split

Input Code:

```
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Original dataset:")
print(f"  X shape: {X.shape}")
print(f"  y shape: {y.shape}")
print("\nTraining set:")
print(f"  X_train shape: {X_train.shape}")
print(f"  y_train shape: {y_train.shape}")
print("\nTesting set:")
print(f"  X_test shape: {X_test.shape}")
print(f"  y_test shape: {y_test.shape}")
print(f"\nSplit ratio: {len(X_train)/len(X):.0%} tra
in, "
      f"{len(X_test)/len(X):.0%} test")
```

Output:

```
Original dataset:
  X shape: (150, 4)
```

```
    y shape: (150,)
Training set:
    X_train shape: (120, 4)
    y_train shape: (120,)

Testing set:
    X_test shape: (30, 4)
    y_test shape: (30,)

Split ratio: 80% train, 20% test
```

8.4.2 Stratified Splitting

Stratified splitting ensures that the proportion of classes in the training and testing sets matches the original dataset.

Input Code:

```
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load imbalanced dataset
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

print("Original class distribution:")
print(f"  Malignant (0): {np.sum(y == 0)} ({np.mean(y == 0):.1%})")
print(f"  Benign (1): {np.sum(y == 1)} ({np.mean(y == 1):.1%})")

# Regular split (might not preserve proportions)
X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("\nRegular split - Test set distribution:")
print(f"  Malignant (0): {np.sum(y_test1 == 0)} "
      f"({np.mean(y_test1 == 0):.1%})")
```

```
print(f" Benign (1): {np.sum(y_test1 == 1)} "
      f"({np.mean(y_test1 == 1):.1%})")

# Stratified split (preserves proportions)
X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("\nStratified split - Test set distribution:")
print(f" Malignant (0): {np.sum(y_test2 == 0)} "
      f"({np.mean(y_test2 == 0):.1%})")
print(f" Benign (1): {np.sum(y_test2 == 1)} "
      f"({np.mean(y_test2 == 1):.1%})")
```

Output:

Original class distribution:

Malignant (0): 212 (37.3%)

Benign (1): 357 (62.7%)

Regular split - Test set distribution:

Malignant (0): 63 (36.8%)

Benign (1): 108 (63.2%)

Stratified split - Test set distribution:

Malignant (0): 64 (37.4%)

Benign (1): 107 (62.6%)

8.4.3 Train-Validation-Test Split

For hyperparameter tuning, you often need a validation set in addition to training and testing sets.

Input Code:

```
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load dataset
iris = datasets.load_iris()
X = iris.data
```

```
y = iris.target

# First split: 80% train+val, 20% test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Second split: 75% train, 25% val (of the 80%)
# This gives 60% train, 20% val, 20% test overall
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42
)

print("Dataset split summary:")
print(f"  Training set: {len(X_train)} samples ({len(
X_train)/len(X):.0%})")
print(f"  Validation set: {len(X_val)} samples ({len(
X_val)/len(X):.0%})")
print(f"  Test set: {len(X_test)} samples ({len(X_te
st)/len(X):.0%})")
print(f"\n  Total: {len(X_train) + len(X_val) + len(
X_test)} samples")
```

Output:

```
Dataset split summary:
  Training set: 90 samples (60%)
  Validation set: 30 samples (20%)
  Test set: 30 samples (20%)

Total: 150 samples
```

8.4.4 Cross-Validation

Cross-validation provides a more robust estimate of model performance by using multiple train-test splits.

Input Code:

```
from sklearn.model_selection import cross_val_score,
KFold
from sklearn.linear_model import LogisticRegression
```

```
from sklearn import datasets

# Load dataset
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Create model
model = LogisticRegression(max_iter=200, random_state=42)

# 5-fold cross-validation
cv_scores = cross_val_score(model, X, y, cv=5)

print("5-Fold Cross-Validation Results:")
print(f"  Scores: {cv_scores}")
print(f"  Mean accuracy: {cv_scores.mean():.4f}")
print(f"  Standard deviation: {cv_scores.std():.4f}")

# 10-fold cross-validation
cv_scores_10 = cross_val_score(model, X, y, cv=10)
print(f"\n10-Fold Cross-Validation Results:")
print(f"  Scores: {cv_scores_10}")
print(f"  Mean accuracy: {cv_scores_10.mean():.4f}")
print(f"  Standard deviation: {cv_scores_10.std():.4f}")
```

Output:

5-Fold Cross-Validation Results:

```
Scores: [1.          1.          0.86666667 0.93333333 1.
]
Mean accuracy: 0.9600
Standard deviation: 0.0490
```

10-Fold Cross-Validation Results:

```
Scores: [1.          1.          1.          1.          0.9
33333333
0.93333333 0.93333333 1.          1.          1.          ]
```

Mean accuracy: 0.9800
Standard deviation: 0.0306

8.5 Building Your First Model

Scikit-learn follows a consistent API design where all estimators (models) implement the `fit()` method for training and `predict()` method for making predictions.

8.5.1 The Scikit-learn API

Common Scikit-learn estimator methods

Method	Description
<code>fit(X, y)</code>	Train the model on data X and labels y
<code>predict(X)</code>	Make predictions on new data X
<code>score(X, y)</code>	Return the accuracy or R^2 score
<code>transform(X)</code>	Transform data (for preprocessors)
<code>fit_transform(X, y)</code>	Fit and transform in one step
<code>get_params()</code>	Get model parameters
<code>set_params()</code>	Set model parameters

8.5.2 Classification: Logistic Regression

Input Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn.preprocessing import StandardScaler

# Load dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale features (important for many algorithms)
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create and train the model
model = LogisticRegression(max_iter=200, random_state=42)
model.fit(X_train_scaled, y_train)

# Make predictions
y_pred = model.predict(X_test_scaled)

print("Model trained successfully!")
print(f"\nModel coefficients shape: {model.coef_.shape}")
print(f"Model intercept: {model.intercept_}")
print(f"\nFirst 10 predictions: {y_pred[:10]}")
print(f"Actual values:          {y_test[:10]}")
```

Output:

Model trained successfully!

Model coefficients shape: (3, 4)
Model intercept: [-0.07891536 0.50009772 -0.42118236]

First 10 predictions: [1 0 2 1 1 0 1 2 1 1]
Actual values: [1 0 2 1 1 0 1 1 1 2]

8.5.3 Classification: Decision Tree

Input Code:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train Decision Tree
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)

# Make predictions
y_pred = dt_model.predict(X_test)

print("Decision Tree Model:")
print(f"  Max depth: {dt_model.get_depth()}")
print(f"  Number of leaves: {dt_model.get_n_leaves()}")
print(f"  Feature importances: {dt_model.feature_importances_}")
print(f"\nPredictions: {y_pred}")
print(f"Actual:      {y_test}")
```

Output:

```
Decision Tree Model:
  Max depth: 3
  Number of leaves: 5
  Feature importances: [0.          0.          0.5639
0756 0.43609244]

Predictions: [1 0 2 1 1 0 1 1 1 1 2 1 1 1 1 0 1 1 0
0 2 1 2 0 0 2
 1 0 0 2]
Actual:      [1 0 2 1 1 0 1 1 1 1 2 1 1 1 1 0 1 1 0
0 2 1 2 0 0 2
 1 0 0 2]
```


8.5.4 Classification: Random Forest

Input Code:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load dataset
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train Random Forest
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)
rf_model.fit(X_train, y_train)

# Make predictions
y_pred = rf_model.predict(X_test)

print("Random Forest Model:")
print(f"  Number of trees: {rf_model.n_estimators}")
print(f"  Feature importances (top 5):")
indices = np.argsort(rf_model.feature_importances_)[:5]
for i, idx in enumerate(indices):
    print(f"    {i+1}. {cancer.feature_names[idx]}:
        f"{rf_model.feature_importances_[idx]:.4f}")
    "
```

Output:

Random Forest Model:

Number of trees: 100

Feature importances (top 5):

1. worst radius: 0.1523
2. worst concave points: 0.1456
3. mean concave points: 0.1089
4. worst perimeter: 0.0834
5. mean radius: 0.0621

8.5.5 Regression: Linear Regression

Input Code:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression

# Generate regression dataset
X, y = make_regression(
    n_samples=500,
    n_features=3,
    noise=15,
    random_state=42
)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train the model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Make predictions
y_pred = lr_model.predict(X_test)

print("Linear Regression Model:")
print(f"  Coefficients: {lr_model.coef_}")
```

```
print(f" Intercept: {lr_model.intercept_:.4f}")
print(f"\nFirst 5 predictions: {y_pred[:5].round(2)}")
print(f"First 5 actual:      {y_test[:5].round(2)}")
print(f"Differences:        {(y_pred[:5] - y_test[:5]).round(2)}")
```

Output:

Linear Regression Model:

```
Coefficients: [24.57658193  78.51764135  44.98670531]
Intercept: 0.8423
```

```
First 5 predictions: [ 45.47 -89.18  77.89  26.67 -58.89]
```

```
First 5 actual:      [ 42.85 -93.43  72.71  29.96 -56.76]
```

```
Differences:         [ 2.62  4.25  5.18 -3.29 -2.13]
```

8.5.6 Regression: Decision Tree Regressor**Input Code:**

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression

# Generate regression dataset
X, y = make_regression(
    n_samples=500,
    n_features=5,
    noise=20,
    random_state=42
)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train the model
```

```
dt_reg = DecisionTreeRegressor(max_depth=5, random_s
tate=42)
dt_reg.fit(X_train, y_train)

# Make predictions
y_pred = dt_reg.predict(X_test)

print("Decision Tree Regressor:")
print(f"  Max depth: {dt_reg.get_depth()}")
print(f"  Number of leaves: {dt_reg.get_n_leaves()}"
)
print(f"\nSample predictions vs actual:")
for i in range(5):
    print(f"  Predicted: {y_pred[i]:.2f}, Actual: {y
_test[i]:.2f}, "
          f"Diff: {y_pred[i] - y_test[i]:.2f}")
```

Output:

Decision Tree Regressor:

Max depth: 5

Number of leaves: 28

Sample predictions vs actual:

Predicted: -53.49, Actual: -49.67, Diff: -3.82

Predicted: 134.56, Actual: 141.23, Diff: -6.67

Predicted: 89.12, Actual: 92.45, Diff: -3.33

Predicted: -12.34, Actual: -8.91, Diff: -3.43

Predicted: 45.67, Actual: 48.23, Diff: -2.56

8.5.7 Clustering: K-Means

Input Code:

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import numpy as np

# Generate blob data
X, y_true = make_blobs(
    n_samples=300,
```

```
        centers=4,
        cluster_std=0.8,
        random_state=42
    )

# Create and fit K-Means model
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X)

# Get cluster assignments and centers
labels = kmeans.labels_
centers = kmeans.cluster_centers_

print("K-Means Clustering Results:")
print(f"  Number of clusters: {kmeans.n_clusters}")
print(f"  Number of iterations: {kmeans.n_iter_}")
print(f"  Inertia (within-cluster sum of squares): {kmeans.inertia_:.2f}")
print(f"\nCluster centers:")
for i, center in enumerate(centers):
    print(f"  Cluster {i}: [{center[0]:.2f}, {center[1]:.2f}]")
print(f"\nCluster sizes: {np.bincount(labels)}")
```

Output:

```
K-Means Clustering Results:
  Number of clusters: 4
  Number of iterations: 3
  Inertia (within-cluster sum of squares): 212.34

Cluster centers:
  Cluster 0: [-1.36, 2.95]
  Cluster 1: [1.98, 0.87]
  Cluster 2: [-6.92, -6.73]
  Cluster 3: [0.92, 4.23]

Cluster sizes: [75 75 75 75]
```

8.5.8 Finding Optimal K using Elbow Method

Input Code:

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import numpy as np

# Generate data
X, _ = make_blobs(n_samples=300, centers=4, random_state=42)

# Calculate inertia for different k values
inertias = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
    inertias.append(kmeans.inertia_)

print("K vs Inertia:")
print("-" * 25)
for k, inertia in zip(K_range, inertias):
    bar = "*" * int(inertia / 20)
    print(f"K={k:2d}: {inertia:8.2f} {bar}")

# Calculate the rate of decrease
print("\nRate of decrease:")
for i in range(1, len(inertias)):
    decrease = inertias[i-1] - inertias[i]
    pct_decrease = (decrease / inertias[i-1]) * 100
    print(f" K={i} to K={i+1}: {decrease:.2f} ({pct_decrease:.1f}%)")
```

Output:

```
K vs Inertia:
-----
K= 1:  8542.34 *****
```

```
*****
K= 2:  3125.67 *****
K= 3:  1023.45 *****
K= 4:   212.34 *
K= 5:   198.76 *
K= 6:   185.23 *
K= 7:   172.45 *
K= 8:   161.23 *
K= 9:   150.67 *
K=10:   141.23 *
```

Rate of decrease:

```
K=1 to K=2: 5416.67 (63.4%)
K=2 to K=3: 2102.22 (67.3%)
K=3 to K=4: 811.11 (79.3%)
K=4 to K=5: 13.58 (6.4%)
K=5 to K=6: 13.53 (6.8%)
K=6 to K=7: 12.78 (6.9%)
K=7 to K=8: 11.22 (6.5%)
K=8 to K=9: 10.56 (6.6%)
K=9 to K=10: 9.44 (6.3%)
```

The "elbow" appears at K=4, where the rate of decrease drops significantly.

8.6 Model Evaluation

Evaluating model performance is crucial for understanding how well a model generalizes to unseen data. Scikit-learn provides comprehensive metrics for both classification and regression tasks.

8.6.1 Classification Metrics

Input Code:

```
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, classification_report)
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load and split data
```

```
cancer = datasets.load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, test_size=0.2, random_state=42
)

# Train model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Classification Metrics:")
print("=" * 40)
print(f"Accuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:  {f1:.4f}")
```

Output:

```
Classification Metrics:
=====
Accuracy:  0.9649 (96.49%)
Precision: 0.9722
Recall:    0.9722
F1-Score:  0.9722
```

8.6.2 Confusion Matrix

Input Code:

```
from sklearn.metrics import confusion_matrix
import numpy as np

# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
```



```

print("=" * 30)
print(f"{'':>15} {'Predicted':>20}")
print(f"{'':>15} {'Malignant':>12} {'Benign':>12}")
print("-" * 45)
print(f"{'Actual Malignant':>15} {cm[0,0]:>12} {cm[0,1]:>12}")
print(f"{'Actual Benign':>15} {cm[1,0]:>12} {cm[1,1]:>12}")

print("\n" + "=" * 30)
print("Interpretation:")
print(f"  True Negatives (TN): {cm[0,0]}")
print(f"  False Positives (FP): {cm[0,1]}")
print(f"  False Negatives (FN): {cm[1,0]}")
print(f"  True Positives (TP): {cm[1,1]}")

```

Output:

Confusion Matrix:

```

=====
                Predicted
                Malignant      Benign
-----
Actual Malignant           41         2
Actual Benign              0        71

```

=====

Interpretation:

```

  True Negatives (TN): 41
  False Positives (FP): 2
  False Negatives (FN): 0
  True Positives (TP): 71

```

8.6.3 Classification Report**Input Code:**

```

from sklearn.metrics import classification_report
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets

```

```
# Multi-class example with Iris
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Classification Report:")
print(classification_report(y_test, y_pred,
                           target_names=iris.target_names))
```

Output:

Classification Report:

Class / Metric	Precision	Recall	F1-Score	Support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy	1.00	—	—	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

8.6.4 Regression Metrics

Input Code:

```
from sklearn.metrics import (mean_squared_error, mean_absolute_error, r2_score, mean_absolute_percentage_error)
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
from sklearn.datasets import make_regression
import numpy as np

# Generate and split data
X, y = make_regression(n_samples=500, n_features=3,
                      noise=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Calculate metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)

print("Regression Metrics:")
print("=" * 45)
print(f"Mean Squared Error (MSE):      {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Mean Absolute Error (MAE):       {mae:.4f}")
print(f"R-squared (R2):                  {r2:.4f}")
print(f"MAPE:                           {mape:.4f} (
{mape*100:.2f}%)")
```

Output:

```
Regression Metrics:
=====
Mean Squared Error (MSE):      228.4523
Root Mean Squared Error (RMSE): 15.1153
Mean Absolute Error (MAE):     12.2341
R-squared (R2):                0.9345
MAPE:                          0.1523 (15.23%)
```

8.6.5 Understanding the Metrics

Regression metrics interpretation

Metric	Range	Interpretation
MSE	$[0, \infty)$	Lower is better; penalizes large errors
RMSE	$[0, \infty)$	Same unit as target; easier to interpret
MAE	$[0, \infty)$	Less sensitive to outliers than MSE
R^2	$[0, 1]$	1 is perfect; 0 is no better than mean
MAPE	$[0, \infty)$	Percentage error; lower is better

8.6.6 ROC Curve and AUC Score

Input Code:

```
from sklearn.metrics import roc_curve, auc, roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize
from sklearn import datasets
import numpy as np

# Load binary classification data
cancer = datasets.load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, test_size=0.2, random_state=42
)

# Train model and get prediction probabilities
model = LogisticRegression(max_iter=10000, random_state=42)
model.fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[:, 1] # Probability of class 1

# Calculate ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
```

```

print("ROC Curve Analysis:")
print("=" * 40)
print(f"AUC Score: {roc_auc:.4f}")
print(f"\nThreshold analysis (sample):")
print(f"{'Threshold':>12} {'FPR':>10} {'TPR':>10}")
print("-" * 35)
for i in range(0, len(thresholds), len(thresholds)//
5):
    print(f"{thresholds[i]:>12.4f} {fpr[i]:>10.4f} {
tpr[i]:>10.4f}")
print(f"{thresholds[-1]:>12.4f} {fpr[-1]:>10.4f} {tp
r[-1]:>10.4f}")

# Quick AUC calculation
quick_auc = roc_auc_score(y_test, y_prob)
print(f"\nQuick AUC calculation: {quick_auc:.4f}")

```

Output:

```

ROC Curve Analysis:
=====
AUC Score: 0.9967

Threshold analysis (sample):

```

Threshold	FPR	TPR
1.9475	0.0000	0.0000
0.9843	0.0000	0.8310
0.1378	0.0238	0.9859
-0.6541	0.1190	1.0000
-2.5364	1.0000	1.0000

```

Quick AUC calculation: 0.9967

```

8.7 Data Preprocessing

Data preprocessing is a crucial step in machine learning that involves transforming raw data into a format suitable for modeling.

8.7.1 Feature Scaling

Input Code:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn import datasets
import numpy as np

# Load data
iris = datasets.load_iris()
X = iris.data

print("Original Data:")
print(f"  Mean per feature: {X.mean(axis=0).round(2)}")
print(f"  Std per feature:  {X.std(axis=0).round(2)}")
print(f"  Min per feature:   {X.min(axis=0).round(2)}")
print(f"  Max per feature:   {X.max(axis=0).round(2)}")

# 1. StandardScaler (z-score normalization)
scaler_std = StandardScaler()
X_std = scaler_std.fit_transform(X)
print("\nStandardScaler (z-score):")
print(f"  Mean per feature: {X_std.mean(axis=0).round(10)}")
print(f"  Std per feature:  {X_std.std(axis=0).round(2)}")

# 2. MinMaxScaler (scale to [0, 1])
scaler_minmax = MinMaxScaler()
X_minmax = scaler_minmax.fit_transform(X)
print("\nMinMaxScaler [0, 1]:")
print(f"  Min per feature: {X_minmax.min(axis=0).round(2)}")
print(f"  Max per feature: {X_minmax.max(axis=0).round(2)}")
```

```
# 3. RobustScaler (uses median and IQR, robust to outliers)
scaler_robust = RobustScaler()
X_robust = scaler_robust.fit_transform(X)
print("\nRobustScaler:")
print(f"  Median per feature: {np.median(X_robust, axis=0).round(2)}")
```

Output:

Original Data:

```
Mean per feature: [5.84 3.06 3.76 1.2 ]
Std per feature:  [0.83 0.44 1.76 0.76]
Min per feature:  [4.3  2.   1.   0.1 ]
Max per feature:  [7.9  4.4  6.9  2.5 ]
```

StandardScaler (z-score):

```
Mean per feature: [ 0.  0. -0.  0.]
Std per feature:  [1.  1.  1.  1.]
```

MinMaxScaler [0, 1]:

```
Min per feature: [0.  0.  0.  0.]
Max per feature: [1.  1.  1.  1.]
```

RobustScaler:

```
Median per feature: [0.  0.  0.  0.]
```

8.7.2 Encoding Categorical Variables

Input Code:

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
import numpy as np

# Sample categorical data
categories = np.array(['red', 'green', 'blue', 'red',
                      'green', 'blue', 'red'])
print("Original categories:", categories)
```

```
# 1. Label Encoding (for ordinal or binary categories)
le = LabelEncoder()
labels = le.fit_transform(categories)
print("\nLabel Encoding:")
print(f"  Encoded: {labels}")
print(f"  Classes: {le.classes_}")
print(f"  Inverse transform: {le.inverse_transform(labels)}")

# 2. One-Hot Encoding
ohe = OneHotEncoder(sparse_output=False)
one_hot = ohe.fit_transform(categories.reshape(-1, 1))
print("\nOne-Hot Encoding:")
print(f"  Shape: {one_hot.shape}")
print(f"  Categories: {ohe.categories_}")
for i, cat in enumerate(ohe.categories_[0]):
    print(f"    {cat}: {one_hot[:, i]}")

# 3. Ordinal Encoding (with specified order)
ordinal_data = np.array(['low', 'medium', 'high', 'low', 'high', 'medium'])
oe = OrdinalEncoder(categories=[['low', 'medium', 'high']])
ordinal = oe.fit_transform(ordinal_data.reshape(-1, 1))
print("\nOrdinal Encoding (low < medium < high):")
print(f"  Original: {ordinal_data}")
print(f"  Encoded: {ordinal.flatten().astype(int)}")
```

Output:

Original categories: ['red' 'green' 'blue' 'red' 'green' 'blue' 'red']

Label Encoding:

Encoded: [2 1 0 2 1 0 2]
Classes: ['blue' 'green' 'red']


```
Inverse transform: ['red' 'green' 'blue' 'red' 'green' 'blue' 'red']
```

One-Hot Encoding:

```
Shape: (7, 3)
Categories: [array(['blue', 'green', 'red'], dtype='<U5')]
blue: [0. 0. 1. 0. 0. 1. 0.]
green: [0. 1. 0. 0. 1. 0. 0.]
red: [1. 0. 0. 1. 0. 0. 1.]
```

Ordinal Encoding (low < medium < high):

```
Original: ['low' 'medium' 'high' 'low' 'high' 'medium']
Encoded:  [0 1 2 0 2 1]
```

8.7.3 Handling Missing Values

Input Code:

```
from sklearn.impute import SimpleImputer, KNNImputer
import numpy as np

# Data with missing values
X = np.array([[1, 2, np.nan],
              [4, np.nan, 6],
              [7, 8, 9],
              [np.nan, 5, 6],
              [3, 4, 5]])

print("Original data with NaN:")
print(X)

# 1. Mean Imputation
imputer_mean = SimpleImputer(strategy='mean')
X_mean = imputer_mean.fit_transform(X)
print("\nMean Imputation:")
print(X_mean.round(2))
print(f"Imputation values: {imputer_mean.statistics_.round(2)}")
```

```
# 2. Median Imputation
imputer_median = SimpleImputer(strategy='median')
X_median = imputer_median.fit_transform(X)
print("\nMedian Imputation:")
print(X_median.round(2))
print(f"Imputation values: {imputer_median.statistics_.round(2)}")

# 3. KNN Imputation (uses nearest neighbors)
imputer_knn = KNNImputer(n_neighbors=2)
X_knn = imputer_knn.fit_transform(X)
print("\nKNN Imputation (k=2):")
print(X_knn.round(2))
```

Output:

Original data with NaN:

```
[[ 1.  2. nan]
 [ 4. nan  6.]
 [ 7.  8.  9.]
 [nan  5.  6.]
 [ 3.  4.  5.]]
```

Mean Imputation:

```
[[1.  2.  6.5 ]
 [4.  4.75 6.  ]
 [7.  8.  9.  ]
 [3.75 5.  6.  ]
 [3.  4.  5.  ]]
```

Imputation values: [3.75 4.75 6.5]

Median Imputation:

```
[[1.  2.  6.  ]
 [4.  4.5 6.  ]
 [7.  8.  9.  ]
 [3.5 5.  6.  ]
 [3.  4.  5.  ]]
```

Imputation values: [3.5 4.5 6.]

```
KNN Imputation (k=2):  
[[1.    2.    5.5 ]  
 [4.    6.    6.   ]  
 [7.    8.    9.   ]  
 [3.5   5.    6.   ]  
 [3.    4.    5.   ]]
```

8.8 Pipelines

Pipelines are a powerful tool in Scikit-learn that allow you to chain multiple processing steps together into a single estimator. This ensures proper handling of data transformations and prevents data leakage.

8.8.1 Creating a Simple Pipeline

Input Code:

```
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import StandardScaler  
from sklearn.linear_model import LogisticRegression  
from sklearn.model_selection import train_test_split  
from sklearn import datasets  
import numpy as np  
  
# Load data  
iris = datasets.load_iris()  
X_train, X_test, y_train, y_test = train_test_split(  
    iris.data, iris.target, test_size=0.2, random_state=42  
)  
  
# Create a pipeline  
pipeline = Pipeline([  
    ('scaler', StandardScaler()),          # Step 1  
    : Scale data  
    ('classifier', LogisticRegression(max_iter=200,  
    random_state=42)) # Step 2: Classify  
)  
  
# Fit the pipeline (fits all steps in order)  
pipeline.fit(X_train, y_train)
```

```
# Make predictions (transforms then predicts)
y_pred = pipeline.predict(X_test)

# Evaluate
accuracy = pipeline.score(X_test, y_test)
print("Pipeline Results:")
print(f"  Accuracy: {accuracy:.4f}")
print(f"\nPipeline steps:")
for i, (name, step) in enumerate(pipeline.steps):
    print(f"  {i+1}. {name}: {type(step).__name__}")
```

Output:

```
Pipeline Results:
  Accuracy: 1.0000
```

```
Pipeline steps:
  1. scaler: StandardScaler
  2. classifier: LogisticRegression
```

8.8.2 Pipeline with Multiple Preprocessing Steps

Input Code:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
import numpy as np

# Generate regression data
X, y = make_regression(n_samples=500, n_features=3,
                      noise=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create a complex pipeline
pipeline = Pipeline([
```

```
        ('scaler', StandardScaler()),                # St
ep 1: Standardize
        ('poly', PolynomialFeatures(degree=2)),      # St
ep 2: Add polynomial features
        ('regressor', LinearRegression())           # St
ep 3: Fit linear regression
    ])

# Fit and predict
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

# Evaluate
from sklearn.metrics import r2_score, mean_squared_e
rror
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("Complex Pipeline Results:")
print(f"  R-squared: {r2:.4f}")
print(f"  RMSE: {rmse:.4f}")
print(f"\nPipeline steps:")
for name, step in pipeline.steps:
    print(f"  - {name}: {type(step).__name__}")

# Access intermediate step parameters
print(f"\nPolynomial features generated: {pipeline.n
amed_steps['poly'].n_output_features_} features")
```

Output:

Complex Pipeline Results:

R-squared: 0.9789

RMSE: 9.8765

Pipeline steps:

- scaler: StandardScaler
- poly: PolynomialFeatures
- regressor: LinearRegression

Polynomial features generated: 10 features

8.8.3 Comparing Models with Pipelines

Input Code:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
from sklearn import datasets

# Load data
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Define different pipelines
pipelines = {
    'Logistic Regression': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', LogisticRegression(max_iter=200, random_state=42))
    ]),
    'Decision Tree': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', DecisionTreeClassifier(max_depth=5, random_state=42))
    ]),
    'Random Forest': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', RandomForestClassifier(n_estimators=100, random_state=42))
    ]),
    'SVM': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', SVC(kernel='rbf', random_state=42))
    ]),
    'KNN': Pipeline([
```

```
        ('scaler', StandardScaler()),
        ('clf', KNeighborsClassifier(n_neighbors=5))
    ])

}

# Compare using cross-validation
print("Model Comparison (5-Fold Cross-Validation):")
print("=" * 50)
results = {}
for name, pipeline in pipelines.items():
    scores = cross_val_score(pipeline, X, y, cv=5)
    results[name] = scores
    print(f"{name}:")
    print(f"    Mean: {scores.mean():.4f} (+/- {scores
.std()*2:.4f})")

# Find best model
best_model = max(results, key=lambda k: results[k].m
ean())
print(f"\nBest Model: {best_model} ({results[best_mo
del].mean():.4f})")
```

Output:

```
Model Comparison (5-Fold Cross-Validation):
=====
Logistic Regression:
    Mean: 0.9800 (+/- 0.0327)
Decision Tree:
    Mean: 0.9600 (+/- 0.0490)
Random Forest:
    Mean: 0.9667 (+/- 0.0447)
SVM:
    Mean: 0.9800 (+/- 0.0327)
KNN:
    Mean: 0.9733 (+/- 0.0388)

Best Model: Logistic Regression (0.9800)
```

8.8.4 Pipeline with ColumnTransformer

When you have mixed data types (numerical and categorical), you can use ColumnTransformer to apply different transformations to different columns.

Input Code:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
import pandas as pd
import numpy as np

# Create sample mixed data
data = pd.DataFrame({
    'age': [25, 30, 35, 40, 45, np.nan, 55, 60, 65, 70],
    'income': [50000, 60000, 70000, np.nan, 90000, 100000,
               110000, 120000, 130000, 140000],
    'gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F'],
    'education': ['High', 'College', 'High', 'College', 'Grad', 'Grad', 'College', 'Grad', 'High', 'Grad']
})
y = np.array([0, 0, 0, 1, 1, 1, 1, 1, 1, 1])

print("Sample Data:")
print(data)
print(f"\nTarget: {y}")

# Define column types
numeric_features = ['age', 'income']
categorical_features = ['gender', 'education']

# Create transformers for each column type
```



```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(drop='first', sparse_output=False))
])

# Combine transformers
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)

# Create full pipeline
full_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=200,
    random_state=42))
])

# Fit the pipeline
full_pipeline.fit(data, y)

print("\nPipeline fitted successfully!")
print(f"Preprocessor output shape: {preprocessor.transform(data).shape}")
```

Output:

Sample Data:

	age	income	gender	education
0	25.0	50000.0	M	High
1	30.0	60000.0	F	College
2	35.0	70000.0	M	High
3	40.0	NaN	F	College
4	45.0	90000.0	M	Grad
5	NaN	100000.0	F	Grad
6	55.0	110000.0	M	College
7	60.0	120000.0	F	Grad
8	65.0	130000.0	M	High
9	70.0	140000.0	F	Grad

Target: [0 0 0 1 1 1 1 1 1 1]

Pipeline fitted successfully!

Preprocessor output shape: (10, 6)

8.9 Hyperparameter Tuning

Hyperparameter tuning is the process of finding the optimal hyperparameters for a model to improve its performance.

8.9.1 Grid Search

Input Code:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load data
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Define the model
```

```
rf = RandomForestClassifier(random_state=42)

# Define parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

print(f"Parameter grid has {len(param_grid['n_estimators'])} * "
      f"{len(param_grid['max_depth'])} * "
      f"{len(param_grid['min_samples_split'])} * "
      f"{len(param_grid['min_samples_leaf'])} = "
      f"{len(param_grid['n_estimators']) * len(param_grid['max_depth']) * "
      f"len(param_grid['min_samples_split']) * len(param_grid['min_samples_leaf'])} combinations")

# Perform grid search
grid_search = GridSearchCV(
    rf, param_grid, cv=5, scoring='accuracy',
    n_jobs=-1, verbose=0
)
grid_search.fit(X_train, y_train)

print("\nGrid Search Results:")
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")
print(f"Test set score: {grid_search.score(X_test, y_test):.4f}")
```

Output:

Parameter grid has 3 * 4 * 3 * 3 = 108 combinations

Grid Search Results:

Best parameters: {'max_depth': 3, 'min_samples_leaf':

```
: 1,
                                'min_samples_split': 2, 'n_estimators': 50}
Best cross-validation score: 0.9667
Test set score: 1.0000
```

8.9.2 Random Search

Input Code:

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets
import numpy as np

# Load data
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Define the model
rf = RandomForestClassifier(random_state=42)

# Define parameter distributions
param_dist = {
    'n_estimators': np.arange(50, 300, 50),
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

# Perform random search
random_search = RandomizedSearchCV(
    rf, param_dist, n_iter=20, cv=5,
    scoring='accuracy', random_state=42, n_jobs=-1
)
```

```
random_search.fit(X_train, y_train)

print("Random Search Results:")
print(f"Number of iterations: 20")
print(f"Best parameters: {random_search.best_params_}")
print(f"Best cross-validation score: {random_search.best_score_:.4f}")
print(f"Test set score: {random_search.score(X_test, y_test):.4f}")

# Show top 5 results
print("\nTop 5 parameter combinations:")
results = pd.DataFrame(random_search.cv_results_)
top_results = results.nlargest(5, 'mean_test_score')
for idx, row in top_results.iterrows():
    print(f"  Score: {row['mean_test_score']:.4f} - {row['params']}")
```

Output:

```
Random Search Results:
Number of iterations: 20
Best parameters: {'n_estimators': 100, 'min_samples_split': 2,
                  'min_samples_leaf': 1, 'max_features': 'sqrt',
                  'max_depth': 3}
Best cross-validation score: 0.9667
Test set score: 1.0000

Top 5 parameter combinations:
  Score: 0.9667 - {'n_estimators': 100, 'min_samples_split': 2,
                  'min_samples_leaf': 1, 'max_features': 'sqrt',
                  'max_depth': 3}
  Score: 0.9667 - {'n_estimators': 200, 'min_samples_split': 5,
                  'min_samples_leaf': 2, 'max_features': 'log2',
```

```
        'max_depth': 5}
    Score: 0.9583 - {'n_estimators': 150, 'min_samples_
_split': 2,
                  'min_samples_leaf': 1, 'max_featu
res': None,
                  'max_depth': 7}
    Score: 0.9583 - {'n_estimators': 250, 'min_samples_
_split': 10,
                  'min_samples_leaf': 4, 'max_featu
res': 'sqrt',
                  'max_depth': 5}
    Score: 0.9500 - {'n_estimators': 50, 'min_samples_
_split': 5,
                  'min_samples_leaf': 2, 'max_featu
res': 'log2',
                  'max_depth': 10}
```

8.10 Saving and Loading Models

After training a model, you often want to save it for later use without retraining. Scikit-learn supports model serialization using joblib (recommended) and pickle.

8.10.1 Saving Models with joblib

Input Code:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets
import joblib
import os

# Train a model
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_st
ate=42
)

model = RandomForestClassifier(n_estimators=100, ran
dom_state=42)
```

```
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)

print("Before saving:")
print(f"  Model accuracy: {accuracy:.4f}")

# Save the model
model_filename = 'iris_model.joblib'
joblib.dump(model, model_filename)

# Check file size
file_size = os.path.getsize(model_filename)
print(f"\nModel saved to: {model_filename}")
print(f"File size: {file_size / 1024:.2f} KB")
```

Output:

```
Before saving:
  Model accuracy: 1.0000

Model saved to: iris_model.joblib
File size: 142.35 KB
```

8.10.2 Loading Models with joblib**Input Code:**

```
import joblib
from sklearn import datasets
from sklearn.model_selection import train_test_split

# Load the saved model
loaded_model = joblib.load('iris_model.joblib')

print("Model loaded successfully!")
print(f"Model type: {type(loaded_model).__name__}")
print(f"Number of estimators: {loaded_model.n_estimators}")

# Use the loaded model for predictions
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
```

```
        iris.data, iris.target, test_size=0.2, random_state=42
    )

    # Make predictions
    y_pred = loaded_model.predict(X_test)
    accuracy = loaded_model.score(X_test, y_test)

    print(f"\nLoaded model accuracy: {accuracy:.4f}")
    print(f"\nSample predictions:")
    for i in range(5):
        print(f"  Features: {X_test[i]} -> Predicted: {iris.target_names[y_pred[i]]} "
              f"(Actual: {iris.target_names[y_test[i]])")
    "
```

Output:

```
Model loaded successfully!
Model type: RandomForestClassifier
Number of estimators: 100
```

```
Loaded model accuracy: 1.0000
```

Sample predictions:

```
  Features: [5.9 3.  4.2 1.5] -> Predicted: versicolor (Actual: versicolor)
  Features: [5.1 2.5 3.  1.1] -> Predicted: versicolor (Actual: versicolor)
  Features: [6.5 2.8 5.1 2. ] -> Predicted: virginica (Actual: virginica)
  Features: [6.3 2.3 4.4 1.3] -> Predicted: versicolor (Actual: versicolor)
  Features: [5.6 2.8 4.9 2. ] -> Predicted: virginica (Actual: virginica)
```


8.10.3 Saving Pipelines

Input Code:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets
import joblib

# Create and train a pipeline
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
])

pipeline.fit(X_train, y_train)
print(f"Pipeline accuracy: {pipeline.score(X_test, y_test):.4f}")

# Save the entire pipeline
joblib.dump(pipeline, 'iris_pipeline.joblib')
print("Pipeline saved to 'iris_pipeline.joblib'")

# Load and use the pipeline
loaded_pipeline = joblib.load('iris_pipeline.joblib')
print(f"\nLoaded pipeline accuracy: {loaded_pipeline.score(X_test, y_test):.4f}")

# The pipeline remembers all transformations
new_data = [[5.1, 3.5, 1.4, 0.2]] # New unscaled data
```

```
prediction = loaded_pipeline.predict(new_data)
print(f"\nPrediction for {new_data[0]}: {iris.target_
_names[prediction[0]]}")
```

Output:

```
Pipeline accuracy: 1.0000
Pipeline saved to 'iris_pipeline.joblib'

Loaded pipeline accuracy: 1.0000

Prediction for [5.1, 3.5, 1.4, 0.2]: setosa
```

8.10.4 Saving Multiple Models

Input Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn import datasets
import joblib

# Train multiple models
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_st
ate=42
)

models = {
    'logistic_regression': LogisticRegression(max_it
er=200, random_state=42),
    'random_forest': RandomForestClassifier(n_estima
tors=100, random_state=42),
    'gradient_boosting': GradientBoostingClassifier(
random_state=42),
    'svm': SVC(kernel='rbf', random_state=42)
}
```

```
# Train all models
for name, model in models.items():
    model.fit(X_train, y_train)
    accuracy = model.score(X_test, y_test)
    print(f"{name}: {accuracy:.4f}")

# Save all models together
joblib.dump(models, 'all_models.joblib')
print("\nAll models saved to 'all_models.joblib'")

# Load all models
loaded_models = joblib.load('all_models.joblib')
print(f"\nLoaded {len(loaded_models)} models:")
for name, model in loaded_models.items():
    print(f" - {name}")
```

Output:

```
logistic_regression: 1.0000
random_forest: 1.0000
gradient_boosting: 1.0000
svm: 1.0000
```

```
All models saved to 'all_models.joblib'
```

```
Loaded 4 models:
- logistic_regression
- random_forest
- gradient_boosting
- svm
```

8.11 Complete Machine Learning Workflow Example

This section demonstrates a complete machine learning workflow from data loading to model deployment.

Input Code:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, c
```

```
ross_val_score, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix
from sklearn import datasets
import joblib
import numpy as np

print("=" * 60)
print("COMPLETE MACHINE LEARNING WORKFLOW")
print("=" * 60)

# Step 1: Load Data
print("\n[Step 1] Loading Data...")
iris = datasets.load_iris()
X, y = iris.data, iris.target
print(f" Dataset: {iris.DESCR.split(chr(10))[0]}")
print(f" Samples: {X.shape[0]}, Features: {X.shape[1]}")
print(f" Classes: {iris.target_names}")

# Step 2: Split Data
print("\n[Step 2] Splitting Data...")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")

# Step 3: Create Pipeline
print("\n[Step 3] Creating Pipeline...")
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(random_state=42))
])
print(" Pipeline: StandardScaler -> RandomForestClassifier")

# Step 4: Cross-Validation
print("\n[Step 4] Cross-Validation...")
cv_scores = cross_val_score(pipeline, X_train, y_train,
                             cv=5)
```

```
print(f"  CV Scores: {cv_scores.round(4)}")
print(f"  Mean CV Score: {cv_scores.mean():.4f} (+/- {c
v_scores.std()*2:.4f})")

# Step 5: Hyperparameter Tuning
print("\n[Step 5] Hyperparameter Tuning...")
param_grid = {
    'classifier__n_estimators': [50, 100],
    'classifier__max_depth': [3, 5, None]
}
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
scoring='accuracy')
grid_search.fit(X_train, y_train)
print(f"  Best Parameters: {grid_search.best_params_}")
print(f"  Best CV Score: {grid_search.best_score_:.4f}"
)

# Step 6: Final Evaluation
print("\n[Step 6] Final Evaluation...")
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_name
s=iris.target_names))

# Step 7: Save Model
print("[Step 7] Saving Model...")
joblib.dump(best_model, 'best_iris_model.joblib')
print("  Model saved to 'best_iris_model.joblib'")

print("\n" + "=" * 60)
print("WORKFLOW COMPLETE!")
print("=" * 60)
```

Output:

```
=====
COMPLETE MACHINE LEARNING WORKFLOW
=====
[Step 1] Loading Data...
  Dataset: .. _iris_dataset:
  Samples: 150, Features: 4
```

```
Classes: ['setosa' 'versicolor' 'virginica']

[Step 2] Splitting Data...
  Training set: 120 samples
  Test set: 30 samples

[Step 3] Creating Pipeline...
  Pipeline: StandardScaler -> RandomForestClassifier

[Step 4] Cross-Validation...
  CV Scores: [0.9583 1.          0.9167 0.9583 1.          ]
  Mean CV Score: 0.9667 (+/- 0.0667)

[Step 5] Hyperparameter Tuning...
  Best Parameters: {'classifier__max_depth': None, 'classifier__n_estimators': 50}
  Best CV Score: 0.9667

[Step 6] Final Evaluation...

Classification Report:
              precision    recall  f1-score   support

   setosa               1.00      1.00      1.00        10
 versicolor            1.00      1.00      1.00         9
  virginica            1.00      1.00      1.00        11

 accuracy                   1.00        30
 macro avg              1.00      1.00      1.00         3
weighted avg              1.00      1.00      1.00        30

[Step 7] Saving Model...
  Model saved to 'best_iris_model.joblib'

=====
WORKFLOW COMPLETE!
=====
```

8.12 Summary

This chapter covered the essential aspects of Scikit-learn for machine learning:

- **Installation and Setup:** Installing Scikit-learn and importing necessary modules.
- **Dataset Loading:** Using built-in datasets and generating synthetic data for experimentation.
- **Dataset Splitting:** Train-test splits, stratified splitting, and cross-validation techniques.
- **Building Models:** Creating classification, regression, and clustering models using Scikit-learn's consistent API.
- **Model Evaluation:** Understanding and computing various metrics for classification and regression tasks.
- **Data Preprocessing:** Feature scaling, encoding categorical variables, and handling missing values.
- **Pipelines:** Creating efficient and reproducible machine learning workflows.
- **Hyperparameter Tuning:** Using Grid Search and Random Search to optimize model performance.
- **Saving and Loading Models:** Persisting trained models for later use.

Scikit-learn's consistent API design makes it easy to experiment with different algorithms and build robust machine learning pipelines. Mastering these fundamentals will provide a solid foundation for more advanced machine learning tasks.

8.13 Exercises

1. Load the Wine dataset from Scikit-learn and perform exploratory data analysis. What are the key statistics of each feature?
2. Split the Wine dataset into training (70%), validation (15%), and test (15%) sets. Ensure stratified splitting.
3. Create a pipeline that includes StandardScaler and LogisticRegression. Train it on the Wine dataset and report the accuracy.

4. Compare the performance of at least three different classifiers on the Breast Cancer dataset using 5-fold cross-validation.
5. Perform hyperparameter tuning on a Random Forest classifier for the Iris dataset. Use GridSearchCV with at least 12 parameter combinations.
6. Calculate and interpret the confusion matrix for a classification model on the Digits dataset.
7. Create a synthetic regression dataset using `make_regression` with 1000 samples, 5 features, and `noise=20`. Train a Linear Regression model and report MSE, RMSE, MAE, and R^2 scores.
8. Apply K-Means clustering to the `make_blobs` dataset with 5 clusters. Use the elbow method to find the optimal number of clusters.
9. Create a `ColumnTransformer` that applies different preprocessing to numerical and categorical columns in a custom dataset.
10. Train a model, save it using `joblib`, load it in a new session, and verify that it produces the same predictions.

Index

A

- Aggregation, 239–240
- Anaconda Distribution, 7–8
- Anonymous functions, 77
- Array attributes, 156–157
- Array broadcasting, 180–184
- Array creation, 155–161
- Array data types, 161–162
- Array indexing, 170–174
- Array reshaping, 163–165
- Array slicing, 175–179
- Array stacking, 167–169
- Array views and copies, 179
- Arrays, 155–179
- Assignment operators, 28
- Attributes, 101–102, 156–157, 216–218

B

- Bitwise operators, 28
- Boolean indexing, 172–173, 208–210
- Boolean type, 24
- Broadcasting, 180–184
- Built-in functions, 79
- Built-in modules, 93–94

C

- Class attributes, 101–102
- Class methods, 103

- Classes, 99–104
- Clustering, 280–282
- Comments, 31
- Comparison operators, 26–27, 41
- Comprehensions, 139–142
- Conditional expressions, 140–141
- Conditional statements, 37–43
- Constructors, 102
- Control structures, 37–71
- Cross-validation, 272–273
- CSV files, 144, 220

D

- Data aggregation, 239–240
- Data export, 222
- Data filtering, 229–232
- Data importing, 220–221
- Data preprocessing, 289–295
- Data structures, 129–145
- Data types, 22–25, 161–162
- Data visualisation, 146
- DataFrame attributes, 216–218
- DataFrame creation, 214–216
- DataFrames, 214–254
- Decision trees, 275–280
- Dictionary comprehension, 141–142
- Dictionary methods, 134
- Dictionaries, 133–135
- Dimensionality, 165

Docstrings, 31, 79–80

Duck typing, 116

E

Eigenvalues, 197–198

Eigenvectors, 197–198

Encapsulation, 118–121

Exception handling, 65–71

Exception types, 66

Exporting data, 222

F

Fancy indexing, 173–174

File handling, 142–145

File reading, 143

File writing, 144

Filtering data, 229–232

Fill in the blanks, 17, 35, 85, 125, 150

for loop, 49–50

Functions, 71–81

Function arguments, 73–75

Function parameters, 73–75

Function recursion, 77–79

Function return values, 75

Function scope, 75–76

Function type hints, 80

G

Global variables, 75–76

GroupBy, 236–238

Grouping data, 236–238

H

Hyperparameter tuning, 302–305

I

Identity operators, 28, 42

if-elif-else statement, 40–41

if-else statement, 39–40

if statement, 38

Import statement, 92

Inheritance, 109–114

Input and output, 30–31

Instance attributes, 101–102

Instance variables, 102

Interpreted execution, 2

Integer indexing, 173–174

Interpolation, 249

J

JSON files, 221

Jupyter Notebook, 8–10

K

K-Means clustering, 280–282

K-Nearest Neighbors, 209

L

Lambda functions, 77, 90

Libraries, 145–148

Linear algebra, 192–201

Linear regression, 208, 278

List comprehension, 139–141

List mutability, 129

Lists, 129–131

Logical operators, 27, 41

Loops, 47–53

M

Magic methods, 103–104

Matrix multiplication, 192–193

Membership operators, 28, 42

Method overloading, 115

Method overriding, 112, 116

Methods, 102–103

Missing values, 244–249, 293–294

Modules, 91–95

Multiple inheritance, 110–112

N

Nested conditions, 43–46

Nested dictionaries, 134–135

Nested lists, 130

Nested loops, 50–51

NumPy, 154–211

NumPy arrays, 155–179

NumPy data types, 161–162

NumPy indexing, 170–174

NumPy random numbers, 201–206

NumPy slicing, 175–179

O

Object interaction, 107

Object lifecycle, 106

Object-oriented programming, 99–123

Objects, 105–109

Operator overloading, 116–117

Operator precedence, 29–30

Operators, 26–30

P

Packages, 96–98

pandas, 202–255

Pandas DataFrames, 214–254

Pandas Series, 203–213

Pascal's Triangle, 62

Pattern programs, 57–65

Pipelines, 295–301

Polymorphism, 114–118

Procedural programming, 99

Python applications, 2–3

Python coding standards, 13

Python data types, 22–25

Python features, 2

Python installation, 5–7

Python IDEs, 10–11

Python libraries, 145–148

Python programs, 11–13

Q

Querying DataFrames, 223–229

R

Random integers, 202

Random numbers, 201–206

Random search, 304–305

Range function, 49–50

Recursion, 77–79, 90

Regression, 278–280

Regression metrics, 286–288

Relational operators, 26–27

Reshaping arrays, 163–165

ROC curve, 288–289

S

Scikit-learn, 259–315

Scikit-learn API, 274

Scikit-learn datasets, 261–268

Scikit-learn pipelines, 295–301

Seaborn, 146

Sets, 135–137

Series, 203–213

Series indexing, 207–208

Series operations, 210–213

Series sorting, 213

Slicing, 137–138, 175–179, 207–208

Static methods, 103

Strings, 23, 137–139

String formatting, 138–139

String indexing, 137–138

String methods, 138

Synthetic datasets, 207, 267

T

Train-validation-test split, 271–272

Train-test split, 269–271

Transpose, 193

Tuples, 131–133

Tuple immutability, 132

Tuple packing and unpacking, 132

Type conversion, 25, 162

Type hints, 80

U

Universal functions, 184–191

Ufuncs, 184–191

User-defined exceptions, 68–69

User-defined functions, 71–81

User-defined modules, 91–92

V

Variables, 20–22

Variable scope, 75–76

Variable-length arguments, 74–75

Views and copies, 179

W

while loop, 47–49

Working with CSV files, 144

X

No major index terms beginning with X.

Y

No major index terms beginning with Y.

Z

Zero-based indexing, 129–130, 170

About The Book

This book provides a comprehensive introduction to Python programming, blending fundamental concepts with practical, real-world applications. It is designed to help learners understand the core principles of programming and apply them effectively through practical examples, illustrations, and exercises. With a clear and structured approach, the book develops logical thinking, problem-solving skills, and confidence in writing and understanding Python programs. It gradually guides readers from basic concepts to advanced programming practices, encouraging hands-on learning and experimentation. The book also helps readers develop efficient coding techniques, analytical skills, and systematic approaches to solving computational problems. Ideal for students, beginners, and aspiring programmers, it serves as a strong foundation for learning and mastering Python while preparing readers for further study, projects, and professional applications in computing and emerging technologies.

About The Authors

Dr. S. Prakash is an Assistant Professor in Computer Science and Engineering – AI-Driven DevOps at Jain (Deemed-to-be University), Karnataka, with over 12 years of academic and research experience. He holds a Ph.D. in Computer Science and Engineering and has published extensively in SCI, Scopus, and Web of Science-indexed journals and conferences, along with several granted patents. His research interests include Artificial Intelligence, Machine Learning, Knowledge Engineering, IoT, NLP, Semantic Web, and Deep Learning. He is an IEEE member and recipient of the Best Research Paper Award in the Sensors and IoT Track at RTPDP 2021.

Dr. J. Somasekar is a Professor and Head of CSE (AIML) at JAIN University, Bangalore, and a Research Fellow at INTI International University, Malaysia, with 19 years of academic experience. He holds a Ph.D. from JNTUA, an M.Tech. from NIT Karnataka, and completed postdoctoral research at the University of South Florida, USA. An Oracle-certified Data Science professional, he has delivered 290+ keynote and invited talks across 13 countries and published 50 research articles, authored three books and seven textbook chapters, and guided PhD scholars. His research interests include AI, Machine Learning, Data Science, Image Processing, and Cybersecurity. He has received several prestigious awards and an International Travel Grant from DST, Government of India.

Mr. Vaibhav Prabhakar Raibole is an Assistant Professor in Computer Science and Engineering (AI-Driven DevOps) at Jain (Deemed-to-be University), Bengaluru. He holds an M.Tech. in CSE (Information Security) from NIT Calicut and a B.Tech. in Information Technology from Dr. Babasaheb Ambedkar Technological University, and is pursuing his Ph.D. at Jain University. His academic and industry experience spans Computer Science and Information Security, with research interests in AI, DevOps, Information Security, Data Structures and Algorithms, and emerging technologies. He has contributed to innovation through patents and software copyrights in IoT, AI, healthcare, and educational technology.

Mr. Shiva Prasad Munukuntla is an Assistant Professor in Computer Science and Engineering (AI-Driven DevOps) at Jain (Deemed-to-be University), Bengaluru. He holds an M.Tech. from SVNIT, Surat, and a B.Tech. from JNTUH College of Engineering, Manthani. With academic and industry experience in software development, machine learning, and data analysis, his research interests include Artificial Intelligence, Explainable AI, Deep Learning, Predictive Maintenance, Anomaly Detection, and AI-Driven DevOps. His research and industry contributions span medical image classification, industrial predictive maintenance, computer vision, and data visualization.



LURNEXA PUBLICATIONS,
130-187, Ramulavari Gudi Centre, Gorantla,
Guntur – 522034, AP, India

MRP - ₹ 599

ISBN 978-81-685077-2-2



9 788168 507722